1. INTRODUCTION TO THE ANALYSIS OF ALGORITHMS

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

The Analysis of Algorithms course aims to introduce formal frameworks in which we can explore various issues related to computer science, in an unambiguous and rigorous manner. Having had experience programming, you should posses some intuitive understanding of what computers are, what they can do and how well they can do it. But, while intuition does go a long way, we assert that a formal approach is needed to complement it and provide us with certainty about what we're doing and how we're doing it.

The course is structured into three distinct chapters:

1. **Computability Theory** is the branch of Computer Science concerned with addressing questions about computers are and what they can do. By now, you should be familiar with some things that computers *can* do: multiply two numbers, sort a list, find the shortest path in a graph etc. Perhaps less intuitive is what computers *cannot* do; in a future lecture we will present some problems that are impossible for a computer to solve.

2. **Complexity Theory**. Having established precisely what computers *can* do, we move on to assess *how efficiently* they can do it. In this chapter, we will address questions related to what efficiency is, what computational resources are there, how to evaluate an algorithm without getting bogged down in the details, how to know if a solution is optimal etc.

3. **Correctness**. The first two chapters are somewhat constructive in nature, providing us with techniques for building solutions to given problems. But once we have an algorithm, how can we make sure that it does actually solve the problem that it's supposed to? In this final chapter, we look at some methods that allow us to obtain *formal guarantees* about the behavior of our code.

As one can infer from the course title, *algorithms* will play a central role in our discussion. But what exactly is an algorithm?

1 Algorithms

You are most likely already familiar with some algorithms and how to represent them. For our first example, consider this pseudocode of Euclid's algorithm:

```
while (a ≠ b)
    if (a > b)
        a := a − b
    else
        b := b − a

    return a
```

Euclid's repeated-subtraction algorithm

The problem at hand is finding the greatest common divisor between two natural numbers a and b. The algorithm is a "recipe" for doing so, referring to the numbers themselves in a symbolic manner, such that if you were to set the initial value of a to 49 and the initial value of b to 84 and follow all instructions, at the end you'd get the answer: 7.

For now, we shall rely on the following informal definition:

> *"An algorithm is a finite sequence of precise steps that can be used to solve a problem in a finite amount of time".*

There are some key elements of this description that we need to unpack:

- *"finite sequence of steps"* refers to the fact that the description of the algorithm itself has to be finite. In the example above, you can think of each "step" as being one line of the pseudocode.

- *"precise steps"* – while not very precise in itself, this requirement tries to match the intuition that this isn't an algorithm for finding the GCD of two numbers: *determine the GCD of a and b*.

  This is a finite sequence of steps, but it doesn't tell us anything about *how* exactly we can do that, whereas it's very clear what each line of the pseudocode entails.

  Or is it? You could ask: "given two numbers a and b, how can I check if $a \neq b$?" Indeed, we could break this operation into a sequence of more basic ones: for example, if a and b are represented as strings of digits, we first compare their lengths and if they match, we go through the pairs of symbols at corresponding positions in the two strings – if at any point we find a mismatch, the numbers are different, else they're the same.

  For now, we won't dwell on the precise definition of "precise", but having such concerns is the right attitude in our quest to obtain a formal description of algorithms.

- *a finite amount of time* – obviously, for an algorithm to be useful it has to give us the answer, without making us wait forever.

Remember that our aim is to establish a rigorous framework for computational matters so we will need to provide a *formal* definition of *"algorithm"*, described in the language of mathematical notation.

But before we get there, we need to address a keyword that was mentioned in our informal definition: *"problem"*.

This is the aim of the first lecture. You will see that our definition is simple and intuitive, requiring only already-familiar mathematical notions.

Having established what a problem is and thus what exactly we can hope to *solve*, in the next lecture we will address algorithms, by introducing a formal *computational model* – the Turing Machine.

2 Problems

For uniformity, we shall phrase all our problems as questions. Some familiar tasks that computers solve are more naturally expressed as imperative statements, e.g. "sort the list [43, 1, 18]!" We will rephrase such a statement as: "what is the sorted form of the list [43, 1, 18]?"

Consider the following questions:

- Is 0 prime?
- Is 1 prime?
- Is 2 prime?
- Is 451 prime?
- etc.

We can see that much of the question structure remains the same; only one part of it – the number – changes. We can generalize the question above as: "Is x prime?"; we call x *"the input"*. Thus problems are "templates" that ask a question about some generic object. If we substitute the generic object with a concrete one, we get one problem instance, with a particular answer.

The generic form of the sorted-list problem from above is: "What is the sorted form of the list l?"

Each problem has one answer, which we call *the output*.

The *meaning* of a problem is such that each possible input corresponds to exactly one output. There's a mathematical notion that fits this ideas very well: *a function*.

Definition 2.1. A *problem* is a function $f : I \to O$ from a set of *inputs* to a set of *outputs*. Although the input and output sets can be infinite (and the input set usually is), *each member of the set is finite.*

We require inputs and outputs to be finite in length to match the final part of our intuitive definition: *"[...] in a finite amount of time"*. If a problem statement contained an object of infinite length[†], then we couldn't ever finish *reading* it, so we could not start solving it. Similarly, if the output is infinite in length, we wouldn't ever finish *writing* it, so we cannot say that we provided the answer in a finite amount of time.

Note 2.1. You might well object at this point that computers can deal with irrational numbers with infinite decimal representations, such as π. Most of the time, though, that is not true; they only work with *finite approximations*.

But computers can indeed handle actual irrational numbers. However they cannot handle *random real numbers*, just a subset of the reals.

π does have finite representations, it's just that they're not strings of decimal digits. For example, consider the Weierstrass definition: $\pi = 4\int_0^1 \frac{1}{1+x^2}dx$.

This point will make more sense later; for now, you don't need to focus on it.

2.1 Decision problems

Definition 2.2. A *decision problem* is a problem with a boolean answer: `TRUE` or `FALSE`.

In other words, a decision problem is a "yes" or "no" question.

Examples:

- Is this number prime?
- Is this graph fully connected?
- Can this program ever trigger a `SEGFAULT`?

During our future discussions on Computability, Complexity and Correctness, we will focus almost exclusively on decision problems. This is because decision problems have the simplest possible answer structure, which will make our analyses succinct.

Obviously, there are many relevant problems in the real world which are not decision problems. We will later see how what we learn about decision problems can be adapted to other types.

2.2 Function problems

Function problems are basically everything else, i.e. problems whose answer is a mathematical object, other than a boolean.

For example:

- What are the prime factors of n?
- What is b to the power of e?
- What is the largest element in list l?

Note 2.2. Our classification of problems is not exhaustive. For example, there are practically useful problems for whom multiple correct answers are possible and thus cannot be modelled as functions. Consider the question "What is one prime factor of 12?" Both 2 and 3 are valid answers and it should be obvious that it makes sense in practice to ask such questions.

There are other possible models for problems (in particular, the one above is an example of a *search problem*, which can be modeled as a binary relation), but they are outside the scope of `AA` .

For simplicity, we shall limit ourselves to only those kinds of problems that fit Definition 2. Most of the time, we'll discuss decision problems specifically.

Our definitions are too inexact – they don't say much about either the domain or the range of functions.

[†]There are areas that explore the notion of infinite-length input, but they are well outside the scope of `AA` . The key term is "type 2 computability theory".

Some problems concern numbers, some concern lists, other concern graphs. These are vastly different objects; we would like a common way of representing all of them, so that we can have a more precise view on how our inputs (and outputs) look like.

In search of a common representation, we will now take a short detour through the domain of formal languages. The concepts presented in the following section will be studied in more detail at the `LFA` course in the third year.

3 Alphabets and strings

Definition 3.1. An *alphabet* is a finite set whose members we call *"symbols"*.

In Computer Science, the term "alphabet" is used by analogy with natural language alphabets, such as the Latin Alphabet, the Cyrillic Alphabet etc.

However, alphabets are just sets; they're not some fancier mathematical object, but just a convention that makes things easier to interpret in some contexts.

Alphabets are usually denoted by a capital sigma, Σ; here are some examples:

$$\Sigma_1 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$$

$$\Sigma_2 = \{0, 1\}$$

$$\Sigma_3 = \{\square, \triangle, \circ\}$$

Definition 3.2. A *string* over alphabet Σ is a finite sequence of symbols from Σ.

Example strings:

$$s_1 = algorithms$$

$$s_2 = 1001110$$

$$s_3 = \square\square\triangle\square \circ \triangle$$

(Usually we will not explicitly state the alphabet from which a string's symbols are drawn, as it will be clear from the context).

The string of length 0, consisting of no symbols is called *"the empty string"* and is denoted by ϵ[‡].

Definition 3.3. Let Σ be an alphabet. Then Σ^* is *the set of all possible strings over* Σ.

Note that this includes the *empty string*, ϵ.

For $\Sigma_1 = \{a, b, c\}$:

$$\Sigma_1^* = \{\epsilon, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, aab, ...\}$$

For $\Sigma_2 = \{\square, \triangle\}$:

$$\Sigma_2^* = \{\epsilon, \square, \triangle, \square\square, \square\triangle, \triangle\square, \triangle\triangle, \square\square\square, \square\square\triangle, ...\}$$

4 Representing objects with strings

Any finite object can be represented as a string of symbols. This may be somewhat obscured by how we use pen-and-paper to create complex 2D combinations of symbols, but a structure like:

$$\prod_{i=1}^{n} \sqrt{\frac{a_i^2}{b_i}}$$

[‡]We'll avoid confusion between the empty string and a regular symbol by never using Greek Letters in our alphabets.

can easily be converted to a string like "$\Pi(i = 1; n)(\sqrt{}((a_i\hat{}2)/(b_i)))$".

From now on, we shall consider only the string representation of inputs; the alphabet is irrelevant and we will choose it to suit our needs. We shall see, in a future lecture, that the binary alphabet $\Sigma = \{0, 1\}$ is sufficient for all our purposes.

We can now say that a problem is a function from strings to outputs:

$$f : \Sigma^* \to O$$

For decision problems, because there are only two outputs (`FALSE` and `TRUE`), we shall treat them in a special way.

So we can revisit Definition 2.1 and say that a decision problem is a function from strings to $\{FALSE, TRUE\}$:

$$f : \Sigma^* \to \{FALSE, TRUE\}$$

For functional problems, because outputs are arbitrary objects, we can also talk instead about their *string encoding*. So we can say a function problem is a function from strings to strings.

$$f : \Sigma^* \to \Sigma^*$$

THE TURING MACHINE

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

In the first part of the AA course we will focus our attention on what exactly is it that computers can or cannot do. We shall study these issues within the framework of "computability theory", a branch of mathematics. This means that our approach will bear all the characteristics of mathematical rigor: we will try to capture intuitive concepts as formal definitions and logical axioms and we will use inference rules to derive new statements that are *provably true* in our framework.

Our main question can be informally phrased as: *"What problems can algorithms solve?"* In order to approach this question, we shall first need to precisely define what we mean when we use the vague terms: "problems", "algorithms" and "solve".

The concept of "problem" has been addressed in the previous lecture; for our purposes, a problem is a function from a set of strings to another. A key part of this definition is that both the domain and the range can be infinite (and the domain usually is), but all their member strings must be *finite* in length.

We now turn our attention towards what an algorithm is and what it means for an algorithm to solve a problem.

1 Informal view

A Turing Machine is a *computational model* proposed by Alan Turing in 1936. It is a rigorous mathematical abstraction, designed to capture the intuitive notion of "computability". However, it can be visualized as a physical device with tangible components and visible *movement*, which makes it very easy to think about without delving into symbolic notation.

- A Turing Machine consists of a **tape**, a reading **head** and an internal **state**.
- The tape is one-dimensional and split into equally-sized **cells**. It extends indefinitely in either direction, so we can never "run out of tape".
- The machine works by going through a series of **steps**. At each step:
  - The machine is in some state and its head is reading some cell.
  - Depending on the combination of internal state and currently read symbol, the machine makes a particular **transition**.
  - When transitioning, the machine does three things:
    1. It goes to a new internal state; or remains in the same one.
    2. It writes a new symbol on the cell under the head; or leaves it unchanged.
    3. It moves the head to point to the cell immediately on the left or right of the current one; or leaves it where it is.

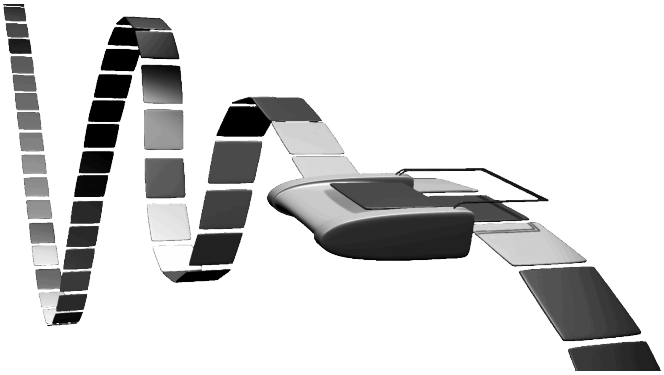


Figure 1: Artistic visualization of a Turing Machine. You can see the tape, split into cells and unbounded in both directions. Here, tape symbols are illustrated as different shades of gray. The printer-like object depicts the "internal control" of the machine – the part that knows how to transition and which memorizes the current internal state. The thin rectangle on its right, hovering above a single cell, is the head.
Image taken from: `https://commons.wikimedia.org/wiki/File:Maquina.png`

2 Formal definition

We shall now summarize the informal presentation from section 1 into a formal definition.

Definition 2.1. A Turing Machine is a 9-tuple: $(Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$. Where:

- Q is a finite set of **internal states**.
- Σ is a finite set of **symbols** used for the machine input (the word that is initially present on the tape) called **the input alphabet**
- Γ is a finite set of **symbols** that the machine can use during its operation, called **the tape alphabet**; $\boxed{\Sigma \subseteq \Gamma}$
- B is **the blank symbol**, the default state of a cell that hasn't been yet written, or that has been erased; $\boxed{B \in \Gamma}$
- q_1 is **the initial state** of the machine before any step is taken; $\boxed{q_1 \in Q}$
- Y is **the accepting state**; once the machine reaches this state, its run is over and it signals that the input is **accepted**, i.e. the answer is "yes".
- N is **the rejecting state**; once the machine reaches this state, its run is over and it signals that the input is **rejected**, i.e. the answer is "no".
- H is **the halting state**; once the machine reaches this state, its run is over and it signals that the computation is done and the answer to the problem is encoded as the current contents of the tape, starting from the first non-blank character and ending with the last. This is useful for function problems, where the answer is a mathematical object other than a boolean.
- δ is **the transition function** which dictates the behavior of the machine during a single-step (note that the machine can transition into the special states Y, N, H, but cannot "move on" from any of those states):

  $$\boxed{\delta : Q \times \Gamma \to Q \cup \{Y, N, H\} \times \Gamma \times \{\leftarrow, -, \rightarrow\}}$$

  The domain captures the idea that a machine's transition is influenced only by the current state and the current symbol read by the head. Based on this pair, the machine makes a transition, changing its state (possibly into one that halts computation), changing the symbol under the head and moving the head: either one cell to the left/right, or keeping it stationary.

  To make our notation more succinct, we shall use $Q' = Q \cup \{Y, N, H\}$ from now on.

3 Machine description

Having established a formal definition of Turing Machines, we can now start describing individual machines that accomplish various tasks.

Here is our first machine, which we shall name `isEven`. It take its input – a big-endian[†] binary number and checks whether it is even (i.e. the last digit is 0).

$isEven = \{\{q_1, q_2\}, \{0, 1\}, \{\square, 0, 1\}, \square, Y, N, H, \delta_{isEven}\}$

$\delta_{isEven}(q_1, 0) = (q_1, 0, \rightarrow)$

$\delta_{isEven}(q_1, 1) = (q_1, 1, \rightarrow)$

$\delta_{isEven}(q_1, \square) = (q_2, \square, \leftarrow)$

$\delta_{isEven}(q_2, 0) = (Y, 0, -)$

$\delta_{isEven}(q_2, 1) = (N, 1, -)$

$\delta_{isEven}(q_2, \square) = (N, \square, -)$

A machine that checks if a number is even.

Before trying to explain how the machine works, we should remark how arcane and intimidating this notation looks like. Fortunately, there are clearer ways of representing a Turing Machine!

3.1 Transition table

The transition function of a Turing Machine is the core of its behavior, so we shall use it as our focal point when designing a clearer representation. The function takes two arguments, which means we can represent it as a 2D table: the rows are states, the columns are tape symbols. Acell contains the triple (next state, written symbol, direction) given by the corresponding combination.

Here is the table representation for δ_{isEven}:

	0	1	$\square$
q_1	$q_1, 0, \rightarrow$	$q_1, 1, \rightarrow$	$q_2, \square, \leftarrow$
q_2	$Y, 0, -$	$N, 1, -$	$N, \square, -$

Looking at the table itself, other information about our machine can be easily extracted: namely the set of tape symbols Γ and the set of states Q.

To be able to figure out the other elements of the 9-tuple, we establish the following conventions:

- the initial state will always be on the *first row* of the table
- the blank symbol will be denoted $\square$
- the halting, accepting and rejecting states, will always be Y, N and H respectively
- the order of the columns will be such that first come all the input symbols, then the blank symbol $\square$, then all other tape symbols

Now we can represent our entire Turing Machine 9-tuple with a concise transition table; conversely, we can look at such a transition table and identify each element of the 9-tuple.

[†]Most significant digit first.

4 `isEven` explained

The machine starts in the initial state, q_1; this state corresponds to the intuitive concept of "searching the end of the input" – thus, at each step, the head is moved one position to the *right*, leaving each symbol as it is, until a blank symbol is found. This is captured by the first two entry of the table.

Once on the blank symbol, the whole input must have been traversed, so the head needs to be moved one cell *left* onto the last symbol of the input and the machine transitions to state q_2, which corresponds to the intuitive concept of "checking what the last symbol is". This is captured by the last entry in the first row.

If this symbol is a 0, the number is even, so it's accepted by transitioning to state Y, providing a "yes" answer. The other two elements of the transition action (namely the written symbol and the movement direction of the head) are irrelevant because computation halted. But we choose the intuitive option of keeping the symbol as it was (a 0) and holding the head still.

If the symbol is a 1, the number is odd, not even, so it's rejected by transitioning to state N, providing a "no" answer.

Note that in state q_1 the head is only moved right, traversing the whole input and stopping at *the very first* blank symbol. This means that, in q_2, if the blank symbol is encountered, there is only one possibility: the input was empty.

5 The empty input and other arbitrary strings

In the beginning, we said that inputs to our machines are strings of symbols from the input alphabet. When constructing `isEven`, we interpreted the input to be "a number in base 2". However, not every string of 0s and 1s corresponds straightforwardly to a number; for example, what about "00101"?

We can adopt the simple convention of allowing leading 0s, such that "00101", "00000000101" and "101" all represent the same number: five. But there's still the issue of the empty input, consisting of no symbols.

To address this, we'll simply refine the *problem statement*, to ask: *"is the input string a binary representation of a number with optional leadings 0s? If yes, is it even?"*[‡]. The empty string is not the binary representation of a number, so the answer is clearly "no".

From now on, such subtleties shall be implicit.

6 Using states as memory

We shall soon see that there is a reasonable, coherent definition of *"expressive"* that allows us to say that "a Turing Machine is as expressive as any generic-purpose programming language". At the moment, this hardly seems to be the case and you're excused to suspect that there's not much that a Turing Machine can do.

We take now one step towards convincing you of the expressive power of this model, by showing a technique through which we can *"memorize"* information. Examine the following machine, `sameFirstLast`, which checks whether a binary strings begins and ends with the same symbol:

	0	1	□
$start$	$remember0, 0, \rightarrow$	$remember1, 1, \rightarrow$	$Y, \square, -$
$remember0$	$remember0, 0, \rightarrow$	$remember0, 1, \rightarrow$	$expect0, \square, \leftarrow$
$remember1$	$remember1, 0, \rightarrow$	$remember1, 1, \rightarrow$	$expect1, \square, \leftarrow$
$expect0$	$Y, 0, -$	$N, 1, -$	$N, \square, -$
$expect1$	$N, 0, -$	$Y, 1, -$	$N, \square, -$

This time, we have chosen to give each of the five states a suggestive name, instead of generic ones like q_1, q_2 etc. From the machine's point of view, the names are irrelevant; their purpose is simply to make things clearer for us.

We can see here an example of *memorization*: from the **start** state, depending on the contents of the first cell of the input, the machine will transition to one of two states: **remember0** or **remember1**. The states themselves acts as memory, in this case retaining one bit of information.

[‡]Note that we could also rephrase this in a way that avoids the concept of "number representation" entirely: *"is the last symbol of the input 0?"*

Some entries in the table might surprise you. As with our first example, we need to think of a way to deal with the empty input; we choose to accept it, by transitioning to the Y state, but this is just a convention, we could just as well transition to N. But what about the transition on $(expect0, \square)$? Such a situation can never occur:

- if the input was empty, the machine goes from **start** directly to N and halts, never reaching any other state
- if the input starts with a 1, then the machine will transition to **remember1** and we can see, by looking at the table, that **expect0** can never be reached.
- if the input starts with a 0, then the machine will transition to **rememmber0**, will remain in this state while the head goes right until the end of the input, then the machine will transition to **expect0**, as the head moves left onto the last input symbol. Then the machine will transition to either Y or N and halt.

So the machine will never need to "know what to do" if it's in state **expect0** and reads a $\square$ – this situation cannot occur for any input.

We arbitrarily choose to halt computation with a negative answer (i.e. transition to N), leave the symbol as it is and keep the head still. From now on, we shall omit explicitly describing such transitions, by leaving the corresponding entry empty, to keep our tables lighter.

6.1 Multiple bits of memory

In the previous example we showcased how a machine can use states to remember one bit of information. This technique can be extended to an arbitrary number of bits. To illustrate this, consider the following machine, `samePairFirstLast` that checks whether the input starts and ends with the same combination of two bits. For example, 011101011001 is accepted because it starts and ends with "01", while 101111 is rejected, because it starts with "10" and ends with "11".

	0	1	$\square$
$start$	$remember0, 0, \rightarrow$	$remember1, 1, \rightarrow$	$N, \square, -$
$remember0$	$remember00, 0, \rightarrow$	$remember01, 1, \rightarrow$	$N, \square, -$
$remember1$	$remember10, 0, \rightarrow$	$remember11, 1, \rightarrow$	$N, \square, -$
$remember00$	$remember00, 0, \rightarrow$	$remember00, 1, \rightarrow$	$expect00, \square, \leftarrow$
$remember01$	$remember01, 0, \rightarrow$	$remember01, 1, \rightarrow$	$expect10, \square, \leftarrow$
$remember10$	$remember10, 0, \rightarrow$	$remember10, 1, \rightarrow$	$expect01, \square, \leftarrow$
$remember11$	$remember11, 0, \rightarrow$	$remember11, 1, \rightarrow$	$expect11, \square, \leftarrow$
$expect00$	$expect0, 0, \leftarrow$	$N, 1, -$	
$expect01$	$expect1, 0, \leftarrow$	$N, 1, -$	
$expect10$	$N, 0, -$	$expect0, 1, \leftarrow$	
$expect11$	$N, 0, -$	$expect1, 1, \leftarrow$	
$expect0$	$Y, 0, -$	$N, 1, -$	
$expect1$	$N, 0, -$	$Y, 1, -$	

6.2 Checking for palindromes

We move on to a slightly more complicated task: answering the question *"is this binary string a palindrome?"*

When is a string a palindrome? One neat way to think of it is with a recursive definition: a string is a palindrome if:

- it's empty
- its first and last symbols are the same and there's a palindrome between them

Our machine, `isPalindrome` will thus need to check if the word is empty; if not, it will check the first and last symbols for equality, then restart the same process for the string in between:

	0	1	$\square$
$start$	$remember0, \square, \rightarrow$	$remember1, \square, \rightarrow$	$Y, \square, -$
$remember0$	$remember0, 0, \rightarrow$	$remember0, 1, \rightarrow$	$expect0, \square, \leftarrow$
$remember1$	$remember1, 0, \rightarrow$	$remember1, 1, \rightarrow$	$expect1, \square, \leftarrow$
$expect0$	$reset, \square, \leftarrow$	$N, 1, -$	$Y, \square, -$
$expect1$	$N, 0, -$	$reset, 1, \leftarrow$	$Y, \square, -$
$reset$	$reset, 0, \leftarrow$	$reset, 1, \leftarrow$	$start, \square, \rightarrow$

Notice that state **start** here is almost the same as in the example of section 6, except the initial symbol is *erased* – this is done so that we can later easily find the beginning of "the inner substring".

remember0 and **remember1** are identical to those of `sameFirstLast`; **expect0** and **expect1** are similar, but when a match is found, we can't just accept, we need to start checking the inner substring. So we remove the last symbol (now the tape only contains the inner substring) and move on to a state called **reset**, whose purpose is to bring the head onto the leftmost non-blank symbol, then redo the whole thing by transitioning to **start**.

6.3 Same number of 1s and 0s

We will now showcase a technique for *"marking"* symbols, by making use of Γ, the tape alphabet.

We need to answer the question: *"does this binary string have an equal number of 0s and 1s?"*. How would you solve this, given a very long string written on a piece of paper and a pen?

You would probably look at the first symbol, cross it off, then go symbol by symbol to the right looking for a match[§]. When you find a match, you cross it off as well, then go back to the first unmarked symbol and start over.

If you reach the end of the string when seeking for a match, the answer is "no" – there was some unmatched symbol.

If all symbols are matched, the answer is "yes".

This is exactly how the following machine, `isMatched` works, by using two extra tape symbols: $\emptyset$ and $\not{1}$ to represent crossed out symbols.

	0	1	$\square$	$\not{\emptyset}$	$\not{1}$
$start$	$find_1, \not{\emptyset}, \rightarrow$	$find_0, \not{1}, \rightarrow$	$Y, \square, -$	$start, \not{\emptyset}, \rightarrow$	$start, \not{1}, \rightarrow$
$find_0$	$reset, \not{\emptyset}, \leftarrow$	$find_0, 1, \rightarrow$	$N, \square, -$	$find_0, \not{\emptyset}, \rightarrow$	$find_0, \not{1}, \rightarrow$
$find_1$	$find_1, 0, \rightarrow$	$reset, \not{1}, \leftarrow$	$N, \square, -$	$find_1, \not{\emptyset}, \rightarrow$	$find_1, \not{1}, \rightarrow$
$reset$	$reset, 0, \leftarrow$	$reset, 1, \leftarrow$	$start, \square, \rightarrow$	$reset, \not{\emptyset}, \leftarrow$	$reset, \not{1}, \leftarrow$

Note that we could also erase the first character of the string, instead of crossing it out.

We also don't actually need to distinguish between crossed out symbols; on paper, that is just an artifact of our pen strokes. But the machine can just as easily overwrite both 0s and 1s with an X, like this:

	0	1	$\square$	X
$start$	$find_0, X, \rightarrow$	$find_1, X, \rightarrow$	$Y, \square, -$	$start, X, \rightarrow$
$find_0$	$reset, X, \leftarrow$	$find_0, 1, \rightarrow$	$N, \square, -$	$find_0, X, \rightarrow$
$find_1$	$find_1, 0, \rightarrow$	$reset, X, \leftarrow$	$N, \square, -$	$find_1, X, \rightarrow$
$reset$	$reset, 0, \leftarrow$	$reset, 1, \leftarrow$	$start, \square, \rightarrow$	$reset, X, \leftarrow$

7 Representing Turing Machines as prose

So far, we have designed fairly simple machines. But even the 2-bit memorizer in subsection 6.1 had quite a large, unwieldy table. In future lectures, we will need larger machines, for which explicitly presenting the transition table would be a gargantuan, excruciating task, more laborious than illuminating.

To make our lives easier, we shall be using English prose to describe the functionality of larger machines, by combining descriptive phrases that should evoke familiar transition tables.

For example, after going through the transition tables presented above, we can now say about a machine:

"First, move the head on the last symbol of the input."

and we can easily envision how the explicit transitions would look like.

Now that you have seen the transition table for a machine which checks if an input starts and ends with the same 2-symbol combination, you can easily see that the following machine is possible, and you can imagine how its transition table would look like:

"Check if the input starts and ends with the same 4-symbol combination."

[§] A 1 if the first symbol was 0, a 0 if the first symbol was 1.

And we can write this simple sentence, instead of presenting a 61×3 table!

Our goal is not to design Turing Machines for the fun of it. The machines are just a useful tool that we employ to get rigorous formal answers to questions related to computability. When designing a machine we want our description to be succinct (so that we don't waste much time with it), but explicit enough to be convinced that such a machine can exist and to have an idea of how its explicit transition table would look like. For example, given the problem: *"does this binary string have more 0s than 1s?"*, it is not very convincing (with what we've seen so far) to describe a machine that solves it like this:

"Check if the input has more 0s than 1s."

But we can employ the techniques already discussed and come up with:

---

1 *If the current symbol is blank, reject.*
2 *Mark the current symbol and move right, searching for a complement symbol.*
2 *If none is found:*
3 *If the marked symbol was a 1, accept.*
4 *If the marked symbol was a 0, reject.*
5 *Else, mark the found symbol, move to the first non-marked symbol and go to 1.*

In the future, if we encounter such a problem, we can rely on this description and confidently describe our machine as:

"Check if the input has more 0s than 1s."

As we continue studying examples of Turing Machines, especially at the labs, we shall be able to abstract more and more functionality into short phrases.

8 References and further reading

Turing Machines were first described by Alan Turing (1912-1954) in his seminal 1936 article: "On computable numbers, with an application to the Entscheidungsproblem" [1]. The paper provides a negative answer to an important mathematical question and introduces the Machines as a way to formalize what a mathematician with a pen and paper can actually do.

While "the Entscheidungsproblem" itself is outside the scope of `AA` , Turing's paper also introduces the concepts of decidability, undecidability, reductions, the Universal Machine etc. which will be the subject of our study during the next four lectures.

The article is freely available online[††], although we recommend reading Charles Petzold's "The Annotated Turing" [2] – a book built around the article, which includes the necessary historical context and motivation, relevant details about Turing's life and education, as well as later corrections to the mistakes present in the 1936 version.

Bibliography

[1] Alan Mathison Turing et al. "On computable numbers, with an application to the Entscheidungsproblem". In: *J. of Math* 58.345-363 (1936), p. 5.

[2] Charles Petzold. *The annotated Turing: a guided tour through Alan Turing's historic paper on computability and the Turing machine*. Wiley Publishing, 2008.

[††] `https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf`

COMPUTING

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

In the introductory lecture, we discussed what "problems" are and classified them into two classes:

- *"decision problems"* – those who admit a "yes" or "no" answer
- *"function problems"* – those whose answer is some mathematical object other than a boolean, such as a number, a set, a graph etc.

In the second lecture, we introduced the concept of *Turing Machines* and presented some particular examples of machines that solve problems. The answer to a decision problem was given by transitioning to one of the final states Y (a "yes" answer) or N (a "no" answer), while the answer to a function problem was given by encoding the answer in the tape contents and transitioning to the final state H.

In all cases mentioned above, the computation ends by reaching a final state. But consider the following machine:

	0	1	$\square$
q_1	$q_1, 0, -$	$q_1, 1, -$	$q_1, \square, -$

The machine starts in the initial state q_1. Regardless of what symbol it reads, it remains in the same state, keeping the head stationary. Its computation will never end: it can never reach any of the three final states.

This should evoke the more familiar concept of `while (1) {}` from a language like C. The machine does work; it goes through successive discrete stages, each time transitioning to a state, writing a symbol on the tape and moving the tape head. It just so happens that the next state is always the same as the current one, the written symbol is always the same as the read one and the movement direction of the head is to keep it stationary, so there is no interesting effect of any transition.

However, if the machine above seems too odd, here's one that moves the head and changes the contents of the tape:

	0	1	$\square$
q_1	$q_1, 1, \rightarrow$	$q_1, 1, \rightarrow$	$q_1, 1, \rightarrow$

Regardless of the input, the machine will go on forever, writing one more 1 on the tape at each step. By analogy with C, you can think of it as: `while (1) { printf("1"); }`.

Here is one more example of a machine that goes on forever, but also changes state:

	0	1	$\square$
q_1	$q_1, 1, \rightarrow$	$q_1, 0, \rightarrow$	$q_2, \square, \leftarrow$
q_2	$q_2, 1, \leftarrow$	$q_2, 0, \leftarrow$	$q_1, \square, \rightarrow$

This machine keeps going left-to-right, then right-to-left through the input, flipping symbols.

The three examples above always run forever, regardless of the input. But we can easily conceive of a machine that halts for some inputs and doesn't for others:

	0	1	$\square$
q_1	$q_1, 0, H$	$q_2, 1, \rightarrow$	$q_2, 1, \rightarrow$
q_2	$q_2, 1, \rightarrow$	$q_2, 1, \rightarrow$	$q_2, 1, \rightarrow$

This one halts in one transition for any input that begins with a 0, or goes on to write a never-ending sequence of 1s for any input that begins with 1, as well as for the empty input.

The key takeaway here is that, during its run, a machine can either make a finite number of transitions until it reaches a final state, or make an infinite number of transitions, never reaching a final state. In the first case, we say that the machine *"halts"*, in the other that it *"doesn't halt"*.

We will now formalize the notion of a machine's *"run"*.

1 Turing Machine configurations

At any given *step* of computation, the following is true:

- the tape contains a finite number of non-blank cells
- the head is scanning exactly one cell
- the machine is in some state $\mathbf{q}$

These three elements together (tape contents, head position, state) uniquely characterize a computation step – together, they form a *"configuration"*.

Definition 1.1. A *configuration* of a Turing Machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ is a triplet $(\gamma_L, \gamma_R, \mathbf{q})$ where:

- γ_L is a finite string of symbols $g_{L_1} g_{L_2} g_{L_3} \cdots g_{L_n}$ $(g_{L_1}, g_{L_2}, g_{L_3}, ..., g_{L_n} \in \Gamma)$ with the contents of the tape, *to the left of the head.*
  The string is *empty* if there are no non-blank symbols to the left of the head. For example, this is how all machines start.
  Note that the character currently read by the head **is not** part of this string.
- γ_R is a finite string of symbols $g_{R_1} g_{R_2} g_{R_3} \cdots g_{R_m}$ $(g_{R_1}, g_{R_2}, g_{R_3}, ..., g_{R_m} \in \Gamma)$ with the contents of the tape, *under and to the right of the head.*
  If the head is positioned on a blank symbol, to the right of *all* non-blank symbols, this string consists of a single blank symbol.
  Note that the character currently read by the head **is** part of this string.
- $\mathbf{q} \in Q$ is the current state

1.1 Configuration types

Definition 1.2. The *initial configuration of a Turing machine* $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ *on input* $w \in \Sigma^*$ is (ϵ, w, q_1). We shall call it $C_0(w)$.

Definition 1.3. A *halting configuration of Turing machine* $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ *on input* $w \in \Sigma^*$ is (ϵ, ϕ, q_f), where ϵ is the empty string, $\phi \in \Gamma^*$ and $q_f \in \{Y, N, H\}$.
Specifically, an *accepting configuration* is one where $q_f = Y$ and a *rejecting configuration* is one where $q_f = N$.

Notice that, in a halting configuration, the head needs to be on the *leftmost non-blank symbol*. In the previous lecture we did not require this and indeed we described machines that leave it in an arbitrary position.

In future lectures, we will want to "chain together" two machines M_1 and M_2 and having halting configurations resemble the initial configuration will make our job easier.

1.2 Configurations example

Recall the machine `isEven` from the previous lecture and let's consider its run over input `10010`.

The first configuration, at the very beginning, before doing any computational step is:

$$(\epsilon, 10010, q_1)$$

The third element of the tuple tells us that the machine is in state q_1. There are no symbols to the left of the head, hence the first element of the tuple is the empty string ϵ. The whole input is to the right of the head, except for the first symbol which is right under it. To figure out the symbol currently read by the head, we need to inspect the first symbol of the second element of the tuple; here, it is 1.

Looking at the transition table of `isEven`, we know that $\delta(q_1, 1) = (q_1, 1, \rightarrow)$. So the next configuration is:

$$(1, 0010, q_1)$$

The state is still q_1, but the head moved one position right, so the first 1 symbol of the input is now to the left of the head, which is positioned over a 0.

The machine then goes through the following configurations, until it transitions into a halting state.

$$(10, 010, q_1)$$
$$(100, 10, q_1)$$
$$(1001, 0, q_1)$$
$$(10010, \square, q_1)$$
$$(1001, 0\square, q_2)$$
$$(1001, 0\square, Y)$$

Note 1.1. Notice that once the head steps outside the input to the right (from the third to the fourth configuration), a new blank symbol needs to be added, which then remains part of the configuration string.

2 Going from one configuration to another

Definition 2.1. Let $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ be a Turing Machine.

We say that $C \vdash_M C'$ (read "from configuration C, machine M goes to configuration C' in one step") if, in a single transition, the machine goes from configuration C to configuration C'.

Depending on the transition function δ, the following situations are possible (for ease of reading, we shall use bolded Latin letters, like $\mathbf{p}$ to represent *states*, monospaced Latin letters from the beginning of the alphabet, like $\mathtt{a}$, to represent *tape symbols* and Greek letters to represent *strings of tape symbols*):

1. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, -)$:
   1.1. $(\phi, \mathtt{a}\mu, \mathbf{p}) \vdash_M (\phi, \mathtt{b}\mu, \mathbf{s})$
2. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, \rightarrow)$:
   2.1. $(\phi, \mathtt{ac}\mu, \mathbf{p}) \vdash_M (\phi\mathtt{b}, c\mu, \mathbf{s})$
   2.2. $(\phi, \mathtt{a}, \mathbf{p}) \vdash_M (\phi\mathtt{b}, \square, \mathbf{s})$
3. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, \leftarrow)$:
   3.1. $(\phi c, \mathtt{a}\mu, \mathbf{p}) \vdash_M (\phi, \mathtt{cb}\mu, \mathbf{s})$
   3.2. $(\epsilon, \mathtt{a}\mu, \mathbf{p}) \vdash_M (\epsilon, \square\mathtt{b}\mu, \mathbf{s})$

Where:

- $\phi, \mu \in \Gamma^\star$ (possibly-empty strings of tape symbols)
- $\mathtt{a}, \mathtt{b}, \mathtt{c} \in \Gamma$
- $\mathbf{q}, \mathbf{s} \in Q$

2.1 Zero or more steps

We now define the operator $\vdash_M^*$ to be the *reflexive and transitive closure* of $\vdash_M$. This means that we extend this operator to yield a configuration "in 0 or more steps", exhibiting the two properties:

- **reflexivity**: For any configuration C: $\quad C \vdash_M^* C$
- **transitivity**: For any configurations C_1, C_2, C_3: $\quad (C_1 \vdash_M^* C_2) \wedge (C_2 \vdash_M^* C_3) \implies C_1 \vdash_M^* C_3$

We now give formal definitions for "the result" of running a machine M on an input w.

3 How Turing Machines run

In all definitions in this section:

- M is a Turing Machine $(Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$
- $w, v \in \Sigma^*$
- $\phi \in \Gamma^*$
- $\beta, \beta' \in \{B\}^*$ are strings of 0 or more blank symbols B

β and β' will be used as "leading"/"trailing" blanks that are needed because of our definition of configurations and of the operator $\vdash_M$; when the head moves past the non-blank symbols, a blank symbol is added to our configuration, as mentioned in Note 1.2.

Definition 3.1. $M[w] \to TRUE \stackrel{\text{def}}{=} C_0(w) \vdash_M^* (\beta, \phi, Y)$

Read this as: *"M accepts input w"*.

Note 3.1. In many Computability textbooks, you may see this denoted by $M(w) = TRUE$, but we want to emphasize here that M is not a *function* that *maps* w to a value, but a **machine** that **performs computation** starting from w.

Definition 3.2. $M[w] \to FALSE \stackrel{\text{def}}{=} C_0(w) \vdash_M^* (\beta, \phi, N)$

Read this as: *"M rejects input w"*.

Definition 3.3. $M[w] \to v \stackrel{\text{def}}{=} C_0(w) \vdash_M^* (\beta, v\beta', H)$

Read this as: *"on input w, M computes the output v"*.

This definition tells us that M computes a value v, if it eventually transitions into state H with the contents of the tape (from the first non-blank, to the last non-blank) consisting of that value.

Note that the output alphabet is the same as the input alphabet. Remember that our goal is to "solve problems", and we defined problems as mapping strings over some alphabet onto strings over *the same alphabet* ($f : \Sigma^* \to \Sigma^*$).

Definition 3.4. $M[w]$ *halts* $\stackrel{\text{def}}{=} M[w] \to X$, where $X \in \Sigma^* \cup \{\texttt{FALSE}, \texttt{TRUE}\}$

This is a generalization of the previous three definitions. If, on input w, M accepts or rejects or computes an output v, we can say generically that *"M halts on input w"*. Otherwise we say that *"M doesn't halt on input w"*.

Going forwards, it will be useful for us to have a short notation for this case:

Definition 3.5. $M[w] \to \perp \stackrel{\text{def}}{=} M[w]$ doesn't halt

(The symbol $\perp$ is called a *"bottom"* and its usage here is a bastardization of its meaning in *Type Theory*.)

The following definition will be very useful for us in future lectures:

Definition 3.6. Let M_1 and M_2 be two Turing Machines.

$$M_1[w] \equiv M_2[w] \stackrel{\text{def}}{=} (M_1[w] \to X \wedge M_2[w] \to X) \vee (M_1[w] \to \perp \wedge M_2[w] \to \perp)$$

where $X \in \Sigma^* \cup \{\texttt{FALSE}, \texttt{TRUE}\}$.

Read this as: *"on input w, M_1 and M_2 yield the same result"*.

The term "result" here is a bit of a stretch, because the machines could also not halt. This is why formal definitions are important, to avoid confusion caused by the vagueness of natural terms.

So two machines "yield the same result" on an input if one of these holds true:

- both machines *accept* the input, by eventually transitioning to state Y
- both machines *reject* the input, by eventually transitioning to state N
- both machines eventually transition to state H leaving the same sequence of non-blanks on the tape
- both machines continue computation forever, never halting

4 Accepting a problem

Definition 4.1. Let $f : \Sigma^* \to \{FALSE, TRUE\}$ be a decision problem and $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ a Turing Machine:

M accepts $f \stackrel{\text{def}}{=} \forall w \in \Sigma^\star, f(w) = TRUE \Leftrightarrow M[w] \to TRUE$

Note that the definition doesn't say anything about the case in which $f(w) = FALSE$. The machine could transition to final state N, or H, or it could loop forever, it doesn't matter.

Definition 4.2. f is acceptable $\stackrel{\text{def}}{=} \exists M$ s.t. M accepts f

We can now define our first important set of decision problems:

Definition 4.3. $RE \stackrel{\text{def}}{=} \{f \mid f \text{ is acceptable } \}$

The name "RE" stands for "recursively-enumerable", which is another term for "acceptable". We use it here (instead of something like A) because it's the classical name for this set, in almost all texts on Computability Theory.

5 Deciding a problem

Definition 5.1. Let $f : \Sigma^* \to \{FALSE, TRUE\}$ be a decision problem and $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ a Turing Machine:

M decides $f \stackrel{\text{def}}{=} \forall w \in \Sigma^\star, (f(w) = TRUE \Leftrightarrow M[w] \to TRUE) \wedge (f(w) = FALSE \Leftrightarrow M[w] \to FALSE)$

Definition 5.2. f is decidable $\stackrel{\text{def}}{=} \exists M$ s.t. M decides f

Notice that for a problem to be *decidable*, there has to exist a Turing machine M which *always halts*, regardless of its input. M matches our intuition of what a *"decision algorithm"* is: a *finite mechanistic procedure* that *always* provide an answer to a yes/no question, in *a finite amount of time.*

Definition 5.3. $R \stackrel{\text{def}}{=} \{f \mid f \text{ is decidable } \}$

The name "R" stands for "recursive", which is another term for "decidable". Like RE, it's a classic name.

At this point in the lecture, the concept of acceptance might seem artificial and meaningless. But we shall soon prove (and provide examples) that there are decision problems for which there exist machines that can *accept* them but there exist no machines that can *decide* them. Thus, acceptance is a notion worthy of study.

What's more, we will also prove (and provide examples) that there are decision problems which are not acceptable: no Turing Machine exists that can accept them.

6 Computing a problem

We move on from decision problems for a bit, and generalize the concept of "deciding". If f is not a decision problem (its answers are not booleans, but other mathematical objects):

Definition 6.1. Let $f : \Sigma^* \to \Sigma^*$ be a function problem and $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ a Turing Machine:
M computes $f \stackrel{\text{def}}{=} \forall w \in \Sigma^\star, M[w] \to f(w)$

Definition 6.2. f is computable $\stackrel{\text{def}}{=} \exists M$ s.t. M computes f

Notice that, per our definitions, decidability is a special case of computability, applicable only to decision problems.

Similarly to what we said about acceptable and decidable problems, we will soon prove that there are some function problems that are not computable.

7 The Church-Turing thesis

We now formulate a very important (and possibly non-intuitive) claim that is essential to the relevance of our approach to Computability Theory: the Church-Turing thesis. One possible formulation of the thesis is:

"The problems that are solveable by a mathematician with pen and paper are precisely those computable by a Turing Machine."

We can rephrase it in a way more relevant to our goals:

"The problems that are solvable by an algorithm are those that are computable by a Turing Machine."

This is a philosophical statement about reality, that asserts a correspondence between the intuitive concept of "algorithm" and the formal concept of "Turing machine". Because "algorithm" (or "solvable by an algorithm") is not a formally defined mathematical object, the Church-Turing thesis **is not** a mathematical statement; thus it can be neither proven or disproven within the realm of mathematics.

However, it can be analyzed, debated, doubted, accepted etc. At this point, you might find little reason to believe this assertion that Turing Machine can solve any problem solvable by an algorithm, but there are some solid arguments in support of it.

7.1 Reasons to believe

7.1.1 Different approaches, same result

At the beginning of the twentieth century, what algorithms are and what can actually be solved with an algorithm became an important subject. This led to the idea of an "effective method", a recipe of steps to carry out a problem. The characteristic of an effective method are:

- It consists of a finite number of instructions, each expressible by a finite amount of symbols.
- When carried out, it will produce a result in a finite number of steps.
- It can be carried out by a human with just pencil and paper.
- It demands no insight, intuition or ingenuity.

A problem that can be solved by an effective method is "effectively calculable". Note that this is an intuitive, informal concept.

There are at least three contemporaneous attempts at formalizing this notion:

- recursive functions
- the Lambda calculus
- Turing Machines

These three widely different models all started as an attempt to rigorously capture which problems are "effectively calculable". They were all shown to be equivalent to each other; that is, they identify precisely the same set of problems.

Since then, many other models of computations (e.g. counter machines, RAM machines, Markov algorithms) were designed for which it was proven that the set of problems *computable in that model* is R.

7.1.2 No known counterexamples

One way to reject the hypothesis would be to provide a counterexample of a problem that can be solved algorithmically, but for which no Turing Machine that decides it exists.

The examples from the previous lecture, as well as the exercises in the first lab, should convince you that there's quite a wide range of problems that these machines can decide. The literature on computability theory is filled with countless other examples of Turing Machines deciding various functions, as well as methods for proving that a Turing Machine which decides a function exists (i.e. even without explicitly presenting such a machine). [†]

8 References and further reading

The "recursive functions" mentioned in subsubsection 7.1.1 were formulated by Kurt Gödel (1906-1978) during a series of lectures at Princeton. The lecture notes can be found, for example, in "Kurt Gödel: Collected Works: Volume I: Publications 1929-1936" [1].

The Lambda calculus was developed by Alonzo Church (1903-1995) and his students. In his 1936 paper "An Unsolvable Problem of Elementary Number Theory" [2], Church set out to formalize the notion of "effective-calculability" and prove the existence of undecidable problems. He also showed the equivalence between lambda calculus and Gödel's recursive functions.

Alan Turing described his machines in 1936, in "On computable numbers, with an application to the Entscheidungsproblem" [3] (which was only published in 1937). Because Church's paper had been published one year before, Turing added an appendix to his own, proving the equivalence between lambda-definable functions and Turing-computable functions.

The Stanford Encyclopedia of Philosophy has an entry on the Church-Turing thesis [4]: its historical context, what it claims and what it doesn't, as well as a more detailed section on why to accept it. [‡]

Bibliography

[1] Kurt Gödel. *Kurt Gödel: Collected Works: Volume I: Publications 1929-1936*. Vol. 1. Oxford University Press, New York, 1986, pp. 338–371.

[2] Alonzo Church. "An unsolvable problem of elementary number theory". In: *American Journal of Mathematics* 58.2 (1936), pp. 345–363.

[3] Alan Mathison Turing et al. "On computable numbers, with an application to the Entscheidungsproblem". In: *J. of Math* 58.345-363 (1936), p. 5.

[4] B. Jack Copeland. "The Church-Turing Thesis". In: *The Stanford Encyclopedia of Philosophy*. Ed. by Edward N. Zalta. Summer 2020. Metaphysics Research Lab, Stanford University, 2020.

[†] There is a branch of Computability Theory that deals with *"hypercomputation"* – computation models which can compute functions that Turing Machines cannot. However, while the models and results are mathematically sound, the computation they describe could not be carried out by a human with pen and paper. For example, some models require performing an infinite number of steps in a finite amount of time.

[‡] `https://plato.stanford.edu/entries/church-turing/`

THE UNIVERSAL TURING MACHINE

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

We have seen various examples of Turing Machines designed to solve some particular problem. For each problem we encountered, we created a new machine: we defined a new 9-tuple (usually implicitly, by just writing the transition table), specifying what states the machine has, what tape symbol it uses, what transitions it can perform etc.

We will now show that it is possible to create a single machine capable of doing anything that some Turing Machine can do. In particular, this one machine could accept/decide/compute any acceptable/decidable/computable problem. We will call this a *"Universal Turing Machine"* (UTM).

A UTM receives as input the *encoding* of a pair (M, w), consisting of some Turing Machine M and some string w, then decodes the description of M in order to simulate how M would run when given input w.

The only thing that a UTM has to "know" how to do is read a Turing Machine description and correctly apply its effects on the string w.

In order to prove such a machine can exist, we need to first design a proper encoding for arbitrary Turing Machines, then specify the implementation of a UTM: how it can simulate another machine based on its encoding.

1 Encoding Turing Machines

1.1 Dealing with different alphabets

Remember that the definition of a Turing Machine involves an *input alphabet*. For machines that compute an output, this is also the *output alphabet*. If each machine can have its own alphabet of arbitrary size, holding arbitrary symbols, how can we hope to design a single machine to simulate all others? We cannot; we would have to specify a single machine with a single input alphabet that is somehow capable of containing every possible alphabet. Our Universal Turing Machine will only be "universal" in regards to all possible machines over a single specific alphabet, namely $\Sigma_b = \{0, 1\}$.

However, the loss in generality is not as great as it might first seem, for reasons we will explore below.

For any alphabet $\Sigma = \{s_1, s_2, ..., s_n\}$, we can create a *"binary encoding"* of possible strings.

Definition 1.1. A binary encoding of an alphabet $\Sigma = \{s_1, s_2, ..., s_n\}$ is a bijection:

$$enc : \Sigma^* \to \{0, 1\}^*$$

such that:

$$\forall w \in \Sigma^* (w = \sigma_1\sigma_2...\sigma_m;\ \sigma_1, \sigma_2, ..., \sigma_m \in \Sigma), enc(w) = enc(\sigma_1)enc(\sigma_2)...enc(\sigma_m)$$

Here we abused notation a bit; in the expression $w = \sigma_1\sigma_2...\sigma_m$, each σ_i refers to some *symbol* from Σ; however, in the final expression $enc(\sigma_1)enc(\sigma_2)...enc(\sigma_m)$, each σ_i refers to *a string consisting of a single symbol*.

Put into words, the encoding of a word is the concatenation of the encodings of its symbols.

We shall call "dec" the *inverse* of enc (i.e. enc^{-1}).

Theorem 1.1. There exists an encoding $enc : \Sigma^* \to \{0, 1\}^*$, such that for any Turing Machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$, there exists a Turing Machine $M_b = (Q_b, \{0, 1\}, \Gamma_b, B_b, q_{1_b}, Y_b, N_b, H_b, \delta_b)$ such that:

$$\forall w \in \Sigma^*, (M[w] \to v) \Rightarrow (M_b[enc(w)] \to enc(v))$$

Proof. We'll denote the symbols of Σ: $\sigma_1, \sigma_2, ..., \sigma_n$, where $n = |\Sigma|$.

Our encoding function will map each σ_i to a string of 1, followed by i 0s. E.g., $enc(\sigma_4) = 10000$

Like in the multitape-to-single-tape simulation, we will construct a machine M_b that simulates, in turn, each transition of M, by going through multiple *phases*.

M_b will have a copy of of all states of M ($Q \subseteq Q_b$), as well as all tape symbols of M ($\Gamma \subseteq \Gamma_b$). The blank symbol of M is the blank symbol of M_b.

The initial state of M_b will be the initial state of M, same for the final states Y, N and H.

1. For any transition of the form:

   $\delta(q, g) = (r, h, D)$, with $q \in Q, r \in Q', g, h \in \Gamma \setminus \Sigma, D \in \{-, \rightarrow\}$

   M_b will have a transition:

   $\delta_b(q, g) = (r, h, D)$

   If the transition doesn't involve input symbols and the head won't move left, there's nothing special that needs to be done.

2. For any transition of the form:

   $\delta(q, g) = (r, h, \leftarrow)$, with $q \in Q, r \in Q', g, h \in \Gamma \setminus \Sigma$

   M_b will have a transition:

   $\delta_b(q, g) = (r_a ux, h, \leftarrow)$

   and $\delta_b(r_a ux, 0) = (r_a ux, 0, \leftarrow)$

   $\delta_b(r_a ux, 1) = (r, 1, -)$

   If the transition doesn't involve input symbols and the head moves left, we need to skip over the 0s of our encoding to place M_b's head over the initial 1.

3. For any transition of the form:

   $\delta(q, g) = (r, \sigma_j, D)$, with $q \in Q, r \in Q', g \in \Gamma \setminus \Sigma, \sigma_j \in \Sigma, D \in \{\leftarrow, -, \rightarrow\}$

   M_b will have a transition: $\delta_b(q, g) = (r_shift_j_D, 1, \rightarrow)$

   Where the purpose of the state $r_shift_j_D$ is to shift the rest of the tape contents right by j symbols, filling the space created with j 0s, transition to state r and move the *simulated head* in direction D.

   Moving the simulated head to the right, means moving the actual head just one cell to the right of the inserted 0.

   Keeping the simulated head still, means moving the actual head to the left, past the inserted 0s, until it reaches a 1.

   Moving the simulated head to the left, means moving the actual head left, past the inserted 0s, until it reaches a 1, then moving it once more to the left; if there is a 0 there, move it to the left until a 1 is encountered.

4. From any state $q \in Q$, when encountering a 1, M_b will need to determine the symbol from Σ whose encoding it just came across. To do this, it will go right, counting each 0 and memorizing the number of 0s (using states). So M_b will have the following transitions:

   - $\delta_b(q, 1) = (q_sym, 1, \rightarrow)$
   - $\delta_b(q_sym, 0) = (q_sym_1, 0, \rightarrow)$
   - $\delta_b(q_sym_1, 0) = (q_sym_2, 0, \rightarrow)$
   - $\delta_b(q_sym_2, 0) = (q_sym_3, 0, \rightarrow)$
   - ...
   - $\delta_b(q_sym_{n-1}, 0) = (q_sym_n, 0, \rightarrow)$

From each q_sym_i on any non-0 symbol, M_b will transition to a state whose purpose is to shift the right part of the tape left, by i symbols, deleting the 0s that form the encoding of σ_i.

Then, looking at $\delta(q, \sigma_i) = (r, g, D)$, it will either overwrite the remaining 1 with g, if $g \in \Gamma \setminus \Sigma$, or apply the same mechanism as in item 3 to shift the right part of the tape to the right and insert a new symbol σ_j.

□

We can formulate and prove a similar theorem for accepting, rejecting or never halting on an input. Now we can focus strictly on machines whose input alphabet is $\{0, 1\}$.

2 Encoding Turing Machines as binary strings

Like in the example above, where we encoded symbols of some alphabet Σ, we shall use a unary encoding for states, symbols and directions.

$M = (Q, \{0, 1\}, \Gamma, B, q_0, H, Y, N, \delta)$

Remember that Q' is a *finite* set of states, so there must be some natural number n, such that $|Q| = n$[†] Thus we can rename all states of Q to $q_1, q_2, q_3, ..., q_n$, with the convention:

- q_1 is the initial state
- q_2 is the accepting state
- q_3 is the rejecting state
- q_4 is the halting state

The index of the other states in Q can be arbitrary.

Similarly, Γ is a *finite* set of tape symbols, so there must be some natural number r, such that $|\Gamma| = r$[‡]. Thus we can rename all symbols of Γ to $g_1, g_2, g_3, ..., g_r$, with the convention:

- g_1 is 0
- g_2 is 1
- g_3 is the blank symbol

The index of the other symbols in Γ can be arbitrary.

Right now, we can encode the first eight elements of our 9-tuple. We now proceed to design an encoding for the transition function, but first, we will employ the following convention about head movement direction:

- d_1 is *left* ($\leftarrow$)
- d_2 is *right* ($\rightarrow$)
- d_3 is *hold* ($-$)

With the above conventions, each transition T has the form:

$\delta(q_i, g_j) = (q_k, g_l, d_m)$

We encode such a transition as:

$enc(T) = 0^i 10^j 10^k 10^l 10^m$

Where "0^i" means "the symbol 0 repeated i times. Note this is just a sequence of the *unary encodings* of each number, separated by a 1.

Remember the domain and range of the transition function:

$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, -, \rightarrow\}$

[†] n must also be at least four, because each TM has at least four states: the initial state, the halting state, the accepting state and the rejecting state.

[‡] r must also be at least three, because our TMs have at least three tape symbols: 0, 1 and □.

Thus, each Turing Machine has a finite number of transitions, exactly $|Q| \cdot |\Gamma|$, or $n \cdot r$.

To encode the entire transition function, we simply take the encoding for all the transitions and concatenate them, separating them with a "11".

$enc(\delta) = enc(T_1)11enc(T_2)11enc(T_3)11...enc(T_{nr-1})enc(T_{nr})$

The order that the transitions appear in is irrelevant and can be arbitrary.

Note that the encoding of a transition cannot contain two consecutive 1s, so there is no ambiguity as to where a transition's encoding ends.

As you will soon see, it turns out that all the relevant information about our machine M is contained in the encoding of the transition function. Thus, we conclude this section with $enc(M) = enc(\delta)$.

3 Example encodings

We provide an example encoding for a Turing Machine previously studied: `isEven`.

	0	1	$\square$
q_1	$q_1, 0, \rightarrow$	$q_1, 1, \rightarrow$	$q_2, \square, \leftarrow$
q_2	$Y, 0, -$	$N, 1, -$	$N, \square, -$

First, we need to map the internal states and tape symbols to new names based on our convention.

$q_1 \Longrightarrow q_1$

$Y \Longrightarrow q_2$

$N \Longrightarrow q_3$

$H \Longrightarrow q_4$

$q_2 \Longrightarrow q_5$

$0 \Longrightarrow g_1$

$1 \Longrightarrow g_2$

$\square \Longrightarrow g_3$

Let's take the transition $\delta(q_2, 0) = (Y, 0, -)$ and rewrite it according to the renaming scheme above:

$\delta(q_5, g_1) = (q_2, g_1, d_3)$

Next, we encode it as a binary string:

$$\underbrace{00000}_{q_2}1\underbrace{0}_{0}1\underbrace{00}_{Y}1\underbrace{0}_{0}1\underbrace{000}_{-}$$

We then to do this for every transition and concatenate all of them, separated by "11"s (remember that the order is irrelevant, but we'll use the ordering in which they appear above):

0101010100110100101001001101000100000100010110000001010010100011000001001000100100 ↩

11001000100010001000

(The line break is not part of the string, it just wouldn't fit inside this page).

Recall that the input to the Universal Turing Machine is a tuple (M, w), where M is a Turing Machine and w is a word on which to simulate M.

To encode this tuple, we will simply concatenate the machine's encoding with that of the input, separated by three consecutive ones, "111"[§].

[§]Our construction is carefully designed so that no string of three consecutive ones can appear inside the encoding of a Turing Machine.

$enc((M, w)) = enc(M)111w$

With this in mind, let's proceed to describing the Universal Turing Machine.

4 Formal definition of the UTM

Definition 4.1. A *"Universal Turing Machine"* U is a Turing Machine such that for any Turing Machine $M = (Q, \{0, 1\}, \Gamma, B, q_1, Y, N, H, \delta)$ and string w, $U[enc((M, w))] \equiv M[w]$.

5 UTM behavior

A UTM is not *unique*, there's an infinite number of machines which respect Definition 4; they may possess various interesting characteristics, but our goal in designing one will be to keep its description clear and concise.

In order to easily visualize our Universal Machine (and write less text) we shall design it as a three-tape Turing Machine. Combined with our single-tape simulation for multitape TMs, we could then obtain a single-tape Universal Turing Machine.

The three-tapes will have the following roles:

1. The first tape will contain the input: the encoding of a (M, w) tuple. The UTM will treat this as a "read-only" tape – it will never modify any symbol on it.

2. The second tape will simulate the contents of M's tape; in any given configuration of U, it will contain the encoding of exactly those symbols that are part of the current configuration of M.

3. The third tape will contain M's current state during the simulation, encoded as a string of zeros. E.g. if M is in state $\mathbf{q_5}$, the third tape will contain five 0s: "00000"

First, the UTM goes through an *initialization* phase, where it properly initializes the second tape with the encoding of w and the third tape with the encoding of M's initial state, q_1.

Recall the single tape simulation of a multitape Turing Machine; similarly, U will precisely simulate the effect of each transition of M, by using multiple transitions. We will refer to the simulation of one transition of M as a *"phase"*.

At the beginning of each phase, U's first and third heads will be positioned over the first non-blank symbol of their respective tape; the second head will match where M's head should be (on the first 0 of the encoding of M's "current symbol"). Each phase consists of the following steps:

1. Move the first and third head in tandem to the right, while they're both reading 0.

2. If the first head reaches a 1 at the same time that the third head reaches a blank, it means that on the first tape we've reached a transition relevant to the current state of M; else go to item 6.

3. On the first tape, skip the following 1, then move the first and second head in tandem, while they're both reading 0.

4. If the first head reaches a 1 at the same time that the second head reaches a 1 or a blank, it means that on the first tape we've reached a transition relevant to the current state and the currently read symbol of M; else go to item 7.

5. Assuming we're in state q_i reading symbol g_j, we must now perform the necessary changes for M's transition:

$$\delta(q_i, g_j) = (q_k, g_l, d_m)$$

   (a) Erase the contents of the third tape. On the first tape, skip over a 1 and start copying each following 0 onto the third tape until reaching another 1. Then reset the third head onto the leftmost non-blank symbol.

(b) on the second tape (the head should be on the 1 immediately after the current symbol's encoding), go left and remove 0s until reaching a 1. By "removal" we mean shifting the entire right part of the tape, one cell to the left.

Once the symbol is erased, on the first tape, skip the 1 and go on the first of the following l 0s. On the second tape, insert a 0 to the right. By "insertion", we mean shifting the entire right part of the tape, one cell to the right, then writing a 0 in the gap that appears.

Repeat these steps (move the first head one cell to the right; insert one 0 on the second tape) until the first head reaches a 1. We have finished overwriting symbol g_j with symbol g_l.

(c) Move the first head one cell to the right and count the 0s (memorizing them with the help of auxiliary states).

We now know the direction the head of M should move:

- If the head of M should stay where it is, move the second head left until it reaches a 1, then one cell to the right (remember that were positioned on the last 0 of the encoding of the written symbol; we do this to go to the first 0 of its encoding).
- If the head of M should move left, move the second head left until it reaches a 1, then once more, then one cell to the right.
- If the head of M should move right, move the second head one cell to the right.

6. If we're not reading the encoding of a transition concerning the current state, we need to continue searching; reset the third head to the leftmost non-blank symbol and move the first head right until it reaches two consecutive ones, then on the cell immediately to the right of them. Go to item 1.

7. If we're not reading the encoding of a transition concerning the current symbol, we need to continue searching; move the second head left until it reaches a 1 (or blank, for the first symbol), then one cell to the right. Move the first head right until it reaches two consecutive ones, then on the cell immediately to the right of them. Go to item 4.

6 References and further reading

The Universal Machine plays a crucial role in Turing's 1936 article: "On computable numbers, with an application to the Entscheidungsproblem" [1]. Turing introduces a special notation that allows him to "metaprogram" machines by parameterizing some state templates; this is described in section 4, "Abbreviated tables". In section 5, he defines an encoding format for machines, as a base-10 number. Then sections 6 and 7 present the detailed construction of such a Universal Turing Machine.

"The annotated Turing" [2] covers these sections in chapters 7, 8 and 9. It also provides later corrections by Emil Post and Donald Davies that fix some bugs in Turing's article.

The construction in this lecture is adapted from the sections 9.1.2 and 9.2.3 of the textbook "Automata Theory, Languages and Computation", third edition [3].

Neither our construction, nor Turing's, actually provides an explicit transition table for the Universal Machine, on the basis that it would be too much effort to do so for no considerable benefit. But it would be interesting to "see" such a machine in a way that allows us to appreciate its complexity. Fortunately, in his 1989 book "The Emperor's New Mind" [4], Roger Penrose dedicates around three physical pages of the last part of his second chapter to printing out in full the binary encoding of a UTM.

Bibliography

[1] Alan Mathison Turing et al. "On computable numbers, with an application to the Entscheidungsproblem". In: *J. of Math* 58.345-363 (1936), p. 5.

[2] Charles Petzold. *The annotated Turing: a guided tour through Alan Turing's historic paper on computability and the Turing machine*. Wiley Publishing, 2008.

[3] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 2006.

[4] Roger Penrose. *The emperor's new mind*. Oxford University Press, 1989.

MULTITAPE TURING MACHINES

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

We introduce a new model of computation – a Turing Machine with k heads, each positioned over one of k tapes, for some natural number $k \geq 2$.

At first glance, this model might seem "more powerful" than a single-tape machine: it can make use of information written on multiple tapes at once. Indeed, there are some problems which can be solved *more efficiently* on a multitape Turing Machine.

However, from the point of view of computability, we shall soon see that there is no difference in "computational power": the classes of problems that can be decided/accepted/computed by a k-tape[†] machine are exactly those that can be decided/accepted/computed by a single-tape machine.

Why bother with multitape Turing Machines in the context of Computability Theory? The main reason is that they can make some proofs easier to follow. Our next relevant goal is to show the existence of a *"Universal Turing Machine"*, a single machine that can compute/accept/decide any function that is computable/acceptable/decidable. Our construction will involve a Turing Machine with 3 tapes, for ease of visualization.

We now informally describe how a multitape machine can be visualized and how it works:

- There is still a single control unit of the machine, a single internal state.
- There are k different tapes (imagine them laid out vertically, one above the other)
- for each tape, there is a read/write head which, at any step of computation, is positioned over some cell of that tape.
- Depending on the combination of internal state and *the k-tuple of currently read symbols*, the machine makes some transition.
- When transitioning, the machine does three things:
  1. It goes to a new internal state; or remains in the same one.
  2. For each of its heads, it writes some new symbol on the cell under it; or leaves that cell unchanged.

     More explicitly, the machine writes a new symbol on each tape, not necessarily the same symbol on all tapes.
  3. It moves each head, independently, to point to the cell immediately on the left or right of the current cell; or leaves it where it is.

     More explicitly, the machine moves each head at most one position, not necessarily all heads in the same direction.

We will now quickly go through some formal definitions to bring us to the same point where we left the single-tape machine (e.g. configurations, acceptance, decision etc.)

If you understood the definitions for the single-tape machine, these should seem to you very straightforward, so few further explanations are provided and you can probably skip straight to section 5.

[†]Regardless of how high the value of k is.

1 Formal definition

Definition 1.1. A multitape Turing Machine is a 9-tuple: $(Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$.

- The first eight members: $Q, \Sigma, \Gamma, B, q_1, Y, N, H$ – have the same significance as for a single-tape Turing Machine.
- $\boxed{\delta : Q \times \Gamma^k \to Q' \times (\Gamma \times \{\leftarrow, -, \rightarrow\})^k}$

2 Configurations

Definition 2.1. A *configuration* of a k-tape Turing Machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ is a $(k+1)$-tuple $((\gamma_L^1, \gamma_R^1), (\gamma_L^2, \gamma_R^2), ...(\gamma_L^k, \gamma_R^k), \mathbf{q})$ where $\mathbf{q}$ is the current state and each γ_L^i and γ_R^i encode the contents to the left and right of the head, for the ith tape.

Definition 2.2. The *initial configuration of a k-tape Turing machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ on input* w is $((\epsilon, w), (\epsilon, \square), ..., (\epsilon, \square), q_1)$. We shall call it $C_0(w)$.

This tells us that, initially, the input is written on the first tape and all the others are empty.

Definition 2.3. A *halting configuration of a k-tape Turing machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ on input* w is $((\mu^1, \phi^1), (\mu^2, \phi^2), ..., (\epsilon, \phi^k), q_f)$, where ϵ is the empty string, $\phi^i, \mu^i \in \Gamma^*$, for $i = \overline{1, k}$ and $q_f \in \{Y, N, H\}$. Specifically, an *accepting configuration* is one where $q_f = Y$ and a *rejecting configuration* is one where $q_f = N$.

At the end of the computation, the "output" will be on the kth tape, with the kth head being positioned over the first output symbol. The content of the other tapes is irrelevant.

3 Going from one configuration to another

Definition 3.1. Let $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ be a k-tape Turing Machine.

We say that $C \vdash_M C'$ (read "from configuration C, machine M goes to configuration C' in one step") if, in a single transition, the machine goes from configuration C to configuration C'.

Remember that a configuration is a tuple consisting of k pairs of strings and a state. For each k pair, depending on the transition function δ, the following changes are applied to determine the next configuration:

1. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, -)$:
   1.1. $(\phi, \mathtt{a}\mu) \vdash_M (\phi, \mathtt{b}\mu)$
2. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, \rightarrow)$:
   2.1. $(\phi, \mathtt{ac}\mu) \vdash_M (\phi\mathtt{b}, c\mu)$
   2.2. $(\phi, \mathtt{a}) \vdash_M (\phi\mathtt{b}, \square)$
3. $\delta(\mathbf{p}, \mathtt{a}) = (\mathbf{s}, \mathtt{b}, \leftarrow)$:
   3.1. $(\phi c, \mathtt{a}\mu) \vdash_M (\phi, \mathtt{cb}\mu)$
   3.2. $(\epsilon, \mathtt{a}\mu) \vdash_M (\epsilon, \square\mathtt{b}\mu)$

Where:

- $\phi, \mu \in \Gamma^\star$ (possibly-empty strings of tape symbols)
- $\mathtt{a}, \mathtt{b}, \mathtt{c} \in \Gamma$
- $\mathbf{q}, \mathbf{s} \in Q$

The next configuration will consist of all k pairs obtained from applying the rules, together with the new state $\mathbf{s}$.

4 How multitape Turing Machines run

In all definitions in this section:

- M is a k-tape Turing Machine $(Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$
- $w, v \in \Sigma^*$
- $\mu^i, \phi^i \in \Gamma^*, i = \overline{1,k}$
- $\beta, \beta' \in \{B\}^*$ are strings of 0 or more blank symbols B

β and β' will be used as "leading"/"trailing" blanks that are needed because of our definition of configurations and of the operator $\vdash_M$; when the head moves past the non-blank symbols, a blank symbol is added to our configuration.

Definition 4.1. $M[w] \to TRUE \stackrel{\text{def}}{=} C_0(w) \vdash_M^* ((\mu^1, \phi^1), (\mu^2, \phi^2), ..., (\beta, \phi^k), Y)$

Definition 4.2. $M[w] \to FALSE \stackrel{\text{def}}{=} C_0(w) \vdash_M^* ((\mu^1, \phi^1), (\mu^2, \phi^2), ..., (\beta, \phi^k), N)$

Definition 4.3. $M[w] \to v \stackrel{\text{def}}{=} C_0(w) \vdash_M^* ((\mu^1, \phi^1), (\mu^2, \phi^2), ..., (\beta, v), H)$

The rest of the definitions in Section 3 of the previous lecture are identical for multitape Turing Machines. Same for definitions 4.1, 5.1 and 6.1.

5 Equivalence with single-tape Turing Machines

We claimed that multitape machines and single-tape machines are computationally equivalent. We now formulate this as a theorem and prove it.

Theorem 5.1. For any k-tape machine: $M_m = (Q_m, \Sigma, \Gamma_m, B_m, q_{1_m}, Y_m, N_m, H_m, \delta_m)$, there exists a single-tape machine $M_s = (Q_s, \Sigma, \Gamma_s, B_s, q_{1_s}, Y_s, N_s, H_s, \delta_s)$ such that, $\forall w \in \Sigma^*$, $M_m[w] \equiv M_s[w]$.

We devise a precise *simulation* of M_m's behavior: M_s will mimic the effect of each transition of M_m on the input. For each transition that M_m makes, M_s will need to make *several* (in fact, quite a large number) of transitions to simulate it.

We will keep the contents of all k tapes concatenated onto the single tape, separated by a new symbol $\diamond$ that we introduce ($\diamond \notin \Gamma_m, \diamond \notin \Sigma$).

We also need to keep track of where each of the k heads are placed onto their corresponding tape. To do this, for each symbol $g \in \Gamma_m$, we introduce a new symbol $\underline{g}$ that stands for "the head being positioned over symbol g".

M_s uses blanks to delimitate the relevant portion of the tape; so does M_m for its k tapes. To avoid confusion, we distinguish between *"the blank symbol of M_m"* and *"the blank symbol of M_s"*.

For example, if $\Gamma_m = \{0, 1, \square_m\}$, then Γ_s will contain at least: $0, 1, \square_m, \underline{0}, \underline{1}, \underline{\square_m}, \diamond, \square$.

For each state q of M_m we will need several states in M_s. First we will make a simple copy; the copy of q_{1_m} will be q_{1_s}, the initial state of M_s.

The halting state H_s of M_s will be the simple copy of the halting state H_m of M_m. Similarly for Y_s and N_s.

The very first thing M_s needs to do when started on input w is to recreate the initial configuration of $M_m[w]$.

For example, consider the input 10011 and $k = 4$. M_s will start with the following tape:

...□□□10011□□□...

Using some auxiliary states, M_s will modify the tape to look like this:

$$...\square\square\square \diamond \underline{1}0011 \diamond \underline{\square_m} \diamond \underline{\square_m} \diamond \underline{\square_m} \diamond \square\square\square...$$

Then it will move its head over the first underlined symbol; in this example, the leftmost 1. From here, the simulation of M_m's transitions begins.

M_s will simulate each transition of M_m, one at a time; for ease of reference, we shall refer to the simulation of one transition of M_m as a *"phase"*. Each phase consists of the following steps:

1. At the start of a phase, M_s will be in some state q_copy, corresponding to a state q of M_m and the head of M_s will be positioned over the leftmost underlined symbol – i.e. where the first head of M_m is.

2. M_s needs to go rightward through the whole tape, finding the k symbols marked with an underline (which indicate the position of one of the k heads of M_m).

   Each time it finds the position of one of M_m's heads, it needs to *memorize* the symbol under it, in order to build the current "symbol k-tuple" which determines the transition. Remember that memorization of a fixed amount of information can be achieved using states:

   - from state q_copy, when we encounter a relevant symbol $\underline{g_1}$ ($g_1 \in \Gamma_m$), we transition to one of $|\Gamma_m|$ possible states $q_copy_{g_1}$
   - from each of the $|\Gamma_m|$ states $q_copy_{g_1}$, when we encounter a relevant symbol $\underline{g_2}$, we transition to one of $|\Gamma_m|$ possible states $q_copy_{g_1 g_2}$
   - etc.

   For other symbols, reamin in the same state and move right.

   In summary, for each state q of M_m we need:

   - a copy of it, q_copy
   - $|\Gamma_m|$ new states $q_copy_{g_1}$
   - $|\Gamma_m|^2$ new states $q_copy_{g_1 g_2}$
   - $|\Gamma_m|^3$ new states $q_copy_{g_1 g_2 g_3}$
   - ...
   - $|\Gamma_m|^k$ new states $q_copy_{g_1 g_2 g_3 ... g_k}$

   A total of $|Q_m|(1 + |\Gamma_m|\frac{(|\Gamma_m|^k - 1)}{|\Gamma_m| - 1})$ new states.

3. Once we reach a state of the form $q_copy_{g_1 g_2 g_3 ... g_k}$, we have to simulate the transition given by

   $$\delta_m(q, (g_1, g_2, ..., g_k)) = (s, (c_1, D_1), (c_2, D_2), ..., (c_k, D_k))$$

   (Where $c_1, c_2, ..., c_k \in \Gamma_m$ and $D_1, D_2, ..., D_k \in \{\leftarrow, -, \rightarrow\}$.)

   We will do this by going back through the tape and performing the necessary changes for each head, by first transitioning into a new state $pc_s^k_{g_1 g_2 g_3 ... g_k}$.

   The k superscript signifies that we are now performing the changes for the kth head; after this, we'll transition to $pc_s^{k-1}_{g_1 g_2 g_3 ... g_k}$ and go left until we find the $(k-1)^{th}$ head and so on.

   The s shows that we need such a collection of new states for each state s of M_m; after M_s will perform all symbol modifications, it will transition to the simple copy of s and start a new phase.

   So for each of the $|Q||\Gamma_m|^k$ states $r_{g_1 g_2 g_3 ... g_k}$, we add $k|Q'|$ more states.

4. Each head can only move at most one position to the left or to the right, so our changes will be *localized* on the cell with the marked symbol and its two immediate neighbors.

   For example, consider the following tape section of M_s that corresponds to some tape i of M_m:

$$... \diamond 00101\underline{0}11011101 \diamond ...$$

The ith head of M_m will change the symbol currently read (or leave it as it is) and move the head onto the cell to the left/right or keep it where it is. If the head moves, we need to change the current symbol to its non-underlined variant, and change the corresponding symbol (either the one to the left, or the one to the right, depending on where M_m's i^{th} head will go), to its underlined variant.

But nothing else on this i^{th} tape can change during this one transition; only three symbols, at most, will differ, but the rest will be the same:

$$... \diamond 0010???1011101 \diamond ...$$

5. We now describe the transitions for states $pc_s^j_{g_1 g_2 g_3 ... g_k}$ with $2 \leq j \leq k$.

   Let $\delta_m(q, (g_1, g_2, ..., g_k)) = (s, (c_1, D_1), (c_2, D_2), ..., (c_k, D_k))$.

   From state $pc_s^j_{g_1 g_2 g_3 ... g_k}$, on symbol $\underline{g_j}$ we write:

   - $\underline{c_j}$, if $D_j = -$
   - c_j, otherwise

   If $D_j = \rightarrow$ we move right into a state $auxR_s^j_{g_1 g_2 g_3 ... g_k}$ for which

   $$\delta(auxR_s^j_{g_1 g_2 g_3 ... g_k}, h) = (pc_s^{j-1}_{g_1 g_2 g_3 ... g_k}, \underline{h}, \leftarrow), \forall h \in \Gamma_m$$

   Simply put, we move one cell to the right and underline whatever is there to mark that that's where the current (j^{th}) head moved; we then move on to find the next head.

   Similarly for $D_j = \leftarrow$, we move left and use a state of the form $auxL$.

   If $D_j = -$, we move left and change state to $pc_s^{j-1}_{g_1 g_2 g_3 ... g_k}$.

6. For $j = 1$, auxiliary states of the form $auxL$ and $auxR$ will transition to state s_{copy}.

   This concludes the current phase.

It follows, by inductive reasoning, that M_s correctly simulates the behavior of M_m.

5.1 Shifting the tape contents

A complication arises when one of the simulated k heads is either on the first or the last *non*-$\square_m$ symbol, i.e. it's immediately to the left/right of a $\diamond$. For example, if the head is on the first symbol and then it moves left, it will end up on our special delimitator $\diamond$. However, from M_m's point of view, the head should find here a blank symbol. So we need to add one on demand; and we can do so by shifting the rest of our tape's contents, one cell to the right.

$$10001\underline{1}01 \diamond 1\underline{0}11 \diamond \underline{0}11000001 \diamond 1101\underline{0}11\diamond$$

$$10001\underline{1}01 \diamond 1\underline{0}11 \diamond \square_m\underline{0}11000001 \diamond 1101\underline{0}11\diamond$$

$$10001\underline{1}01 \diamond 1\underline{0}11 \diamond \underline{\square_m}011000001 \diamond 1101\underline{0}11\diamond$$

UNDECIDABLE PROBLEMS

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

October 2022

We have seen many examples of what Turing Machines can do and strongly suggested that there are things that they *cannot* do. A consequence of the Church-Turing thesis is that the set R represents exactly those decision problems that can be solved by an algorithm; so any decision problem outside of R would not be algorithmically solvable.

Indeed, we claimed that the problem *"Pairs"* introduced at the very first lecture was an example of such a problem, not belonging to R; but until now we have not rigorously proven this claim, nor have we proven that there are any decision problems that are undecidable.

In this lecture, we will formally show that there are *infinitely many* undecidable problems and identify a particular one, *the Halting Problem*.

In order to achieve the first result, we will take a short detour through set theory to introduce (recap?) the notion of *cardinality*, which is one way of extending the concepts of "smaller" or "larger" when comparing *infinite* sets.

1 Cardinality

We say that two sets A and B "have the same cardinality" if there exists some bijection between them. We denote this by the notation $|A| = |B|$.

For *finite sets*, this conforms very well with our intuition of "size of set" which associates a natural number to each set. For example, the two sets: $A = \{"alpha", "beta", "gamma"\}, B = \{2, 4, 9\}$ have the same cardinality as can be seen from the following bijection[†]:

$f_1 : A \to B$

$f_1("alpha") = 1$

$f_1("beta") = 2$

$f_1("gamma") = 3$

But if we were to compare A with some other set like $C = \{a, b, c, d\}$, we can find no bijection between them, so they have *different cardinalities*; $|A| \neq |C|$.

Because there exists *an injection* from set A to set C, we can say that set A is "smaller" than C; $|A| \leq |C|$. One possible injection is:

$f_2 : A \to B$

$f_2("alpha") = a$

$f_2("beta") = b$

$f_2("gamma") = c$

If $|A| \leq |C|$ and $|A| \neq |C|$, we say $|A| < |C|$.

When we move on into the realm of *infinite sets*, cardinality becomes less intuitive (but, nevertheless, very useful). For example, we can show that, cardinality-wise, the set of all integers $\mathbb{Z}$ has the same cardinality as the set of all natural numbers $\mathbb{N}$. Consider the bijection:[‡]

$f_3 : \mathbb{Z} \to \mathbb{N}$

$f_3(0) = 0$

[†]There are other bijections as well, but we only care that there's at least one.

[‡]This can also be expressed succinctly as $f_3(x) = 2 * |x| - sign(x)$, where $|x|$ is the absolute value of the integer and $sign(x)$ is 0 for positive integers and 1 for negative integers.

$f_3(-1) = 1$

$f_3(1) = 2$

$f_3(-2) = 3$

$f_3(2) = 4$

$f_3(-3) = 5$

$f_3(3) = 6$

...

This will probably seem weird, as the natural numbers are a proper subset of the integers ($\mathbb{N} \subsetneq \mathbb{Z}$); but they both have the same cardinality.

What's more, we can show that the *rational numbers* $\mathbb{Q}$ also have the same cardinality as the natural numbers $\mathbb{N}$, but this is outside the scope of this lecture.

An interesting result for us is that the naturals have the same cardinality as the set of all strings over some alphabet $\Sigma = \{\sigma_1, \sigma_2, ..., \sigma_n\}$.

$nts : \mathbb{N} \to \Sigma^*$

$nts(0) = \epsilon$

$nts(1) = \sigma_1$

$nts(2) = \sigma_2$

...

$nts(n) = \sigma_n$

$nts(n+1) = \sigma_1\sigma_1$

$nts(n+2) = \sigma_1\sigma_2$

...

Coming back to familiar sets of numbers, there exists no bijection from the naturals $\mathbb{N}$ to the reals $\mathbb{R}$. The cardinality of $\mathbb{R}$ is larger than that of $\mathbb{N}$.

In the next section, we shall show that the cardinality of the real interval $[0, 1)$ is larger than that of $\mathbb{N}$; that the same holds for $\mathbb{N}$ and $\mathbb{R}$ should then be obvious.

1.1 Diagonalization

Recall the bijection we used to show that the naturals and the integers have the same cardinality. Basically, in order to show that some set S has the same cardinality as the natural numbers, it is sufficient to show that we can *systematically place all elements of that set in a list* – because the items in a list can all be denoted as: 0, 1, 2, ... By the term "systematically", we mean in such a way as to be clear what element of S would be at any position n.

We shall prove, by *reductio ad absurdum*, that we cannot define such a list of all the real numbers in the interval $[0, 1)$.

Theorem 1.1. $|\mathbb{N}| < |[0, 1)|$

Proof. Our proof will consist of two parts; first we will show that there can be no bijection between these two sets, so their cardinalities differ.

Assume that we can make a list of the numbers in $[0, 1)$:

0 $0.d_{00}d_{01}d_{02}d_{03}d_{04}...$

1 $0.d_{10}d_{11}d_{12}d_{13}d_{14}...$

2 $0.d_{20}d_{21}d_{22}d_{23}d_{24}...$

3 $0.d_{30}d_{31}d_{32}d_{33}d_{34}...$

...

Where each d_{ij} is a single digit: the jth decimal figure of the ith number in the list. We consider each number to have infinite decimal digits; for those with a finite representation, such as 0.253, simply consider a neverending trail of 0s, i.e. 0.2530000...

Now consider the following real number:

$X = 0.x_0x_1x_2x_3x_4...$

Where x_i is a single digit, as follows:

- if d_{ii} is 0, then x_i is 1
- if d_{ii} is not 0, then x_i is 0

This number is not in our initial list!

- x_0 is different than d_{00}, so X differs at least in the first decimal place from the first number in the list.
- x_1 is different than d_{11}, so X differs at least in the second decimal place from the second number in the list.
- etc.

Generally, x_i is different than d_{ii} so X differs at least in the ith decimal from the ith number in the list.

Thus X differs from any number in the list, so it's not on the list.

But our initial assumption was that our list contains all numbers in $[0, 1)$ and we forced no constraints on it. So our list cannot exist – there is no bijection between $\mathbb{N}$ and $[0, 1)$; $|\mathbb{N}| \neq |[0, 1)|$.

Consider the following function:

$i : \mathbb{N} \to [0, 1)$

$i(n) = 0.digits(n)$

Where $digits(n)$ represents the sequence of base10 digits used to represent n. For example, $i(12) = 0.12$, $i(734837) = 0.734837$. As i is an injection, $|\mathbb{N}| \leq |[0, 1)|$.

□

2 There exist undecidable problems

Now that we are somewhat familiar with the concept of cardinality, we will apply it in our context of problems and machines, to show that *there are functions for which no Turing Machine that decides them exists.*

2.1 Cardinality of Turing Machines

Let $\mathbb{M}$ be the set of all Turing Machines.

Theorem 2.1. $|\mathbb{M}| \leq |\mathbb{N}|$

(A stronger statement is true: $|\mathbb{M}| = |\mathbb{N}|$, but we have no need for it, so we won't prove it.)

Proof. In the previous lecture we showed how we can encode each Turing Machine as a string over the binary alphabet $\{0, 1\}$, i.e. we provided an injection from $\mathbb{M}$ to $\{0, 1\}^*$; $|\mathbb{M}| \leq |\{0, 1\}^*|$.

In the previous section, we showed how the set of all binary strings has the same cardinality as the set of natural numbers; $|\{0, 1\}^*| = |\mathbb{N}|$.

□

2.2 Cardinality of decision problems

Let $\mathbb{D}$ be the set of all decision problems.

Theorem 2.2. $|\mathbb{N}| < |\mathbb{D}|$

Proof. To represent a decision problem f, we will use an infinite sequence of 0s and 1s; the ith element of the sequence is 1 if $f(nts(i)) = TRUE$, or 0 if $f(nts(i)) = FALSE$, where nts is our bijection from strings to naturals, presented in section 1.

Assume we can make a list of all decision problems.

0 $a_{00}a_{01}a_{02}a_{03}a_{04}...$

1 $a_{10}a_{11}a_{12}a_{13}a_{14}...$

2 $a_{20}a_{21}a_{22}a_{23}a_{24}...$

3 $a_{30}a_{31}a_{32}a_{33}a_{34}...$

...

For example, decision problem f_0, on input $nts(2)$ has an answer corresponding to a_{02}.

Consider the following decision problem $f_d : \Sigma^* \to \{FALSE, TRUE\}$:

$$f_d(w) = \begin{cases} TRUE & f_{nts^{-1}(w)}(w) = FALSE \\ FALSE & f_{nts^{-1}(w)}(w) = TRUE \end{cases}$$

For $w = nts(0)$, the value of $f_d(w)$ is $TRUE$ if $f_0(w) = FALSE$ and vice-versa; so f_d is not f_0, because they differ at least for $w = nts(0)$.

For $w = nts(1)$, the value of $f_d(w)$ is $TRUE$ if $f_1(w) = FALSE$ and vice-versa; so f_d is not f_1, because they differ at least for $w = nts(1)$.

In general, for any $i \in \mathbb{N}$, $w = nts(i)$, the value of $f_d(w)$ is $TRUE$ if $f_i(w) = FALSE$ and vice-versa; so f_i is not f_i, because they differ at least for $w = nts(1)$.

f_d does not appear in the list, so our assumption that a list that can contain all decision problems exists leads to a contradiction; such list can't exist. $|\mathbb{N}| \neq |\mathbb{D}|$.

We will now show an injection from $\mathbb{N}$ to $\mathbb{D}$. For each natural number n, we can obtain a unique binary encoding consisting of the symbols 0 and 1, preceded by an infinite string of leading 0s. Number the last digits "0, the second to last "1", the third to last "2" and so on. Let $f_n(nts(i)) = TRUE$ if the ith digit of n is 1, $FALSE$ otherwise; $\forall i \in \mathbb{N}$.

For example, given the number 6, its binary encoding will be: "...0000110". $f_6(nts(0)) = FALSE$, $f_6(nts(1)) = TRUE$, $f_6(nts(2)) = TRUE$ and, for all other values f_6 is $FALSE$.

The function g that takes a number n and maps it to f_n is an injection from $\mathbb{N}$ to $\mathbb{D}$; $|\mathbb{N}| \leq |\mathbb{D}|$.

□

2.3 There are "fewer" machines than problems

We showed that $|\mathbb{M}| \leq |\mathbb{N}|$, then $|\mathbb{N}| < |\mathbb{D}|$; this gives us$|\mathbb{M}| < |\mathbb{D}|$.

Thus, there is no injection from $\mathbb{D}$ to $\mathbb{M}$ – there are some decision problems to which we cannot associate a Turing Machine. So there are infinitely many decision problems, for which there is no Turing Machine that can decide them.

In fact, we have proven a stronger statement. Remember the weaker concept of "solving a decision problem": *acceptance*. This was also defined in terms of existence of some Turing Machines. The fact that the cardinality of decision problems is strictly greater than that of all Turing Machines means that there are infinitely many decision problems for which there is no Turing Machine that can accept them.

Given an undecidable problem f, we have no hope of devising an algorithm that can solve it; i.e. always produce the correct answer in a finite amount of steps.

But if we know that f is undecidable, but *acceptable,* then, by definition, there exists a Turing Machine M, such that, for all words w for which $f(w) = TRUE$, $M[w] \to TRUE$. When $f(w) = FALSE$, the machine will either provide the correct answer or loop forever. So a clever idea is to check if the machine loops forever; if yes, then we can conclude that the answer to $f(w)$ is $FALSE$.

Unfortunately, such a check cannot itself be performed algorithmically, as we shall show in the next section.

3 The Halting problem

Definition 3.1. The *"halting problem"* is the decision problem:

$$\texttt{HALT}(enc((M,w))) = \begin{cases} TRUE & M[w] \text{ halts} \\ FALSE & otherwise \end{cases}$$

Here is part of the reason why this problem is important to us:

If $HALT$ were decidable, we could build a machine M_{HALT} that decides it. We could then take any machine M and input w, serve them as input to M_{HALT} and find out if $M[w]$ halts. Only if we know it halts, we run $M[w]$ and get the result.

If M is a machine that accepts a problem f, then the fact that $M[w]$ doesn't halt assures us that $f(w) = FALSE$; in effect, we can now decide f, i.e. always obtain the correct answer in a finite amount of time.

3.1 HALT is undecidable

Theorem 3.1. $\texttt{HALT} \notin R$

Proof. Assume that there exists a machine M_H, s.t. M_H decides HALT.

We construct a machine D, which takes as input the encoding of some Turing Machine M; the behavior of D is as follows:

1. on input $enc(M)$, simulate $M_H[enc((M, enc(M))]$
2. if $M_H[enc((M, enc(M)))] \to FALSE$, then transition to state Y
3. else (if $M_H[enc((M, enc(M)))] \to TRUE$) transition to a new state *loop*, s.t. $\forall g \in \Gamma_{M_H}, \delta_{M_H}(loop, g) = (loop, g, \to)$

Thus, for any machine M, $D[enc(M)]$ either halts in state Y or it doesn't ever halt. What about $D[enc(D)]$?

Assume that it halts; then $M_H[enc((D, enc(D)))] \to TRUE$, so (see the 3rd step), D will transition to state *loop* and never halt. This is a contradiction!

Assume $D[enc(D)]$ doesn't halt; then $M_H[enc((D, enc(D)))] \to FALSE$, so (see the 2nd step), D will transition to state Y, halting. This is a contradiction!

There is no consistent behavior for $D[enc(D)]$. Let's look at the "components" of D and check that it was properly constructed:

- it simulates the operation of another machine on some input
- it branches based on the output of the simulation
- it can transition to final state Y
- it can transition to a state that goes to the right forever, leaving the tape unchanged, never halting

From previous lectures and labs, it should be evident that a machine can be designed to perform all these four actions, so D is properly defined.

The first bullet is the most complex operation; but recall the construction from the lab of a machine $M_{f \circ g}$ that computes the functional composition of two computable functions f and g, based on the machines that compute them: M_f and M_g. It's possible to have a machine contain the behavior of another machine *"baked into it"*.

Then the contradiction must stem from the initial assumption about the existence of M_H, the machine that decides HALT. So M_H cannot exist; $\texttt{HALT} \notin R$.

□

3.2 HALT is acceptable

Theorem 3.2. $\texttt{HALT} \in RE$

This should be an intuitive result. Remember what it means for a problem f to be *acceptable*: there has to exist a machine M_f that halts in state Y for any input for which $f(w) = TRUE$; for others it can either halt in state N or loop forever.

We will prove this theorem by constructing a machine that accepts HALT.

Let M_H be a Turing Machine with the following behavior:

- on input v, check if v is a valid encoding of a tuple (M, w) where M is a Turing Machine and w is a word. If it isn't, transition to N.
- simulate $M[w]$; transition to Y.

Theorem 3.3. M_H accepts HALT.

Proof. Remember, from the "Computing" lecture, what it means for a machine to accept a function. For all tuples (M, w) consisting of a machine and a word:

$\texttt{HALT}(enc((M, w)) = TRUE \Leftrightarrow M_H[enc((M, w))] \rightarrow TRUE$

This statement is made of two implications; we will have to prove both separately.

"$\Rightarrow$":

$\texttt{HALT}((enc((M, w))) = TRUE \Rightarrow M[w]$ halts $\Rightarrow$ in the second step, M_H transitions to $Y \Rightarrow M_H[enc((M, w))] \rightarrow TRUE$

"$\Leftarrow$":

$M_H[enc((M, w))] \rightarrow TRUE \Rightarrow M[w]$ halts $\Rightarrow \texttt{HALT}(enc((M, w))) = TRUE$

□

4 References and further reading

The concept of cardinality and the diagonal argument used in 1.1 were first presented by Georg Cantor (1845-1918) in his 1891 article: "Uber eine elementare Frage der Mannigfaltigkeitslehre." [1].

The idea that there are functions for which no machine exists to compute them is central to Turing's 1936 paper [2]. However, Turing's formalism and conventions were very different than in this lecture.

The modern formulation of "the halting problem", originates from Martin Davis's textbook "Computability and unsolvability" [3].

The proof of the undecidability of HALT presented in this lecture is adapted from section "An undecidable language", in chapter 4.2 of Michael Sipser's "Introduction to the Theory of Computation" [4].

Bibliography

[1] Georg Cantor. "Uber eine elementare Frage der Mannigfaltigkeitslehre". In: *Jahresbericht der Deutschen Mathematiker-Vereinigung* 1 (1891), pp. 75–78.

[2] Alan Mathison Turing et al. "On computable numbers, with an application to the Entscheidungsproblem". In: *J. of Math* 58.345-363 (1936), p. 5.

[3] Martin Davis. *Computability and unsolvability*. McGraw-Hill Book Company, 1958, p. 70.

[4] Michael Sipser. *Introduction to the Theory of Computation, Third Edition*. CENGAGE Learning, 2012. Chap. 4 Undecidability, pp. 207–209.

REDUCTIONS

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

November 2022

We have proven that there exist undecidable problems and then exhibited one particular example: *the halting problem*.

But what if we want to systematically classify newly encountered problems based on decidability? We are given a new problem f and we want some method by which we can tell whether it is decidable or undecidable.

At this point, we could prove decidability by designing a machine M_f that decides f. For undecidability, we could attempt a proof by contradiction in the style of creating machine D from the previous lecture, which helped us show that a machine that decides `HALT` cannot exist.

We will now introduce a handy "tool" that allows us to use *known* problems in order to establish the (un)decidability of *new* problems: **reductions**.

We will start with an informal description of reductions, but before we even do that, it should be pointed out that you are already familiar with and have used reductions before in the context of programming!

Consider this programming task:

"Write a program that checks if a string is a palindrome."

You could start writing a solution "from scratch", keeping track of two indices (which start from the beginning and end of the input string, respectively and work their way towards the middle) and checking if the characters at those indices match. Or, you might come up with a very short solution:

```
int isPalindrome(const char *s) {
    return (strcmp(s, reverse(s)) == 0)
}
```

That is, you've *reduced* the problem of determining whether a string is a palindrome, to the problem of finding its reverse. In programming, we write these sort of solutions for a variety of reasons: to keep code readable and maintainable, to modularize it so that we can use bits of it in other places etc.

In the context of Computability Theory, we shall use reductions for the purpose of proving (un)solvability[†]: if `reverse` is solvable, then so is `isPalindrome`.

(Actually, we also make use of the `strcmp` function, so we should say that you've reduced our problem to the one of determining the reverse *and* comparing it with the original.)

1 Intuition for Turing Machines

1.1 Prefix strings

Consider the following decision problem, `PREFIX`:

"Given two binary strings s and p, does s start with p?"

For $s = 0011010010$ and $p = 001101$, the answer is "yes", while for $s = 01010110$ and $p = 0110$, the answer is "no".

Is the problem decidable?

The answer should be an obvious "yes". To prove that it's decidable, we can provide a Turing Machine that decides it.

We will call this machine `isPrefix` and we'll adopt the following conventions:

[†]By which we mean any of: "computability", "acceptability", "decidability".

- the input of the machine will be a string over the alphabet $\Sigma = \{0, 1, \&\}$: it will be of the form $s\&p$ (e.g. 0011010010&001101)

- the machine will have two tapes

The machine will first move right, "moving" s from the first tape onto the second, by deleting each symbol on the first tape and writing it on the second.

Once it reaches the delimiter symbol &, it will reset (move left) the head of the second tape to the first symbol of s, then move the first head onto the first symbol of p.

The machine then moves both heads in tandem to the right:

- if there is a symbol mismatch and the head on the first tape is scanning a 0 or a 1, reject.

- if the head of the first tape reaches a blank symbol, accept.

Note 1.1. In the `resources` directory associated with this lecture, you will find the full description of this machine, written in the syntax for the online simulator.

1.2 Substrings

Let's consider a different, but similar, decision problem `SUBSTRING`:

"Given two binary strings s and p, does s *contain* p?"

For $s = 0011010010$ and $p = 1010$, the answer is "yes"[‡], while for $s = 01010110$ and $p = 0100$, the answer is "no".

Is the problem decidable?

The answer should be an obvious "yes". To prove that it's decidable, we can provide a Turing Machine that decides it... Or, we could show that a solution to the prefix problem entails a solution to this one.

Consider the following proposition, which asserts that a string p is a substring of s iff p is the prefix of some suffix of s:

$$\texttt{SUBSTRING}((s,p)) = TRUE \Leftrightarrow \exists i, 1 \leq i \leq n, \text{ s.t. }, \texttt{PREFIX}(s_i ... s_n, p)) = TRUE$$

Where $s_1, s_2, ..., s_n$ are the symbols that make up s.

`PREFIX` being decidable, we know that there is a machine that decides it (we even presented such a machine, `isPrefix`, in detail in the previous section). Starting from this machine, we can construct another, `isSubstring`, that decides `SUBSTRING`. Below, we give a pseudocode description of the machine:

`isSubstring`$[s\&p]$:
1. for $1 \leq i \leq |s|$:
2. compute $s' = s_i ... s_n$
3. if $|s'| < |p|$ transition to N
4. simulate `isPrefix`$[s'\&p]$
5. if `isPrefix`$[s'\&p] \rightarrow TRUE$
6. transition to Y

There are many techniques of reducing a problem to another; in this course, we will only address what are known as *"mapping reductions"*, in which we take an input for a problem and transform it into the input for another problem.

The reduction above, from `PREFIX` to `SUBSTRING` is **not** a mapping reduction.

2 Mapping reductions

In this section, let $f : \Sigma^* \rightarrow \{FALSE, TRUE\}$ and $g : \Sigma^* \rightarrow \{FALSE, TRUE\}$ be two decision problems over some alphabet Σ.

[‡]The string 1010 can be found starting from the fourth symbol of string 0011010010.

Definition 2.1.

$$f \leq_m g \stackrel{\text{def}}{=} \exists t : \Sigma^* \to \Sigma^*, \text{ s.t. } \begin{cases} t \text{ is computable} \\ \forall w \in \Sigma^\star, f(w) = g(t(w)) \end{cases}$$

Read the left-hand-side as "*f is mapping reducible to g*".

You can think of reducibility as a tool to compare the "hardness" of problems; if $f \leq_m g$, than solving f is "at most as hard as solving g".

The computability of t is important, because it means that if we have a machine that decides M_g, we can construct a machine that decides f: on any input w, first compute $t(w)$, then simulate $M_g[t(w)]$.

Theorem 2.1. (g is decidable) $\wedge$ ($f \leq_m g$) $\Rightarrow$ f is decidable

Proof. Remember what it means for a problem to be *decidable*: there exists some machine that decides it. So this theorem tells us that if there is a machine M_g that decides g and f is reducible to g, there is a machine M_f that decides f.

f being reducible to g means that there is some computable function t such that for any input w, $f(w) = g(t(w))$. t being computable, means there is some machine M_t that *computes* it.

Based on M_g and M_t, we will construct a machine M_f; the proof is similar to the composition exercise from the lab on computing. M_f will contain both the states+tape symbols of M_t and M_g, and almost the same transitions, with the following exception: when M_t would transition to H, M_f will transition to M_g's initial state instead.

M_f's initial state is the same as M_t's initial state.

Expressed as pseudocode:

$M_f[w]$:
1. simulate $M_t[w]$ to compute $t(w)$
2. simulate $M_g[t(w)]$
3. if $M_g[t(w)] \to TRUE$
4. transition to Y
5. else
6. transition to N

□

Theorem 2.2. (g is acceptable) $\wedge$ ($f \leq_m g$) $\Rightarrow$ f is acceptable

Proof. We use the exact same construction as in the proof above. If M_g accepts g, then M_f will accept f (the only difference is that there might be some outputs of t on which $M_g[t(w))]$ doesn't halt; $M_f[w]$ won't halt either, but it's ok because $g(t(w)) = FALSE \Rightarrow f(w) = FALSE$).

□

Theorem 2.3. (f is undecidable) $\wedge$ ($f \leq_m g$) $\Rightarrow$ g is undecidable

This is a direct consequence of Theorem 2.1 and it will be our main tool for proving a new problem is undecidable.

Similarly, we get:

Theorem 2.4. (f is unacceptable) $\wedge$ ($f \leq_m g$) $\Rightarrow$ g is unacceptable

3 More undecidable problems

To see how an undecidability proof using reductions looks like, we introduce two new interesting decision problems: `ALL` and `ANY`.

We need a known undecidable problem to reduce to each of the new ones, to show that they, in turn, are undecidable.

For now, the only such problem we know is `HALT`.

3.1 ALL

Definition 3.1.

$$\texttt{ALL}(enc(M)) = \begin{cases} TRUE, & \forall w \in \Sigma^*, M[w] \text{ halts} \\ FALSE, & otherwise \end{cases}$$

ALL is similar to HALT. Each of them takes as input a machine M; but whereas HALT also takes some specific input w to tell whether $M[w]$ halts, ALL tells if M halts on *"all"* inputs.

Theorem 3.1. $\texttt{ALL} \notin R$

Proof. We will prove that $\texttt{HALT} \leq_m \texttt{ALL}$. This result, combined with that of Theorem 2.1 show that $\texttt{ALL} \notin R$.

Our transformation t takes as input the encoding of a (machine, string) tuple (M, w) and maps it to the encoding of a single machine M'. We need to specify the behavior of M' in such a way that: $\texttt{HALT}(enc((M, w))) = TRUE \Leftrightarrow \texttt{ALL}(enc(M')) = TRUE$.

$M'[v]$:
1. erase v
2. write w
3. simulate $M[w]$

Is M' well-defined?

The erasing of the input v should seem straightforward: you've implemented machines that erase the contents of their tape at the lab.

But what does it mean for M' to refer to w or to M? The answer is that the contents of w and the encoding of M are *hardcoded* into M'.

You can imagine that M' has a series of states: $q_{w_1}, q_{w_2}, ..., q_{w_n}$ – one for each symbol w_i of w. For each of these states and the blank symbol, there is a transition that writes the symbol, goes to the state corresponding to the next symbol and moves the head right: $\delta_{M'}(q_{w_i}, \square) = (q_{w_{i+1}}, w_i, \rightarrow)$.

Similarly, all the states, tape symbols and transitions of M are hardcoded into M'; after writing the last symbol of w on the tape, M' resets its head onto the first symbol, w_1, and transitions to the initial state of M.

But is the transformation good?

If $\texttt{HALT}(enc((M, w))) = TRUE$, then $M[w]$ halts, so on any input v, M', which has the same behavior as $M[w]$ will also halt, thus $\texttt{ALL}(enc(M')) = TRUE$.

Similarly, if $\texttt{ALL}(enc(M')) = TRUE$, then M' halts on all inputs, which means $M[w]$ halts, so $\texttt{HALT}(enc((M, w))) = TRUE$.

But what about the computability of the transformation? At first, you might wonder what happens if $M[w]$ doesn't halt; but that is of no concern to the machine M_t which computes t. M_t simply has to read the encoding of (M, w) and write out the encoding of M', a machine which at some point will simulate $M[w]$. But it is not M_t which runs $M[w]$! Thus M_t can exist, t is computable.

□

Note 3.1. ALL and HALT are decision problems, so their domain is the entire set of strings: Σ^*.

However, the encoding scheme we described for Turing Machines makes it so that not every string is a valid encoding of a machine. So what do we mean when we say $\texttt{HALT}(enc((M, w)))$ and what is the value of $\texttt{HALT}(w)$ when w is not a valid encoding of a (M, w) tuple?

The answer is that $\texttt{HALT}(w)$ is then $FALSE$. Whenever we define a decision problem and write, for the input variable, an encoding of some object, we implicitly mean that for any string that is not a valid encoding of such an object, the answer is $FALSE$.

This is then relevant for reductions $f \leq_m g$; whenever f takes as input an encoded object, the transformation needs to map it to an input for which g is $FALSE$.

But this is always easy to do; for $\texttt{HALT} \leq_m \texttt{ALL}$, any input that is not a valid encoding of a (M, w) tuple can be mapped to ϵ, or any other string that is not a valid encoding of a Turing Machine.

3.2 ANY

Definition 3.2.

$$\texttt{ANY}(enc(M)) = \begin{cases} TRUE, & \exists w \in \Sigma^*, M[w] \text{ halts} \\ FALSE, & otherwise \end{cases}$$

Theorem 3.2. $\texttt{ANY} \notin R$

Proof. As before, we will reduce HALT to our new problem, i.e. we will prove that $\texttt{HALT} \leq_m \texttt{ANY}$. This result, combined with that of Theorem 2.1 show that $\texttt{ANY} \notin R$.

Our transformation t takes as input the encoding of a (machine, string) tuple (M, w) and maps it to the encoding of a single machine M'. We need to specify the behavior of M' in such a way that: $\texttt{HALT}(enc((M, w))) = TRUE \Leftrightarrow \texttt{ANY}(enc(M')) = TRUE$.

$M'[v]$:
1. if $v = 1011$:
2. erase v
3. write w
4. simulate $M[w]$
5. else:
6. loop

M' only simulates $M[w]$ for a particular input, arbitrarily chosen by the transformation. We could've actually used the same description for M' as in the ALL reduction above, but we choose to write it this way in order to emphasize the differences between ALL and ANY.

If $\texttt{HALT}(enc((M, w))) = TRUE$, then $M[w]$ halts, so on input 1011, M' will have the same behavior as $M[w]$, so it will also halt, thus $\texttt{ANY}(enc(M')) = TRUE$.

Similarly, if $\texttt{ANY}(enc(M')) = TRUE$, then there is an input on which M' halts; because the "else" branch always makes it loop, we can only look at the "then" branch of the "if": here M' will behave as $M[w]$, so it means $M[w]$ halts and $\texttt{HALT}(enc((M, w))) = TRUE$.

□

Theorem 3.3. $\texttt{ANY} \in RE$

The first idea that might come up for a machine that accepts ANY is to start from the first string[§] and run the input machine on it; if it doesn't halt, move on to the next string and so on. But this idea doesn't work; if the input machine doesn't halt on the first string, then our simulation will never halt, so we cannot move on to the second string.

What we need to do is to run the input machine *concurrently* on multiple strings. But on how many strings? We cannot simply run one step for each string, then move on to running the second step for each string – we would never finish the strings for which we need to run one step!

We must, at every step, partially run just one more input.

Proof. We prove $\texttt{ANY} \in RE$ by constructing a machine that accepts ANY.

$A_{\texttt{ANY}}[enc(M)]$:
1. $n := 0$
2. $strings := \emptyset$
3. forever:
4. $strings := strings \cup \{n^{th} \text{ string }\}$
5. for each w in $strings$:
6. simulate $M[w]$ for one more step (or the first step, for the string just added)
7. if $M[w]$ halted, transition to Y
8. $n := n + 1$

If there is any string w for which $M[w]$ halts, it will be discovered by $A_{\texttt{ANY}}$, so $A_{\texttt{ANY}}$ accepts ANY.

□

[§]Remember that $|\Sigma^*| = |\mathbb{N}|$, so there is some ordering of strings.

Notice that, in the previous section, we did not present the construction of a machine that accepts ALL. The reason for that is that such machine doesn't exist!

To prove that ALL is not acceptable, we could reduce a known unacceptable problem to it. But, at the moment, we don't know any such problem, so we need to take a slight detour, studying the *complement* of a decision problem, to arrive at an example: $\overline{\texttt{HALT}}$.

4 Complement of a decision problem

Let $f : \Sigma^* \to \{FALSE, TRUE\}$ be a decision problem.

Definition 4.1. The complement of f is a decision problem $\overline{f}$, such that

$$\overline{f}(w) = \begin{cases} TRUE, & f(w) = FALSE \\ FALSE, & f(w) = TRUE \end{cases}$$

In other words, the complement of a decision problem is the problem that maps each string to the opposite $\{FALSE, TRUE\}$ answer.

When it comes to the relationship between a problem and its complement with regards to decidability and acceptability, we can obtain two interesting theorems.

Theorem 4.1. $f \in R \Leftrightarrow \overline{f} \in R$

This tells us that, if a problem is decidable, so is its complement. We can say that the set of decidable problems, R, is *"closed under complementation"*.

Proof. We will prove the right implication: $f \in R \Rightarrow \overline{f} \in R$.

Because the complement of a problem's complement is the original problem ($\overline{\overline{f}} = f$), the proof of the left implication is nearly-identical.

$f \in R$ means that there exists a machine D_f that decides f. Starting from it, we construct a machine $D_{\overline{f}}$ that decides $\overline{f}$.

$D_{\overline{f}}$ will have all the same elements of D_f, except that all transitions to Y will be replaced by transitions to N and vice-versa.

I.e., any transition $\delta_{D_f}(q, g) = (Y, h, d)$ will become $\delta_{D_{\overline{f}}}(q, g) = (N, h, d)$ and any transition $\delta_{D_f}(q, g) = (N, h, d)$ will become $\delta_{D_{\overline{f}}}(q, g) = (Y, h, d)$, for some $q \in Q;\ g, h \in \Gamma;\ d \in \{\leftarrow, -, \rightarrow\}$.

In effect, any input accepted by D_f is rejected by $D_{\overline{f}}$ and any input rejected by D_f is accepted by $D_{\overline{f}}$.

So $D_{\overline{f}}$ decides $\overline{f}$, thus $\overline{f} \in R$.

□

Theorem 4.2. $(f \in RE) \wedge (\overline{f} \in RE) \Leftrightarrow f \in R$

If a problem is acceptable and its complement is acceptable, then the problem is *decidable*. Obviously, combined with the previous theorem, its complement is also decidable.

Proof. We will prove the right implication: $(f \in RE) \wedge (\overline{f} \in RE) \Rightarrow f \in R$.

The left implication should be obvious from the fact that $R \in RE$, combined with the previous theorem.

$f \in RE$ means that there exists a machine A_f that accepts f. Similarly, $\overline{f} \in RE$ means that there exists a machine $A_{\overline{f}}$ that accepts $\overline{f}$.

Starting from these, we construct a machine D_f that decides f.

It's important to note that, for any string w, either $f(w) = TRUE$ or $\overline{f}(w) = TRUE$. Which means that either $A_f[w]$ will halt in the Y state, or $A_{\overline{f}}$ will halt in the Y state.

So we will run these two machines concurrently and notice which one halts. Not that if $A_{\overline{f}}$ halts, we need to reverse its output. We also said that machines that accept a problem don't *necessarily* loop, but may also halt in final state N, whenever the problem's answer is $FALSE$, so we need to take this into account.

$D_f[w]$:
1. forever:
2. simulate one step of $A_f[w]$
3. if $A_f[w] \to TRUE$:
4. transition to Y
5. else if $A_f[w] \to FALSE$:
6. transition to N
7. simulate one step of $A_{\overline{f}}[w]$
8. if $A_{\overline{f}}[w] \to TRUE$:
9. transition to N
10. else if $A_{\overline{f}}[w] \to FALSE$:
11. transition to Y

□

5 Unacceptable problems

Our first example of a problem outside RE is the complement of the halting problem.

Definition 5.1.

$$\overline{\texttt{HALT}}(enc((M,w))) = \begin{cases} TRUE, & M[w] \to \bot \\ FALSE, & otherwise \end{cases}$$

Note 5.1. We now reiterate what was said in Note 3.1. Taken as it is, for any string that is not a valid encoding of a (M, w) tuple, both HALT and $\overline{\texttt{HALT}}$ should be $FALSE$; but this contradicts our definition of a problem's complement.

So we convene that whenever a problem is presented as *"the original"*, then it will be $FALSE$ on invalid strings, while the problem presented as *"its complement"* will be $TRUE$.

This is merely a convention and does not affect any of our results; we could just as well say that HALT is $TRUE$ on invalid strings and $\overline{\texttt{HALT}}$ is $FALSE$.

Why is $\overline{\texttt{HALT}}$ not in RE?

Remember that, in the last lecture, we proved that $\texttt{HALT} \in RE$. Earlier, in this lecture, we proved Theorem 4.2. So, if $\overline{\texttt{HALT}} \in RE$, then $\texttt{HALT} \in R$. But in the last lecture, we also proved that $\texttt{HALT} \notin R$, which leads to a contradiction. Thus our assumption must be false and $\overline{\texttt{HALT}} \notin RE$.

However, there is still some "solvability" to $\overline{\texttt{HALT}}$: we can write a machine that, whenever $\overline{\texttt{HALT}}(enc((M,w)) = FALSE$, halts in the final state N, giving us the correct answer. Whenever $\overline{\texttt{HALT}}(enc((M,w)) = TRUE$, our machine might halt in Y, or loop forever. This is obvious due to the symmetry to HALT: we just take the machine that accepts HALT and swap Y for N.

We might call $\overline{\texttt{HALT}}$ a "rejectable problem"; in Computability Theory, such problems are simply known as "problems whose complement is acceptable", which leads us to the following set:

Definition 5.2. $coRE \stackrel{\text{def}}{=} \{f \in \mathbb{D} \mid \overline{f} \in RE\}$

Note that $R = RE \cap coRE$.

These three sets capture the problems that are, in some sense, "solvable": either we can always produce the correct answer (R), or produce the correct answer whenever it's $TRUE$ (RE), or produce the correct answer whenever it's $FALSE$ ($coRE$).

We shall now see that ALL belongs to neither of these sets.

6 Beyond $RE \cup coRE$

Having discovered an unacceptable problem, $\overline{\texttt{HALT}}$, we can now show that $\texttt{ALL} \notin RE$, using reductions. We will show that $\overline{\texttt{HALT}} \leq_m \texttt{ALL}$.

We cannot have the same reduction as in the case of $\texttt{HALT} \leq_m \texttt{ALL}$; we cannot check if a machine loops and halt if it does. We have to resort to a clever trick:

$M'[v]$:
1. erase v
2. write w
3. simulate $M[w]$ for $|v|$ steps
4. if $M[w]$ halted;
5. loop
6. else:
7. halt

We're not checking whether $M[w]$ loops, but rather we're checking if the machine is still running after i steps, for some natural number i which depends on the input. If $M[w]$ loops, then it will not halt in i steps, for any i.

Note that i doesn't have to be the *length* of the input; we chose this for ease of writing; it could be the result from some other function from strings to naturals. But that function has to be strictly increasing and have no maximum value. Otherwise, it would be possible for $M[w]$ to halt in a number of steps just above that maximum value.

$\overline{\texttt{HALT}}(enc((M, w)) = TRUE$ means that $M[w]$ never halts; thus it will not halt after any number of steps, so M' will always take the "else" branch and halt for every input; $\texttt{ALL}(enc(M')) = TRUE$.

$\overline{\texttt{HALT}}(enc((M, w)) = FALSE$ means that $M[w]$ halts and it will do so in some number of steps n; thus for any input v, such that $|v| \geq n$, the simulation of $M[w]$ will have halted, so M will take the "then" branch, looping forever; so M' doesn't halt on every input and $\texttt{ALL}(enc(M')) = FALSE$.

Together with Theorem 2.4 we obtain: $\texttt{ALL} \notin RE$.

To show that $\texttt{ALL} \notin coRE$, we need to show that $\overline{\texttt{ALL}} \notin RE$.

We will also do this via a reduction from $\overline{\texttt{HALT}}$, proving $\overline{\texttt{HALT}} \leq_m \overline{\texttt{ALL}}$.

But we have already proven this! Remember the definition of a mapping reduction and that of a problem's complement. The transformation required by a mapping reduction maps all inputs for which $\overline{\texttt{HALT}}$ is $TRUE$ onto strings for which $\overline{\texttt{ALL}}$ is $TRUE$; similarly for $FALSE$.

This means that *the same transformation* used to show $\texttt{HALT} \leq_m \texttt{ALL}$, can be employed between their complements. In general, $f \leq_m g \Leftrightarrow \overline{f} \leq_m \overline{g}$.

We have shown that $\texttt{HALT} \leq_m \texttt{ALL}$ in subsection 3.1, thus $\overline{\texttt{HALT}} \leq_m \overline{\texttt{ALL}}$ and so $\overline{\texttt{ALL}} \notin RE$ ($\texttt{ALL} \notin coRE$).

In conclusion, $\texttt{ALL} \notin RE \cup coRE$.

POST'S CORRESPONDENCE PROBLEM

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

November 2022

At the first lecture, one of the three problems presented was "Pairs". The input to the problem was a set of tiles, with a top side and a bottom side. The question was whether you can arrange the tiles, with repetitions, such that the top sequence matches the bottom sequence.

This is actually known as *"Post's Correspondence Problem"* (PCP). It turns out that the problem is undecidable and we shall prove that result in this lecture.

Up until now, all undecidability results seemed to be very "meta" in some way: they all concerned problems whose input was a Turing Machine. What's interesting about PCP is that it will be our first example of a "natural" problem, formally proven to be undecidable.

The proof of PCP's undecidability is quite a laborious task, for which we need to introduce a few new concepts and adopt conventions about the machines considered.

But first, we need formal definitions to capture what the problem asks, what a tileset is, what a solution is etc.

1 Formal definition

1.1 tileset

Definition 1.1. A *"tileset"* is a finite sequence of pairs of strings over some alphabet Σ: $T = ((t_1, b_1), (t_2, b_2), ..., (t_n, b_n))$.

We shall refer to each pair of the sequence T by using a subscript, i.e.:

1. $T_1 = (t_1, b_1)$
2. $T_2 = (t_2, b_2)$
3. ...
4. $T_n = (t_n, b_n)$

Consider the following tileset:

$$\left(\begin{bmatrix}001\\0\end{bmatrix}, \begin{bmatrix}0\\01\end{bmatrix}, \begin{bmatrix}1\\0011\end{bmatrix}\right)$$

Here, $\Sigma = \{0, 1\}$.

This is just a sequence of three pairs; the first pair is formed of strings 001 and 0, the second of strings 0 and 01 and the third of strings 1 and 0011.

So we can formally represent the tileset above as: $((001, 0), (0, 01), (1, 0011))$.

We will define PCP as a decision problem, which means its input has to be a string over some alphabet. The formalism above provides us with a hint of how to encode a tileset as a string.

Let Σ be the alphabet of the strings which appear in the top/bottom part of a tile. We will use a new alphabet $\Sigma' = \Sigma \cup \{(, ,,)\}$; i.e. the original symbols plus the three new symbols for: open parentheses, comma, closed parentheses.

The encoding of the above tileset is: "$(001, 0)(0, 01)(1, 0011)$".

We are interested in whether a particular tileset has a "solution", so we must formally define what a solution is.

1.2 Solutions

Definition 1.2. A *solution* to a tileset $T = ((t_1, b_1), (t_2, b_2), ..., (t_n, b_n))$ is a finite sequence of indices $(i_1, i_2, ..., i_m)$ where:

$$\begin{cases} m \geq 1 \\ 1 \leq i_j \leq n, \quad (1 \leq j \leq m) \\ t_{i_1} t_{i_2} ... t_{i_m} = b_{i_1} b_{i_2} ... b_{i_m}. \end{cases}$$

Intuitively, a solution is an arrangement of one or more tiles (first line), such that if we read all symbols from the top side of the tiles left-to-right we get the exact same thing as reading all symbols from the bottom side of the tiles left-to-right (third line). A solution is allowed to use a tile multiple times.

The formal definition captures the idea that it's sufficient to provide a list of the tile indexes which appear in the solution.

The indexes are relative to the original tileset.

Consider the example:

$$\left(\begin{bmatrix} 001 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 01 \end{bmatrix}, \begin{bmatrix} 1 \\ 0011 \end{bmatrix} \right)$$

One solution for this is:

$$\begin{bmatrix} 001 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 01 \end{bmatrix} \begin{bmatrix} 001 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0011 \end{bmatrix}$$

Which can be expressed as: $(1, 2, 1, 3)$.

Bot the top and bottom parts, concatenated, form the string: 00100011.

We can now present the formal definition of PCP:

Definition 1.3.

$$\texttt{PCP}(enc(T)) = \begin{cases} TRUE, & \text{there exists a solution for tileset } T \\ FALSE, & otherwise \end{cases}$$

Our main result for this lecture will be proving that PCP is undecidable. The key idea is to describe a reduction from HALT. For reasons that will soon be clear, such a reduction is cumbersome, so we opt for an intermediate step. We introduce a modified version of PCP; we reduce HALT to it and, later, we will reduce this version to PCP. By the transitivity of $\leq_m$, we'll get HALT $\leq_m$ PCP, but breaking the transformation into two steps will allow us to better focus on the relevant aspects.

2 The Modified Post's Correspondence Problem (MPCP)

Our modified version of the problem is nearly-identical to the original one; inputs are still tilesets, as defined in subsection 1.1. The only difference is what constitutes a *solution*.

Definition 2.1. A *tile1-first solution* to a tileset $T = ((t_1, b_1), (t_2, b_2), ..., (t_n, b_n))$ is a finite sequence of indices $(i_1, i_2, ..., i_m)$ where:

$$\begin{cases} m \geq 0 \\ 1 \leq i_j \leq n, \quad (1 \leq j \leq m) \\ t_1 t_{i_1} t_{i_2} ... t_{i_m} = b_1 b_{i_1} b_{i_2} ... b_{i_m} \end{cases}$$

A "tile1-first solution", requires any solution to start with tile #1 of the tileset. Note that the list of indices can be empty (in the uninteresting cases where the top and bottom halves of tile #1 are identical).

Definition 2.2.

$$\texttt{MPCP}(enc(T)) = \begin{cases} TRUE, & \text{there exists a tile1-first solution for tileset } T \\ FALSE, & otherwise \end{cases}$$

It is MPCP to whom we'll reduce HALT. This might seem complicated, because the input to HALT is so different than the input to MPCP. To understand the transformation, we need first to introduce the notion of *"a computational history"* and how to encode one.

3 Computational histories

Definition 3.1. The *computational history* of machine M on input w is a sequence of configurations $h = (C_1, C_2, ...)$ where C_1 is the initial configuration of M on input w and, for each i, $C_i \vdash_M C_{i+1}$.
If $M[w]$ halts, then the history is a finite sequence of length n and C_n is a final configuration.

Going forward, it will be useful for us to be able to encode configurations and computational histories.

In order to do this, we employ the following convention: our configuration string will consist of all the symbols of the tape contents, but to the left of the currently scanned symbol, we shall place a new symbol that represents the current state. Thus we shall represent each configuration of a machine $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$, as a string over a new alphabet $\Sigma' = \Gamma \cup Q$.

Remember machine isEven from the first lecture and consider its run on input 10010.

The *initial configuration*, before any computation step takes place is: $\mathbf{q_1}10010$[†]

The machine goes through the following configurations:

$1\mathbf{q_1}0010$

$10\mathbf{q_1}010$

$100\mathbf{q_1}10$

$1001\mathbf{q_1}0$

$10010\mathbf{q_1}\square$

$1001\mathbf{q_2}0\square$

$1001\mathbf{Y}0\square$

For an input $w = \mathtt{w_1w_2w_3...w_n}$, we will denote the *"initial configuration"* as $C_0(w)$. Thus $C_0(w) = \mathbf{q_1}\mathtt{w_1w_2w_3...w_n}$. In the example above, $C_0(10010) = \mathbf{q_1}10010$.

We introduce a new convention: for the empty string ϵ, the initial configuration will be $C_0(\epsilon) = \mathbf{q_1}\square$ (instead of simply $\mathbf{q_1}$.

The encoding of a computational history is simply the encoding of all configurations, interspersed with a #:

$enc(h = (C_1, C_2, ..., C_n)) = \#enc(C_1)\#enc(C_2)\#...\#enc(C_n)\#$[‡].

Before we start describing the transformations from a pair (M, w) to a tileset T, we'll revisit our model for Turing Machines and simplify their structure, so that we have less edge-cases to consider. This is done with no loss of generality and, as you may later infer, will reduce our burden significantly.

4 Restricted machines

To keep our proof simple, our transformation from machines to tilesets will take as input only machines with the following properties:

- the machines never keep the head stationary; every transition moves it either left or right
- the machines never move their head to the left of the initial position (as if the tape was bounded on the left)

We can prove that, for any machine M matching our classic definition (i.e. can hold its head still, has unbounded tape in both directions), we can construct a machine M' with the restrictions above, such that $\forall w \in \Sigma^*, M[w] \equiv M'[w]$.

[†]Don't get too hung-up on the idea that "$\mathbf{q_1}$" is a symbol here. It doesn't matter that to us it looks as if it's two symbols: "q" and "1". In this context, it's a symbol specially chosen to be unambiguous with other states or any tape symbol from Γ.

[‡]You might reasonably wonder: "What about non-halting machines?". We established that an encoding should be *a string* and that strings are *finite sequences* of symbols, so we cannot encode infinite histories; we don't need to, anyway.

At the lab on computability, you have already proven that you can transform a regular machine to an equivalent one with each of the two properties, by describing two transformations. To get a machine that respects both, simply apply both transformations, one after the other.

5 Building a tileset from a (M, w) pair

To prove that HALT $\leq_m$ MPCP, we need to describe a computable transformation t whose input is the encoding of a (M, w) pair and whose output is the encoding of a tileset T, such that: $\texttt{HALT}(enc((M, w))) = TRUE \Leftrightarrow \texttt{MPCP}(enc(T)) = TRUE$.

The basic idea is to create a tileset in which the fragments on the top and bottom side of each tile can be concatenated together to form the computational history of $M[w]$. In any partial solution, the bottom half will be "ahead" in the computational history, while the top part, representing the same computational history, will lag behind, by one configuration. Only if $M[w]$ halts (its computational history is finite), will the top part be able to "catch up to" the bottom part. Completing the match entails creating the entire computational history of $M[w]$ and thus simulating $M[w]$.

Let $M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta)$ and $w = w_1 w_2 w_3 ... w_n$.

First, the transformation creates the initial tile:

$$\left[\begin{matrix} \# \\ \#\mathbf{q_1}\mathtt{w_1}\mathtt{w_2}\mathtt{w_3}...\mathtt{w_n}\# \end{matrix}\right]$$

Remember that we are creating a tileset for MPCP; so a solution, if it exists, must start with this first tile. Notice how this tile illustrates the idea above, that the top part is "lagging behind" the bottom part – in this case, by the entire first configuration. This is the only tile that depends on the word w; the rest of the tiles depend simply on M.

For all $a, b, c \in \Gamma$ and all $r, s \in Q$:

If $\delta(r, a) = (s, b, \rightarrow)$, then add all tiles of the form:

$$\left[\begin{matrix} \mathtt{ra} \\ \mathtt{bs} \end{matrix}\right]$$

If $\delta(r, a) = (s, b, \leftarrow)$, then add all tiles of the form:

$$\left[\begin{matrix} \mathtt{cra} \\ \mathtt{scb} \end{matrix}\right]$$

Remember the single-tape simulation of a multitape Turing machine, where we pointed out that, from one configuration to another, there's a limited number of symbols that can change – those around the head – the changes to the tape contents are *local*.

The two classes of tiles above capture both possible movements of the head (remember that we're now only dealing with machine which can't keep their head stationary).

In any partial solution, the top row would need to be filled in a way to resemble the last configuration of the bottom row (which is one configuration ahead!). Whatever tile we add that contains a state and surrounding symbols will then have its bottom part contain the correct modifications for the *next configuration*, such that the bottom row will still be one configuration ahead.

We then add, for any symbol $a \in \Gamma$, the following tile:

$$\left[\begin{matrix} \mathtt{a} \\ \mathtt{a} \end{matrix}\right]$$

This allows matching the rest of the tape symbols, which are not in the vicinity of the head.

We also need a tile for the configuration delimiter #:

$$\left[\begin{matrix} \# \\ \# \end{matrix}\right]$$

If the head ever reaches the end of the input and read a black symbol, we need to "create" it on demand, so we'll add a special version of the above tile, with one extra blank symbol in the bottom half:

$$\begin{bmatrix} \# \\ \square\# \end{bmatrix}$$

When the bottom half reaches a configuration in which the head has reached the end of the input, in some state r, this tile can be used to advance the partial solution, placing a blank on the bottom.

The top will have to catch up to it and it can only do so with a tile created for the transition $\delta(r, \square)$, namely one which has either `rsquare` or `cr`$\square$ on top, which will force the bottom row to be completed with the correct effects of that transition.

If the bottom half ever reaches a halting configuration (one which contains a final state: Y, N, or H), we then need to let the top row "catch up" to the bottom row. We introduce several tiles to create *artificial configurations* in which the machine is in a final state and its tape symbols disappear one by one.

Thus, for each $X \in \{Y, N, H\}$ we add the tiles:

$$\begin{bmatrix} \mathtt{a}\mathbf{X} \\ \mathbf{X} \end{bmatrix}, \begin{bmatrix} \mathbf{X}\mathtt{a} \\ \mathbf{X} \end{bmatrix}$$

After adding several of these tiles, the bottom part will be ahead of the top part by just one *artificial configuration*: $\#X\#$, where $X \in \{Y, N, H\}$. I.e. the machine is in final state X and there's nothing left on the tape.

We then need to let the top part match the bottom part and we do this with the special tiles (where $X \in \{Y, N, H\}$):

$$\begin{bmatrix} \mathbf{X}\#\# \\ \# \end{bmatrix}$$

If there exists a solution, this has to be the final tile.

The tileset described here only has a solution if the top half can ever catch up to the bottom half, i.e. if the computation of $M[w]$ reaches on of the final states $\{Y, N, H\}$, halting.

The transformation t simply has to read the encoding of (M, w) and output the encoding of the tiles described above, so it is surely computable.

We have thus proven that HALT $\leq_m$ MPCP, so MPCP $\notin R$.

We now need to return to our original problem, PCP.

6 MPCP $\leq_m$ PCP

MPCP and PCP are very similar problems; the key difference is that the solution of MPCP *has to start with the first pair of the tileset*.

To reduce MPCP to PCP means to find a transformation t that takes a tileset TS and maps it to another tileset TS' such that TS' only has a *solution* if TS has a *tile1-first solution*. I.e. $\texttt{MPCP}(enc(T)) = TRUE \Leftrightarrow \texttt{PCP}(enc(T')) = TRUE$.

We must build a tileset that obviously has no solution that doesn't start with the first pair.

Let Σ be an alphabet and $w = w_1 w_2 ... w_n$ be a string consisting of the symbols $w_1, w_2, w_3, ..., w_n \in \Sigma$. We introduce the following string operations:

1. $\star w = \diamond w_1 \diamond w_2 \diamond w_3 ... \diamond w_n$
2. $w\star = w_1 \diamond w_2 \diamond w_3 \diamond ... w_n \diamond$
3. $\star w\star = \diamond w_1 \diamond w_2 \diamond w_3 \diamond ... \diamond w_n \diamond$

Where $\diamond$ is a new symbol, i.e. $\diamond \notin \Sigma$.

The first operation adds a $\diamond$ *before* each symbol of string w, the second operation adds a $\diamond$ *after* each symbol of string w while the third one adds one $\diamond$ *before and after* each symbol of string w.

We also introduce a new "terminator symbol": ■.

The purpose of the three string operations and the terminator symbol is to convert each tileset that is an MPCP input:

$$\left(\begin{bmatrix} t_1 \\ b_1 \end{bmatrix}, \begin{bmatrix} t_2 \\ b_2 \end{bmatrix}, \begin{bmatrix} t_3 \\ b_3 \end{bmatrix}, \dots \begin{bmatrix} t_n \\ b_n \end{bmatrix}\right)$$

To a PCP input that looks like this:

$$\left(\begin{bmatrix} \star t_1 \\ \star b_1 \star \end{bmatrix}, \begin{bmatrix} \star t_1 \\ b_1 \star \end{bmatrix}, \begin{bmatrix} \star t_2 \\ b_2 \star \end{bmatrix}, \begin{bmatrix} \star t_3 \\ b_3 \star \end{bmatrix}, \dots \begin{bmatrix} \star t_n \\ b_n \star \end{bmatrix}, \begin{bmatrix} \diamond ■ \\ ■ \end{bmatrix}\right)$$

Each top-side string starts with a $\diamond$; the only bottom-side string that starts with a $\diamond$ is $\star b_1 \star$. It is thus obvious that, if there is a solution, it must start with the first tile:

$$\begin{bmatrix} \star t_1 \\ \star b_1 \star \end{bmatrix}$$

This is the only tile for which both the top and bottom part start with the same symbol, namely $\diamond$.

Why then did we not just add one $\diamond$ before/after the strings and we went to the trouble of defining the three $\star$ operations that intersperse $\diamond$ among all of a string's symbols? If we did that, we would invalidate solutions in which two or more top/bottom parts are used to match one bottom/top part etc.

The fact that the $\star$ operations interleave all symbols with $\diamond$ means that all matches are unaffected (notice how the first tile of the MPCP input appears in two forms: one with the bottom part surrounded by $\star$, forcing it to be the first tile, and another "normal" one, in which the bottom part is only *succeeded* by a star. This is because the tile could appear multiple times as part of the solution to MPCP, besides its first occurrence.)

Whatever solution exists for the MPCP tileset also applies to the PCP tileset, by taking the corresponding tiles. The strings formed will have the same symbols, but interspersed with $\diamond$. There is only one problem: the bottom string of the solution (the one formed by concatenating the bottom part of the tiles in the solution) will have an extra $\diamond$ than the top string. That is why we need the last special tile with the terminator symbol: it allows the top part to add the missing $\diamond$.

This means that MPCP $\leq_m$ PCP. Combined with the result from section 5 (HALT $\leq_m$ MPCP) and the transitivity of the $\leq_m$ relation, we obtain: HALT $\notin$ PCP. Thus PCP $\notin R$.

7 PCP $\in RE$

The proof of this is very simple, so we will just sketch the main idea: a machine that accepts PCP can, for any tileset T, start trying all possible sequences of length 1 to see if they represent a solution; then move on to all possible sequences of length 2, then to length 3 and so on.

If there is a solution, it will be found at some point, so the machine will halt, accepting. Otherwise, if there is no solution, the machine will keep trying longer and longer sequences, forever.

8 References and further reading

PCP was first presented and proven undecidable by Emil L. Post in his 1945 article: "A variant of a recursively unsolvable problem" [1]. However, Post's proof does not involve a reduction from HALT, using computational histories. In fact, it doesn't use the formalism of Turing Machines at all.

The proof in this lecture is adapted from chapter 5.2 of the textbook "Introduction to the Theory of Computation" [2] and chapter 9.4 of the textbook "Introduction to Automata Theory, Languages, and Computation" [3]. The latter credits it to unpublished notes of Robert W. Floyd.

Bibliography

[1] Emil L Post. "A variant of a recursively unsolvable problem". In: *Bulletin of the American Mathematical Society* 52.4 (1946), pp. 264–268.

[2] Michael Sipser. *Introduction to the Theory of Computation, Third Edition*. CENGAGE Learning, 2012. Chap. 5.2 A simple undecidable problem, pp. 227–233.

[3] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 2007. Chap. 9.4 Post's Correspondence Problem, pp. 401–412.

RICE'S THEOREM

Mihai-Valentin DUMITRU
mihai.dumitru2201@upb.ro

November 2022

As you may have noticed from the lab(s) and lecture on reductions, many problems that take as input the description of a Turing Machine are undecidable.

In this lecture, we will formalize this intuition, rigorously defining a category of decision problems and proving their undecidability. This result is known as *"Rice's theorem"*.

1 Intuition

Let's consider an example of a decision problem whose undecidability you've proven at the lab:

$$\texttt{P}(enc(M)) = \begin{cases} TRUE, & \forall w \in \Sigma^*, M[w] \text{ decides if } w \text{ is a palindrome} \\ FALSE, & otherwise \end{cases}$$

Note that it's crucial to distinguish between this problem and the following one:

$$\texttt{PALINDROME}(w) = \begin{cases} TRUE, & w \text{ is a palindrome} \\ FALSE, & otherwise \end{cases}$$

`PALINDROME` tells us if a particular string is a palindrome. It is a decidable problem; at the lab on Turing Machines, you even had to write a machine that decides it.

`P` tells us if a particular machine decides `PALINDROME`. It is *undecidable*; at the first reductions lab, you probably built a reduction to it from the halting problem: $\texttt{HALT} \leq_m \texttt{P}$.

The transformation t takes an input that encodes a (machine, word) tuple (M, w) and produces the encoding of a single machine M'.

The behavior of M' is as follows:

$M'[v]$:
1. memorize v
2. erase v
3. write w
4. simulate $M[w]$
5. erase any output and write v
6. if v is a palindrome:
7. transition to Y
8. else:
9. transition to N

To "memorize v", you can imagine that M' has some separate tape on which it can copy its input; a tape whose contents are not affected by the later simulation of $M[w]$.

After simulating $M[w]$, M' simply acts like some machine that decides whether v is a palindrome. We know that such machine exists, because $\texttt{PALINDROME} \in R$.

M' will correctly decide whether v is a palindrome, but only if it can go past line 4; i.e. if the simulation of $M[w]$ halts. So if we can tell if M' correctly decides whether its input is a palindrome, we can tell if $M[w]$ halts. Knowing that `HALT` is undecidable, leads us to the conclusion that `P` is also undecidable.

But is there anything special about checking if an input is a palindrome? Couldn't we just as well replace the final "if" of M' with something like "if v ends with a 0", or "if v has more than eight symbols" and obtain a similar

result? Namely, that deciding whether a given Turing Machine decides whether its input ends with 0, or if it has more than eight symbols, is impossible?

This is what Rice's theorem tells us.

2 Informal statement

One way to phrase Rice's theorem is the following:

"Any non-trivial property of acceptable problems is undecidable."

This sentence contains some key terms whose meaning is not obvious, so we will unpack them one by one.

2.1 "... property of acceptable problems..."

A *"property"* is something that an object either has, or doesn't. We can model a property using a set: the elements of the set are exactly the objects who have the property. Having the property means being an element of that set, not having the property means not being an element of that set. A property of "problems" is thus a set of problems.

2.2 "... property [...] is undecidable..."

With each set of decision problems, we can associate a "yes" or "no" question regarding membership to that set. I.e. given a decision problem f and a set of decision problems $P \subseteq \mathbb{D}$, we have the questions: *"$f \in P$?"*.

"Yes" or "no" questions remind us of decision problems, which map their inputs to the range $\{FALSE, TRUE\}$. The problem is that we specified the *domain* of decision problems to be the set of all strings over some alphabet: Σ^*. At the lecture on undecidability, we established that the cardinality of all decision problems is strictly greater than the cardinality of all strings, so we cannot have an encoding that maps an arbitrary decision problem to some string. Thus, an arbitrary decision problem cannot be the *"input"* of some other decision problem.

However, there is an interesting subset of decision problems that **can** be represented by strings, namely the *recursively-enumerable problems*. The string representation of such a problem is simply the encoding of a machine that accepts it.

We can have a decision problem which takes as input the encoding of a Turing Machine and maps it to $TRUE$, if the problem accepted by that machine has a certain property, or to $FALSE$ if the problem accepted by that machine doesn't have a certain property.

We abuse a bit the language and say that a "property is undecidable"; but, more clearly, we mean that the decision problem associated with said property (i.e. "does f have property P") is undecidable.

2.3 "... non-trivial property..."

We use "trivial" here in a precise sense: a property is trivial if either:

- all objects have that property
- no objects have that property

In the previous section we established that our "objects" are acceptable problems (represented by the encoding of a machine) and properties are subsets of RE.

Thus, the "trivial properties" are the following two subsets:

- $\emptyset$
- RE

The decision problems associated with them are, respectively:

- $f_\emptyset(enc(M)) = FALSE, \forall M$ (because no acceptable problem is a member of $\emptyset$)
- $f_{RE}(enc(M)) = TRUE, \forall M$ (because every acceptable problem is a member of RE)

These problems are clearly decidable:

- $f_\emptyset$ is decided by the machine which, from the initial state, upon reading any symbol, transitions to N.

- f_{RE} is decided by the machine which, from the initial state, upon reading any symbol, transitions to Y.

Which is why Rice's theorem mentions that only the **non-trivial** ones are undecidable.

We can now rigorously capture the informal description given about into a formal statement of Rice's theorem.

3 Formal statement

Theorem 3.1. Let S be a subset of the acceptable problems, $S \subseteq RE$.
Let f_S be the decision problem:

$$f_S(enc(A_p)) = \begin{cases} TRUE, & p \in S, \ (p \text{ is the problem accepted by } A_p) \\ FALSE & otherwise \end{cases}$$

Then:

$$f_S \in R \Leftrightarrow (S = \emptyset \vee S = RE)$$

$f_S \in R \Rightarrow (S = \emptyset \vee S = RE)$

$(S = \emptyset \vee S = RE) \Rightarrow f_S \in R$

4 Proof

We've already proven the left implication: $(S = \emptyset \vee S = RE) \Rightarrow f_S \in R$. In subsection 2.3, we constructed machines that decide $f_\emptyset$ and f_{RE}.

We now prove the contrapositive of the right implication:

$$(S \neq \emptyset \wedge S \neq RE) \Rightarrow f_S \notin R$$

Any problem in RE is either part of S, or it is not. Because the left-hand-side contains the condition that $S \neq RE$, then there must be at least some problem which is not part of S.

We'll start our proof with the assumption that the trivial problem whose answer is $FALSE$ for any input is not part of our set: $f_{FALSE} \notin S$. Later on, we'll see how we can address that case where $f_{FALSE} \in S$.

Because the left-hand-side also tells us that $S \neq \emptyset$, then there must be some decision problem $g \in S^\dagger$. Remember that $S \subseteq RE$, so any problem in S is acceptable; this means that there exists a Turing Machine M_g that accepts g.

We show that f_S is undecidable, by reducing the halting problem to it. I.e. we prove this statement:

$$\texttt{HALT} \leq_m f_S$$

Our transformation has to turn any (M, w) pair such that $M[w]$ halts into a machine which accepts a problem from S, and any (M, w) pair such that $M[w]$ doesn't halt into a machine which accepts a problem in $RE \setminus S$.

The transformation will take the encoding of any (M, w) and turn it into the encoding of a machine that acts as follows:

$M'[v]$:
1. memorize v
2. erase v
3. write w
4. simulate $M[w]$
5. erase any output and write v
6. simulate $M_g[v]$

If $\texttt{HALT}(enc((M, w))) = TRUE$, then $M[w]$ halts; this means that $M'[v]$ finishes the simulation on line 4 and proceeds to simulate $M_g[v]$ and have the same behavior. Because M_g accepts $g \in S$, M' also accepts g, thus $f_S(enc(M')) = TRUE$.

[†] There might be more, even an infinite number! We are interested only in one of them, chosen arbitrarily.

On the other hand, if $\texttt{HALT}(enc((M,w))) = FALSE$, then $M[w]$ doesn't halt; this means that $M'[v]$ never finishes the simulation on line 4. $M'[v]$ never reaches the final state Y, so it doesn't accept any input. However, each machine accepts *a problem*; the problem accepted by M' is f_{FALSE}. We started our proof with the assumption that $f_{FALSE} \notin S$, so $f_S(enc(M')) = FALSE$.

The point of the construction is that either M' has the same behavior as M_g, accepting g (which is in S), or never halts for any input, accepting f_{FALSE} (which is not in S).

Our demonstration hinges on the fact that $f_{FALSE} \notin S$. But what if $f_{FALSE} \in S$? Well, then we simply consider $RE \setminus S$ as our set and start from there (i.e. we pick a problem $g \in RE \setminus S$). This would give us a proof that $\texttt{HALT} \leq_m f_{RE\setminus S}$, thus $f_{RE\setminus S} \notin R$. But $f_{RE\setminus S}$ is simply the complement of f_S. Remember from the previous lecture that: $f \notin R \Leftrightarrow \overline{f} \notin R$. So $f_S \notin R$.

Note that our proof relies on the fact that our considered subset S contains at least one problem (g) and doesn't contain at least one problem f_{FALSE}. This is true of any subset that is not $\emptyset$ or RE.

5 References and further reading

The theorem presented here was elaborated by Henry Gordon Rice (1920-2003) in his doctoral disertation and in a 1953 article titled "Classes of Recursively Enumerable Sets and Their Decision Problems" [1]. Having been written so early, the paper uses very different notations and conventions than in modern contexts, which makes it quite difficult to read.

The definition and proof presented in this lecture are adapted from subhchapter 9.3.3 of the textbook "'Introduction to Automata Theory, Languages, and Computation" [2].

Bibliography

[1] Henry Gordon Rice. "Classes of recursively enumerable sets and their decision problems". In: *Transactions of the American Mathematical society* 74.2 (1953), pp. 358–366.

[2] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 2007. Chap. 9.3.3 Rice's Theorem and Properties of the RE Languages, pp. 397–399.

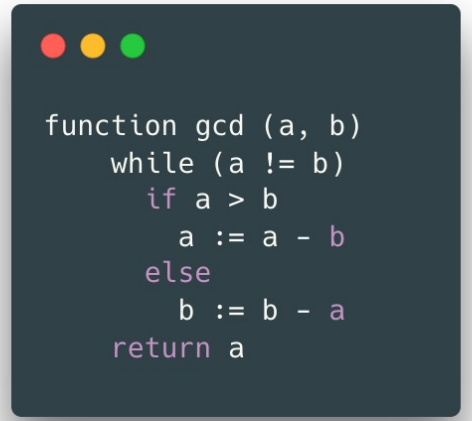


Analiza Algoritmilor

Algorithms & Complexity Theory

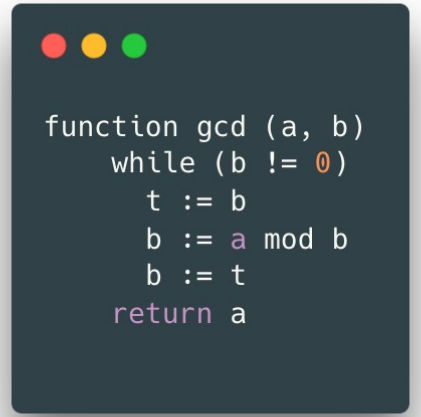
Matei Popovici

Which algorithm is better ?



```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Alg A



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

Alg B

How to we evaluate the performance of an algorithm?

Number of lines of (pseudo)code

How to we evaluate the performance of an algorithm?

We implement the algorithm in some programming language

We measure the execution time on very big inputs

How to we evaluate the performance of an algorithm?

We determine how the number of performed instructions scale with respect to the size of the input.

How to we evaluate the performance of an algorithm?

We determine how the number of performed instructions scale with respect to the size of the worst-case input.

How to we evaluate the performance of an algorithm?

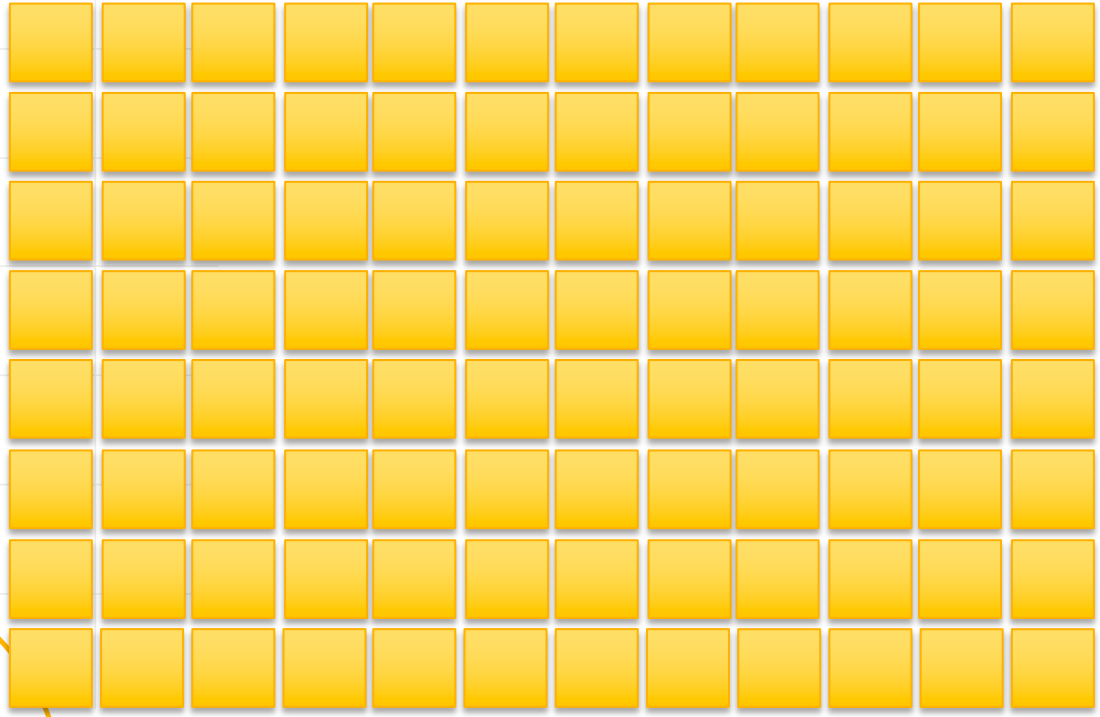
We determine how the consumed memory scales with respect to the size of the worst-case input.

Two variants of Euclid's algorithm

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

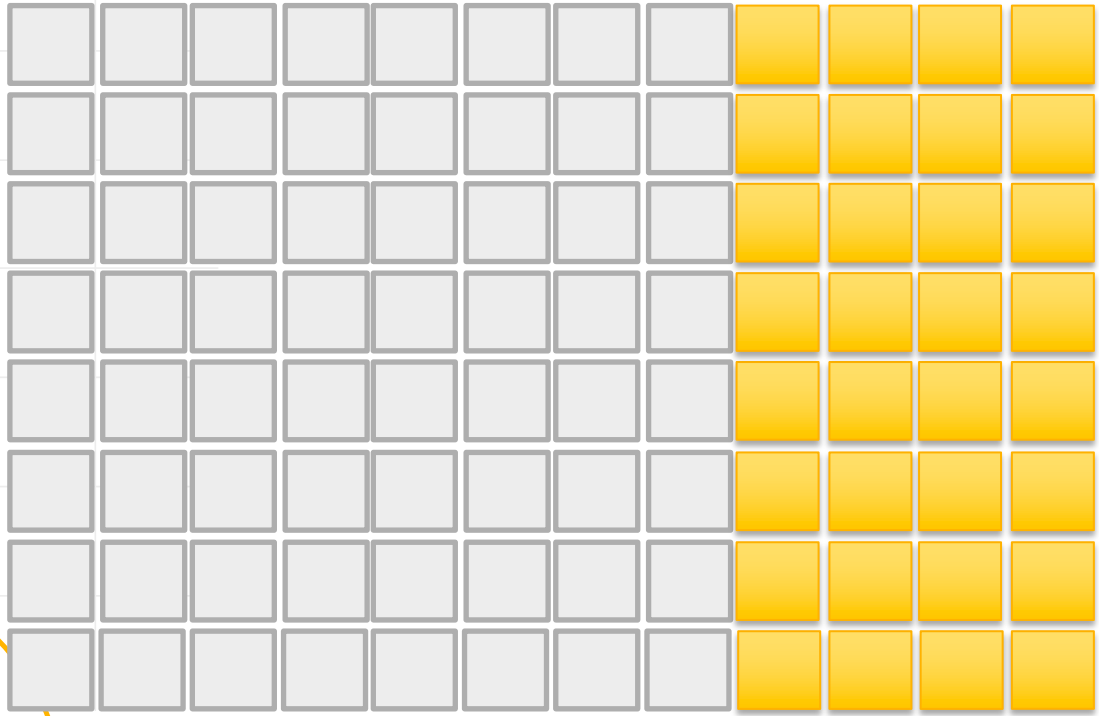
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

finding gcd(8,12)



What is the largest square size S that can be used to pave a 8 x 12 surface?

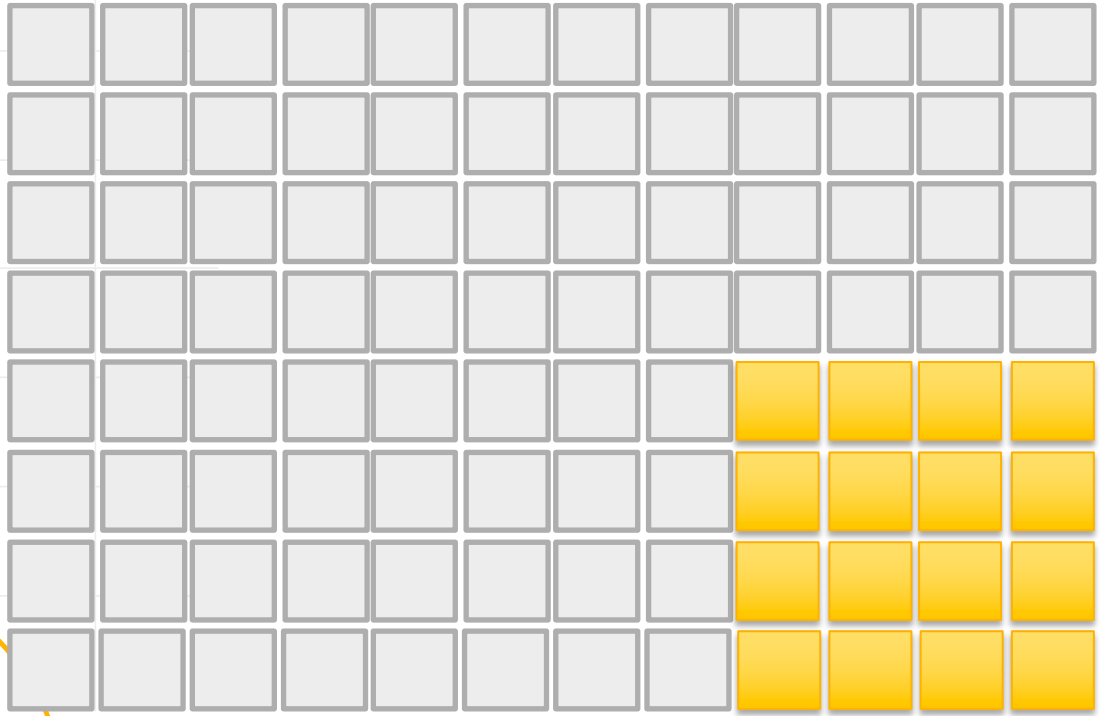
finding gcd(8,12)



If we remove a maximal square from the surface, S doesn't change

gcd(a,b) = gcd(a-b,b)

finding gcd(8,12)

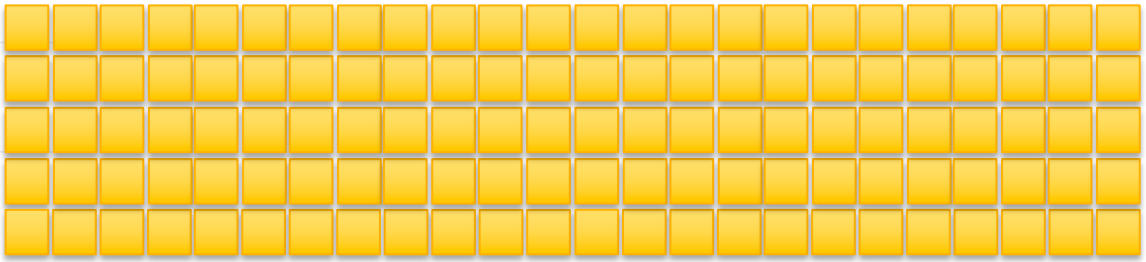


Repeat the process until we end up with a square.

a = b

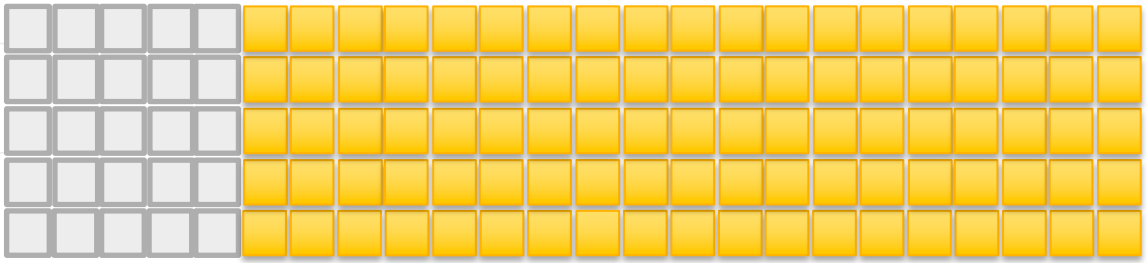
```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

5



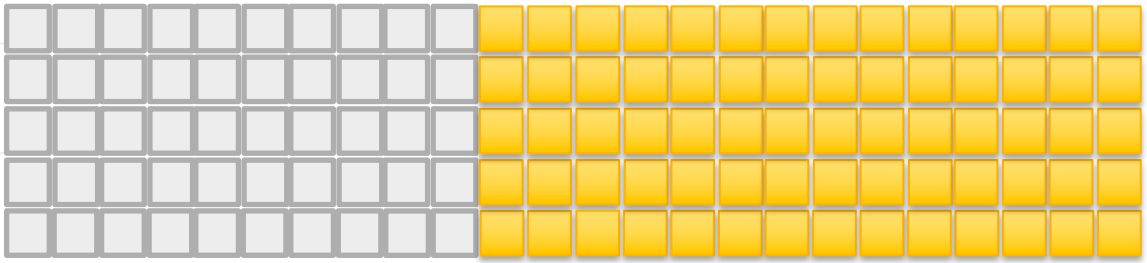
24

5



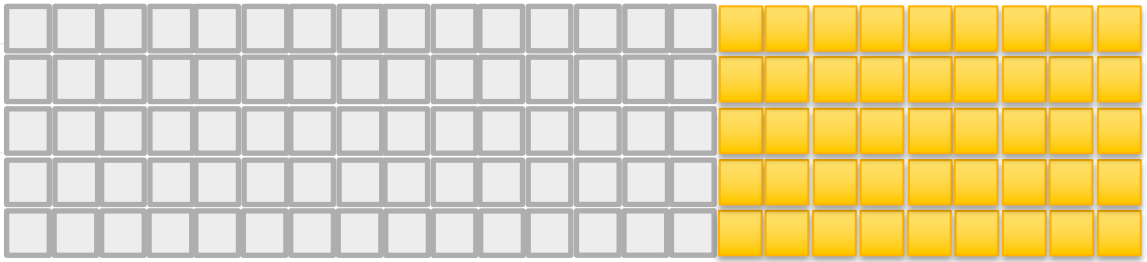
24

5



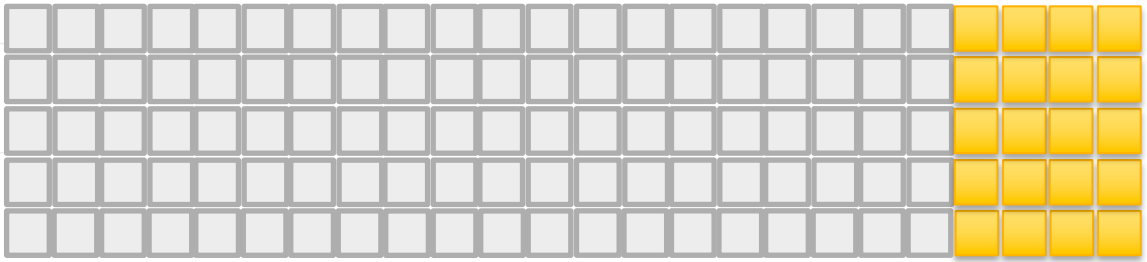
24

5



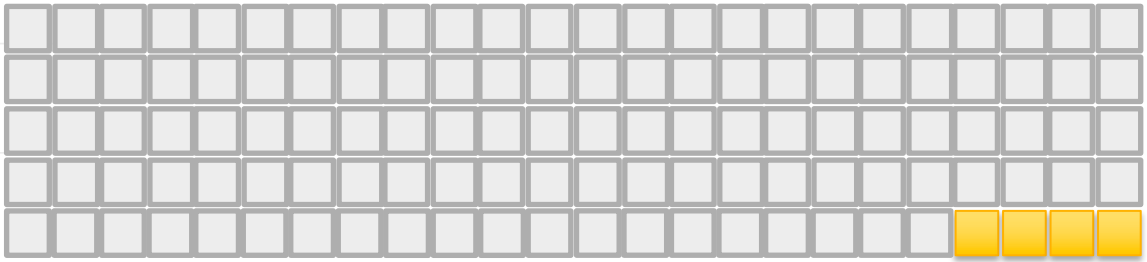
24

5



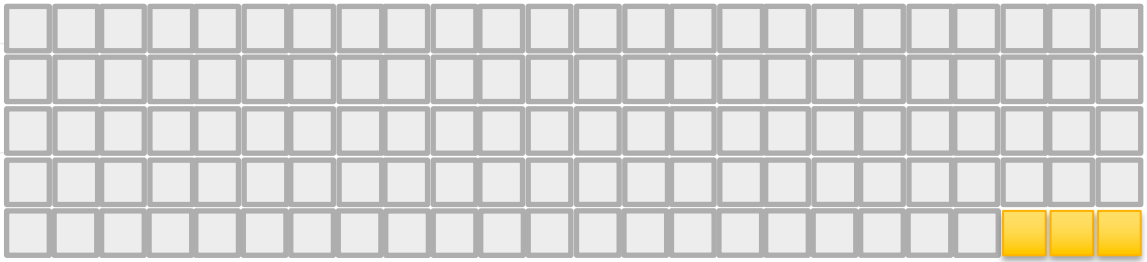
24

5



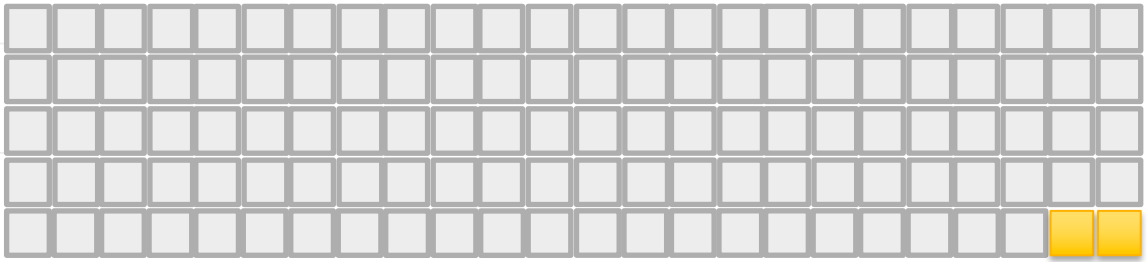
24

5



24

5



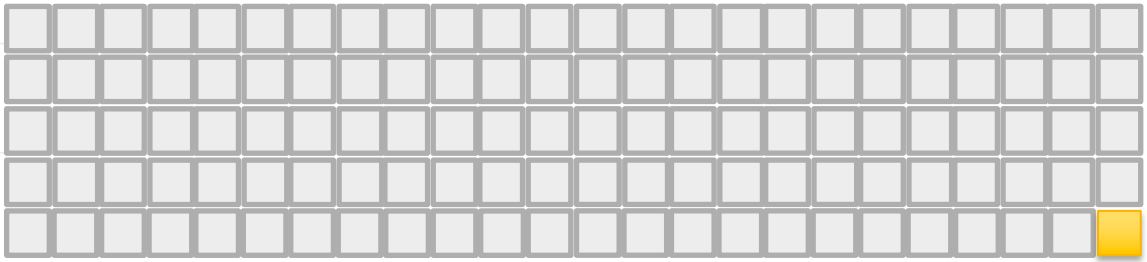
24

We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)



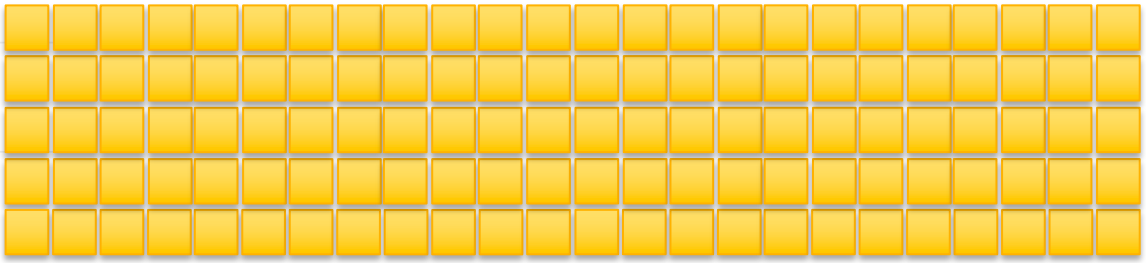
We can substantially improve the number of steps, if we replace:

$gcd(a,b) = gcd(a-b,b)$

by:

$gcd(a,b) = gcd(a \bmod b,b)$

5



24

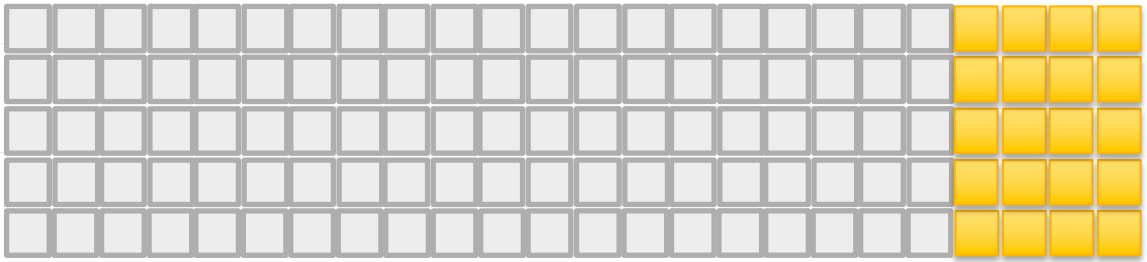
We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)

5



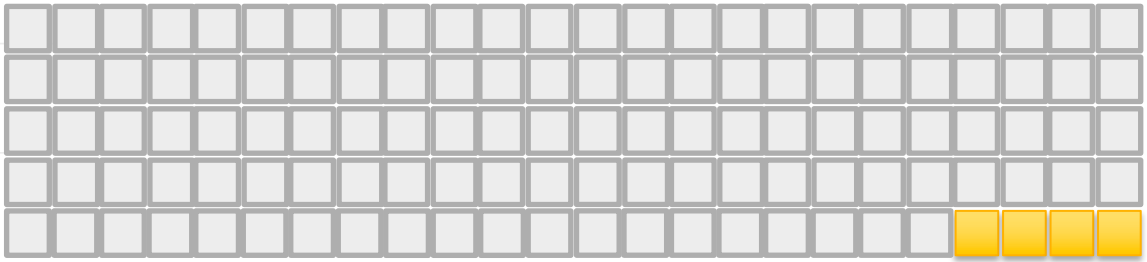
24

We can substantially improve the number of steps, if we replace:

gcd(a,b) = gcd(a-b,b)

by:

gcd(a,b) = gcd(a mod b,b)



5

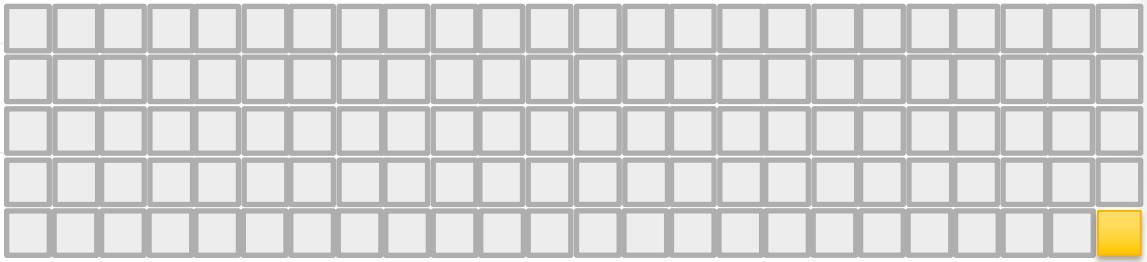
We can substantially improve the number of steps, if we replace:

$$gcd(a,b) = gcd(a-b,b)$$

by:

$$gcd(a,b) = gcd(a \bmod b,b)$$

```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```



5

Measuring the performance of subtraction gcd:

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Worst-case scenario:

b = 1 worst-case, we may have a subtractions

111111111… 11

Maximal value that we can store on n bits?

The size required to store a value n?

We determine how the number of instructions scale with respect to the size of the worst-case input.

Measuring the performance of subtraction gcd:

```
function gcd (a, b)
    while (a != b)
      if a > b
        a := a - b
      else
        b := b - a
    return a
```

Worst-case scenario:

b = 1

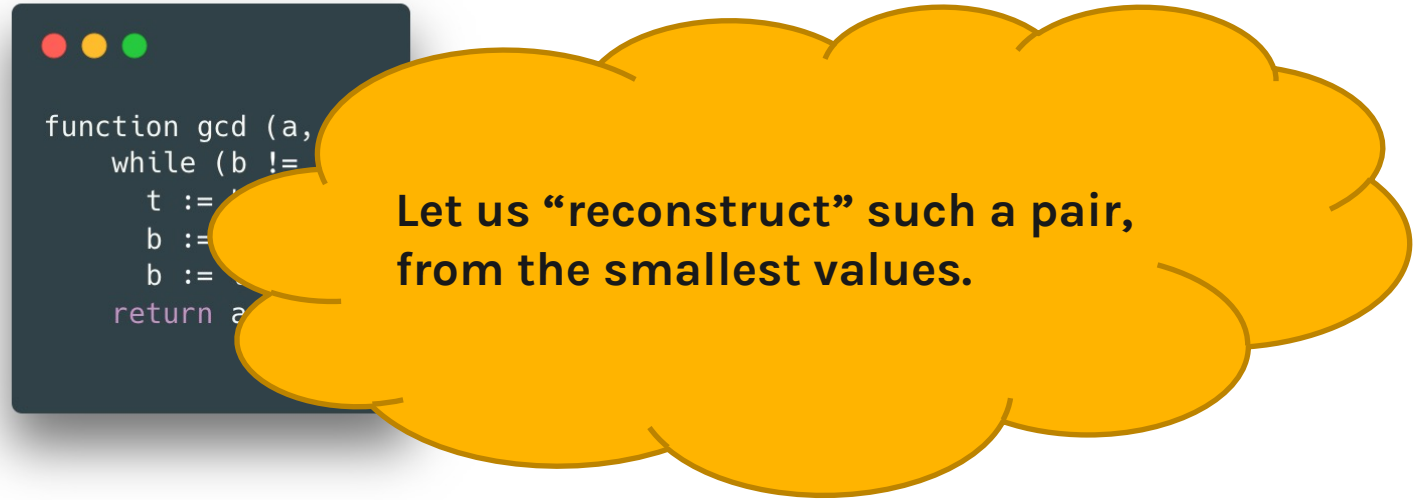
worst-case, we may have 2^n subtractions, where n is the number of bits required to store `max(a,b)`

We determine how the number of instructions scale with respect to the size of the worst-case input.

Measuring the performance of ***modulo*** gcd:

Worst-case scenario:

Pairs of numbers such that
`“a div b” = 1` (no speedup)



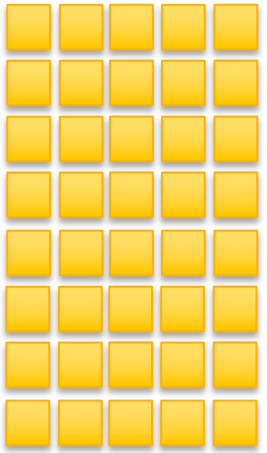
Measuring the performance of ***modulo*** gcd:

Worst-case scenario:

Pairs of numbers such that `"a div b" = 1` (no speedup)

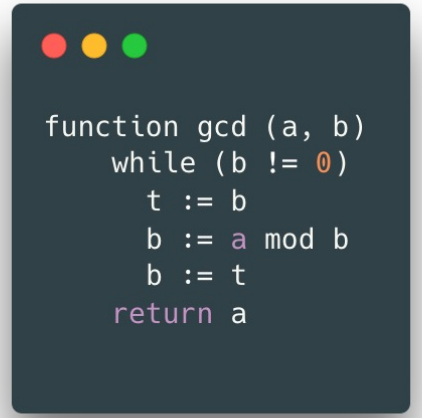
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

These pairs are the Fibonacci numbers!



gcd(8,5)
gcd(5,3)
gcd(3,2)
gcd(2,1)
gcd(1,1)

Measuring the performance of ***modulo*** gcd:



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

Worst-case scenario:

gcd(f_{n+1}, f_n) will iterate n times over the while

$$f_{n+1} \simeq 1,6^{n+1}$$

$$f_{n+1} \leq 2^{n+2}$$

The number of bits required to store 2^{n+1} is n+1.

gcd will perform at most n iterations where max(a,b) is represented on n+1 bits.

subtraction gcd

modulo gcd



We need a language to facilitate expressing algorithm performance.

`Subtracti` at most 2^n subtractions input of size **n**.

`gcd` will perform at most **n** iterations for an input of size **n+1**.

`subtraction gcd` runs in exponential time

`modulo gcd` runs in linear time

An execution time for an algorithm **A** is a function $T : \mathbb{N} \to \mathbb{N}$ such that:

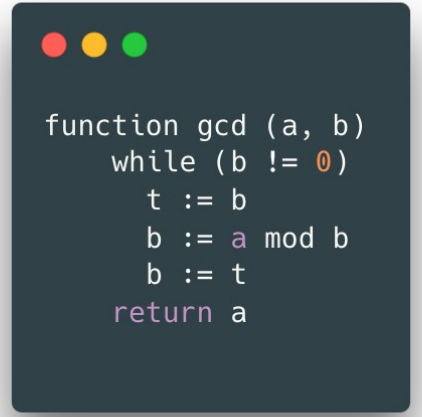
For any input of size n, $T(n) \geq$ the number of instructions performed by A on that very input

We expect T to be increasing (larger inputs, larger execution time)

We generally don't care about $T(n)$ for smaller n.

We are interested in how T increases as n increases, rather than the actual definition of T

We are interested in how `T` increases as `n` increases, rather than the actual definition of `T`

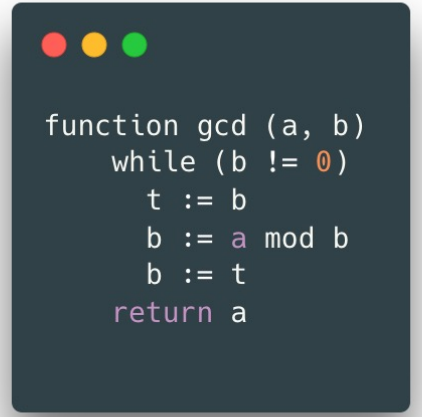


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

$$T(n) = 4*(n-1) \text{ if } n > 1$$

$$T(n) = 1 \text{ otherwise}$$

We are interested in how `T` increases as `n` increases, rather than the actual definition of `T`



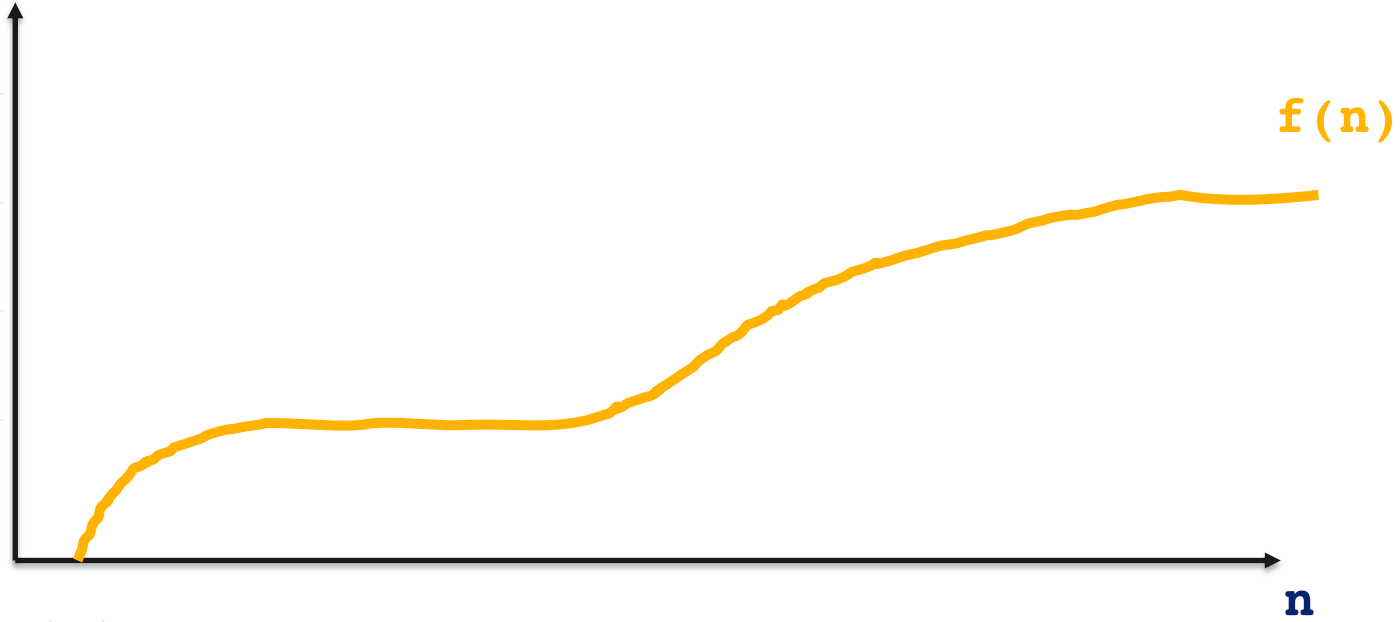
```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

$$T(n) = 4*(n-1) \text{ if } n > 1$$

`T` grows linearly with respect to the size of the input

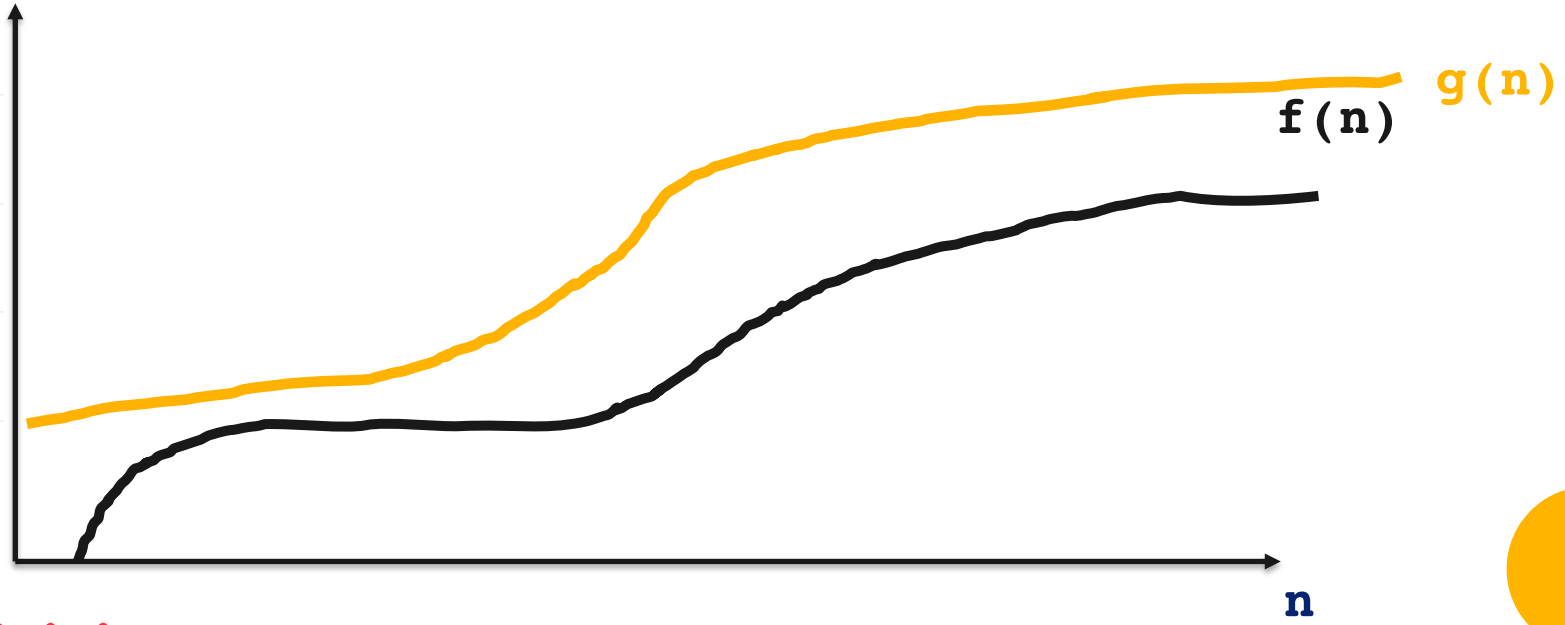
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \exists c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



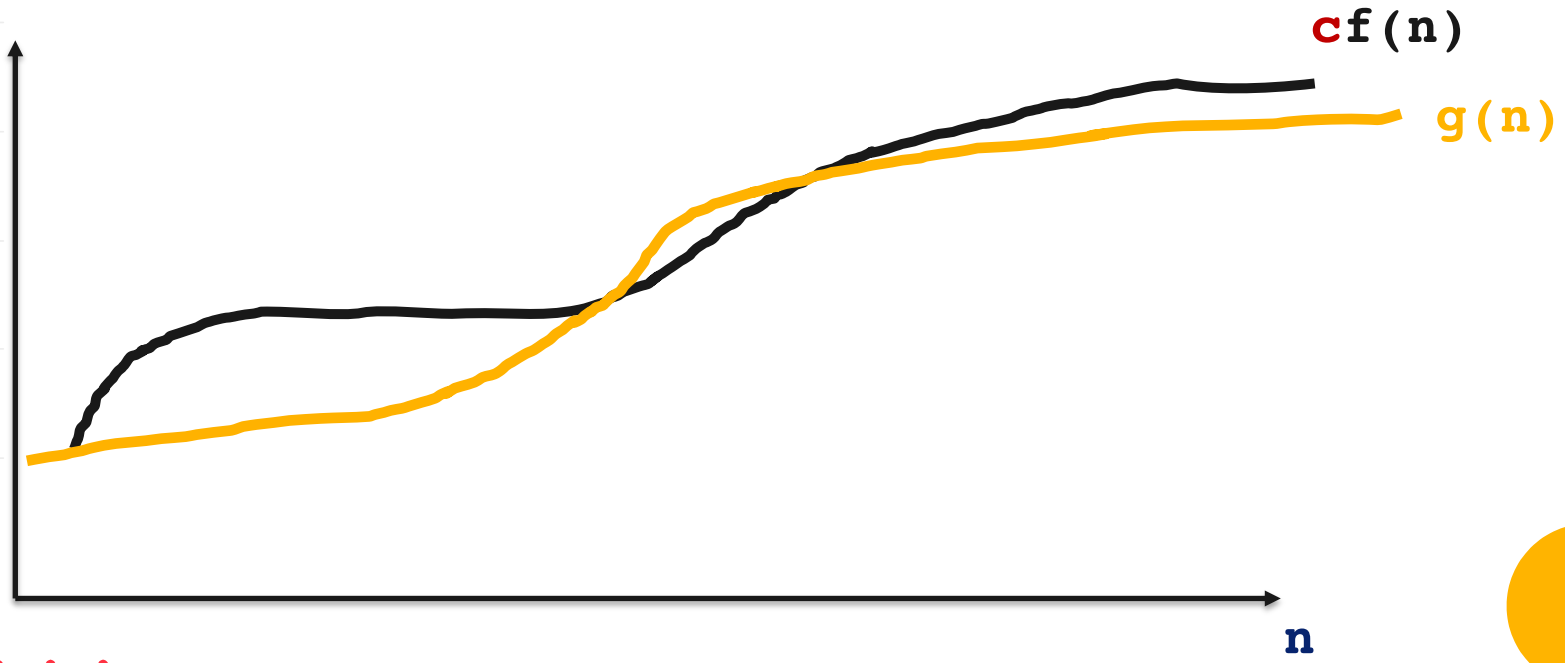
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



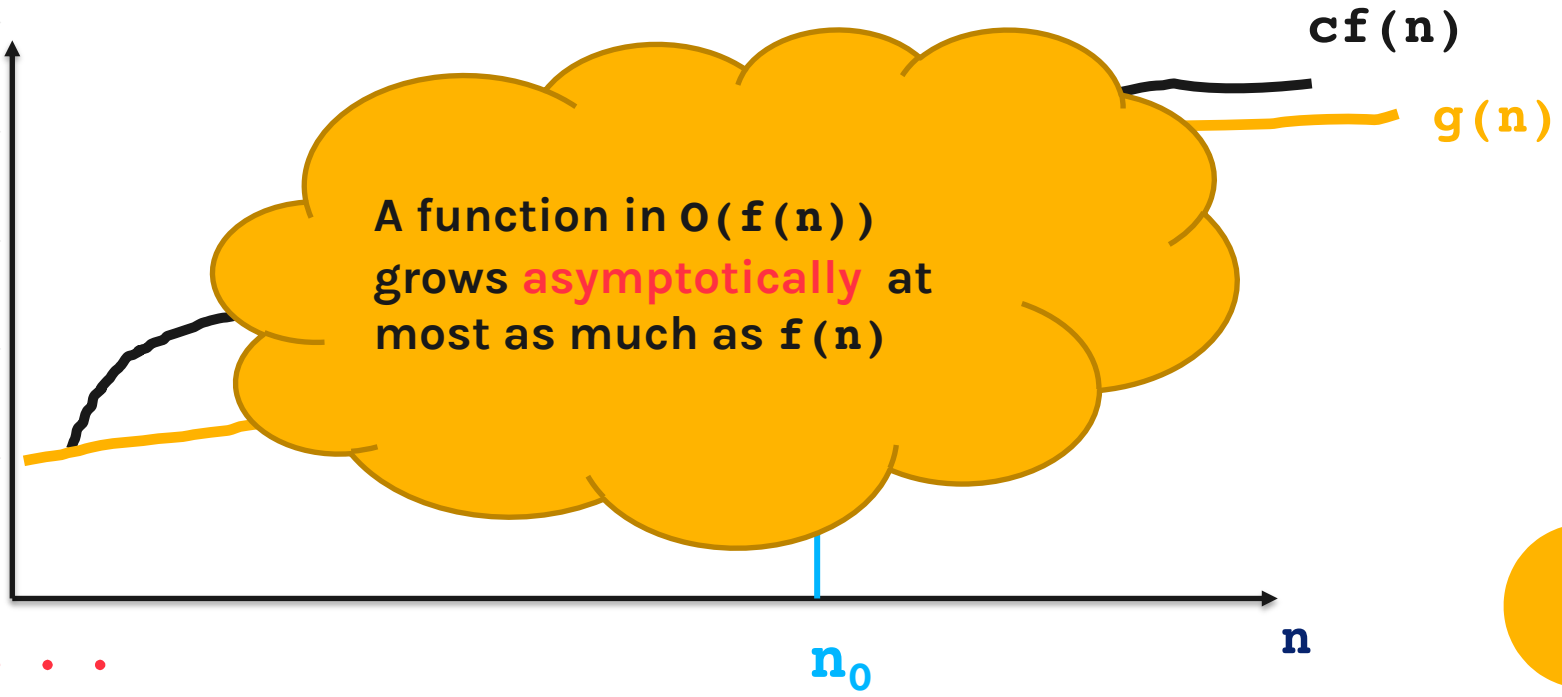
Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



Big O notation

$$O(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq g(n) \leq cf(n) \ \}$$



Big O notation

$O(\log(n))$

A class (or set) of functions

All members of $O(\log(n))$ grow asymptotically at most as $\log(n)$

We consider all members of $O(\log(n))$ to be equivalent w.r.t. how they grow

Saying that A has execution time $O(\log(n))$ means that A runs in logarithmic time, but we don't really care about its exact execution time.

Sometimes we care about the constant c. We may say that it is small, or give its exact value.

$T_{gcd_modulo}(n) \in O(n)$

$T_{gcd_modulo}(n) \in O(2n)$

Formally correct – doesn't tell us much more

$\log_2(n) \in O(n)$

true

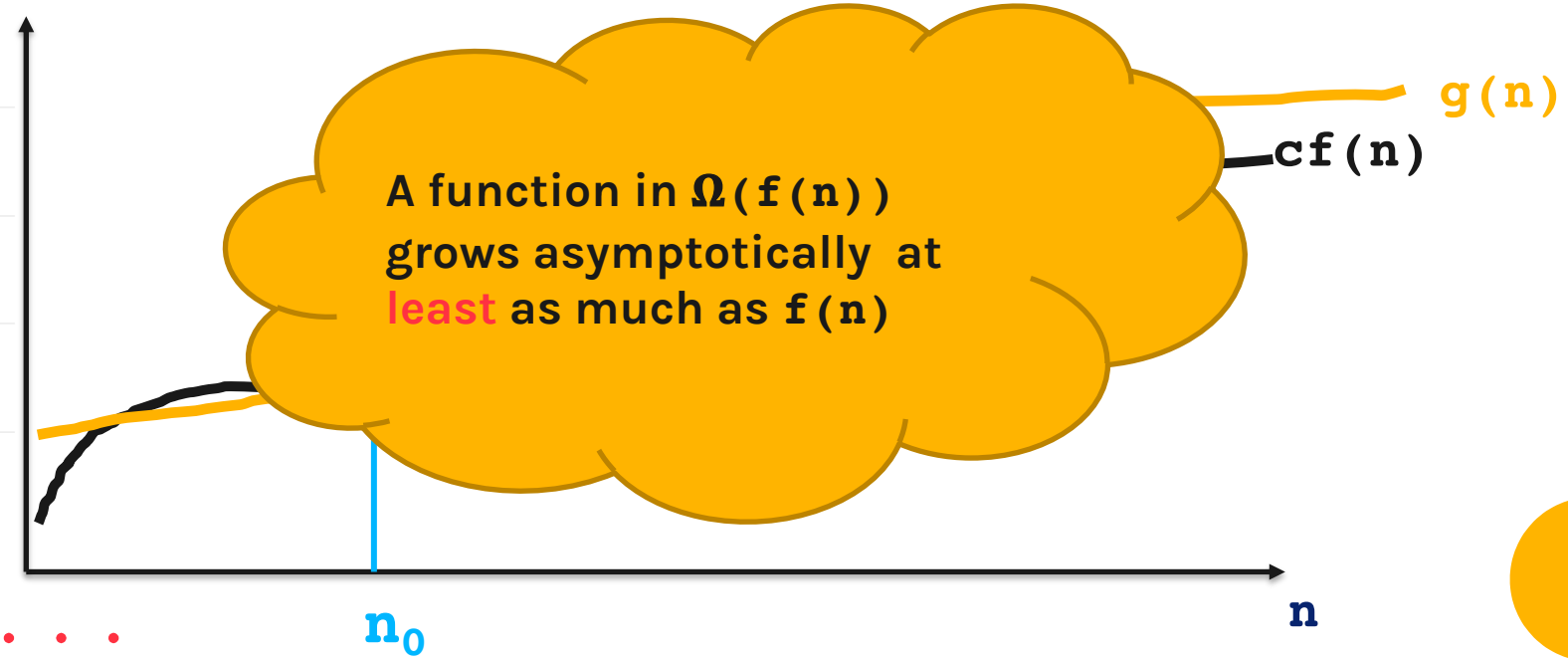
if $\lim_{n \to \infty} g(n)/f(n) = 0$ then

$g(n) \in O(f(n))$?

true

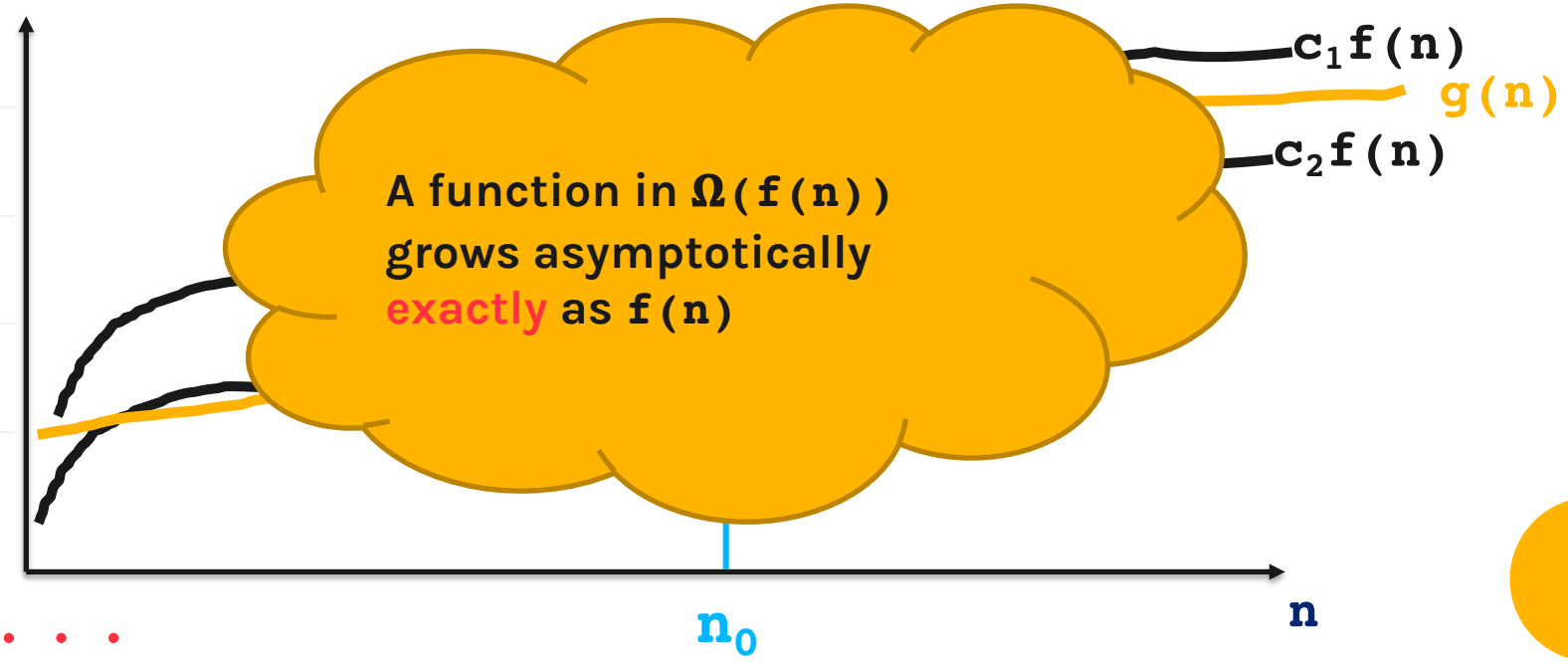
Omega (Ω) notation

$$\Omega(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \exists c > 0, \ \exists n_0 \in \mathbb{N}, \ \forall n \geq n_0, \ 0 \leq cf(n) \leq g(n) \ \}$$



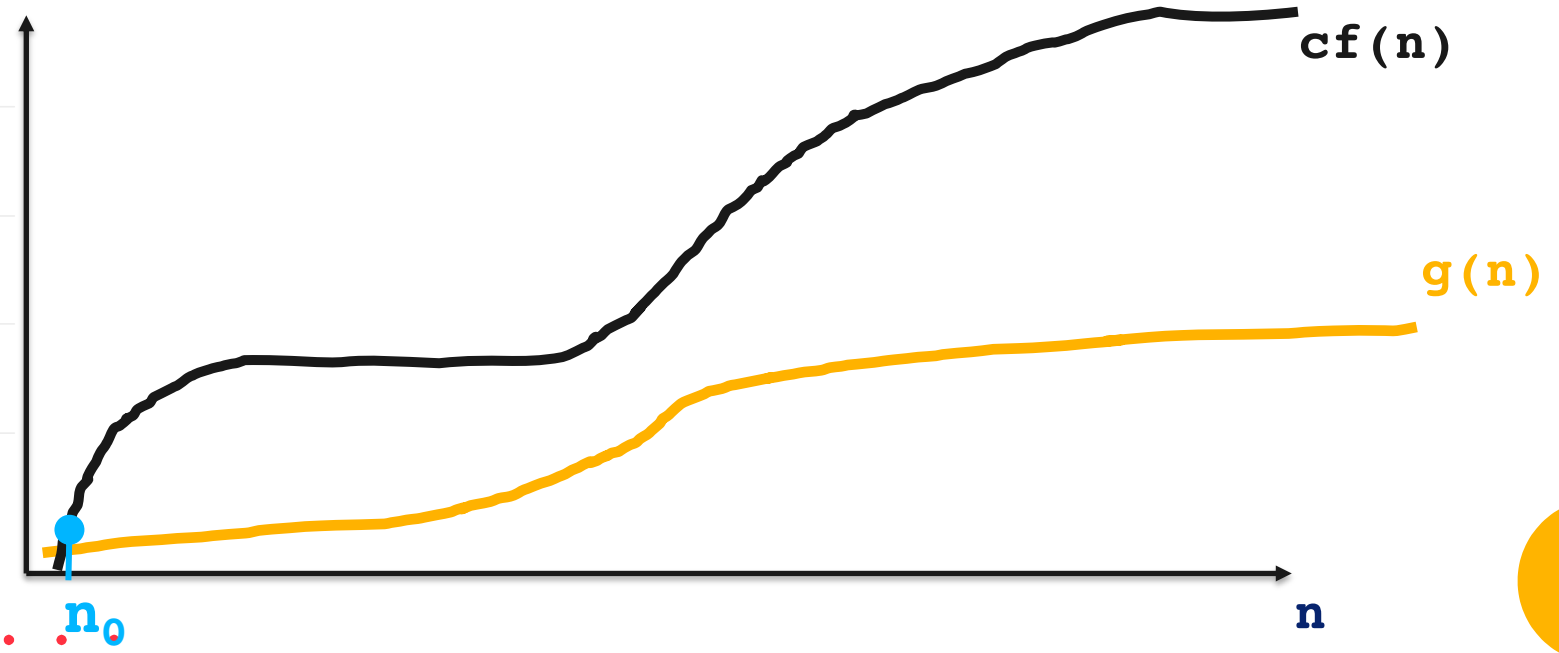
Theta (Θ) notation

$$\Theta(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+,\mathbb{R}^+) \;.\; \exists c_1,c_2 > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)\}$$



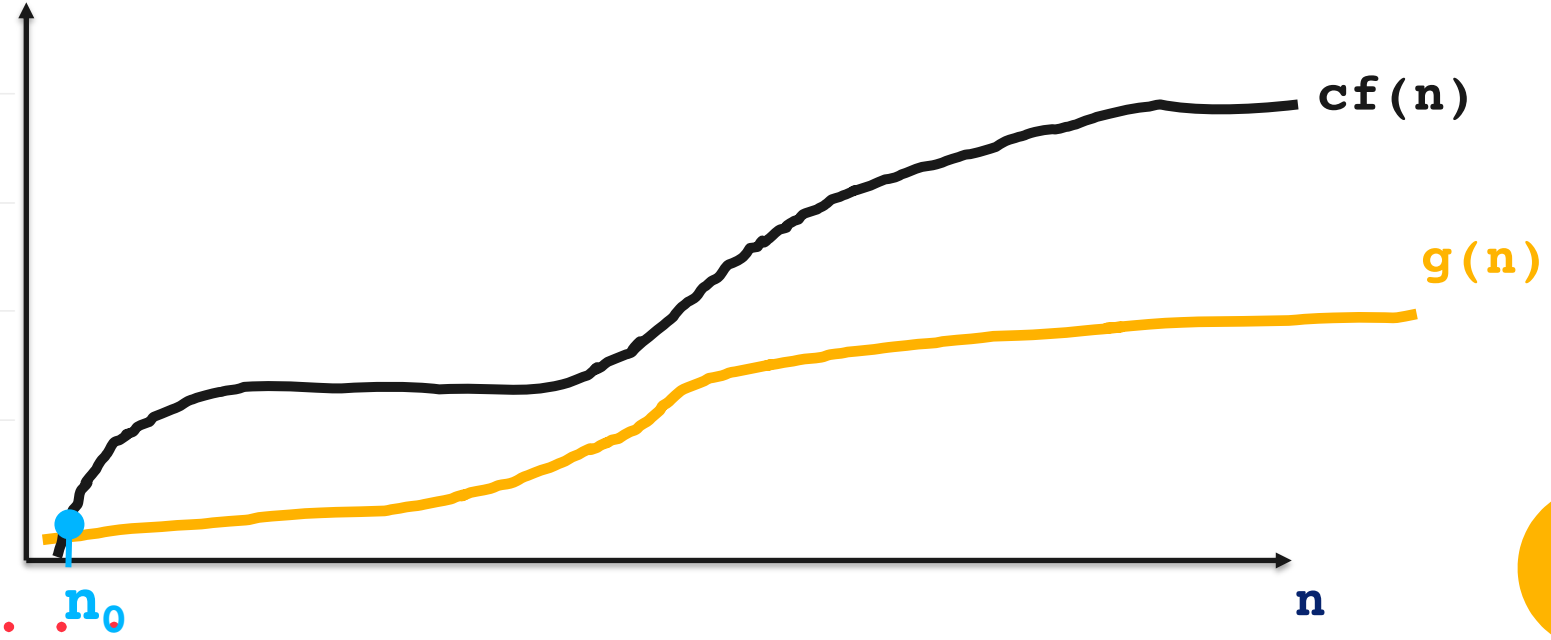
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



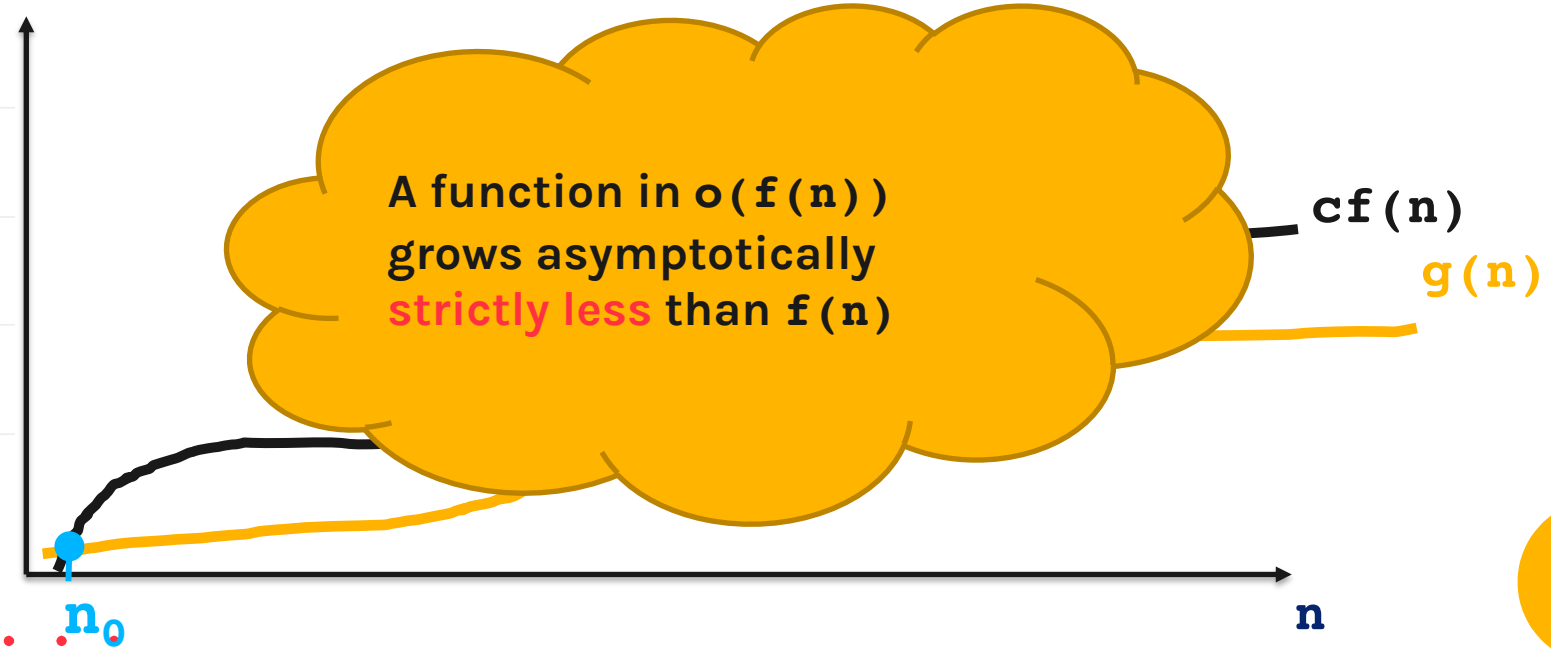
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



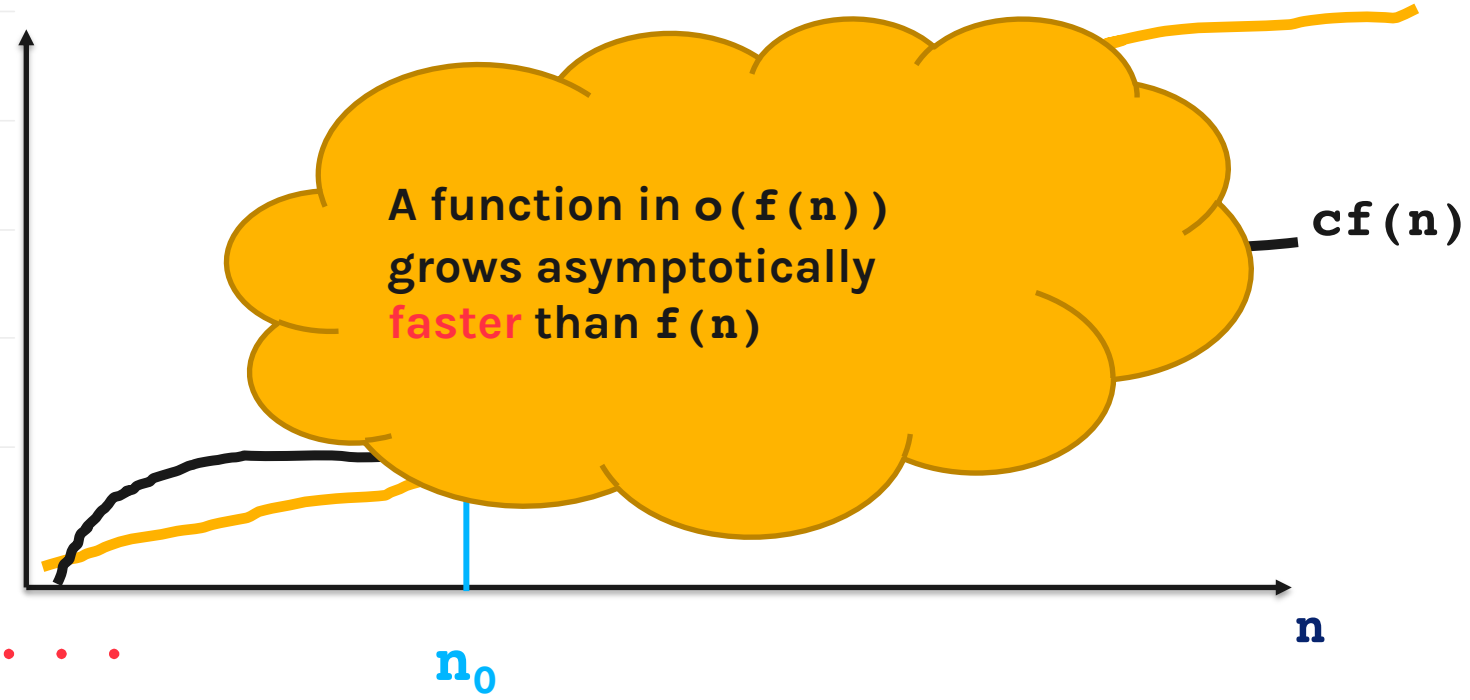
Small o notation

$$o(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \;.\; \forall c > 0,\ \exists n_0 \in \mathbb{N},\ \forall n \geq n_0,\ 0 \leq g(n) \leq cf(n)\ \}$$



Small ω notation

$$\omega(f(n)) = \{g \in \mathbb{H}om(\mathbb{R}^+, \mathbb{R}^+) \ . \ \forall c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, 0 \leq cf(n) \leq g(n)\}$$



Practice

$\log(n) \in \Omega(n)$?

$\text{sqrt}(n) \in \Omega(n)$?

if $f(n) \in O(n)$ then
$f(n) \in \Omega(n)$?

if $f(n) \in O(n)$ and $f(n) \in \Omega(n)$
then $f(n) \in \Theta(n)$?

Give an example of a function in $o(n)$

Give an example of a function in $o(n^2)$ and also in $\omega(n)$

Practice

Give an example of a function in $O(n)$ but not in $o(n)$ and neither in $\Theta(n)$.

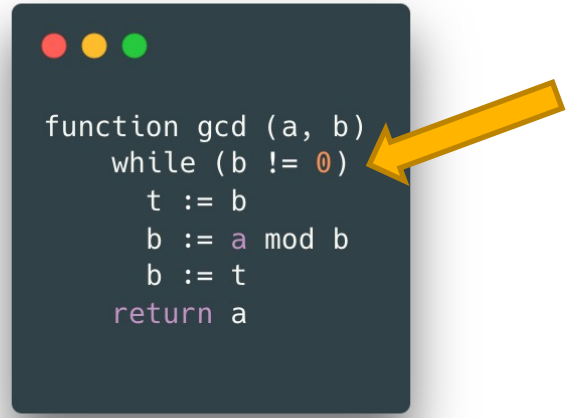
Practice

Give an example of a function in $\Omega(n)$ but not in $o(n)$ and neither in $\Theta(n)$.

Syntactic sugars

A formal language for
combining execution times

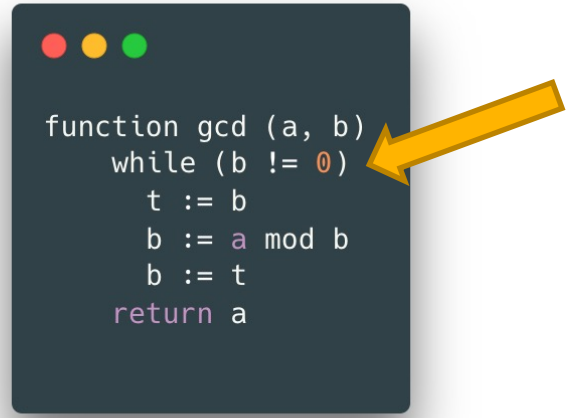
Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`n` or `n+1` iterations?

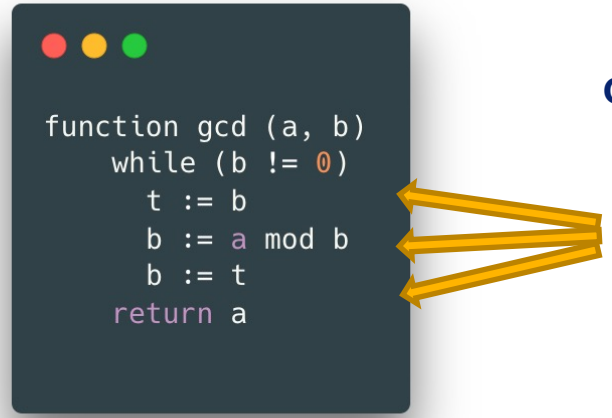
Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

Syntactic sugars

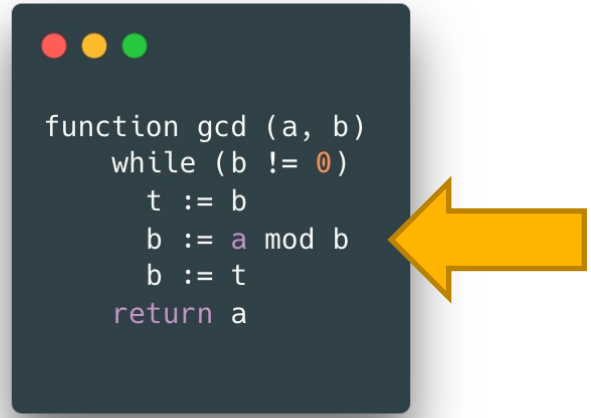


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

three instructions?

Syntactic sugars

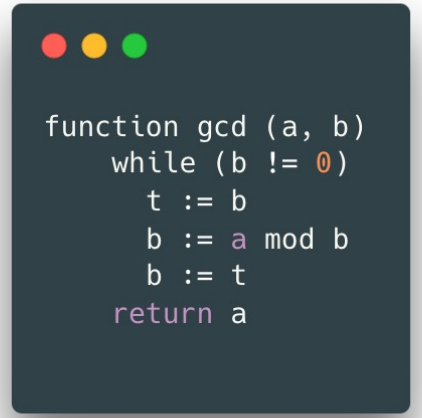


```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

`O(1)` (or constant) time

Syntactic sugars



```
function gcd (a, b)
    while (b != 0)
      t := b
      b := a mod b
      b := t
    return a
```

`O(n)` (or order n) iterations

`O(1)` (or constant) time

$$T(n) = O(n) * O(1) = O(n)$$

Syntactic sugars

$T(n) = O(n) * O(1) = O(n)$



Syntactic sugars

$$T(n) = O(n) * O(1) = O(n)$$

Syntactic sugars

$$T(n) = f(n) * O(1) = O(n)$$

$$f(n) \in O(n)$$

Syntactic sugars

$$T(n) = f(n) * g(n) = O(n)$$

$$f(n) \in O(n)$$

$$g(n) \in O(1)$$

Syntactic sugars

$T(n) = f(n) * g(n) = h(n)$

if $f(n) \in O(n)$ and $g(n) \in O(1)$

then $h(n) \in O(n)$

Practice with syntactic sugars

$O(n) * \Theta(\log n) = ?$

$\Theta(\log(n)) + \Theta(n) = ?$

$\Theta(n) / O(n) = ?$

$n*O(\log(n)) / \Theta(n) = ?$

$\Theta(n) / \omega(\sqrt{n}) = ?$

Lessons learned

Algorithm performance = how the consumed space asymptotically increases with respect to the size of the input, in the worst case.

Asymptotic notations & syntactic sugars = a rigorous language for talking about algorithm execution times.

Thank you!



Analiza Algoritmilor

Algorithms & Complexity Theory

Matei Popovici

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

Many algorithms rely on recursion

Recursion requires a special treatment for determining complexity

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

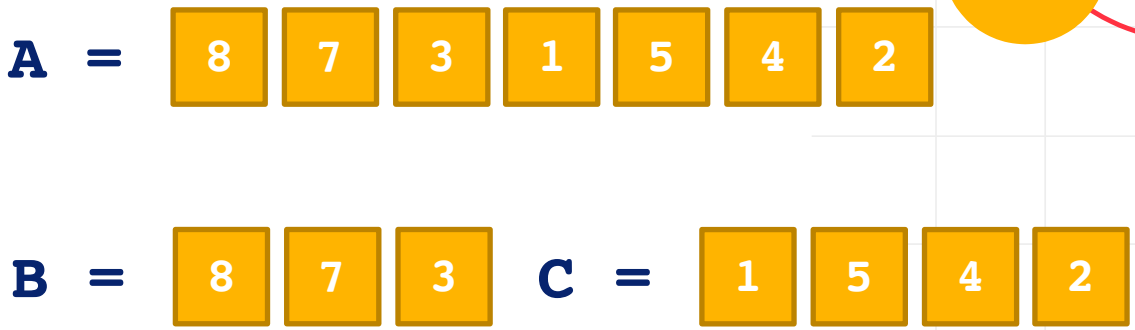


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

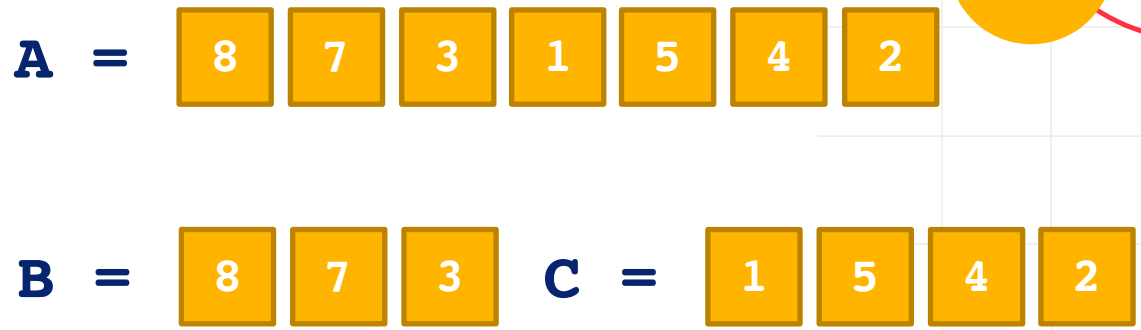


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```



Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

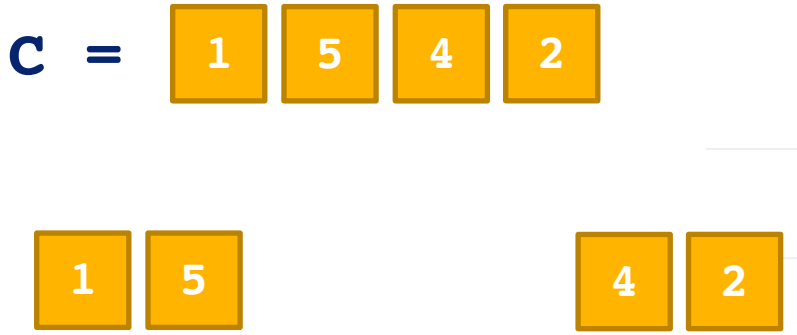
C = | 1 | 5 | 4 | 2 |

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

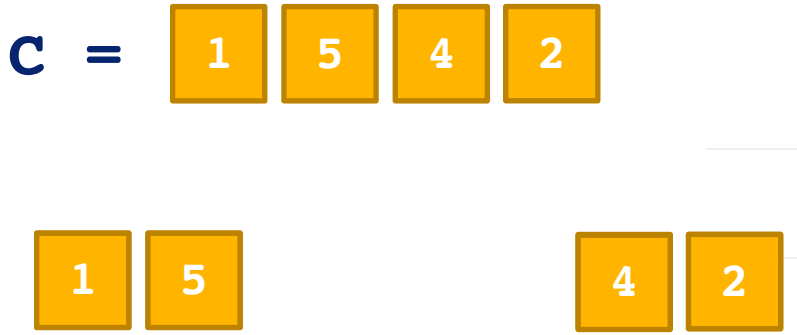


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

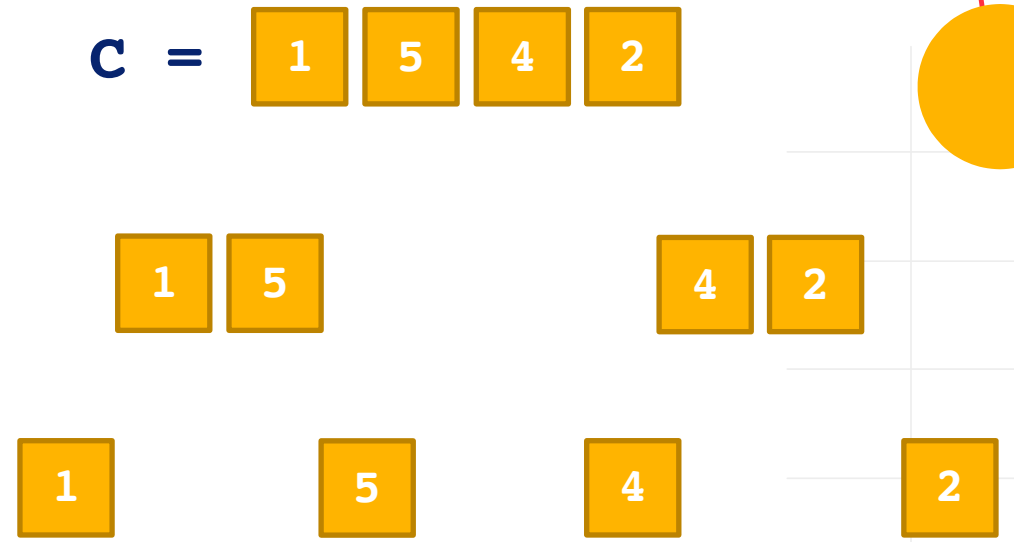


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

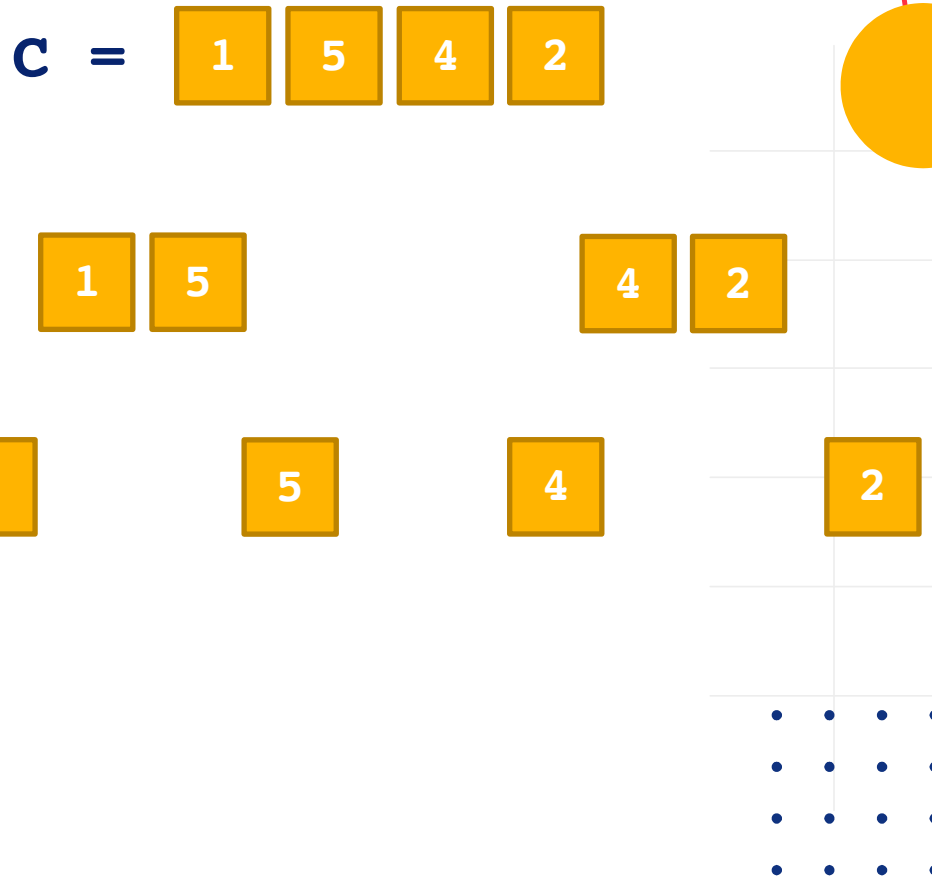


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

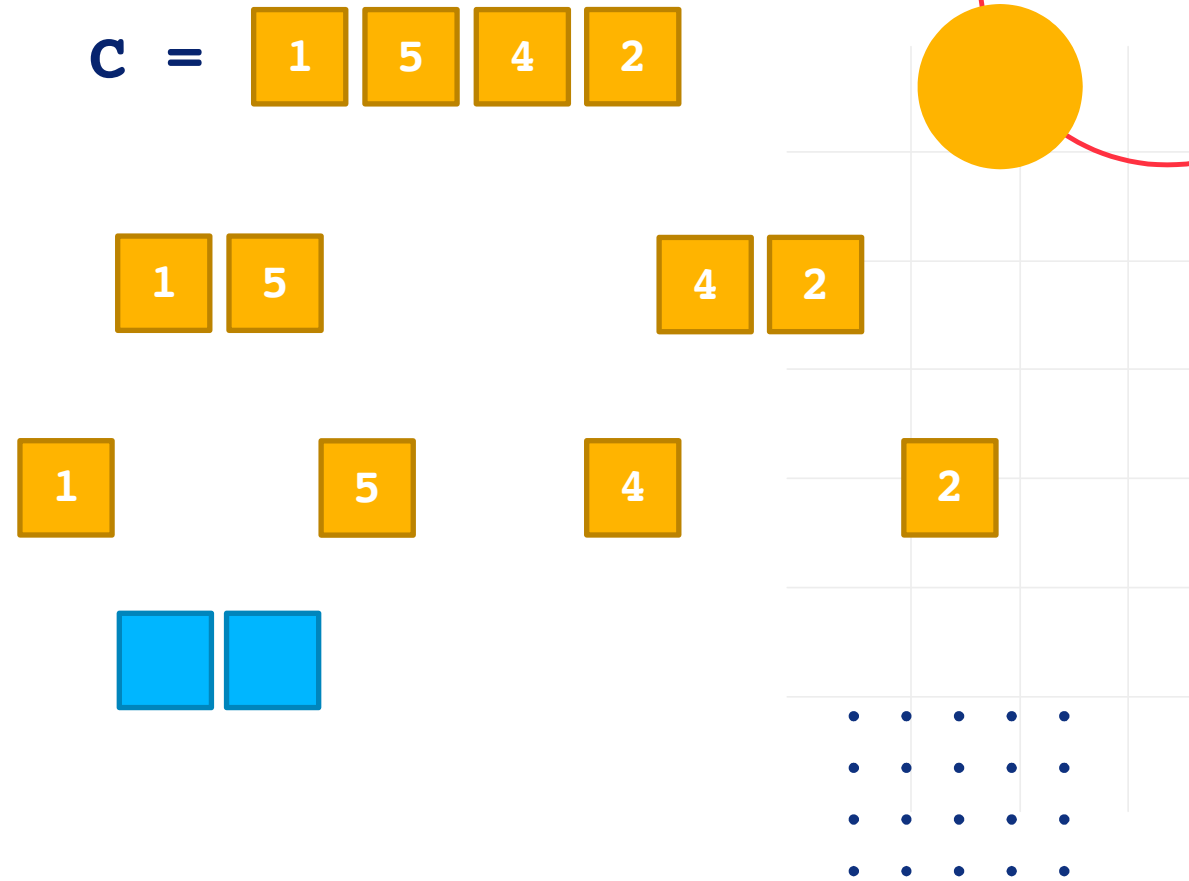
      return merge(B,C)
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

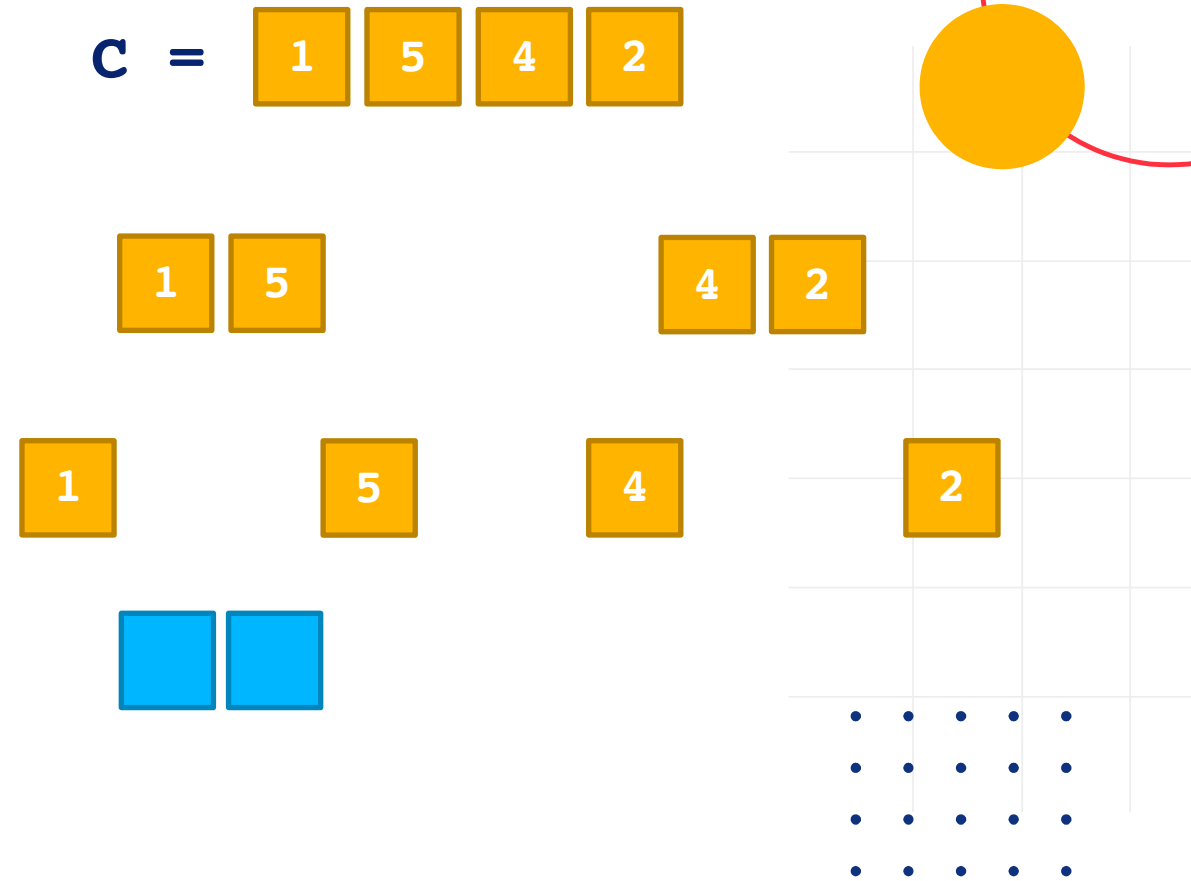
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

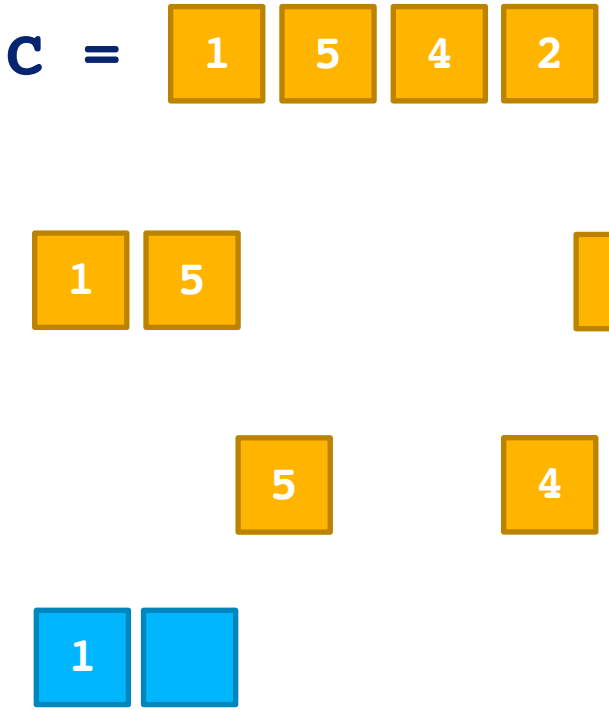
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```









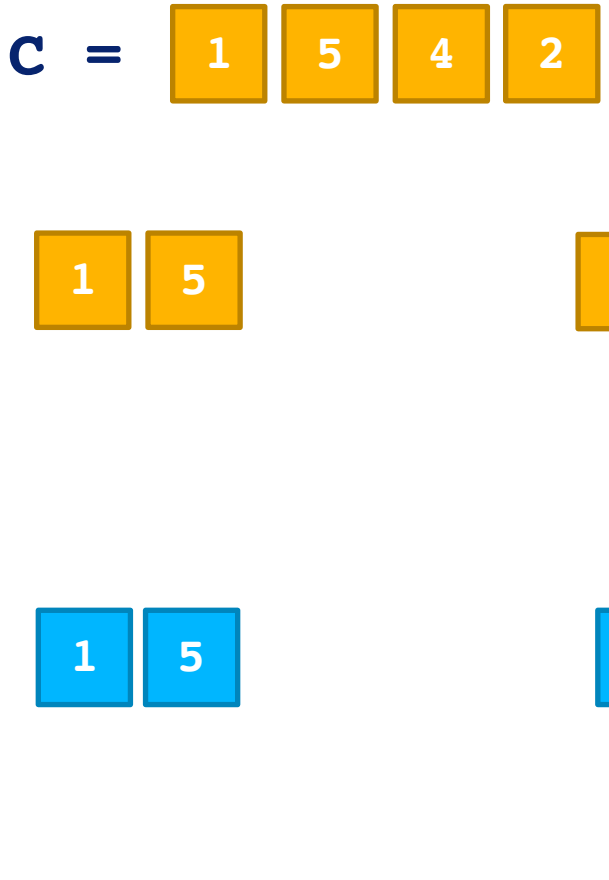




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

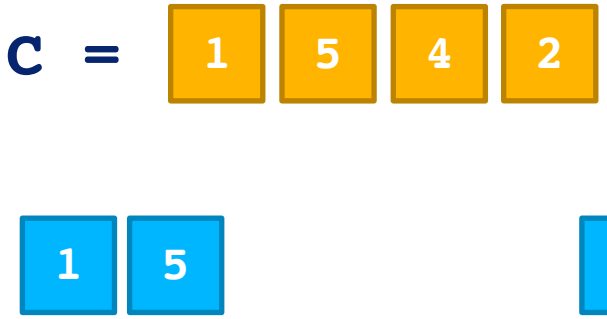
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

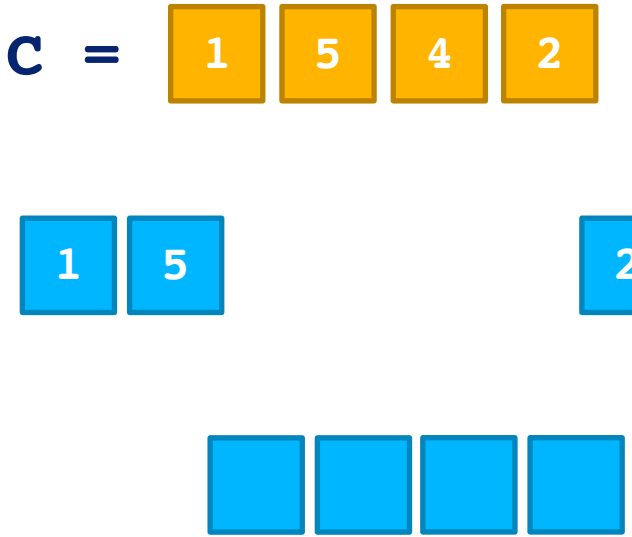
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

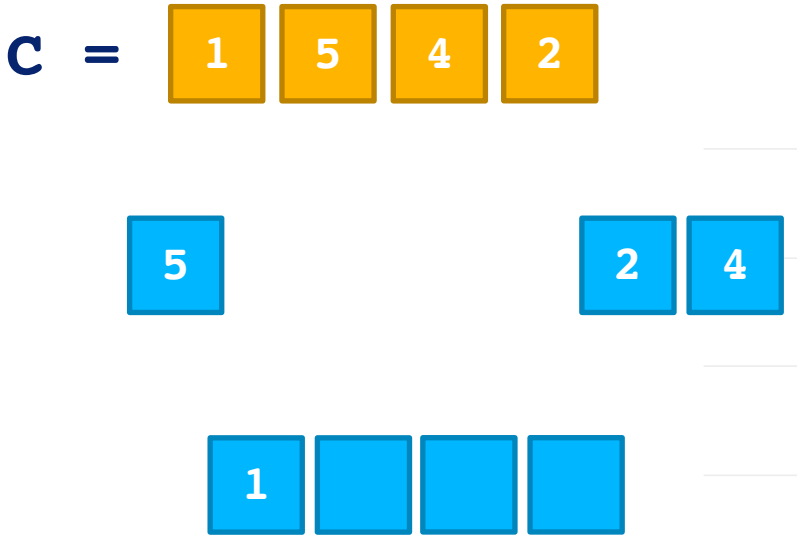
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

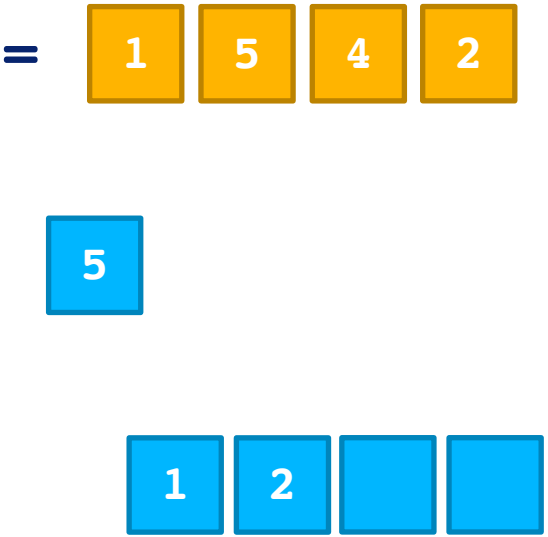
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

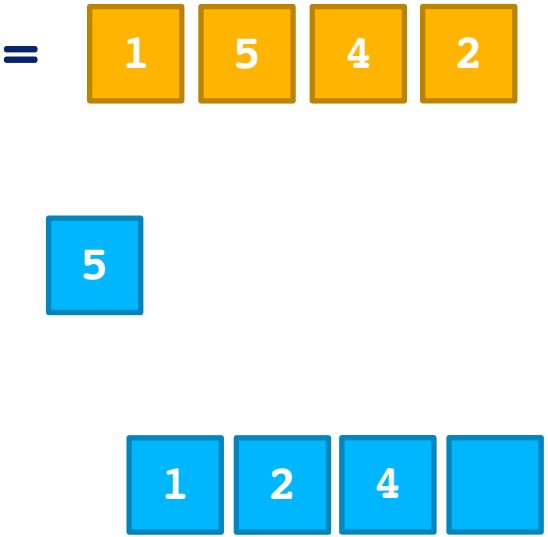
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

C =

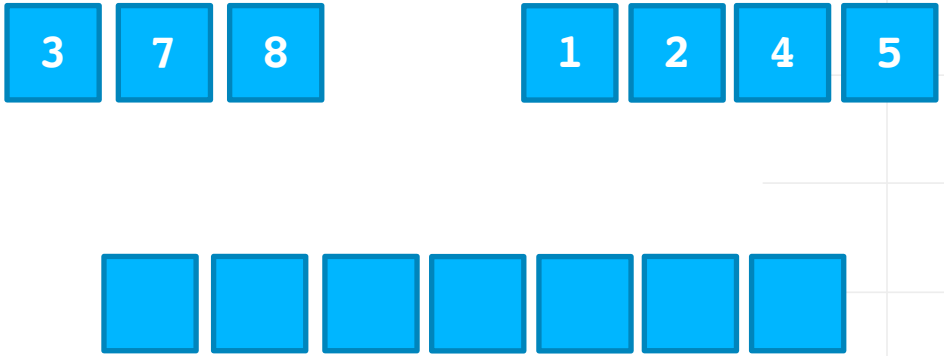




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

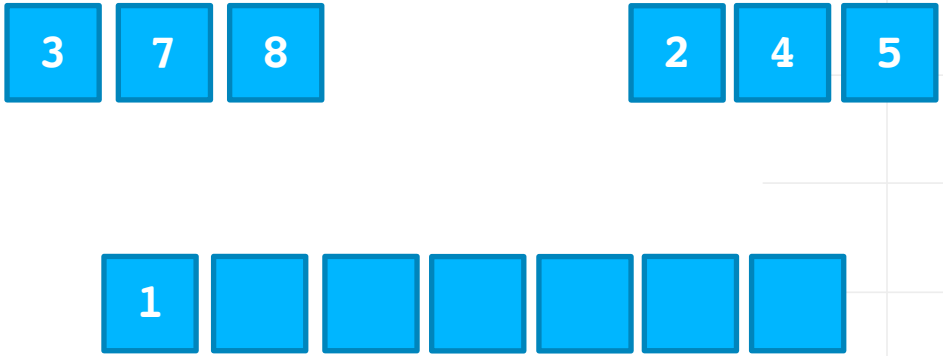
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

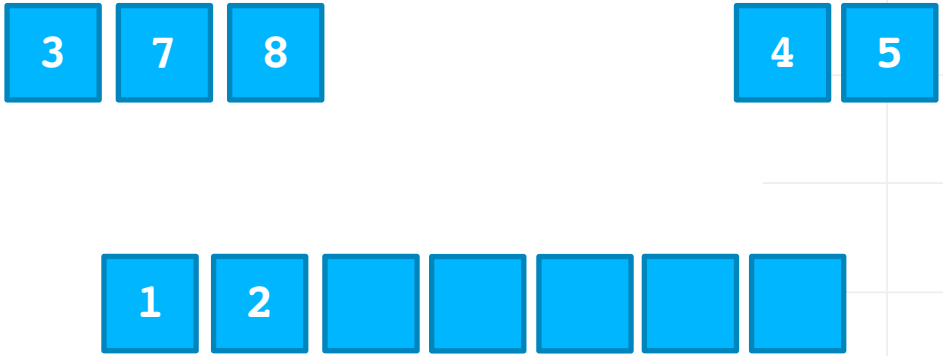
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

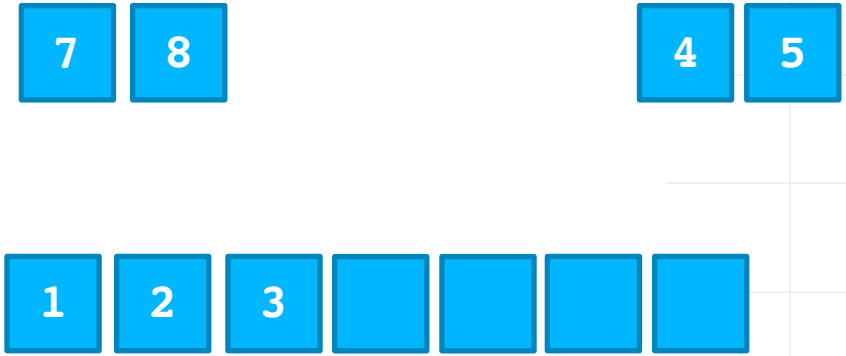
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

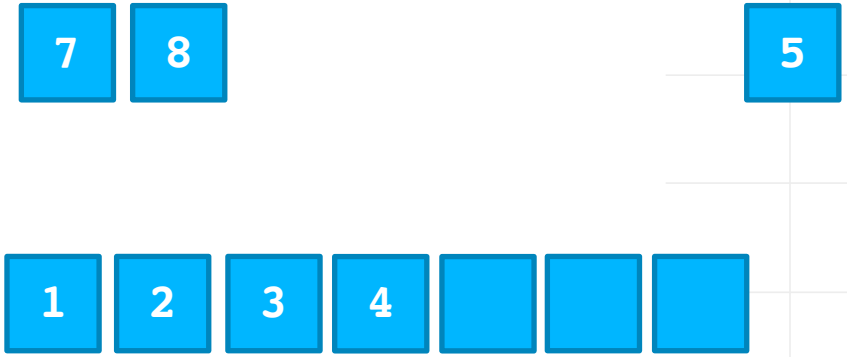
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

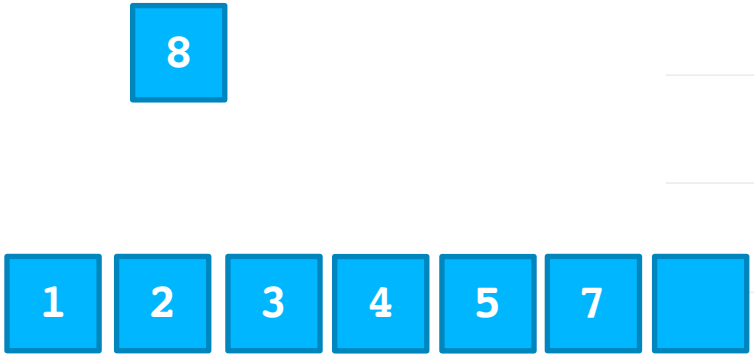




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

The complexity of `merge`:

$\Theta(n+m)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

$\Theta(n)$

The complexity of `mergesort`:

We always assume
constant time for small n

$T_{mergesort}(n) =$

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

$\Theta(n)$

The complexity of `mergesort`:

$$T_{mergesort}(n) = T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil) + \Theta(n)$$



n/2 is a very good estimation for both

The complexity of `mergesort`:

$$T_{\texttt{mergesort}}(n) = 2T_{\texttt{mergesort}}(n/2) + \Theta(n)$$

The complexity of `mergesort`:

$$T_{\texttt{mergesort}}(n) = 2T_{\texttt{mergesort}}(n/2) + n$$

Let us drop the theta for a short while

Method 1 of solving the recurrences:

Recurrence trees

$$T_{mergesort}(n) = 2T_{mergesort}(n/2) + n$$

Practice:

Recurrence trees

$$T_{\text{mergesort}}(n) = 2T_{\text{mergesort}}(n/2) + n$$

Method 1 of solving the recurrences:

Substitution method

$$T_{mergesort}(n) = 2T_{mergesort}(n/2) + n$$

Method 1 of solving the recurrences:

Master method



Analiza Algoritmilor

Algorithms & Complexity Theory

Matei Popovici

Mergesort

Split the array in half
Make copies of each half

Recursively sort the array

Quicksort

Choose some value x from the array
Reorder the array into values ≤x and ≥x.

Recursively sort in place.

Quicksort partitioning




```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```

Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



A

pivot = 5

Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



A

i

j

pivot = 5

Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



A

i

j

pivot = 5

Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



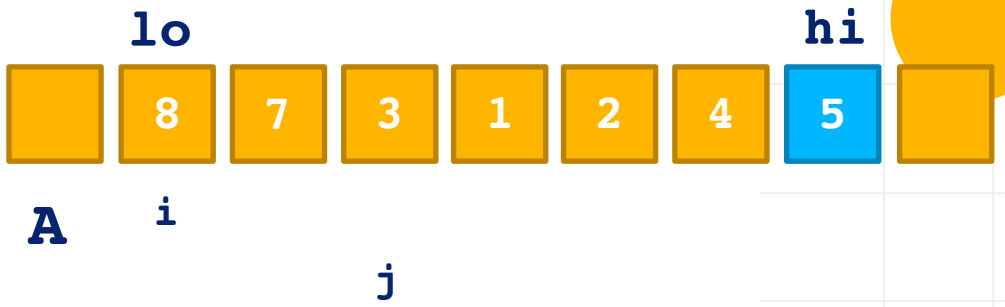
A i

j

pivot = 5

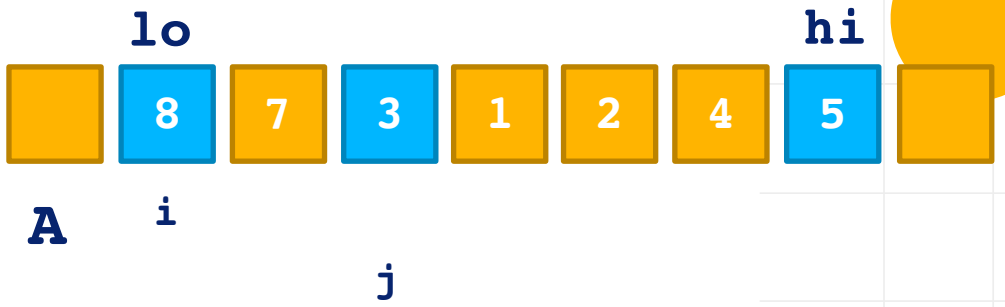
Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



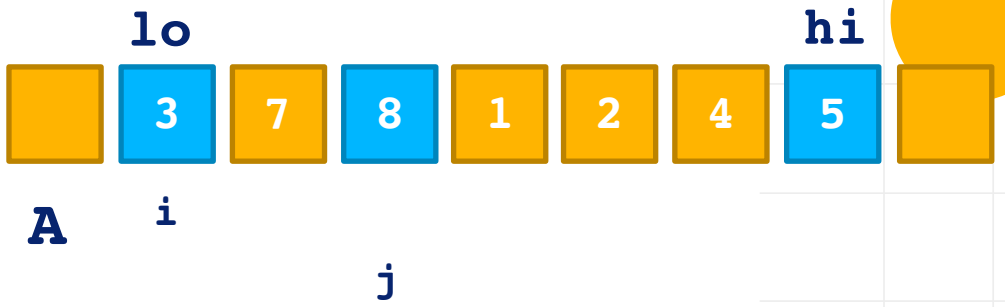
Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



Quicksort partitioning

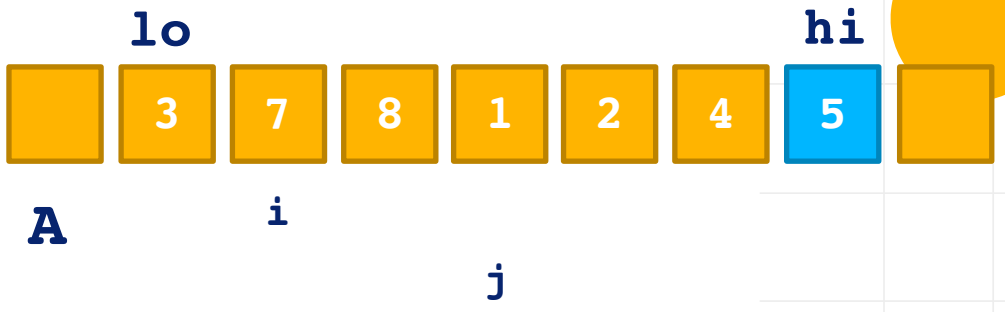
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

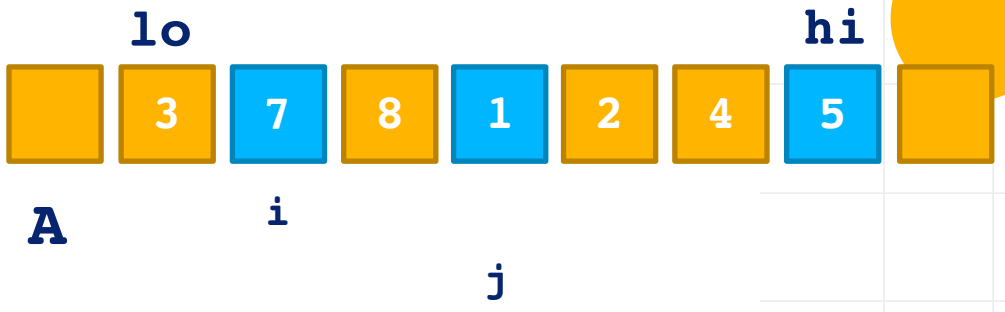
Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



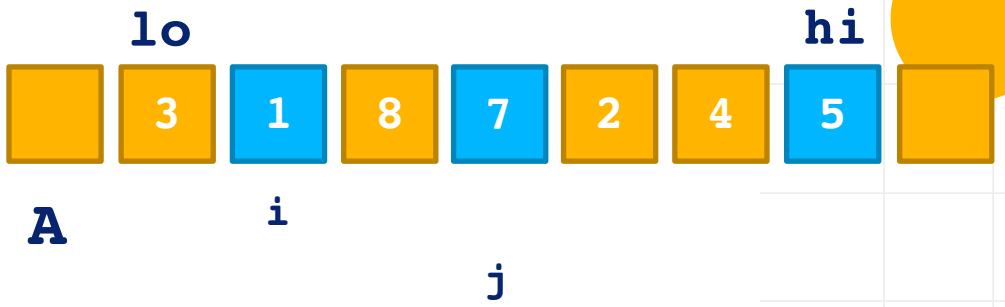
Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



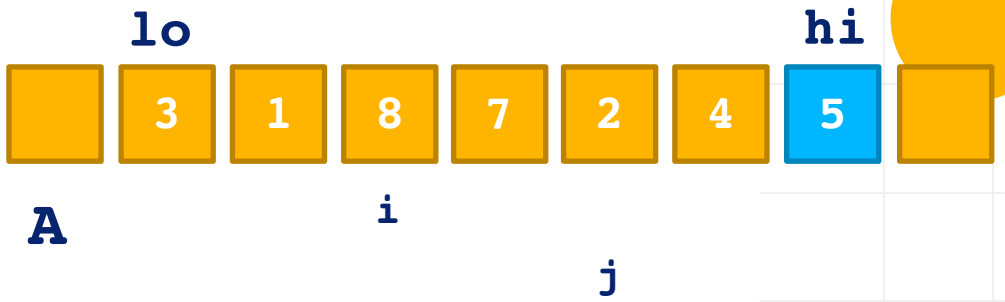
Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



Quicksort partitioning

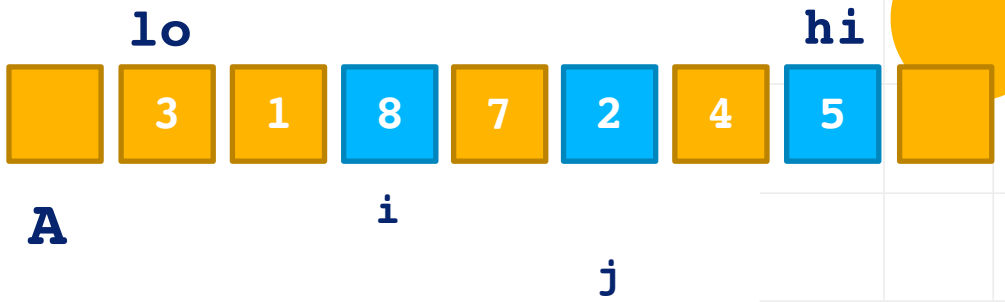
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

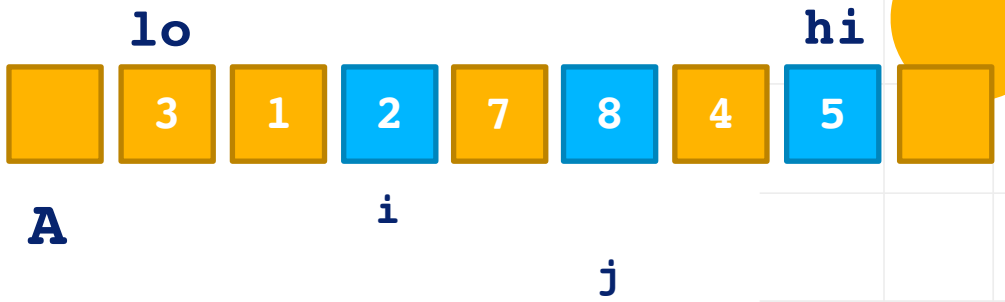
Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



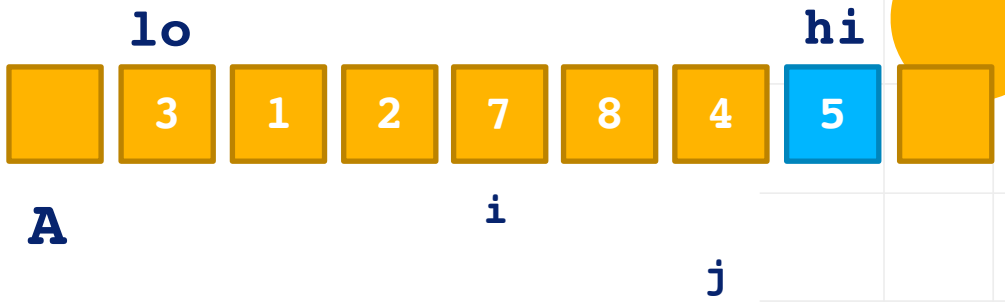
Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



Quicksort partitioning

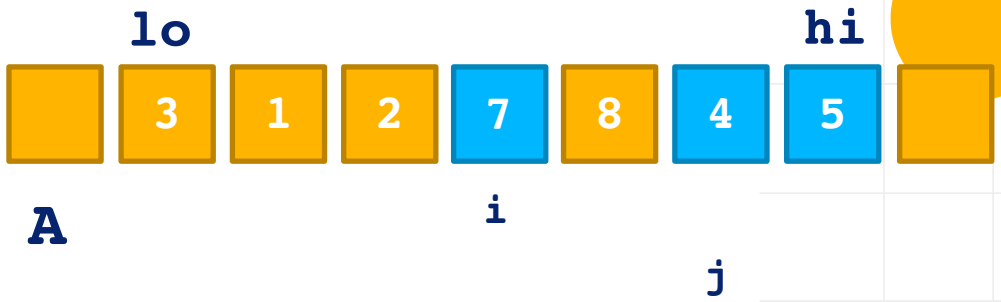
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

Quicksort partitioning

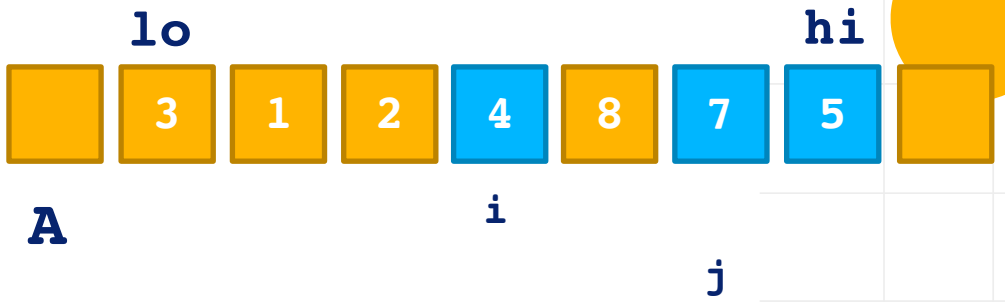
```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

Quicksort partitioning

```
function partition(A, lo, hi)
    pivot := A[hi]
    i := lo
    for j := lo to hi do
        if A[j] < pivot then
            swap A[i] with A[j]
            i := i + 1
    swap A[i] with A[hi]
    return i
```



Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

Quicksort partitioning

```
function partition(A, lo, hi)
   pivot := A[hi]
   i := lo
   for j := lo to hi do
      if A[j] < pivot then
         swap A[i] with A[j]
         i := i + 1
   swap A[i] with A[hi]
   return i
```



pivot = 5

Quicksort

```
algorithm quicksort(A, lo, hi)
    if lo < hi then
        p := partition(A, lo, hi)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)
```

Complexity analysis

Worst-case

Pivot is the largest element

$$T(n) = T(n-1) + \Theta(n)$$

$$T(n) = \Theta(n^2)$$

Best-case

The array is split in half

$$T(n) = 2T(n/2) + \Theta(n)$$

$$T(n) = \Theta(n\log(n))$$

Quicksort

```
algorithm quicksort(A, lo, hi)
    if lo < hi then
        p := partition(A, lo, hi)
        quicksort(A, lo, p - 1)
        quicksort(A, p + 1, hi)
```

Complexity analysis

Average-case

The array is split at 9/10 of its size

$$T(n) = T(n/10) + T(9n/10) + \Theta(n)$$

$$T(n) = \Theta(n\log(n))$$

Probabilistic analysis of Quicksort

Bike universe



I go to UPB by bike in a sunny day



I go to UPB on foot in a rainy day



I go to UPB by bike in a rainy day



I go to UPB by bike in a sunny day and I get a flat tire

Dice universe



A two-dice roll
36 different outcomes

Random variables

$$X : U \longrightarrow \mathbb{R}$$

U – universe

a set of events that might take place

a set of outcomes resulting from an experiment

X(e) – a value associated to event e

a metric

a measurement

Bike universe

	D how much time does it take to go to UPB
 I go to UPB by bike in a sunny day	12 min
 I go to UPB on foot in a rainy day	25 min
 I go to UPB by bike in a rainy day	16 min
 I go to UPB by bike in a sunny day and I get a flat tire	40 min

Dice universe



$S(e)$ = the sum of the dice faces

Probability distribution

$$P : U \longrightarrow [0,1]$$

$P(e)$ – the probability that event e takes place

Bike universe

P



I go to UPB by bike in a sunny day

60%



I go to UPB on foot in a rainy day

24%



I go to UPB by bike in a rainy day

15%



I go to UPB by bike in a sunny day and I get a flat tire

1%

Dice universe



$P(e) = 1/36$

Expectation of a random variable

$$\mathbb{E}[X] = \sum P(e) * X(e)$$

Repeat an experiment very many times
Observe a process for a long time

Record the values $X(e)$ that you observe

Compute the average value of X, over all observed events e

If you repeat the experiment long-enough, each event e will occur with probability $P(e)$

Bike universe:

$\mathbb{E}[D]$ - how long does a trip to UPB last, on average?

	D	P	
 I go to UPB by bike in a sunny day	12 min	60%	=7,2
 I go to UPB c… rainy day		…%	=6
 I go to UPB… rainy day		15%	=2,4
 I go to UPB by bike in a sunny day and I get a flat tire	40 min	1%	=0,4

$\mathbb{E}[D] = 16min$

Dice universe

$\mathbb{E}[S]$- what is the average sum of dice-faces?



Dice universe



$\mathbb{E}[S]$ - what is the average sum of dice-faces?

The smart way is to use ***linearity of expectation***

$$\mathbb{E}[S]=\mathbb{E}[S_1+S_2]$$

Random variables for the sums of each face

Dice universe



$\mathbb{E}[S]$ - what is the average sum of dice-faces?

The smart way is to use ***linearity of expectation***

$$\mathbb{E}[S]=\mathbb{E}[S_1+S_2] = \mathbb{E}[S_1] + \mathbb{E}[S_2]$$

$$\mathbb{E}[S_i]= (1+2+3+4+5+6)/6 = 3,5$$

$$\mathbb{E}[S]= 3,5 + 3,5 = 7$$

Quicksort universe

Assume for simplicity that elements in the array are unique

Event e_i means that the i–th element is the pivot

Probability $P(e_i) = 1/n$ where n is the size of the array

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

Quicksort universe

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + T(i-1) + T(n-i)$$

Quicksort universe

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + T(i-1) + T(n-i)$$

The ***expected*** execution time of Quicksort

Quicksort universe

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

The ***expected*** execution time of Quicksort

Quicksort universe

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

$$\mathbb{E}(Q_n) = \Theta(n) + 1/n * \Sigma\ (\mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i}))$$

Quicksort universe

Random variable Q_n where $Q_n(e_i)$ is the running time of Quicksort, if the current pivot is the i-th element.

$$Q_n(e_i) = \Theta(n) + \mathbb{E}(Q_{i-1}) + \mathbb{E}(Q_{n-i})$$

$$T(n) = \Theta(n) + 1/n * \Sigma\ (T(i-1) + T(n-i))$$

Quicksort universe

$$T(n) = \Theta(n) + 1/n * \Sigma (T(i-1) + T(n-i))$$

Quicksort universe – algebrical part

$$T(n) = \Theta(n) + 1/n * \sum_{0<i\leq n} (T(i-1) + T(n-i))$$

$$T(n) = \Theta(n) + 2/n * \sum_{0\leq i<n} T(i)$$

Quicksort universe – algebrical part

$$T(n) = \Theta(n) + 1/n * \sum_{0<i\leq n} (T(i-1) + T(n-i))$$

$$T(n) = n + 2/n * \sum_{0\leq i<n} T(i)$$

$$nT(n) = n^2 + 2 * \sum_{0\leq i<n} T(i)$$

$$(n-1)T(n-1) = (n-1)^2 + 2 * \sum_{0\leq i<n-1} T(i)$$

Quicksort universe – algebrical part

$$nT(n) = n^2 + 2 * \sum_{0 \le i < n} T(i)$$

$$(n-1)T(n-1) = (n-1)^2 + 2 * \sum_{0 \le i < n-1} T(i)$$

$$nT(n) - (n-1)T(n-1) = n^2 - (n-1)^2 + 2T(n-1)$$

$$nT(n) = 2n-1 + (n+1)T(n-1)$$

$$nT(n) \le 2(n+1) + (n+1)T(n-1)$$

Quicksort universe – algebrical part

$$nT(n) \leq 2(n+1) + (n+1)T(n-1)$$

$$T(n)/(n+1) \leq 2/n + T(n-1)/n \qquad F(n) = T(n)/(n+1)$$

$$F(n) \leq 2/n + F(n-1)$$

$$F(n) \leq 2(1/n + 1/(n-1) + \ldots + 1/2 + 1)$$

$$F(n) \leq 2H_n \simeq 2\lg(n)$$

$$T(n) \leq 2(n+1)\lg(n) \qquad T(n)=O(n \log(n))$$



Analiza Algoritmilor

P vs NP

Matei Popovici




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++]  := B[k++]
    while (j < n)

C[i++]  := A[j++]
    while (k < m)
        C[i++]  := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

11110



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

00010



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    a := size of A
    n := size of B

C := new array of size a + n
    i,j,k := 0
    while (j < a and k < n)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < n)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    a := size of A
    n := size of B

C := new array of size a + n
    i,j,k := 0
    while (j < a and k < n)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < n)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```


1 1 1 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 1 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 0 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 0 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 1 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

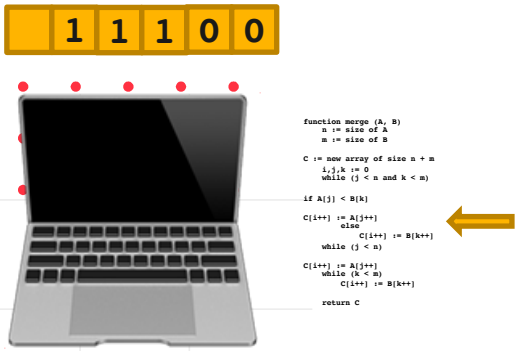

1 1 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

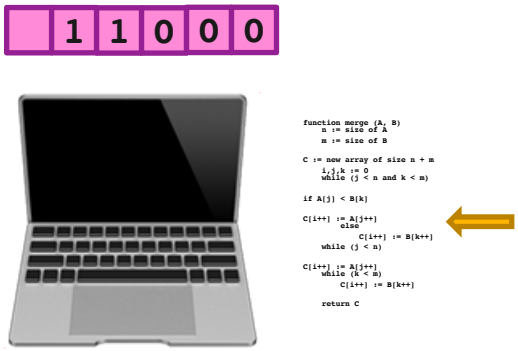

0 1 1 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

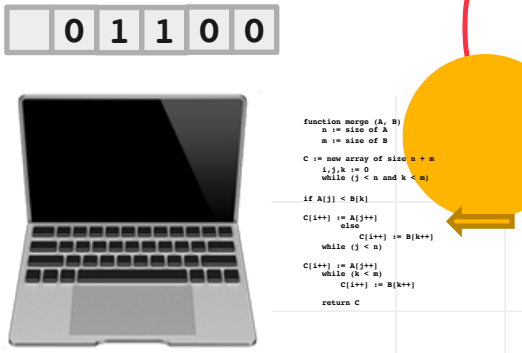

0 0 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

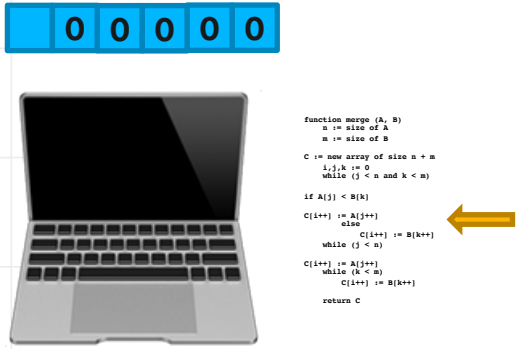

1 0 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

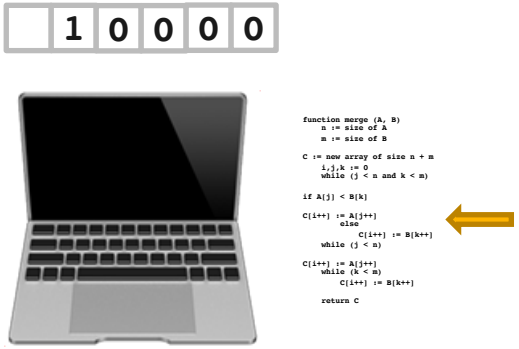

0 1 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

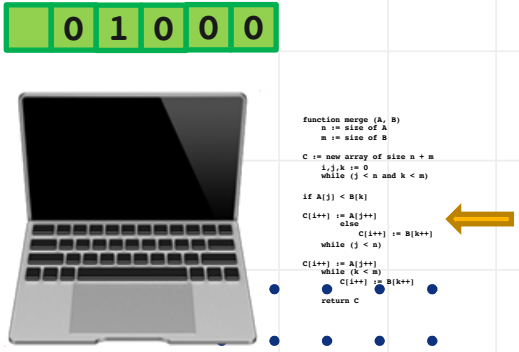

11100
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


11000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


01100
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


00000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


10000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


01000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```


1 1 1 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 1 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 0 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 0 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

There is an unlimited number of copies the machine can make

Each machine works on its local data

Each machine has access to data computed before it was ***spawned***

Different copies cannot communicate or share data during their computations

Lecture outline

machine = algorithm

Formal definition

Can these machines solve undecidable problems?

Can these machines solve problems more efficiently?

Are there problems which still cannot be solved efficiently by these machines?

We will use these idealised machines to classify problems



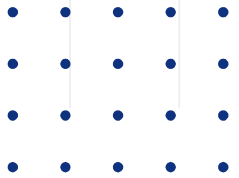
Lecture outline

We will use these idealised machines to classify problems

Problems which can be solved efficiently on non-parallel algorithms

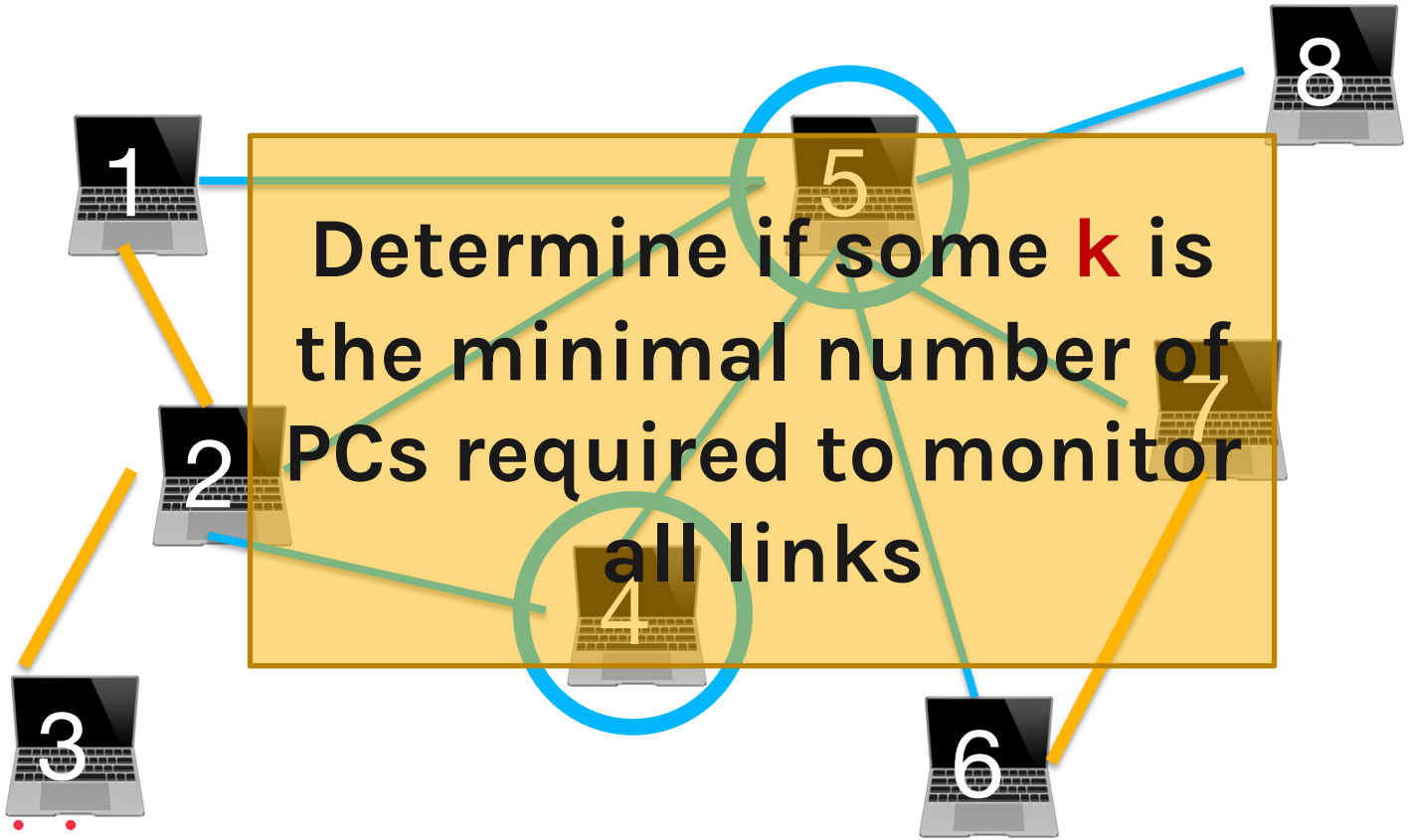
Problems which can be solved efficiently on parallel algorithms

Problems which cannot be solved efficiently on parallel algorithms



Parallel algorithms – informal definition

Problem 2: k-Vertex-Cover



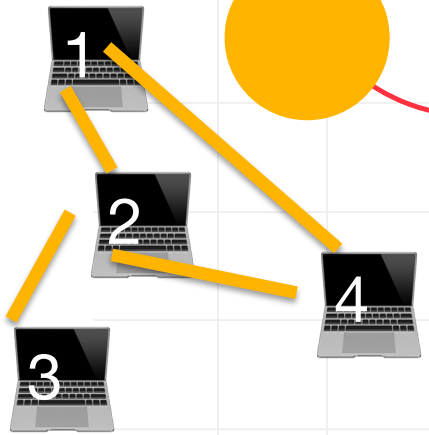
Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

$C = \emptyset,\ k = 2$



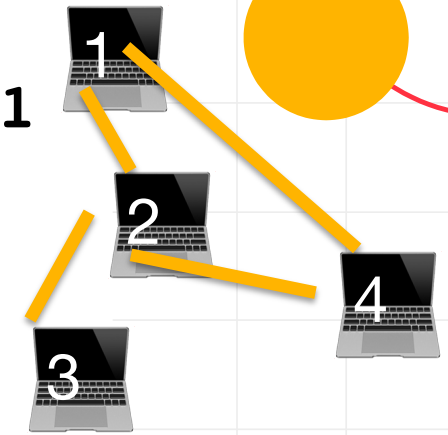
Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

$C = \emptyset,\ k = 1,\ v = 1$



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

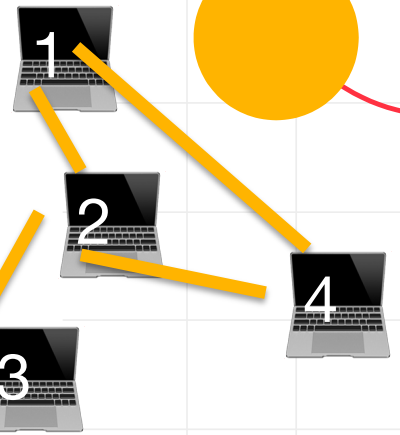
    success
```

c = ∅, k = 1, v = 1

c = ∅, k = 1, v = 2

c = ∅, k = 1, v = 3

c = ∅, k = 1, v = 4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

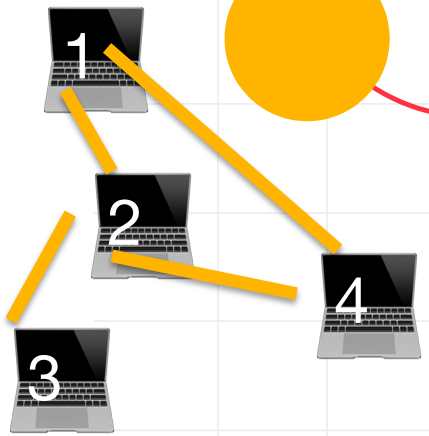
    success
```

C = {1}, k = 1

C = {2}, k = 1

C = {3}, k = 1

C = {4}, k = 1



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
  while (k > 0)
    k := k - 1
    v := choice(V)
    if C contains v
      fail
    C := {v} U C

  for each (u,v) in E
    if C does not contain u nor v
      fail

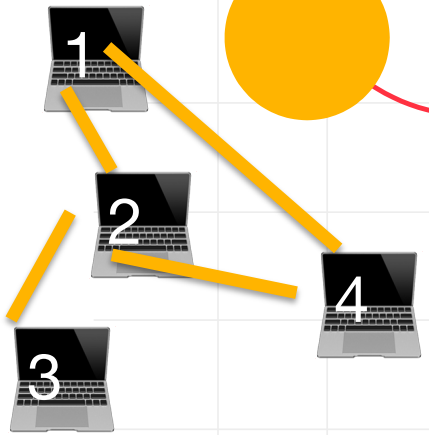
  success
```

C = {1}, k = 0

C = {2}, k = 0

C = {3}, k = 0

C = {4}, k = 0



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

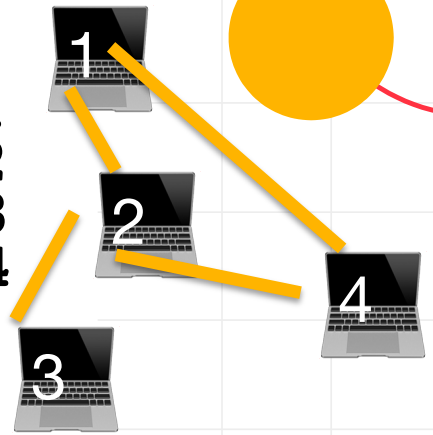
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0

C = {3}, k = 0

C = {4}, k = 0



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

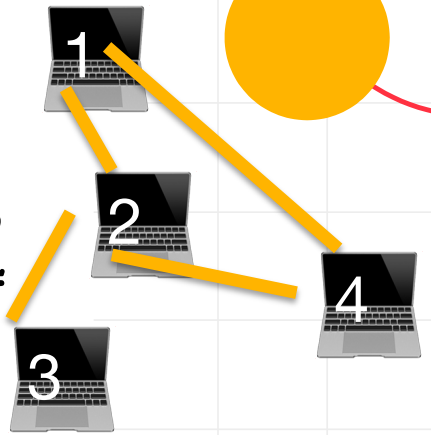
    success
```

```
C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0

C = {4}, k = 0
```



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

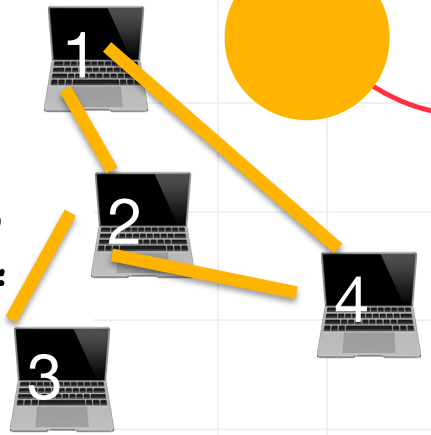
    success
```

```
C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0
```



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

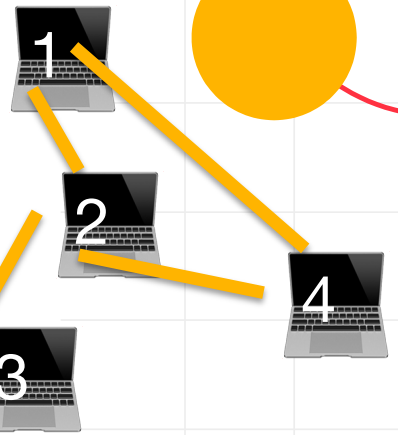
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

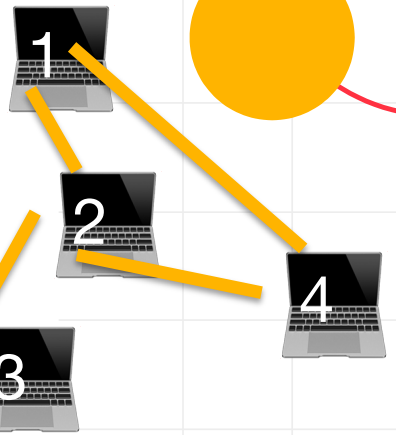
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {4}, k = 0, v=2
C = {4}, k = 0, v=3

C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1

C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2

C = {3}, k = 0, v=4

C = {4}, k = 0, v=1



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1,2}
C = {1,3}
C = {1,4}

C = {2,1}

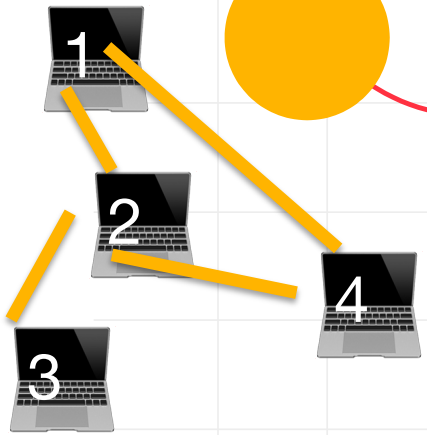
C = {2,3}
C = {2,4}

C = {3,1}
C = {3,2}

C = {3,4}

C = {4,1}

C = {4,2}
C = {4,3}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1,2}
C = {1,3}
C = {1,4}

C = {2,1}

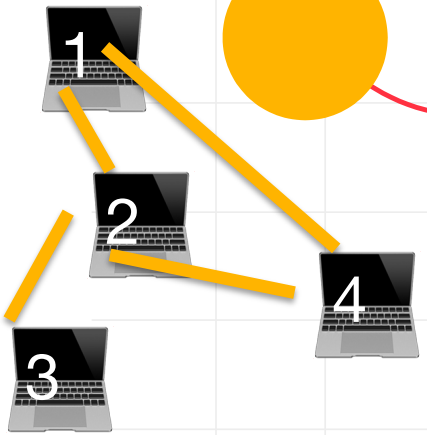
C = {2,3}
C = {2,4}

C = {3,1}
C = {3,2}

C = {3,4}

C = {4,1}

C = {4,2}
C = {4,3}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

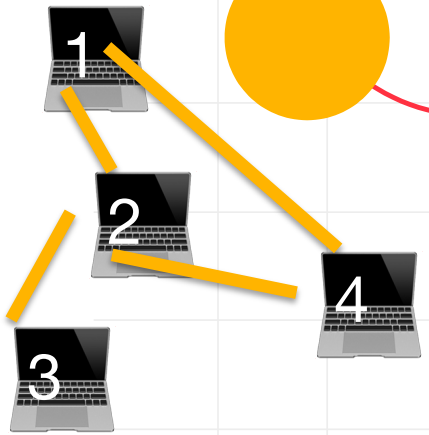
    success
```

C = {1,2}

C = {2,1}

C = {2,4}

C = {4,2}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

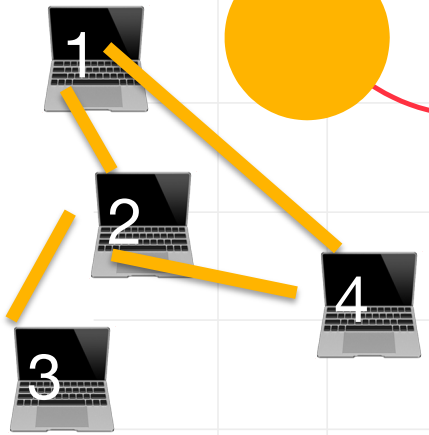
    success
```

C = {4,2}

C = {1,2}

C = {2,1}

C = {2,4}



Parallel algorithms – informal definition

Parallel algorithm = any type of pseudocode enhanced with:

`choice(A)`

Creates $|A|$ copies. For each value v_i in A, the algorithm spawns a clone. In each clone, this instruction evaluates to v_i.

`success`

Terminates the current copy with success.

`fail`

Terminates the current copy with failure.

Parallel algorithms – informal definition

A parallel algorithm algorithm returns true, iff at least one copy returns success.

A parallel algorithm algorithm returns false, iff all copies fail.

Parallel algorithms – informal definition

A parallel algorithm is nondeterministic, in the sense that its execution is not uniquely determined by the input data.

Parallel algorithms – complexity

nondeterministic complexity

angelic complexity

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

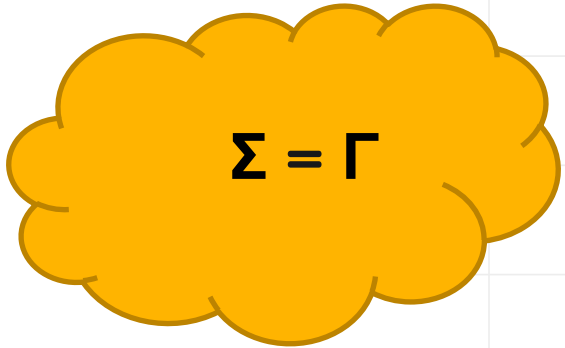
`choice(A)` constant time $O(1)$

Angelic complexity for this algorithm is $O(n^2)$

Deterministic Turing Machines

$$M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta).$$

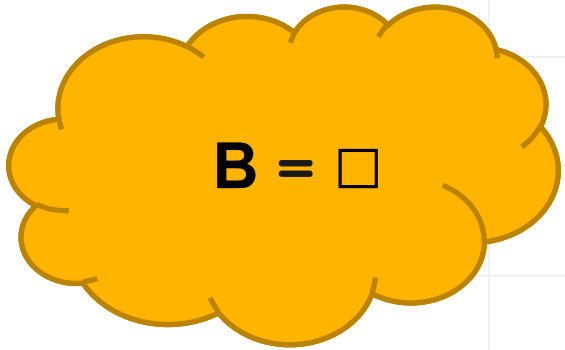
- Q – set of states
- Σ – input alphabet
- Γ – tape alphabet
- B – blank symbol
- q_1 – initial state
- Y, N, H – accepting, rejecting, halting states
- δ – transition function



Deterministic Turing Machines

$$M = (Q, \Sigma, B, q_1, Y, N, H, \delta).$$

- Q – set of states
- Σ – input alphabet
- B – blank symbol
- q_1 – initial state
- Y, N, H – accepting, rejecting, halting states
- δ – transition function



Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, H, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N, H – accepting, rejecting, halting states

δ – transition function

B = □

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, H, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N, H – accepting, rejecting, halting states

δ – transition function

We do not need a halting state for decision problems

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N – accepting, rejecting states

δ – transition function

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

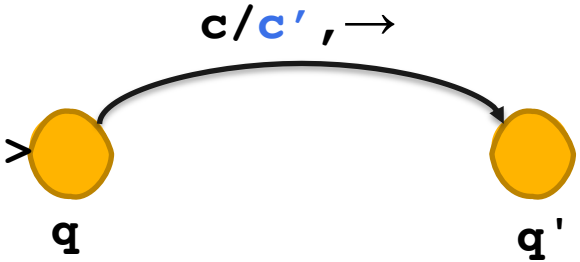
Q – set of states

Σ – input alphabet

q_1 – initial state

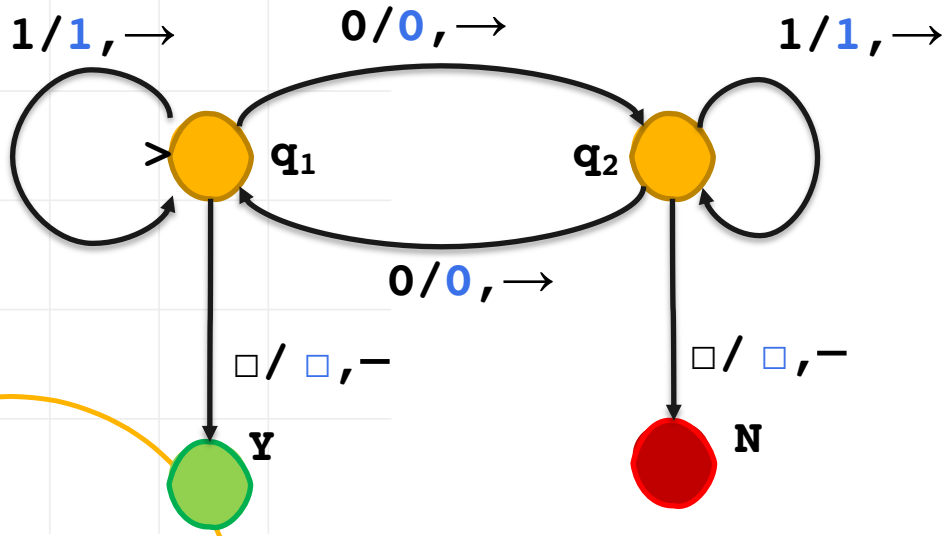
Y, N – accepting, rejecting states

$\delta: Q \times \Sigma \longrightarrow Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition function



Deterministic Turing Machines

$M = (Q, \Sigma, q_1, Y, N, \delta)$.



	0	1	□
q_1	$q_2, 0, \rightarrow$	$q_1, 1, \rightarrow$	$Y, \square, -$
q_2	$q_1, 0, \rightarrow$	$q_2, 1, \rightarrow$	$N, \square, -$

Nondeterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N – accepting, rejecting states

$\delta: Q \times \Sigma \longrightarrow Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition function

Nondeterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

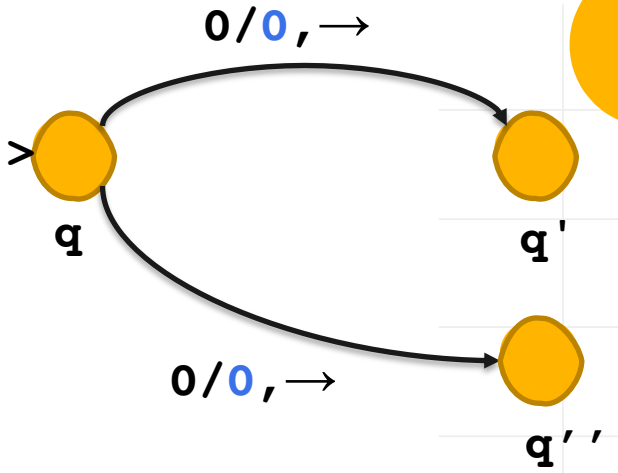
Y, N – accepting, rejecting states

~~$\delta: Q \times \Sigma \Longrightarrow Q \cup \{Y,N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$~~ ~~transition function~~

$\Delta: Q \times \Sigma \times Q \cup \{Y,N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ transition relation

Nondeterministic Turing Machines

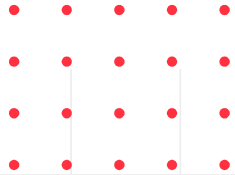
$M = (Q, \Sigma, q_1, Y, N, \Delta)$.

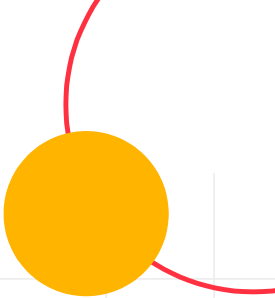


- Q – set of states
- Σ – input alphabet
- q_1 – initial state
- Y, N – accepting, rejecting states

$\Delta: Q \times \Sigma \times Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition relation

The Turing Machine can now be in two or more states ***at the same time.***

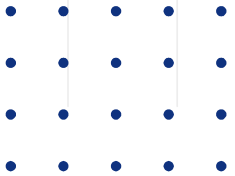




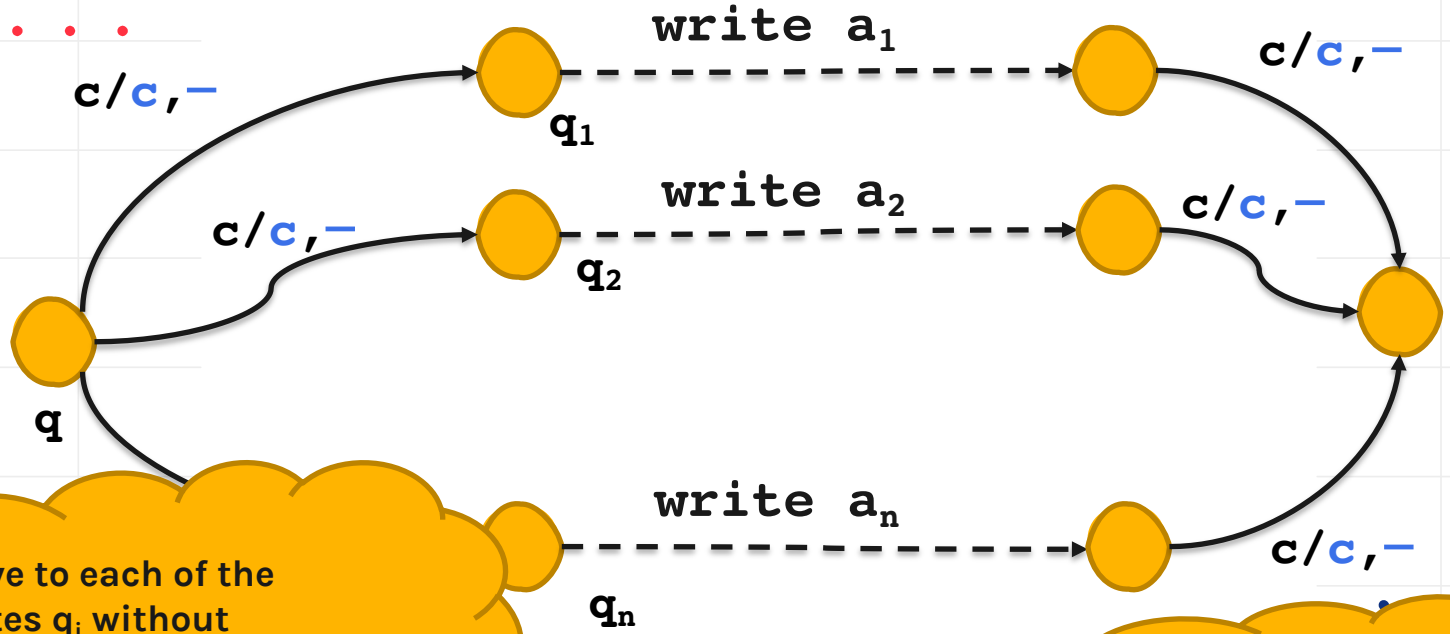
The relationship between ***Nondeterministic Turing Machines*** and ***Parallel Algorithms***

They are one and the same!



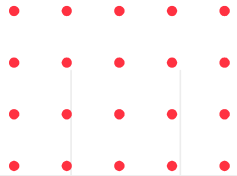


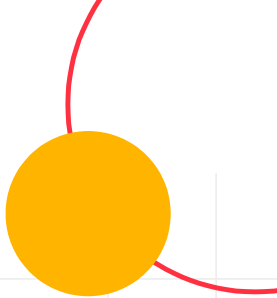
choice(A) where A = {a_1, a_2, …, a_n}



Move to each of the states q_i without changing the contents of the tape

Continue with the processing....

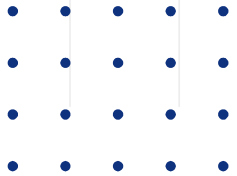




The relationship between ***Nondeterministic Turing Machines*** and ***Parallel Algorithms***

We will prefer writing parallel algorithms because they are more legible and easy to follow

Any parallel algorithm is a Nondeterministic Turing Machine

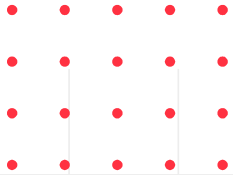




Complexity Theory – an introduction

Classify problems (not algorithms!) based on the existence of efficient algorithms for them.

If a problem can be solved by some algorithm in polynomial time, then we do not care about the exponent of the polynomial



polynomial

worse than polynomial

Turing Machines

graph reachability

is prime

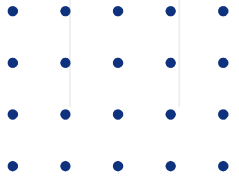
sorted sequence

Nondeterministic Turing Machines

graph reachability

is prime

sorted sequence



Turing Machines	Nondeterministic Turing Machines
polynomial	
worse than polynomial	k-vertex-covering
	hamiltonian path
	SAT
	All-k-vertex-covering
	forall-SAT

Turing Machines	Nondeterministic Turing Machines
polynomial	
worse than polynomial	k-vertex-covering
	hamiltonian path
	SAT
	All-k-vertex-covering
	forall-SAT

Turing Machines

Nondeterministic Turing Machines

It is expensive to go over possible candidate solutions

It is cheap to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

SAT

***All*-k-vertex-covering**

***forall*-SAT**

Turing Machines

Nondeterministic Turing Machines

It is expensive to go over possible candidate solutions

It is cheap to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

SAT

***All*-k-vertex-covering**

***forall*-SAT**



Turing Machines

Nondeterministic Turing Machines

It is **expensive** to go over possible candidate solutions

It is **cheap** to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

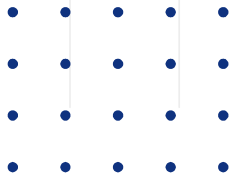
SAT

***All*-k-vertex-covering**

***forall*-SAT**

***All*-k-vertex-covering**

***forall*-SAT**

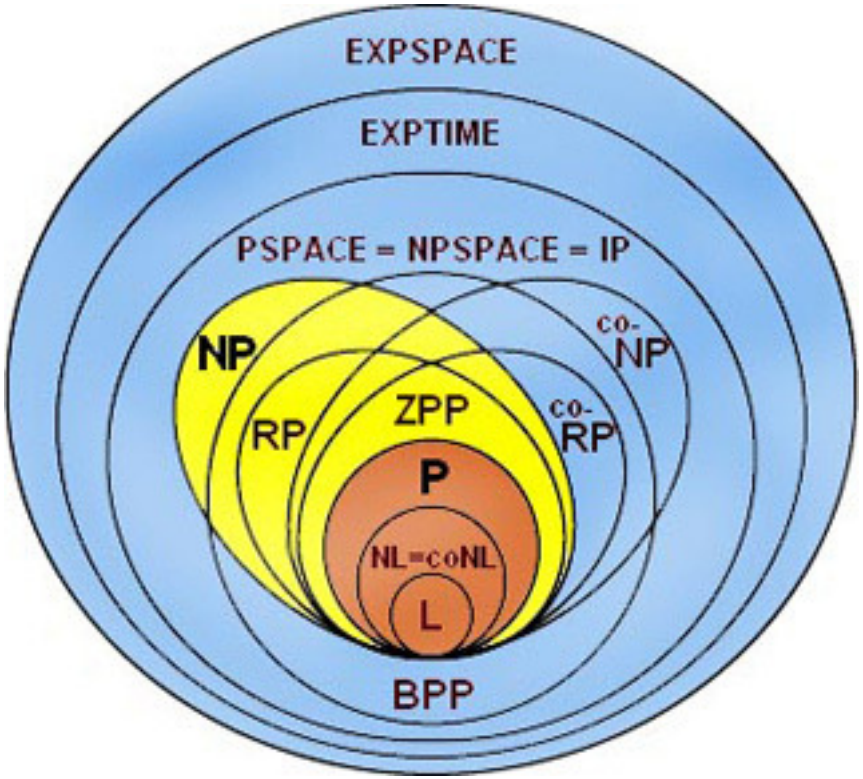


Complexity Theory – classifies problems into:

Problems that can be solved in deterministic polynomial time

Problems that can be solved in NON-deterministic polynomial time

Problems that can be solved in deterministic exponential time


EXPSPACE
EXPTIME
PSPACE = NPSPACE = IP
NP
co-NP
RP
ZPP
co-RP
P
NL=coNL
L
BPP

Complexity Theory – classifies problems into:

Problems that can be solved in deterministic poly

Problems tha NON-determin

Problems that can b deterministic exponential time

What happens when we need better algorithms for our problem at hand?

What happens when we need better algorithms?

Problems that can be solved in deterministic polynomial time

improve data-structures
run in data-center

Problems that can be solved in NON-deterministic polynomial time

Use SAT solvers
Use approximation algorithms
Use random algorithms

Problems that can be solved in deterministic exponential time

Use SMT solvers
Identify tractable cases

Complexity classes

Problems that can be solved in deterministic polynomial time

Problems that can be solved in NON-deterministic polynomial time

Problems that can be solved in deterministic exponential time

P = The set of all **decision problems** f for which there exists a Turing Machine/ algorithm that runs in **polynomial time** and **decides** f.

NP = The set of all **decision problems** f for which there exists a **Nondeterministic** Turing Machine/ algorithm that runs in **polynomial time** and **decides** f.

EXP = The set of all **decision problems** f for which there exists a Turing Machine/ algorithm that runs in **exponential time** and **decides** f.

Relationship between classes

$$P \subseteq NP \subseteq EXP$$

A deterministic Turing Machine is a special case of a nondeterministic Turing Machine

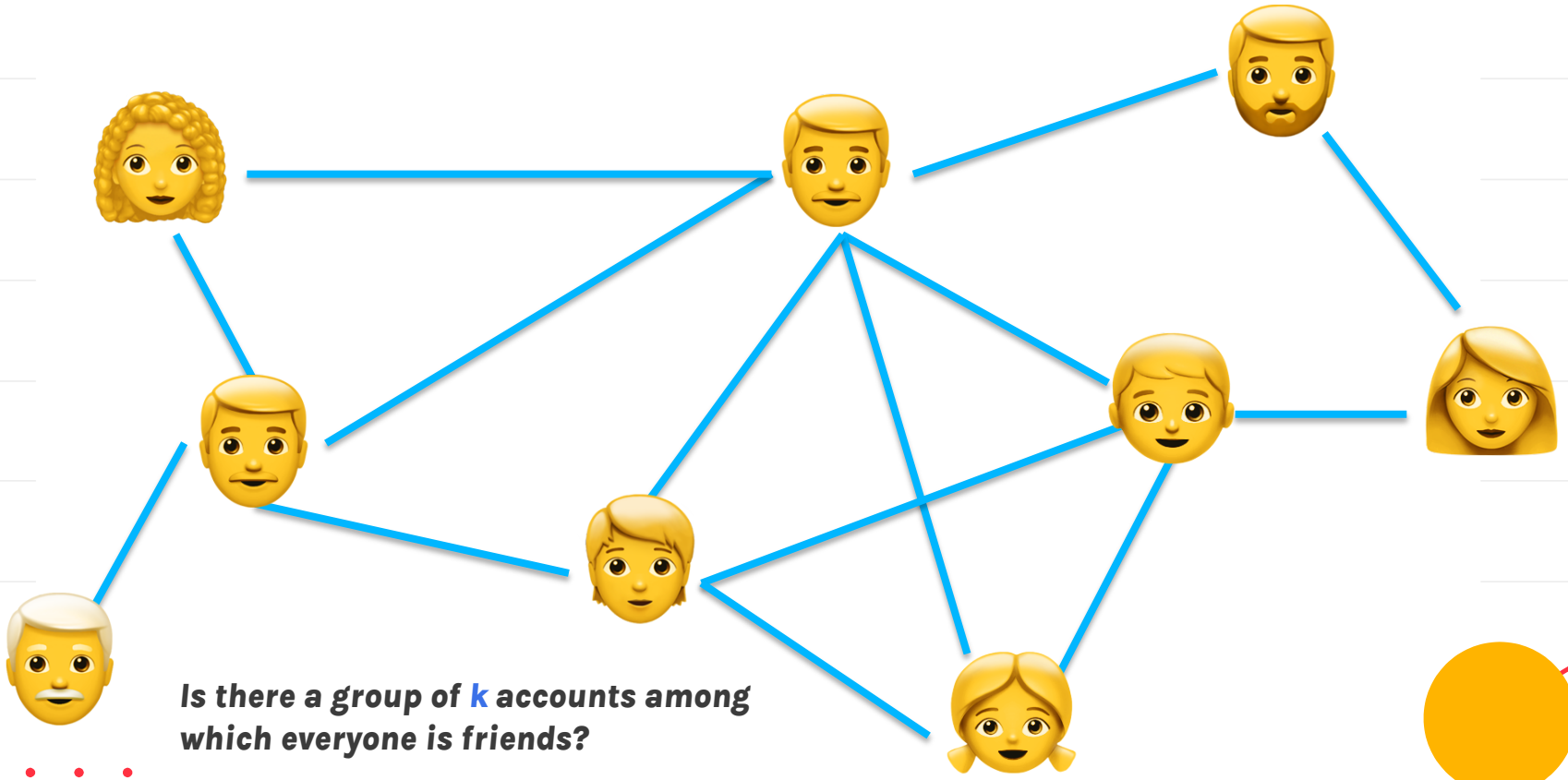
If a problem is solvable in deterministic polynomial time, it is also solvable in nondeterministic polynomial time (i.e. by a NTM)

Relationship between classes

$$P \subseteq NP \subseteq EXP$$

We can simulate any nondeterministic Turing Machine running in polynomial time or a parallel algorithm via backtracking, as a conventional Turing Machine/conventional algorithm that runs in exponential time.

Practice: social network

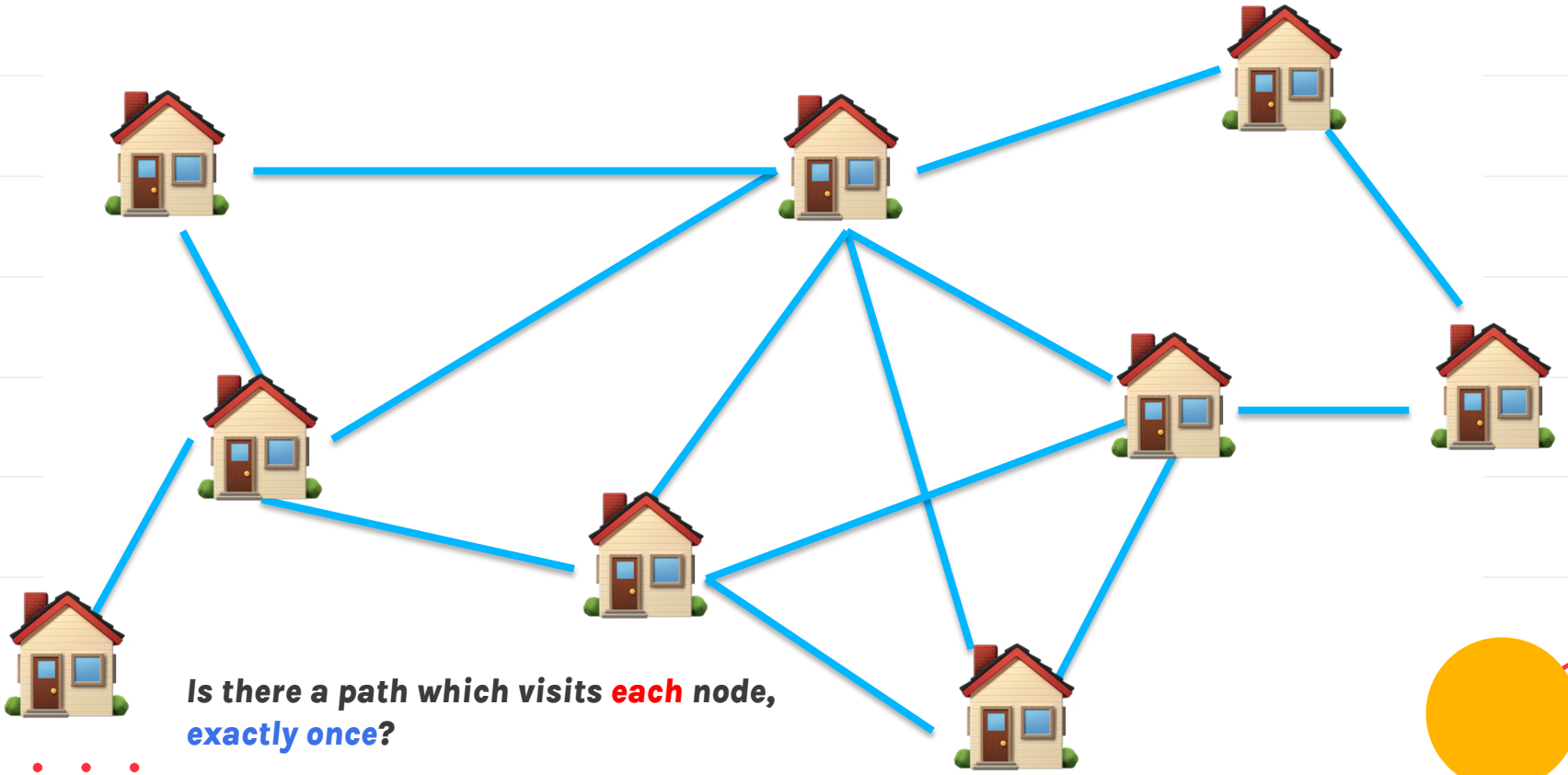


Is there a group of k accounts among which everyone is friends?

Practice: k clique

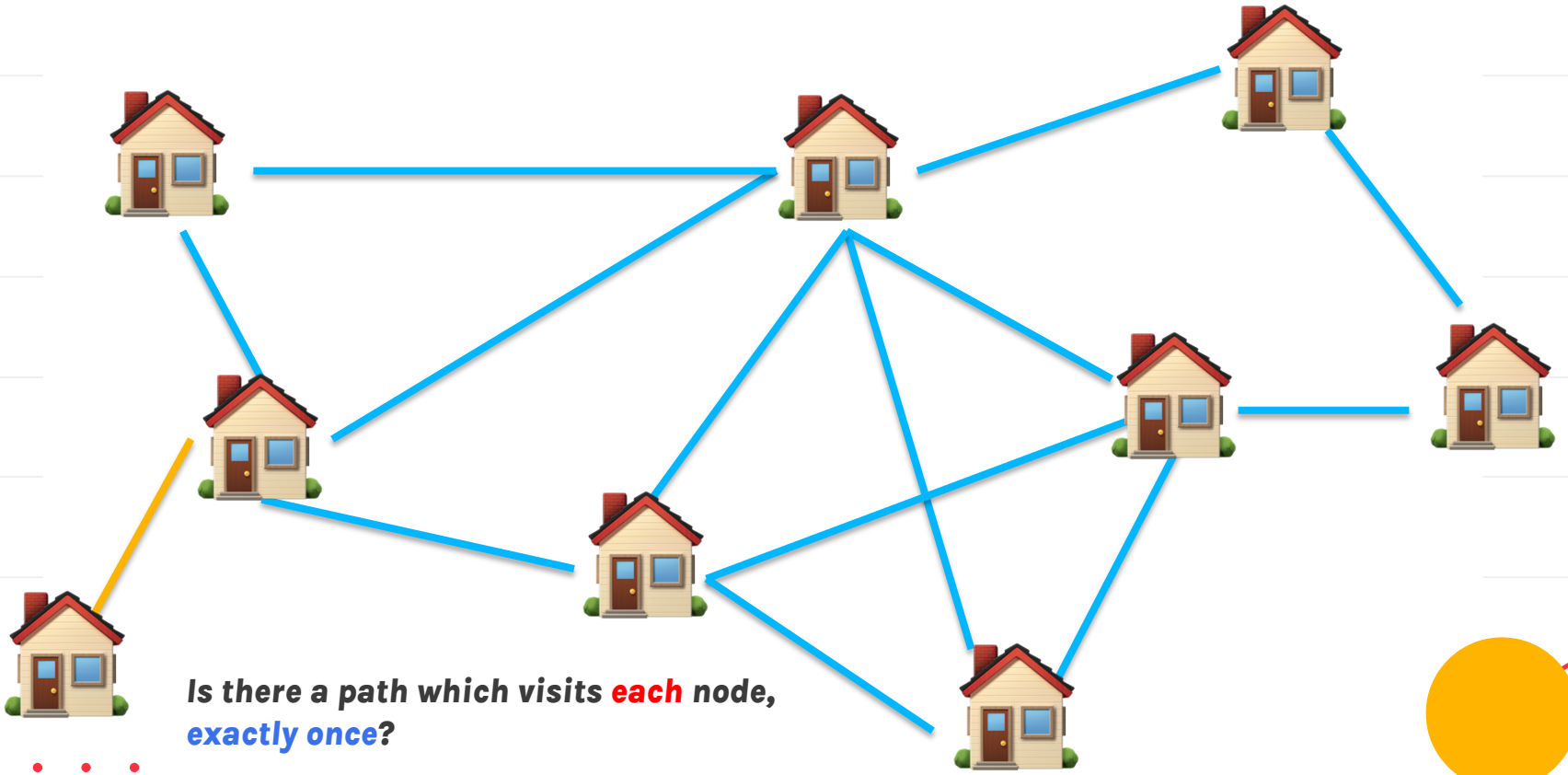
Let $G = (V,E)$ *be an unoriented graph and* k *be a natural number.*
There exists a subset $C \subseteq V$ *of size* k *such that, for each pair of nodes* $u, v \in C$, *there is an edge* $(u,v) \in E$?

Practice: Hamiltonian Path



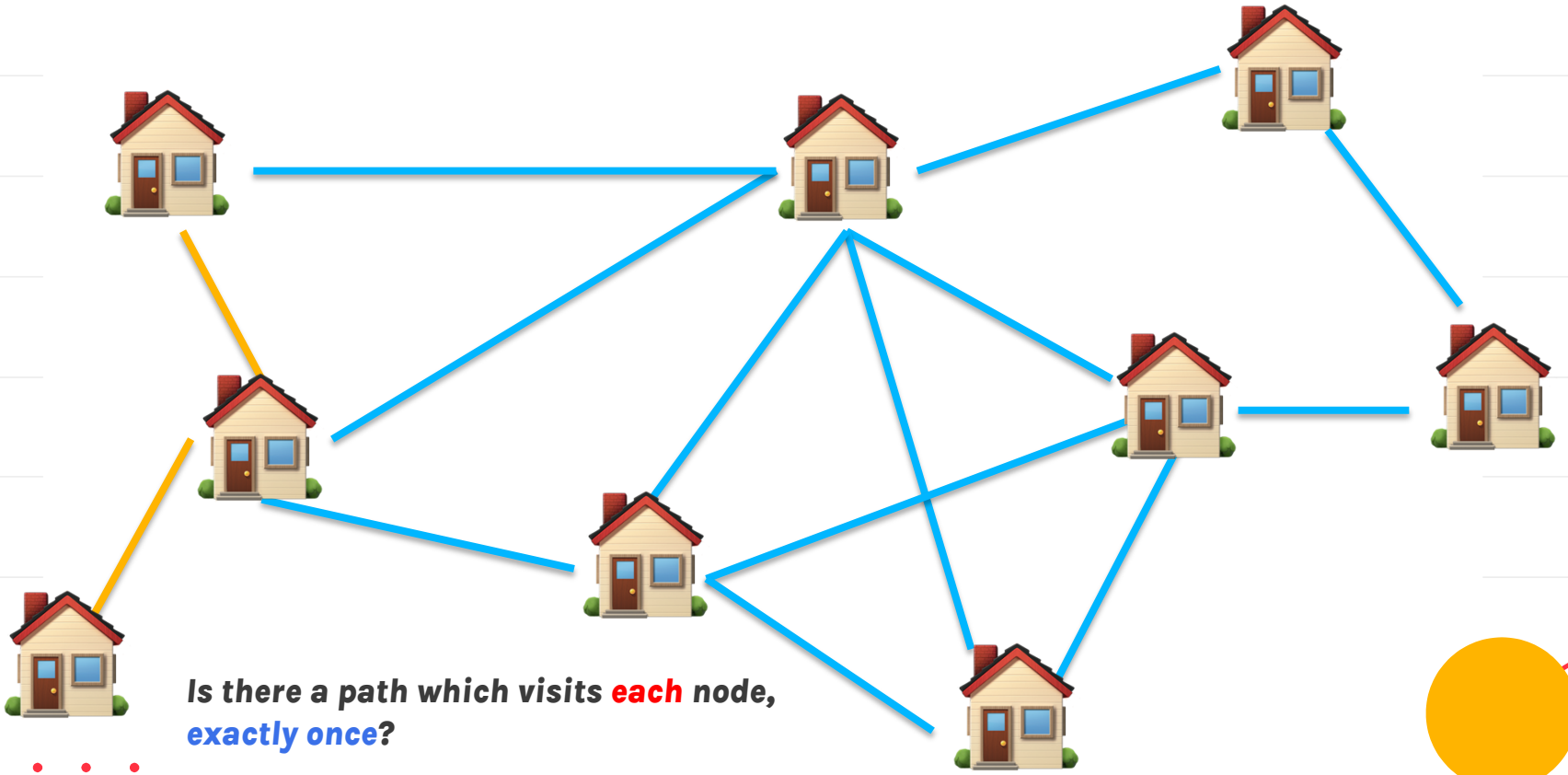
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



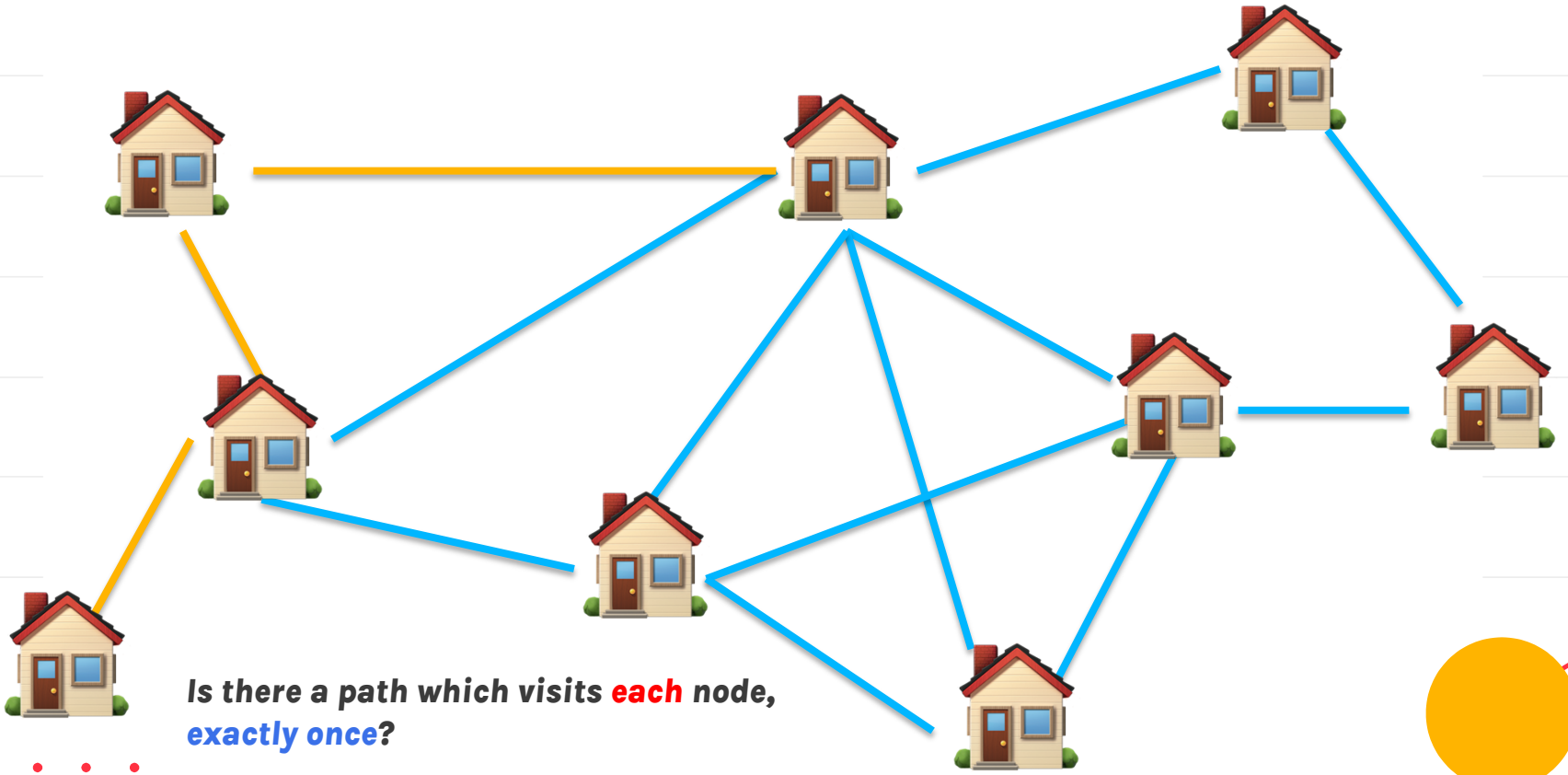
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



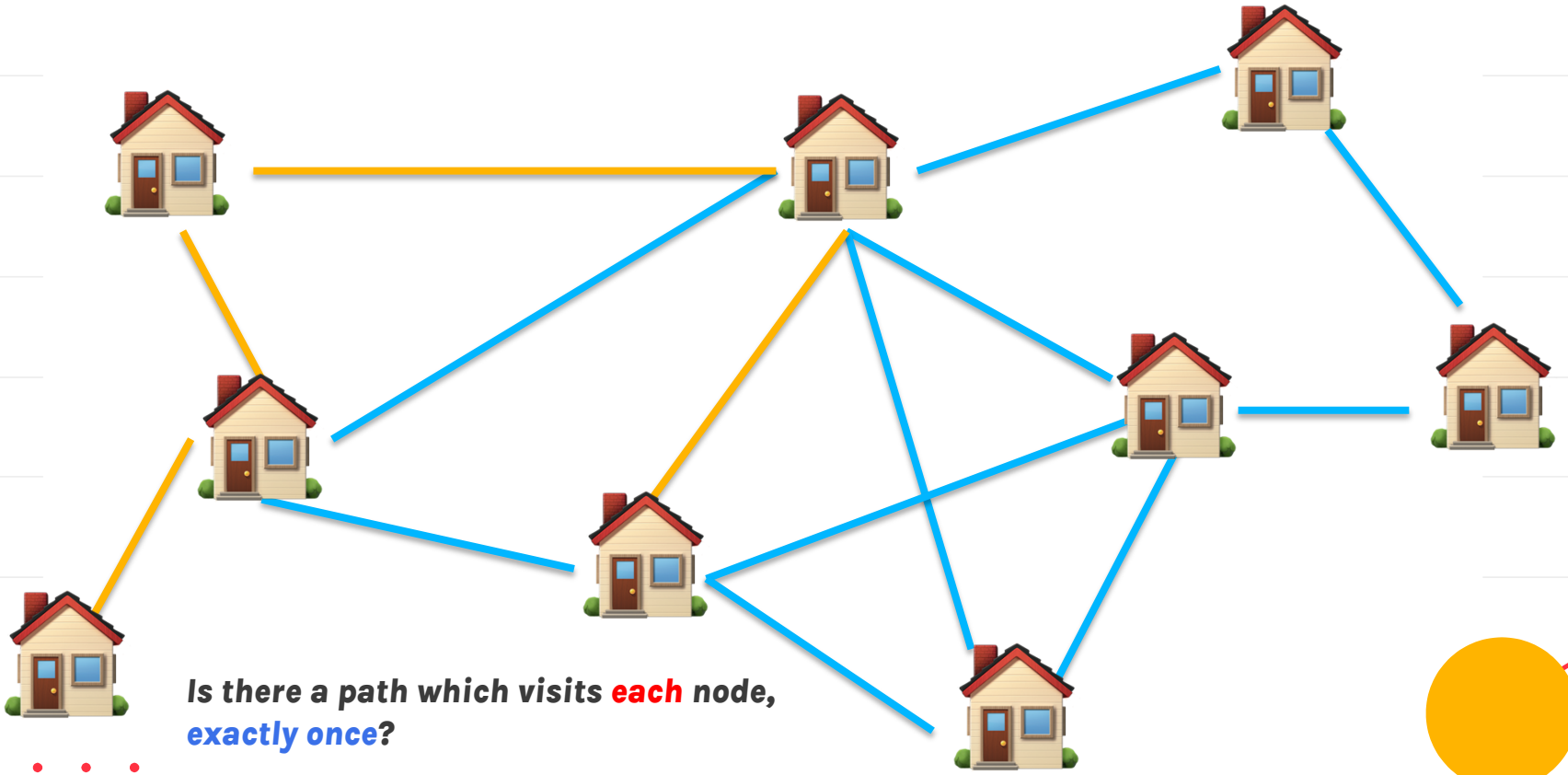
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



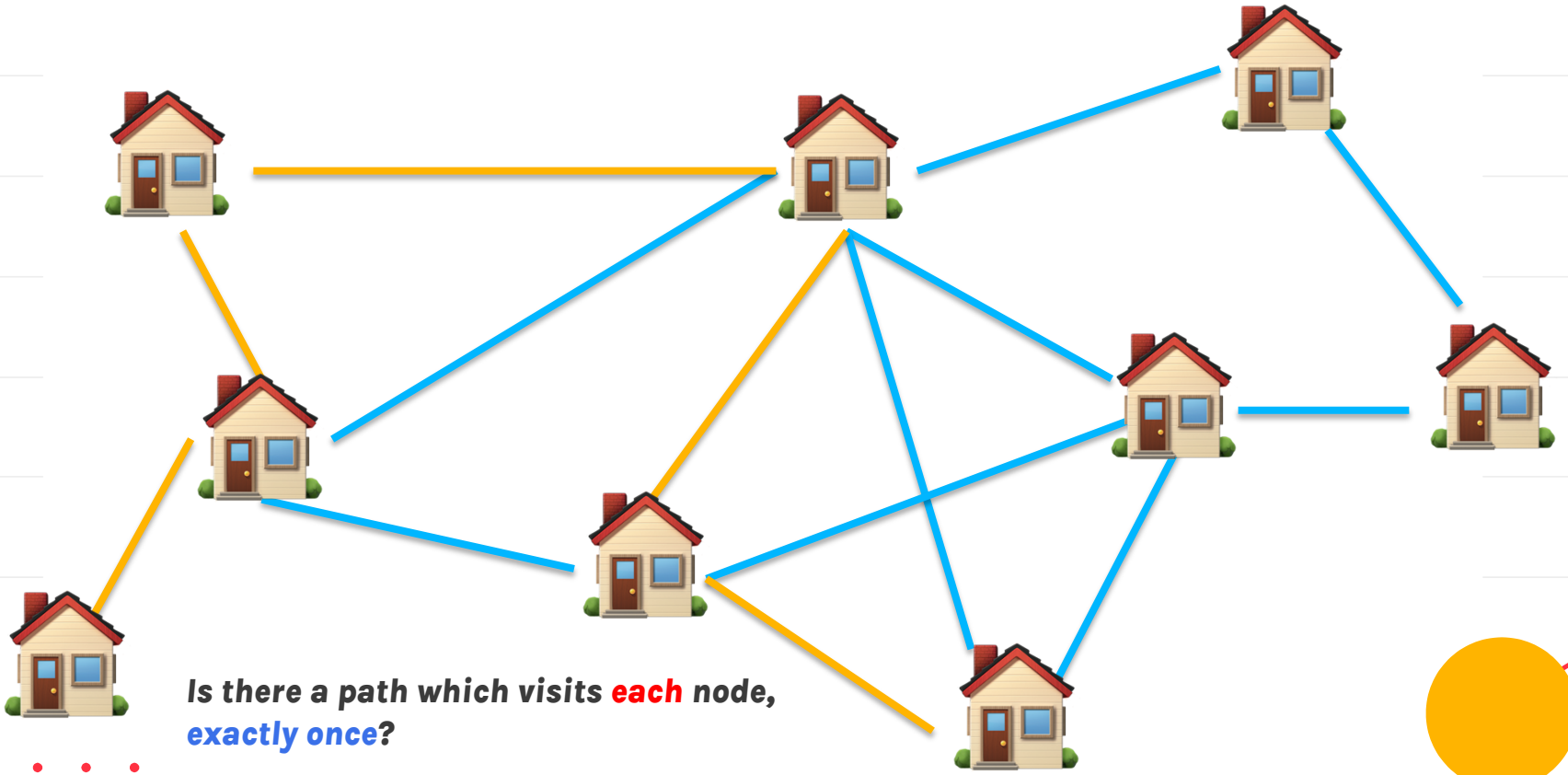
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



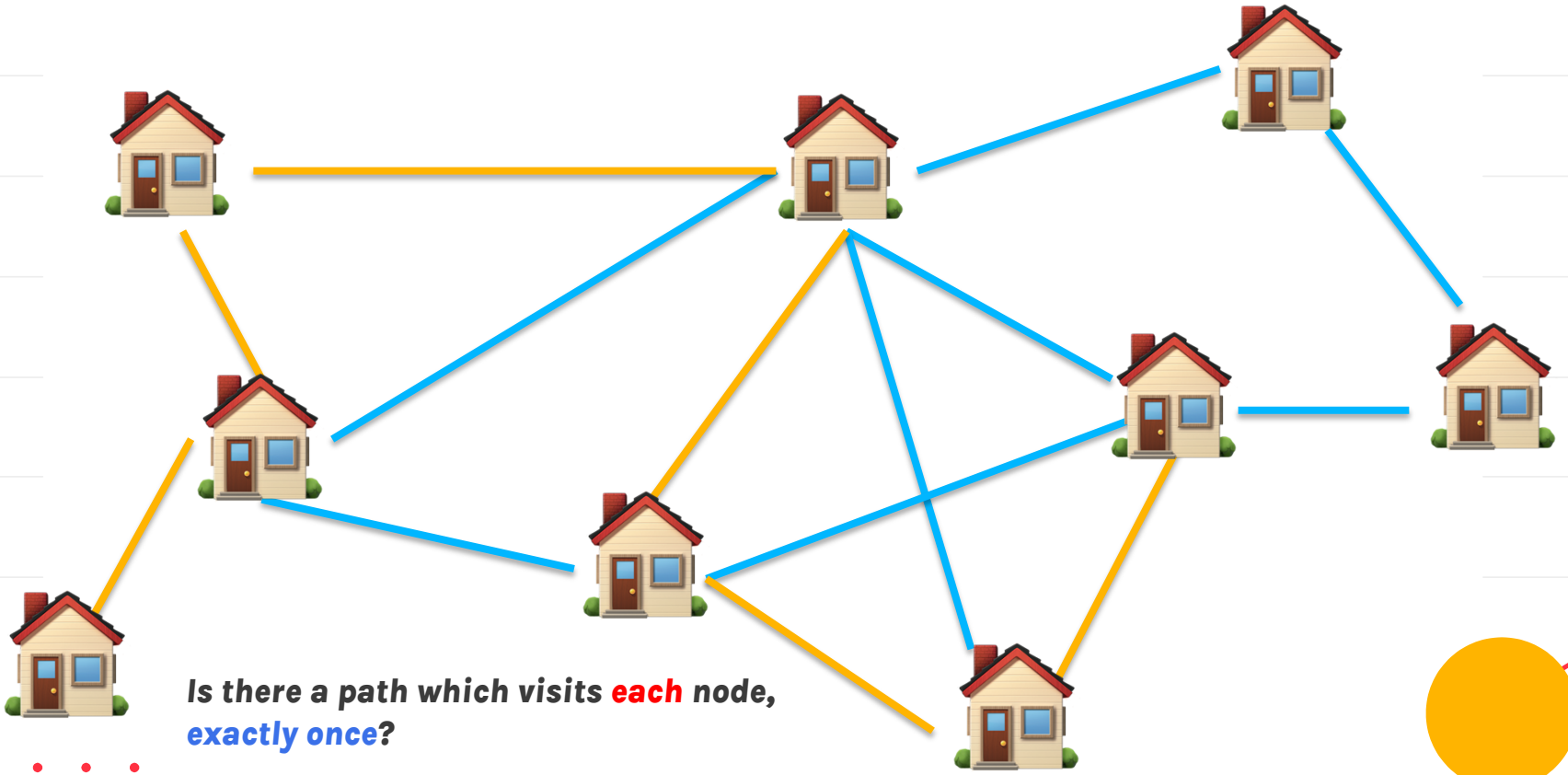
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



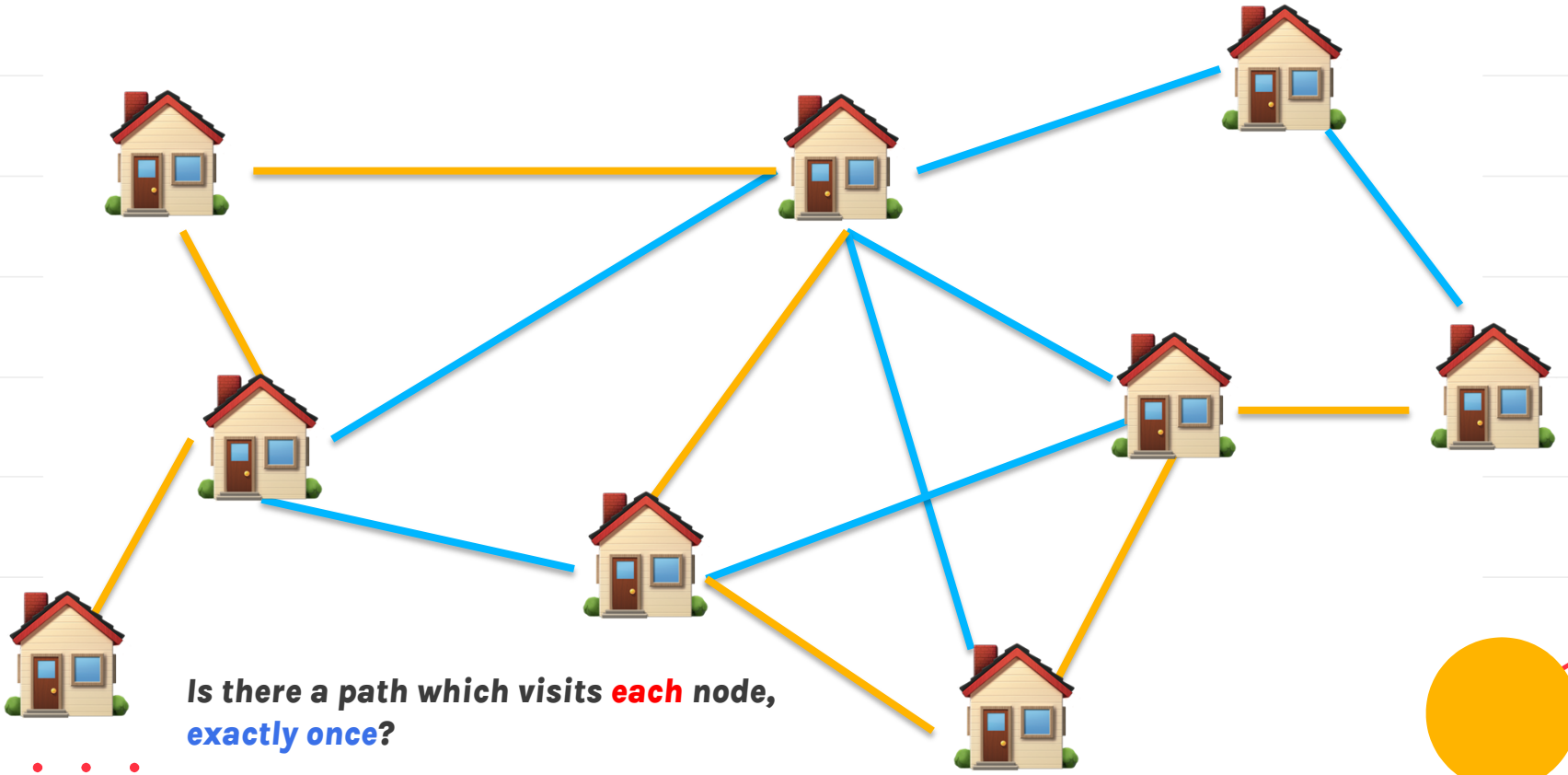
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



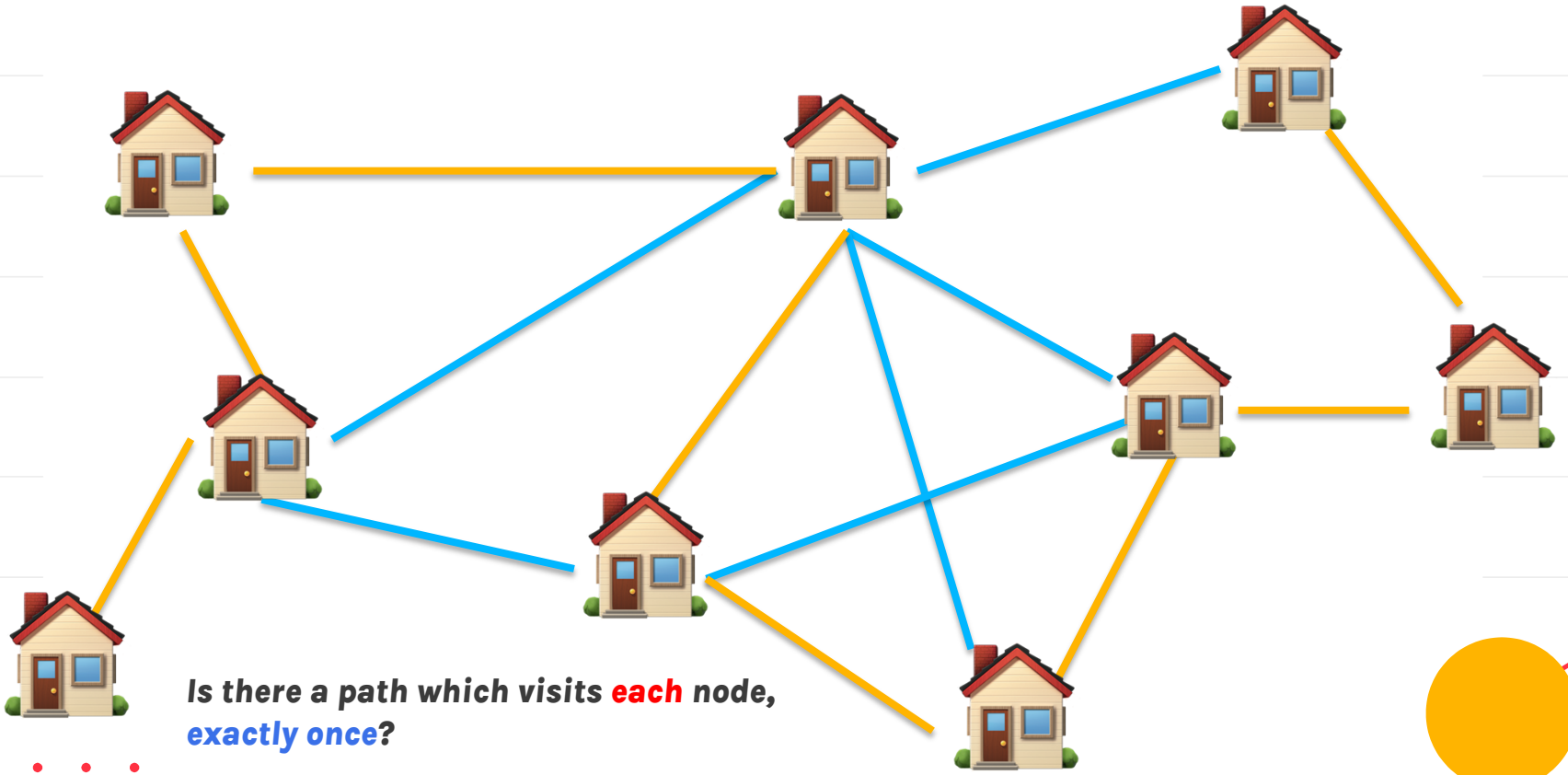
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



Is there a path which visits each node, exactly once?



Analiza Algoritmilor

NP-hard & NP-complete problems

Matei Popovici

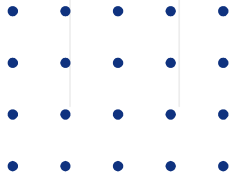


We often use algorithms for existing problems to solve new problems.

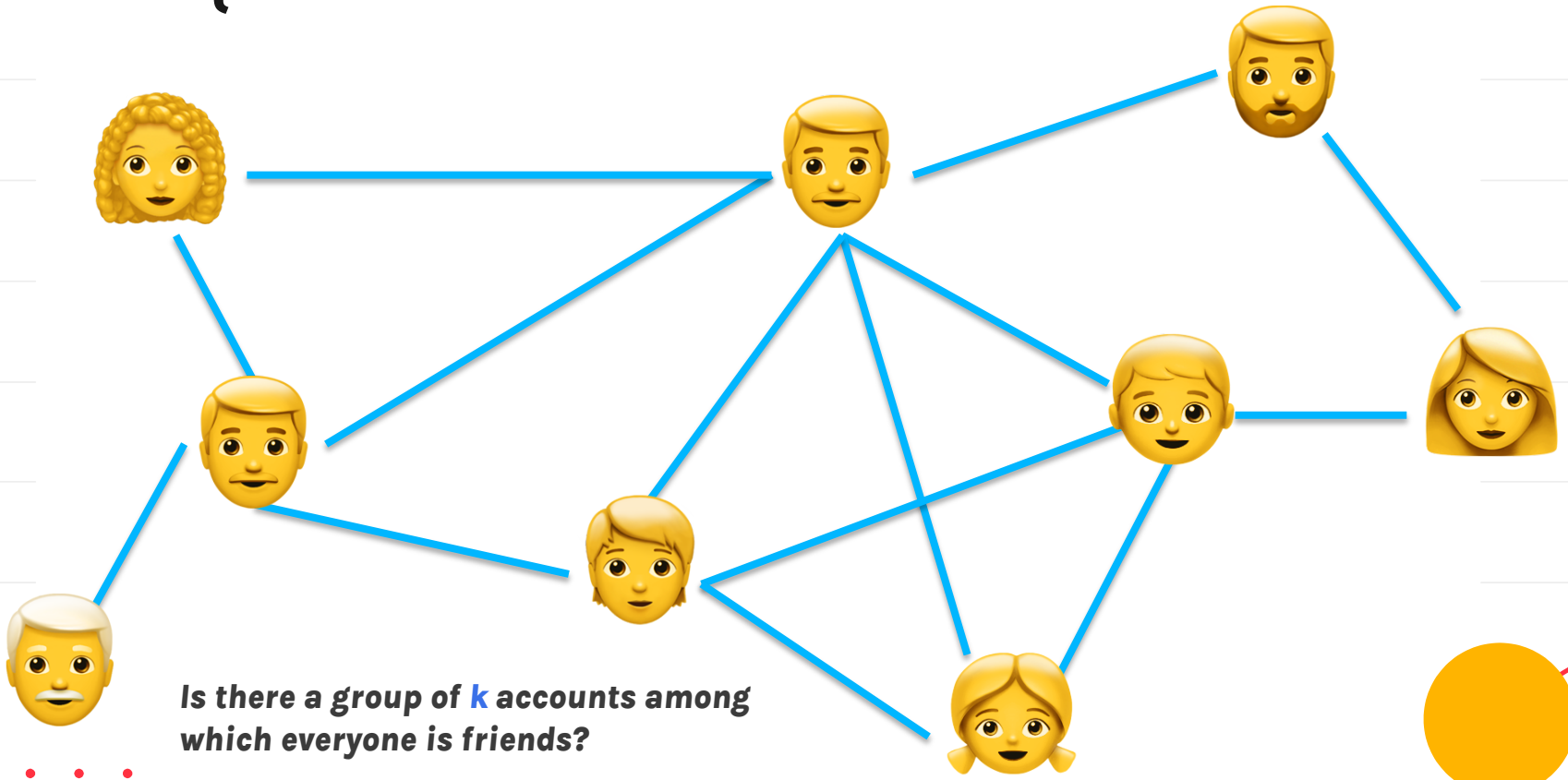
isPalindrome(L) = reverse(L) == L

max(L) = sort(last(L))

k-clique(G,k) = k-Vertex-Cover(complement(G),|V|-k)

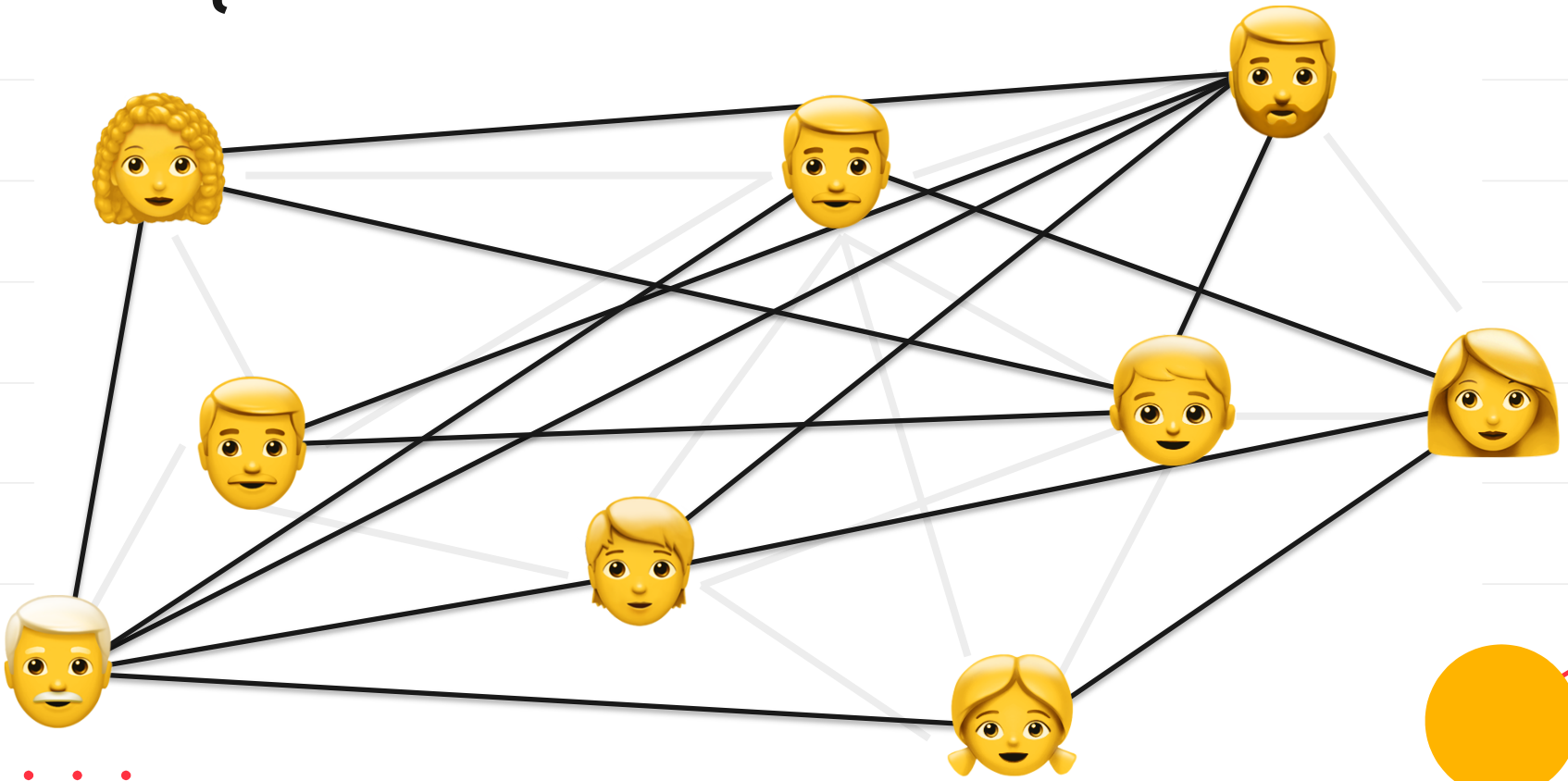


K-Clique reduction to k-Vertex-Cover

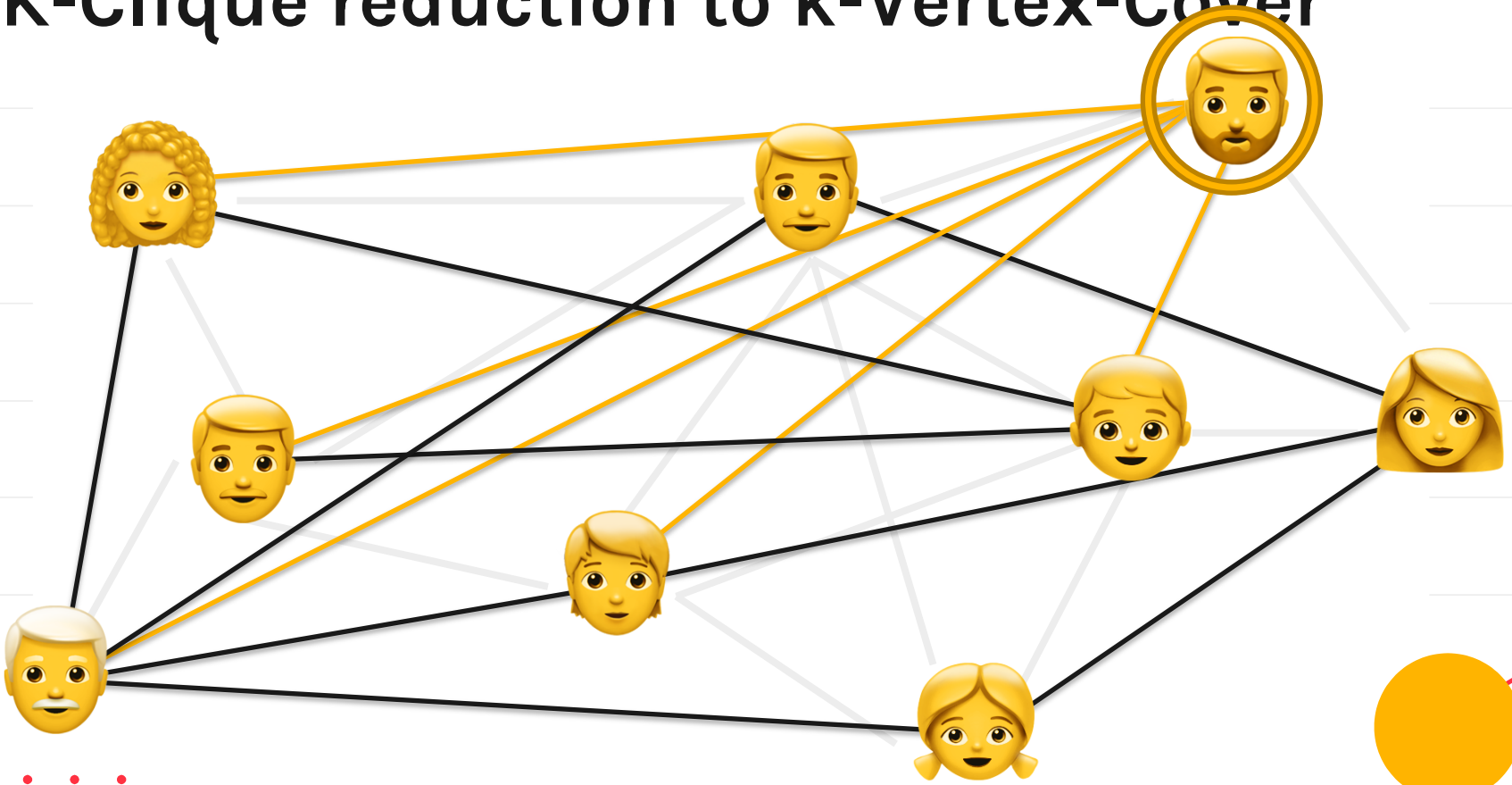


Is there a group of k accounts among which everyone is friends?

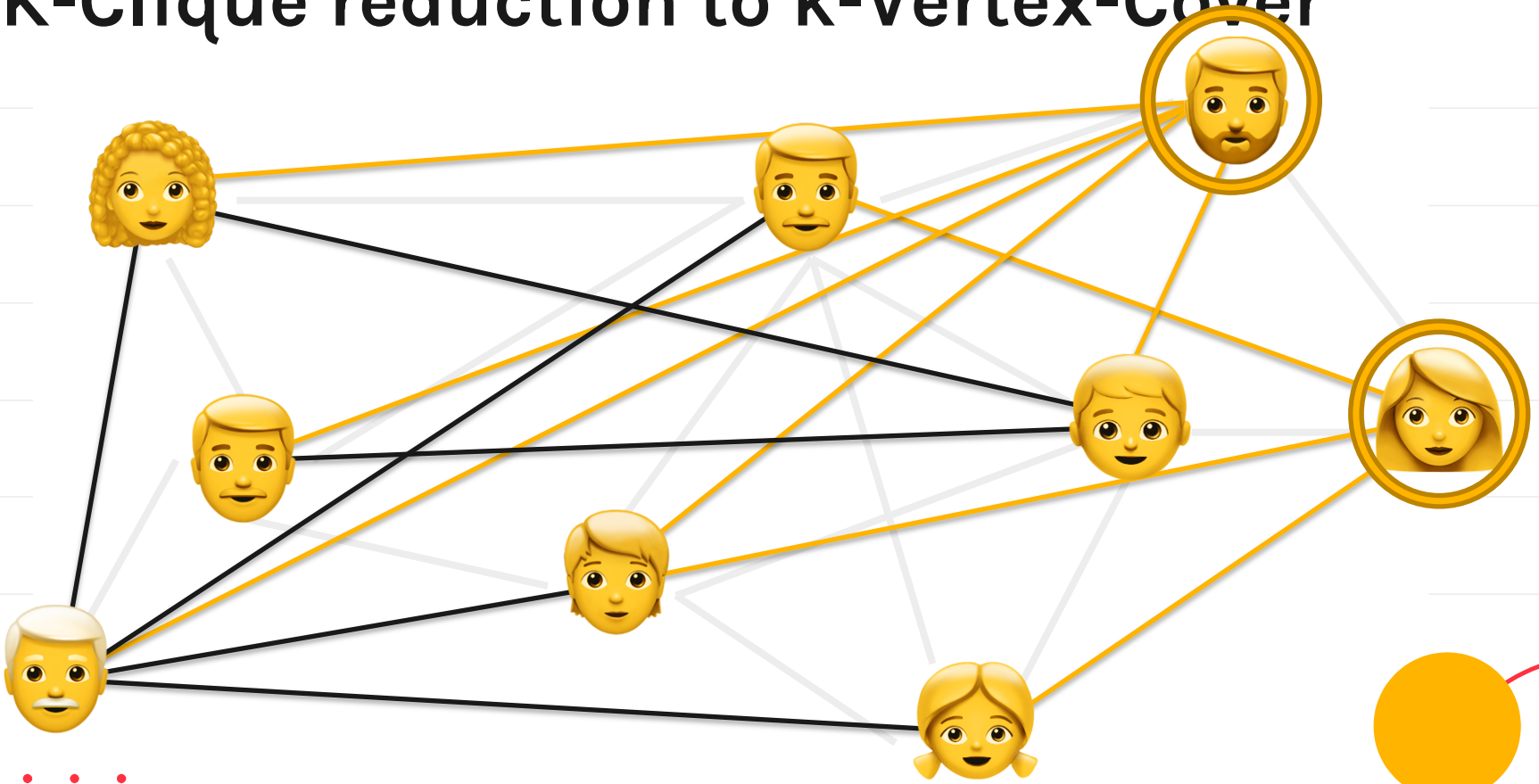
K-Clique reduction to k-Vertex-Cover



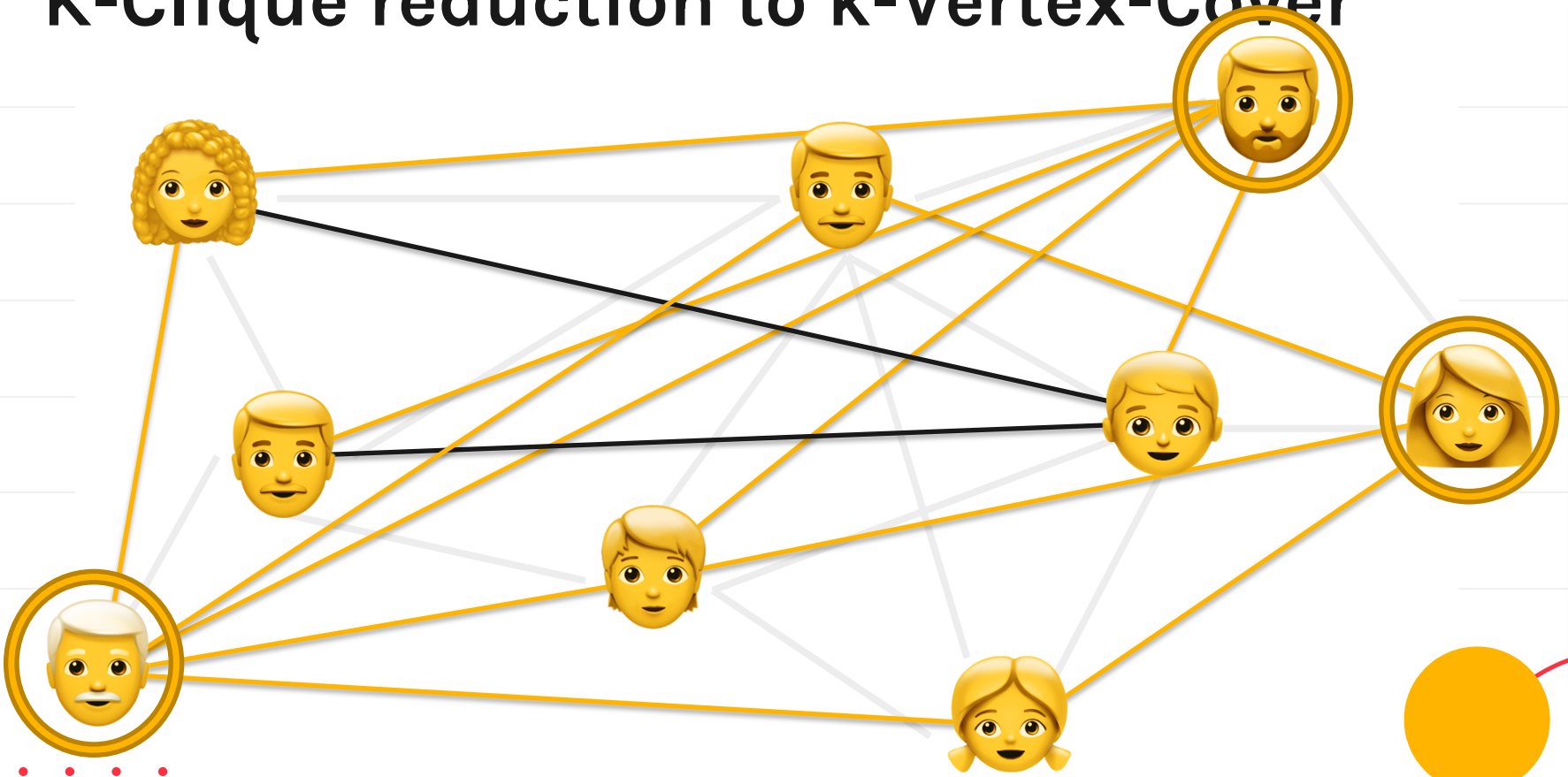
K-Clique reduction to k-Vertex-Cover



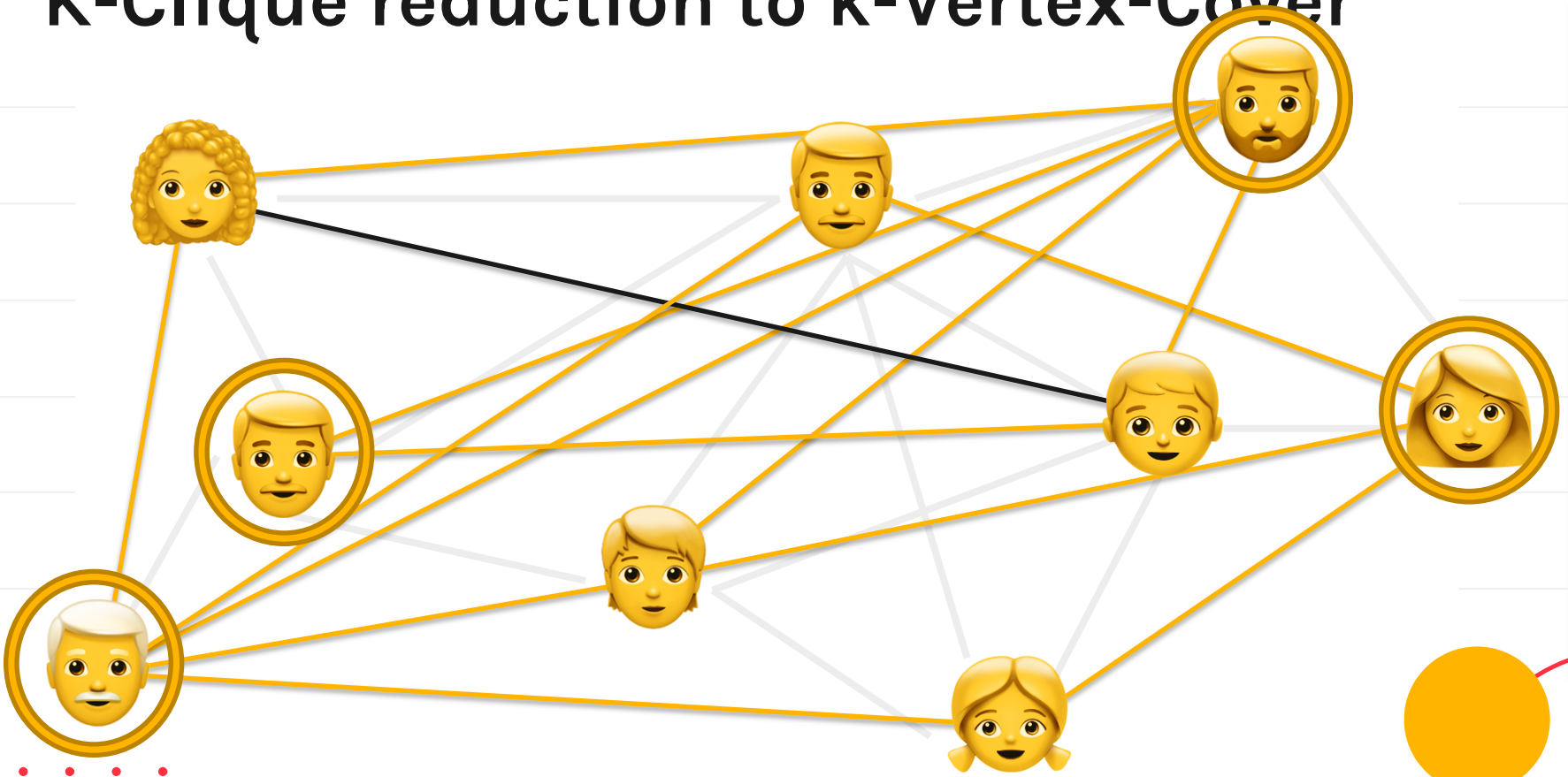
K-Clique reduction to k-Vertex-Cover



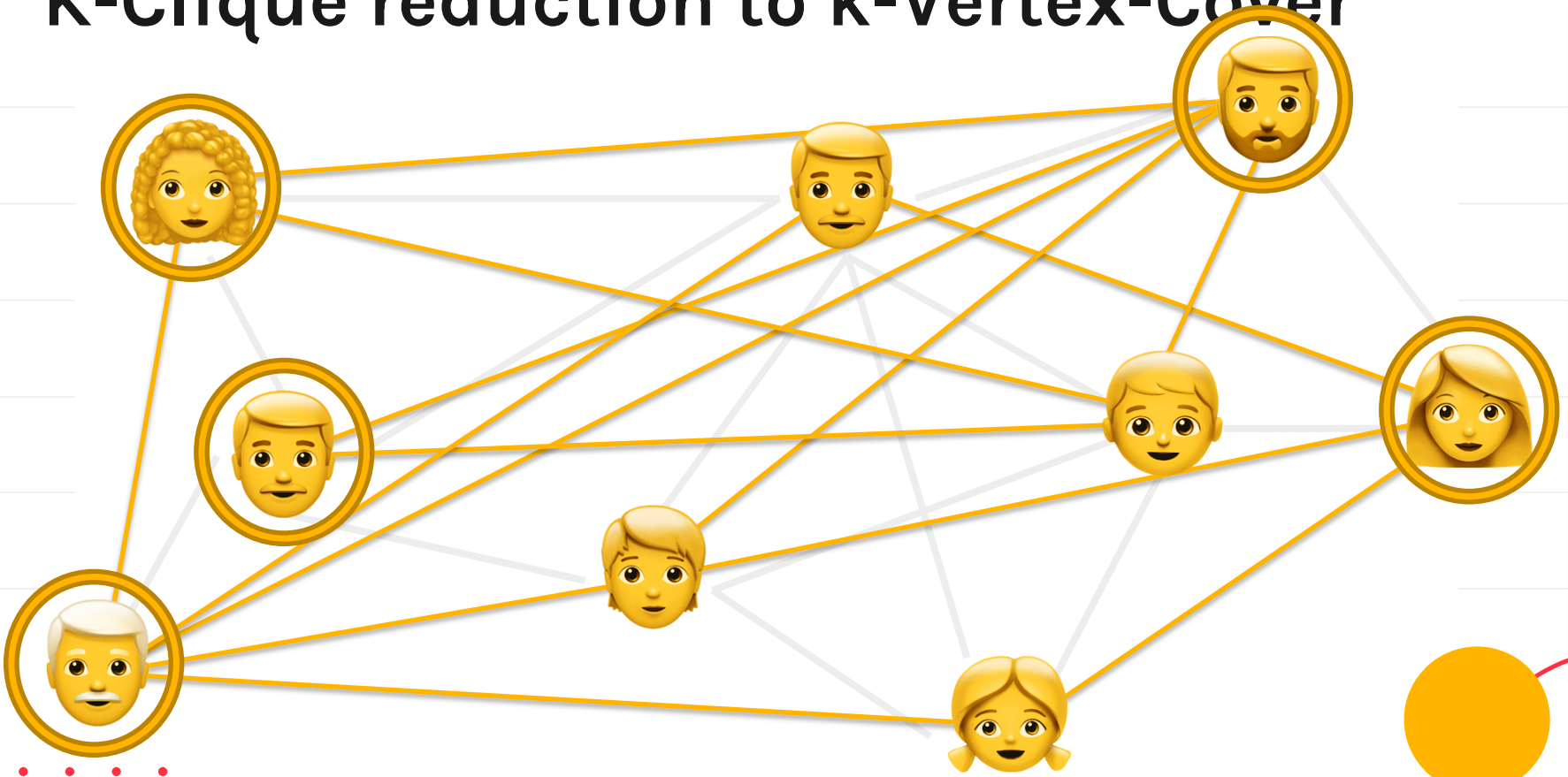
K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover



K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

Start with an instance $G=(V,E)$ and k for the problem k-Clique

Build a new graph $G'=(V',E')$ as follows:

- $V = V'$ – the new graph has the same nodes as the original one
- $E' = \text{complement}(E)$ – for each pair of nodes $u,v \in V$ such that $(u,v) \notin E$, add an edge $(u,v) \in E$.

If G has a clique C of size k, then G' has a vertex covering $V \setminus C$ of size $|V|-k$.

K-Clique reduction to k-Vertex-Cover

K-Clique $\leq_p$ k-Vertex-Cover

Start with an instance $G=(V,E)$ and k for the problem k-Clique

Build a new graph $G'=(V',E')$ as follows:

- $V = V'$ – the new graph has the same nodes as the original one
- $E' = \text{complement}(E)$ – for each pair of nodes $u,v \in V$ such that $(u,v) \notin E$, add an edge $(u,v) \in E$.

G has a clique C of size k, $\Leftrightarrow$ G' has a vertex covering $V \setminus C$ of size $|V|-k$.

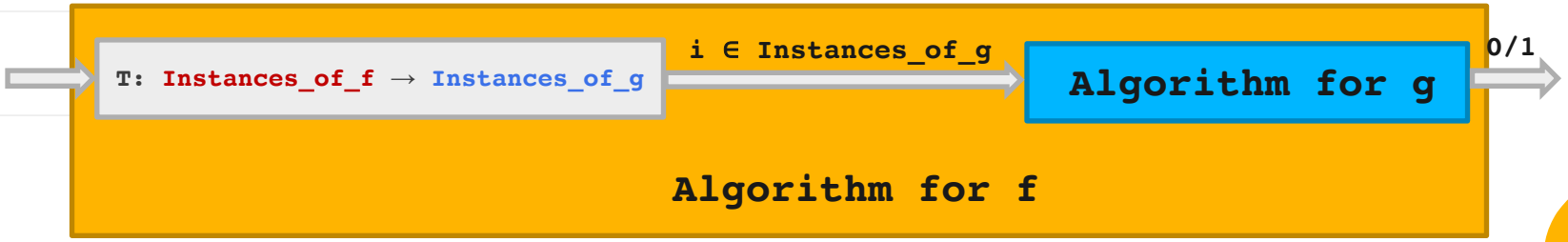
Polynomial reduction $f \leq_p g$

If we have an algorithm for g, then we can also solve f.

Suppose i is an instance for problem f.

We find a transformation `T: Instances_of_f → Instances_of_g`

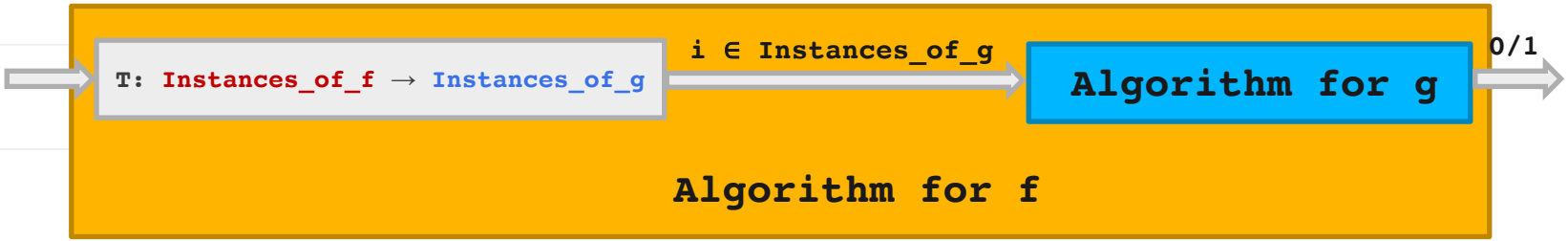
such that `f(i) = 1 ⇔ T(f(i)) = 1`



Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`

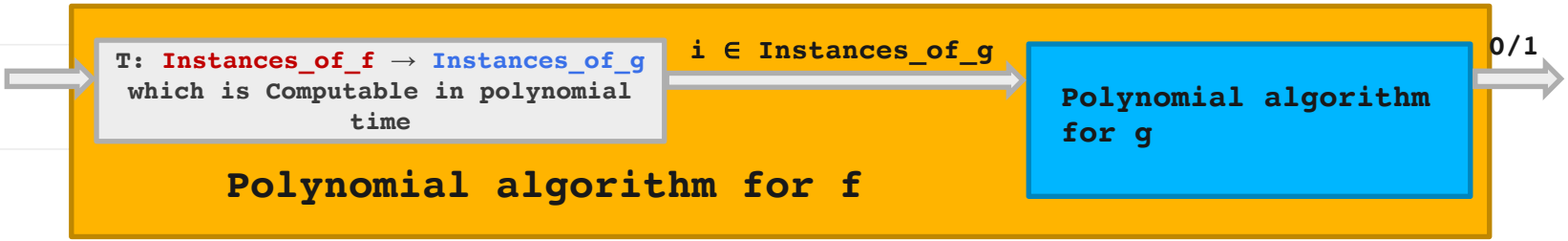


If T is computable in polynomial time, then...

Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`



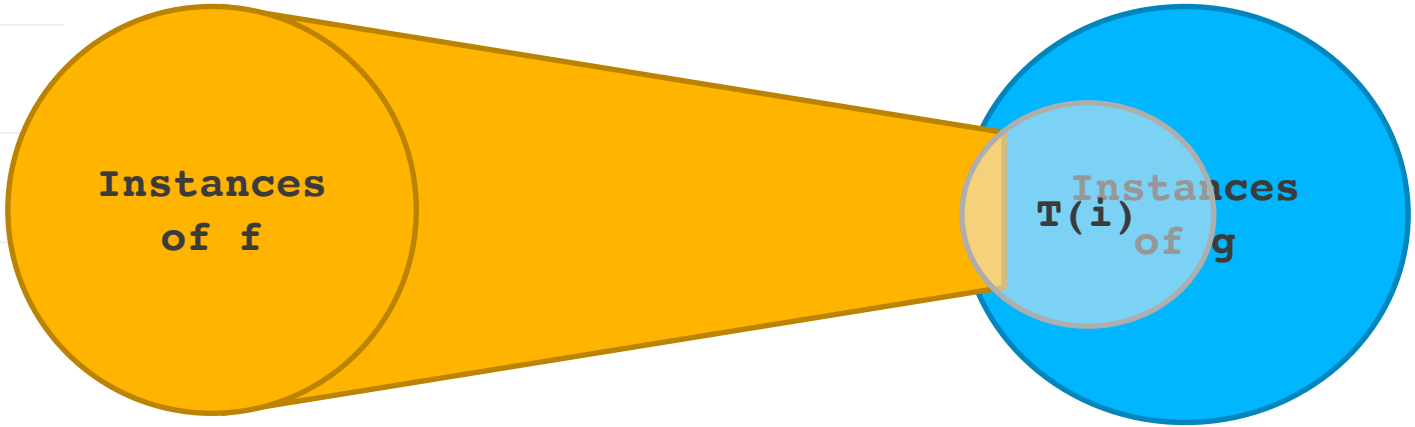
If T is computable in polynomial time, then...

Polynomial reduction $f \leq_p g$

We find a transformation `T: Instances_of_f → Instances_of_g`

such that `f(i) = 1 ⇔ T(f(i)) = 1`

T is not total nor inversible
(not all instances of g have a corresponding instance of f)



Polynomial reduction $f \leq_p g$

Proposition. *Let* $f \leq_p g$ *. If* $g \in P$ *then also* $f \in P$.

$$A \Rightarrow B$$

$$\neg B \Rightarrow \neg A$$

Proposition. *Let* $f \leq_p g$ *. If* $f \notin P$ *then also* $g \notin P$.

There is no known method to show $g \notin P$.

A few words about SAT

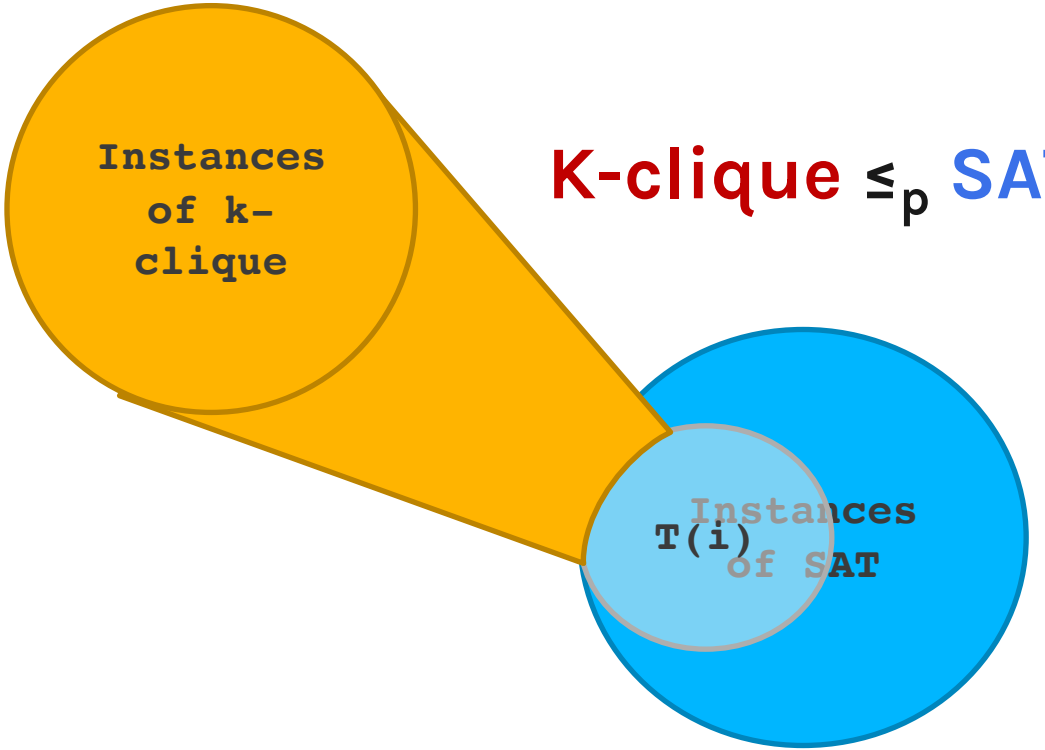
Known as the SATISFIABILITY problem

$(\neg X \vee \neg Y \vee Z) \wedge (X \vee \neg Y \vee \neg Z) \wedge (\neg X \vee \neg Y \vee Z)$

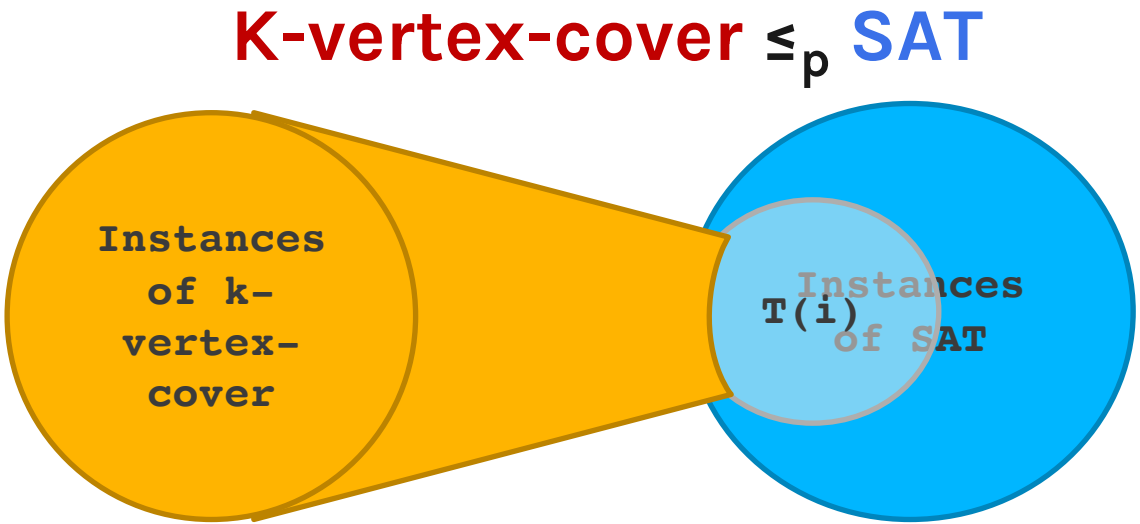
There exists some truth-value assigned to x, y, z ***such that the proposition is true?***

SAT has a special property:

A few words about SAT

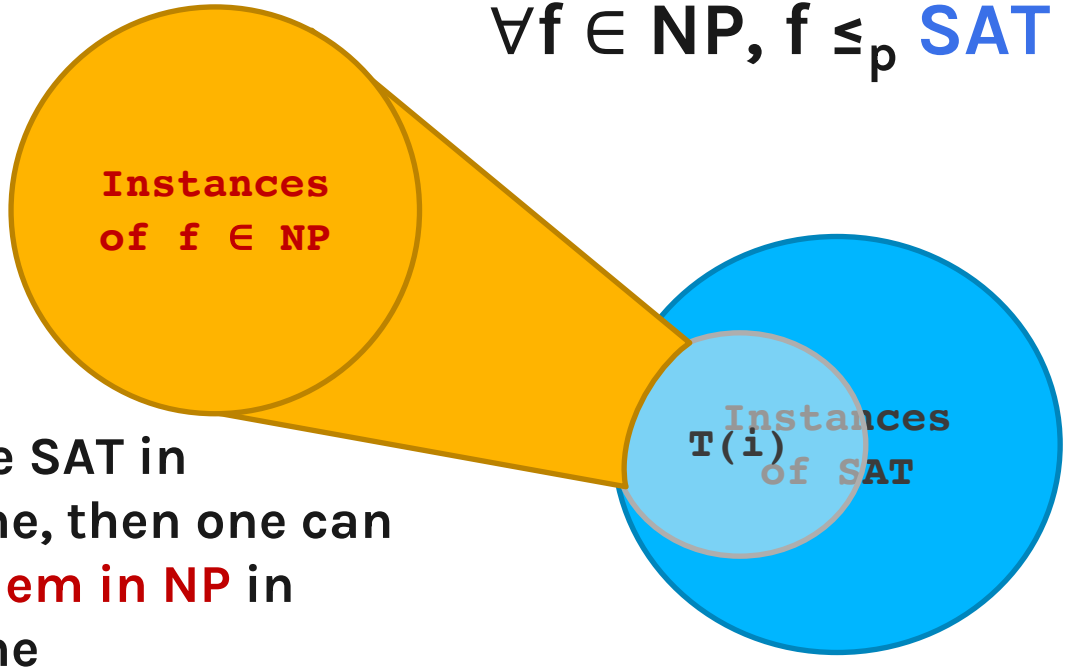


A few words about SAT



A few words about SAT

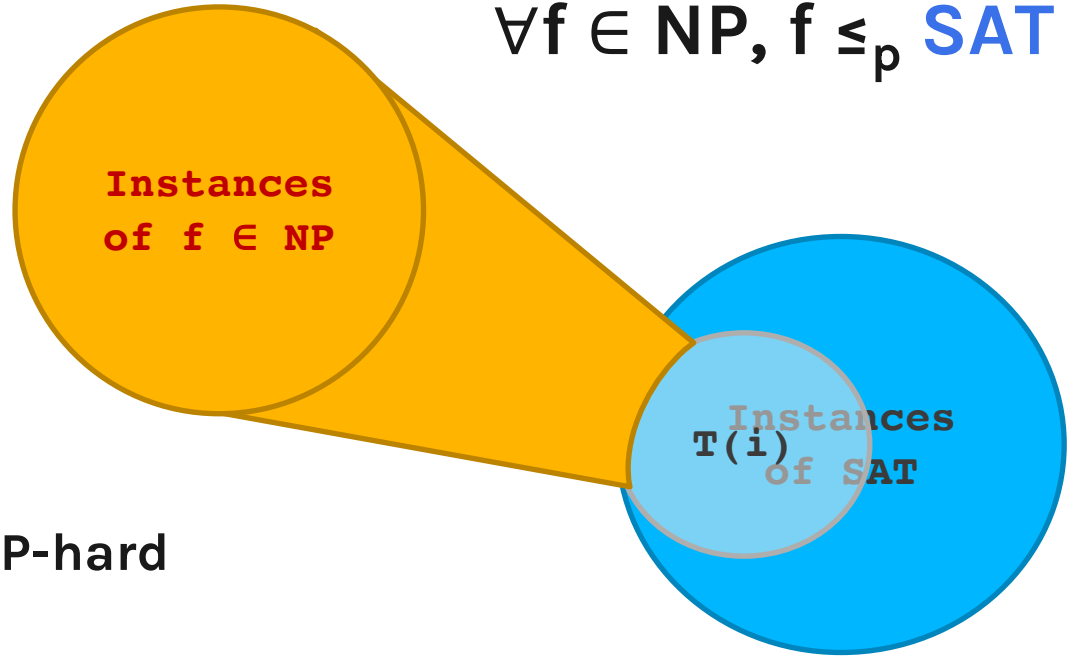
$\forall f \in NP, f \leq_p SAT$



If one can solve SAT in polynomial time, then one can solve any problem in NP in polynomial time

A few words about SAT

$\forall f \in NP, f \leq_p SAT$



SAT is called NP-hard

NP-hard and NP-complete problems

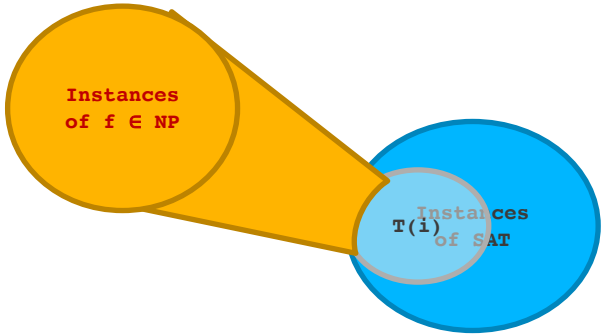
Def. A problem g is called NP-hard iff $\forall f \in NP, f \leq_p g$.

NP-hard problems are problems that are at least as hard as any problem in NP.

Prop. SAT is NP-hard. *Prop. SAT* $\in$ *NP.*

Def. A problem which is NP-hard and a member of NP is called NP-complete.

Prop. SAT is NP-complete.



NP-hard and NP-complete problems

Suppose h is a problem from NP and SAT $\leq_p$ ***h.***

If we find a polynomial algorithm for h, we also have one for SAT.

If we find a polynomial algorithm for SAT, we have one for all problems in NP.

$\forall f \in NP, f \leq_p SAT \leq_p h.$ ***Hence h is also NP-hard.***

Complexity Theory – classifies problems into:

Problems that can be solved in deterministic polynomial time — **The class P**

Problems that can be solved in NON-deterministic polynomial time — **The class NP**

Problems that can be solved in deterministic exponential time

Complexity Theory – classifies p into:

We strongly believe that NP-complete problems are not in P.

We could prove `f` `is NP-complete`

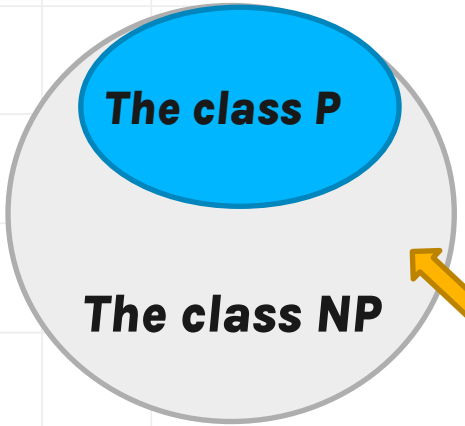
Problems that can be solved in NON-deterministic polynomial time

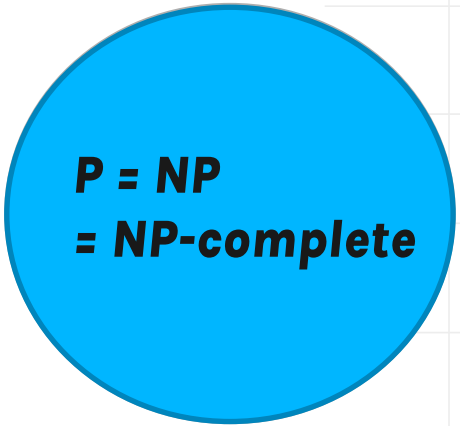
NP-complete problems

Problems that can be solved in deterministic exponential time

Do not know how to prove $f \notin P$

There are two possible scenarios





NP-complete problems

We believe this scenario is not likely!!

Recap

Suppose you cannot write a polynomial algorithm for **f**.

You show that ***most likely*** one does not exist.

You show that ***f is NP-hard***.

You select (from a textbook), a problem g that ***is NP-hard (maybe SAT)***

You show that $g \leq_p f$.

You may also want to show that ***f*** is in NP.

If this doesn't work, maybe ***f*** is even more difficult.

What to remember

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If anyone finds a polynomial algorithm for any of them, P = NP.

It is unlikely that you will discover a new NP-complete problem.

It is very likely that you will need to solve an existing NP-complete problem, or a slight variation for it.

Today, there are very many good algorithms that solve SAT (in the worst-case, exponentially). They are called SAT solvers.

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

Today, there are very many good algorithms that solve SAT (in the worst-case, exponentially). They are called SAT solvers.

A lot of engineers use $f \leq_p SAT$, to solve their problems ***f*** with a SAT solver.

Not to confuse with $SAT \leq_p f$, which shows that ***f*** is NP-hard.

Polynomial reductions are a very practical skill for understanding NP-completeness and how to approach algorithm-writing.

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If your problem is an ***optimisation*** problem (e.g. ***min***-Vertex-Cover)

It has an ***equivalent*** decision problem (k-Vertex-Cover)

If we can solve the decision variant, we can solve the optimisation problem

```
min-Vertex-Cover-alg(G) =
        for k = 0,|V|
              if k-Vertex-Cover-alg(k,G)
                return 1
```

What to remember:

A more careful look at ***SAT*** – the first-known NP-complete problem (early 1970)

More than 3000 problems are known today to be NP-complete.

If your problem is an ***optimisation*** problem (e.g. ***min***-Vertex-Cover)

It has an ***equivalent*** decision problem (k-Vertex-Cover)

and the decision variant is NP-complete:

search approximation algorithms
search random algorithms

What next, in this lecture

Reductions, reductions, reductions…

We will show how some problems f reduce to SAT - $f \leq_p SAT$
We can then use SAT solvers for them.
This approach makes sense only if we believe $f \notin P$, otherwise inefficient.

We will prove some problems g are NP-hard by picking f that is NP-hard and showing $f \leq_p g$.

Usually, both efforts are necessary.



Analiza Algoritmilor

SAT and a few polynomial reductions

Matei Popovici

Boolean sentences

Start with a set of propositions

`Props = {p, q, r, … , x, y, z, …}`

Sometimes they are called variables.
It is not a mistake but it is awkward.

Boolean sentences

Start with a set of propositions

$\mathtt{Props} = \{p, q, r, \ldots, x, y, z, \ldots\}$

A proposition can be true or false.

An interpretation tells us which propositions are true and which are false.

$\mathtt{I} : \mathtt{Props} \rightarrow \{0,1\}$ or

$\mathtt{I} \subseteq \mathtt{Props}$

Propositions in $\mathtt{I}$ are held to be true and the rest – false.

Boolean sentences

A sentence φ is a proposition ($\varphi = p$)

or a conjunction ($\varphi = \varphi_1 \wedge \varphi_2$)

or a disjunction ($\varphi = \varphi_1 \vee \varphi_2$)

or a negation ($\varphi = \neg\varphi_1$)

or parentheses ($\varphi = (\varphi_1)$)

parentheses constructors per se, but they help us disambiguate propositions just like in arithmetic

Boolean sentences

The sentence $\varphi_1 \rightarrow \varphi_2$ means $\neg\varphi_1 \vee \varphi_2$

The sentence $\varphi_1 \leftrightarrow \varphi_2$ means $(\varphi_1 \rightarrow \varphi_2) \wedge (\varphi_2 \rightarrow \varphi_1)$

Conjunctive Normal Form (CNF)

The sentence φ is in CNF iff

$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$ where

$C_i = L_{i1} \wedge L_{i2} \wedge \ldots \wedge L_{ik}$ where

$L_{ij} = p$ or $\neg p$ where p is any proposition

Conjunctive Normal Form (CNF) vs arbitrary form

$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$

Representing a sentence requires a tree

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

We can transform this sentence in CNF

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg(q \vee \neg r))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q \wedge r)))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \\ (p \vee \neg r) \\ (\neg p \vee \neg q) \\ (\neg p \vee r)$$

each line is a conjunction

Conjunctive Normal Form (CNF) vs arbitrary form

$$
\begin{aligned}
\varphi = (\ & p \vee q \quad\ \ ) \\
(\ & p \vee \quad\ \ \neg r) \\
(\neg & p \vee \neg q \quad\ ) \\
(\neg & p \vee \quad\ \ r)
\end{aligned}
$$

each line is a conjunction

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = \begin{matrix} 1 & 1 & 0 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \\ -1 & 0 & 1 \end{matrix}$$

each element L_{ij} tells us if proposition j appears positive, negated, or not at all, in clause i

Boolean sentences

A sentence is called **satisfiable** iff there **exists** an interpretation (for each proposition of the formula), which makes the formula true.

Is this formula **satisfiable**?

$$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

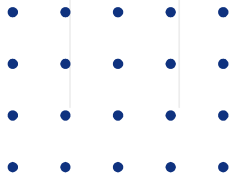
Hard to ***eyeball*** the answer



Boolean sentences

Is this formula **satisfiable**?

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$



Boolean sentences

Is this formula satisfiable?

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

We search for an interpretation which makes all clauses true

In each clause at least one literal must be true

SAT:

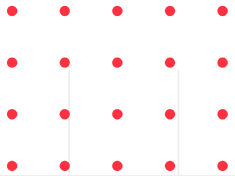
Given a sentence in CNF, is it satisfiable?

SAT ∈ *NP*

```
SAT-algorithm(phi, n, m):
I = array of size m.

for p = 1,m:
    I[p] = choice({-1,1})

for i = 1,n:
    current_clause = false
    for l = 1,m:
        if phi[i][j]*I[j] > 0:
            current_clause = true
    if !current_clause:
        fail
success
```



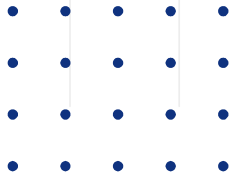
SAT:

Given a sentence in CNF, is it satisfiable?

SAT is ***NP-complete***

Cook-Levine Theorem (1970)





How is SAT useful to us?

usage 1

We want to solve a difficult problem f, for which we do not know an algorithm:

- We find a transformation witnessing $f \leq_p SAT$
  We transform the input to our problem in a SAT formula and run a SAT solver.

usage 2

We want to prove that a problem f, is NP-hard.

- We find a transformation witnessing $SAT \leq_p f$
  - We can also choose other NP-hard problems, not necessarily SAT.

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$T_{11} + T_{12} + \dots + T_{1n1} \leq c_1$$
$$T_{21} + T_{22} + \dots + T_{2n2} \leq c_2$$
$$\dots$$
$$T_{k1} + T_{k2} + \dots + T_{knk} \leq c_k$$

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$x_1 + x_2 + x_3 = 1$ *exactly one variable is 1*

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)$$

not in CNF

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)$$

$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee \text{rest}$

$$(x_1 \vee \text{rest}) \wedge (\neg x_2 \vee \text{rest}) \wedge (\neg x_3 \vee \text{rest})$$

$(x_1 \vee \text{rest}) \wedge$
$(\neg x_2 \vee \text{rest}) \wedge$
$(\neg x_3 \vee \text{rest})$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))$$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))$$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest}) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest}) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest})$$

$$((x_1 \vee \neg x_1 \vee \text{rest}) \wedge (x_1 \vee x_2 \wedge \text{rest}) \wedge (x_1 \vee \neg x_3 \wedge \text{rest})) \wedge$$
$$((\neg x_2 \vee \neg x_1 \vee \text{rest}) \wedge (\neg x_2 \vee x_2 \vee \text{rest}) \wedge (\neg x_2 \vee \neg x_3 \vee \text{rest})) \wedge$$
$$((\neg x_3 \vee \neg x_1 \vee \text{rest}) \wedge (\neg x_3 \vee x_2 \vee \text{rest}) \wedge (\neg x_3 \vee \neg x_3 \vee \text{rest}))$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(x_1 \vee x_2 \vee \text{rest}) \wedge \\
&(x_1 \vee \neg x_3 \vee \text{rest})) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(\neg x_2 \vee x_2 \vee \text{rest}) \wedge \\
&(\neg x_2 \vee \neg x_3 \vee \text{rest})) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(\neg x_3 \vee x_2 \vee \text{rest}) \wedge \\
&(\neg x_3 \vee \neg x_3 \vee \text{rest}))
\end{aligned}
$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(x_1 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(x_1 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_2 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_2 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_3 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_3 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)))
\end{aligned}
$$

$$((x_1 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(x_1 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(x_1 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge$$
$$((\neg x_2 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge$$
$$((\neg x_3 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)))$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee \neg x_1) \wedge (x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_1 \vee x_3)) \wedge \\
&((x_1 \vee x_2 \vee \neg x_1) \wedge (x_1 \vee x_2 \vee \neg x_2) \wedge (x_3 \vee x_1 \vee x_2)) \wedge \\
&((x_1 \vee \neg x_3 \vee \neg x_1) \wedge (x_1 \vee \neg x_3 \vee \neg x_2) \wedge (x_1 \vee \neg x_3 \vee x_3)) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_1 \vee x_3)) \wedge \\
&((\neg x_2 \vee x_2 \vee \neg x_1) \wedge (\neg x_2 \vee x_2 \vee \neg x_2) \wedge (\neg x_2 \vee x_2 \vee x_3)) \wedge \\
&((\neg x_2 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_3)) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_1 \vee x_3)) \wedge \\
&((\neg x_3 \vee x_2 \vee \neg x_1) \wedge (\neg x_3 \vee x_2 \vee \neg x_2) \wedge (\neg x_3 \vee x_2 \vee x_3)) \wedge \\
&((\neg x_3 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_3 \vee x_3)))
\end{aligned}
$$

$$(x_1 \vee \neg x_1 \vee \neg x_1) \wedge (x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_1 \vee x_3) \wedge$$
$$(x_1 \vee x_2 \vee \neg x_1) \wedge (x_1 \vee x_2 \vee \neg x_2) \wedge (x_3 \vee x_1 \vee x_2) \wedge$$
$$(x_1 \vee \neg x_3 \vee \neg x_1) \wedge (x_1 \vee \neg x_3 \vee \neg x_2) \wedge (x_1 \vee \neg x_3 \vee x_3) \wedge$$
$$(\neg x_2 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_1 \vee x_3) \wedge$$
$$(\neg x_2 \vee x_2 \vee \neg x_1) \wedge (\neg x_2 \vee x_2 \vee \neg x_2) \wedge (\neg x_2 \vee x_2 \vee x_3) \wedge$$
$$(\neg x_2 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_3) \wedge$$
$$(\neg x_3 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_1 \vee x_3) \wedge$$
$$(\neg x_3 \vee x_2 \vee \neg x_1) \wedge (\neg x_3 \vee x_2 \vee \neg x_2) \wedge (\neg x_3 \vee x_2 \vee x_3) \wedge$$
$$(\neg x_3 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_3 \vee x_3)$$

Original formula:

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3).$$

CNF transformation is **exponential in size** in the worst case

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$(\neg x_1 \vee \neg x_2) \wedge$ *one of* x_1, x_2 *is* ***zero****, and*
$(\neg x_1 \vee \neg x_3) \wedge$ *one of* x_1, x_3 *is* ***zero****, and*
$(\neg x_1 \vee \neg x_4) \wedge$ *one of* x_1, x_4 *is* ***zero****, and*
$(\neg x_2 \vee \neg x_3) \wedge$ *one of* x_2, x_3 *is* ***zero*** *, and*
$(\neg x_2 \vee \neg x_4) \wedge$ *one of* x_2, x_4 *is* ***zero*** *, and*
$(\neg x_3 \vee \neg x_4)$ *one of* x_3, x_4 *is* ***zero*** *.*

$O(n^2)$ literals for n variables

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

at most one variable is 1

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) = \bigwedge_{i \neq j} (\neg x_i \vee \neg x_j)$$

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$x_1 + x_2 + x_3 = 1$ *exactly one variable is 1*

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

at most one variable is 1

$$x_1 + x_2 + x_3 = 1$$

$$\text{exactlyOne}(x_1, x_2, \ldots, x_n) =$$

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) \wedge$$

$$\text{atLeastOne}(x_1, x_2, \ldots, x_n)$$

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

$$x_1 + x_2 + x_3 = 1$$

at most one variable is 1

$$\text{exactlyOne}(x_1, x_2, \ldots, x_n) =$$

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) \wedge$$

$$(x_1 \vee x_2 \vee \ldots \vee x_n)$$

This set of inequalities is satisfiable

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

$$x_1 + x_2 + x_3 = 1$$

if and only if the following CNF formula is satisfiable

$$(\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_4) \wedge (\neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

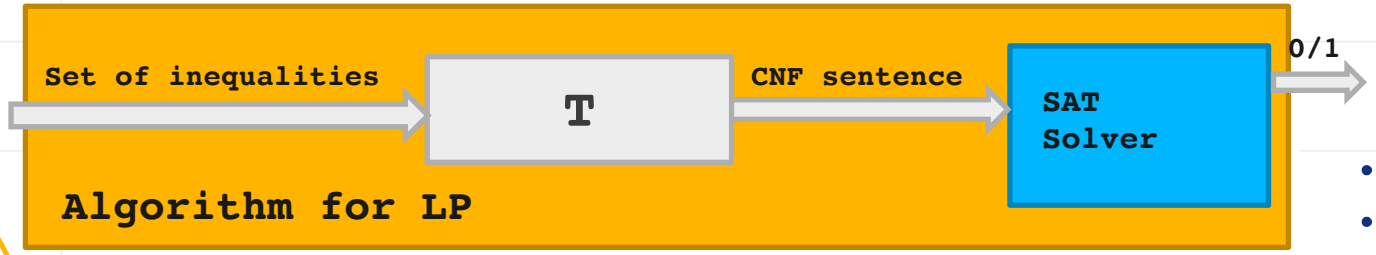
We also know of encodings for more general equations where c > 1

$$T_{11} + T_{12} + \ldots + T_{1n1} \leq c_1$$

They are more elaborate and go beyond the scope of this lecture

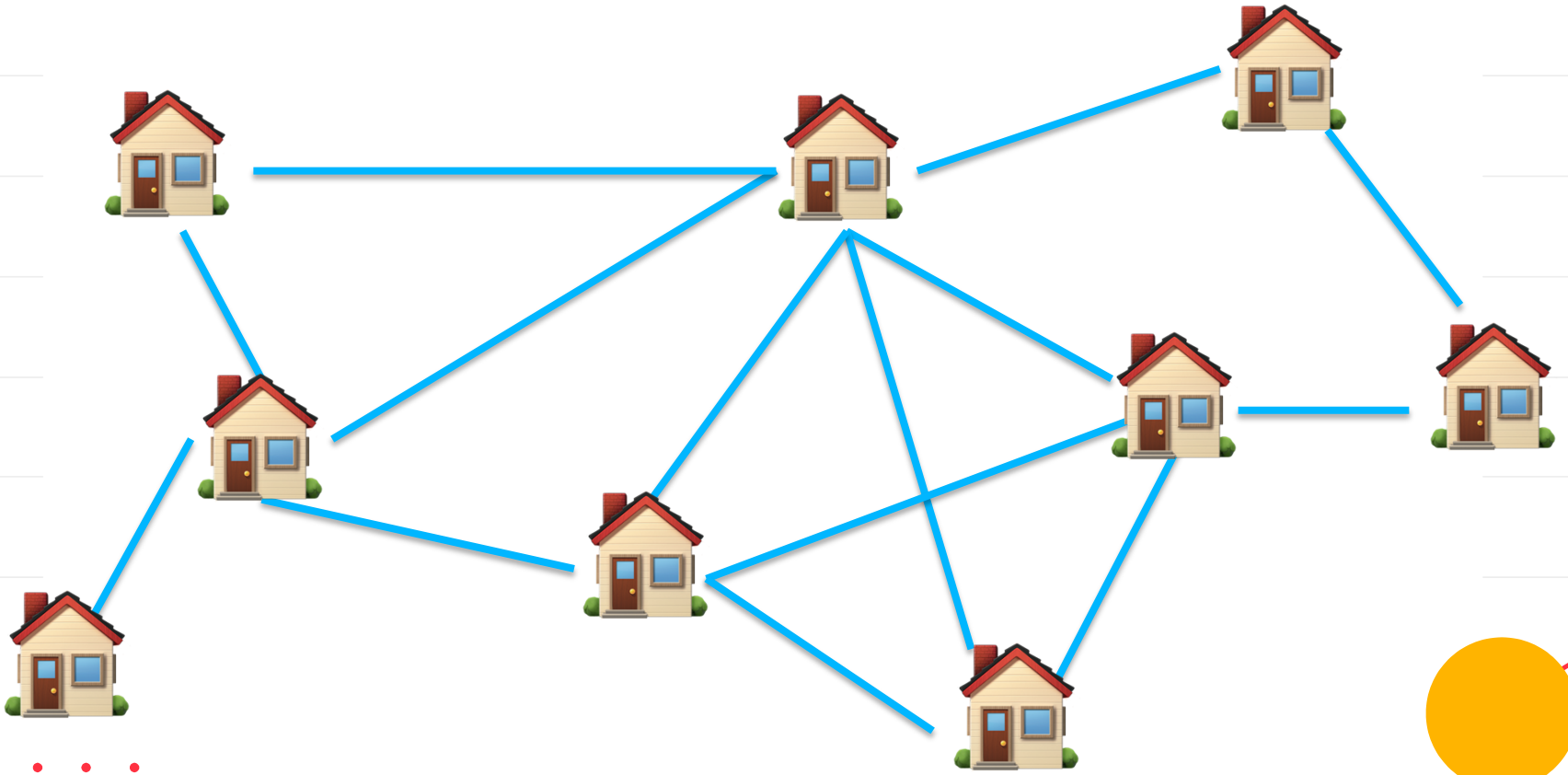
We must prove that the inequalities are satisfiable *iff* the CNF formula we have built is satisfiable.

$$T_{11} + T_{12} + \dots + T_{1n1} \leq c_1$$
$$T_{21} + T_{22} + \dots + T_{2n2} \leq c_2$$
$$\dots$$
$$T_{k1} + T_{k2} + \dots + T_{knk} \leq c_k$$

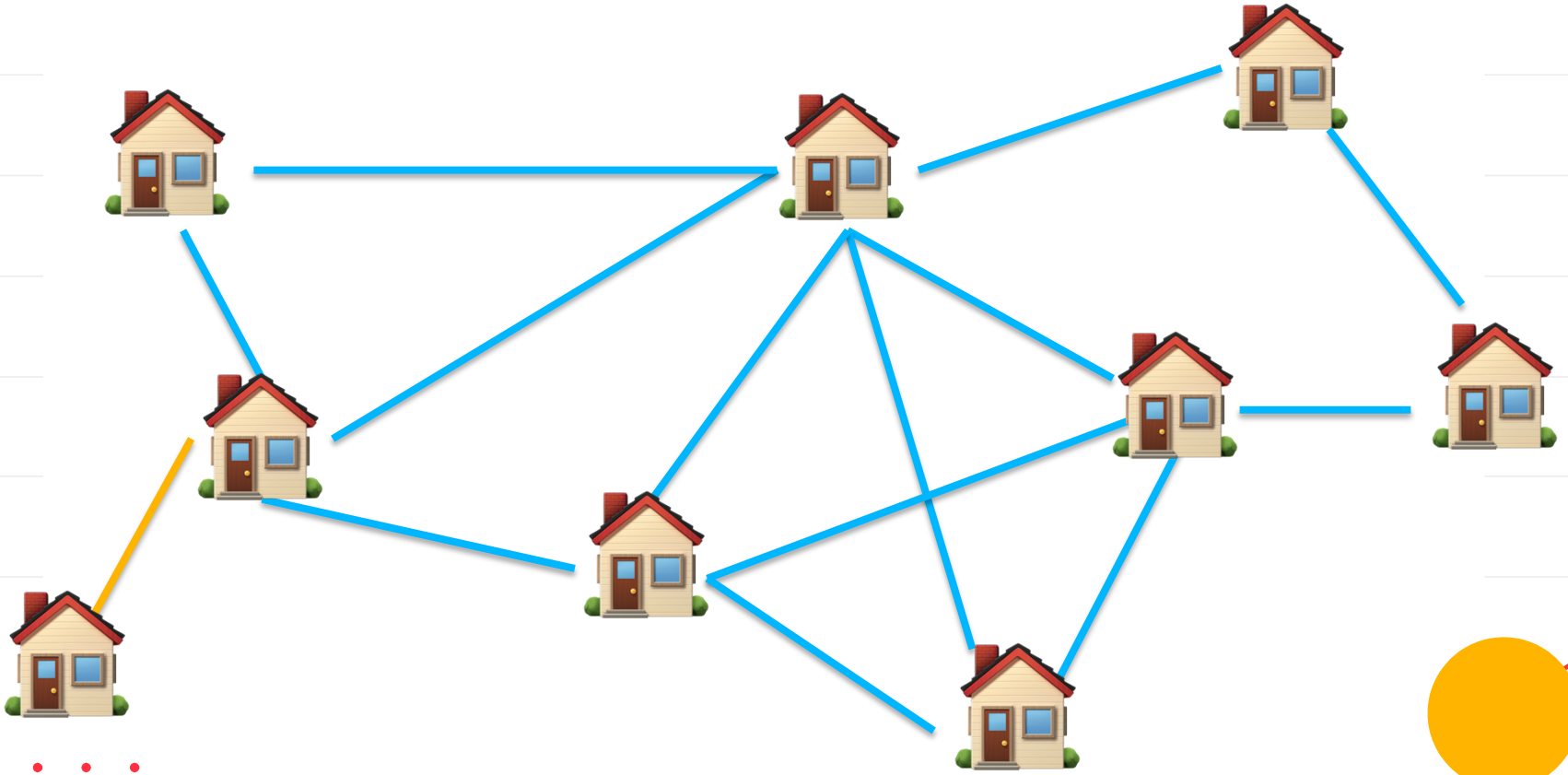


$LP \leq_p SAT$

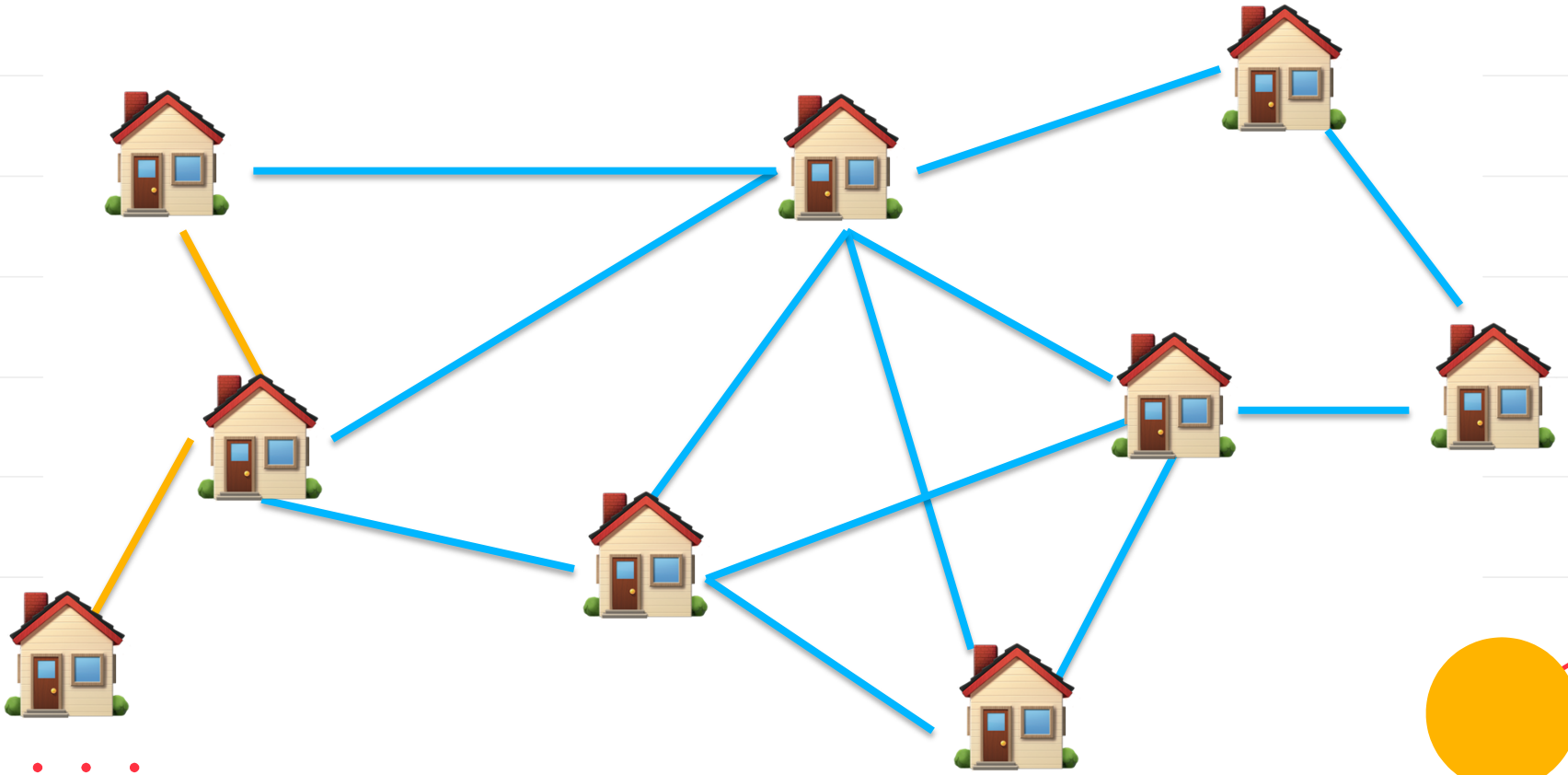
Practice: Hamiltonian Path



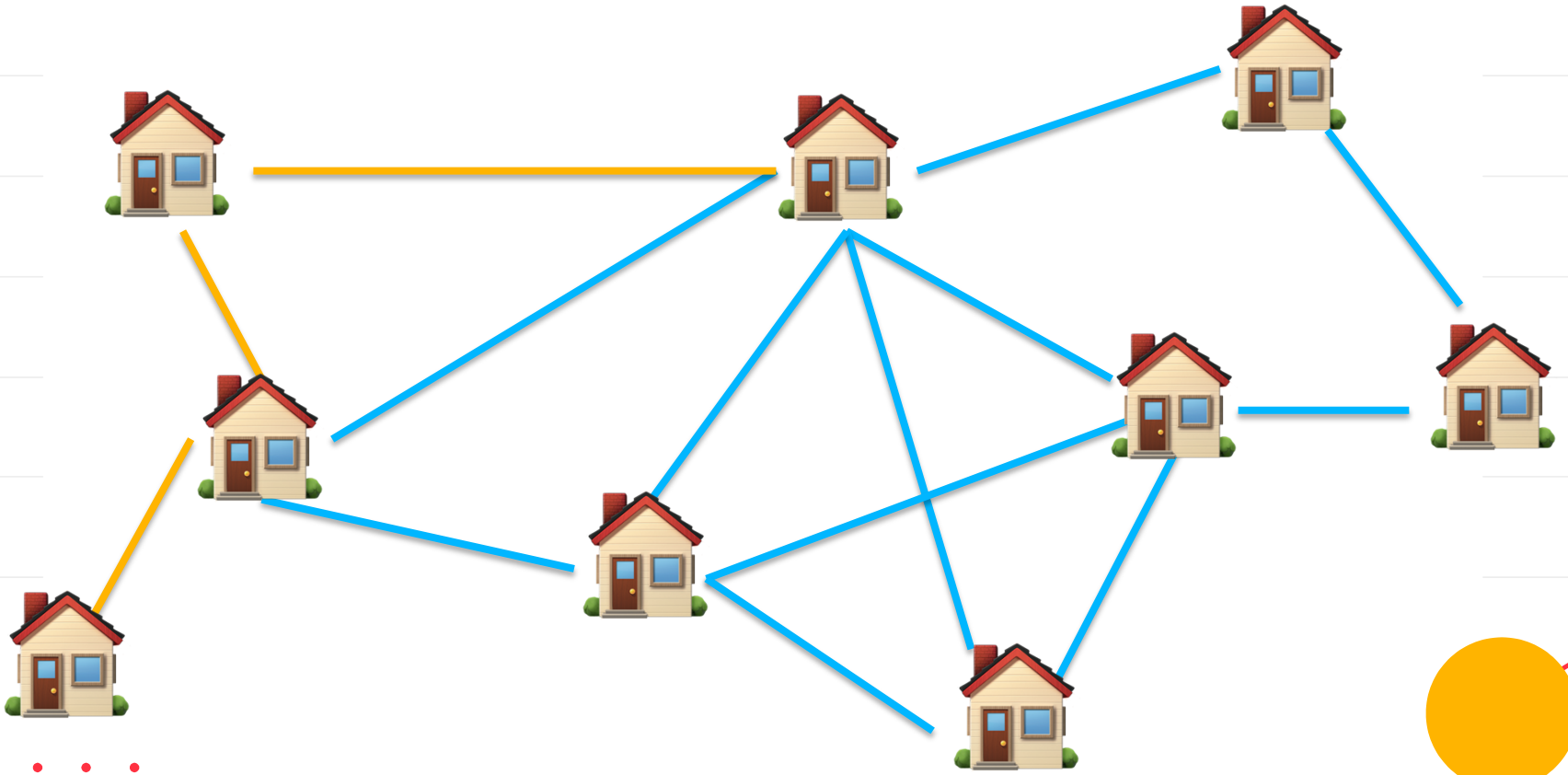
Practice: Hamiltonian Path



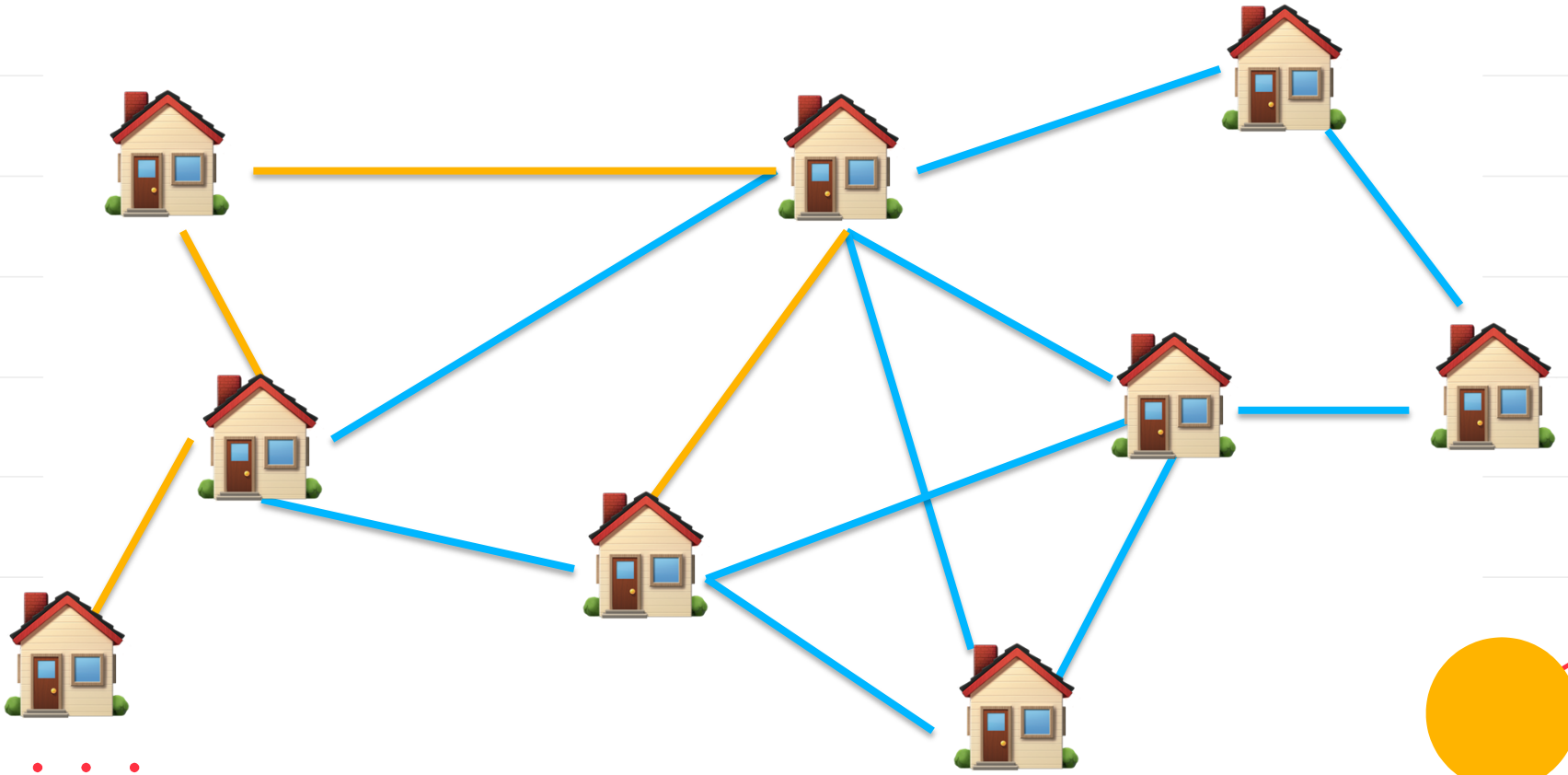
Practice: Hamiltonian Path



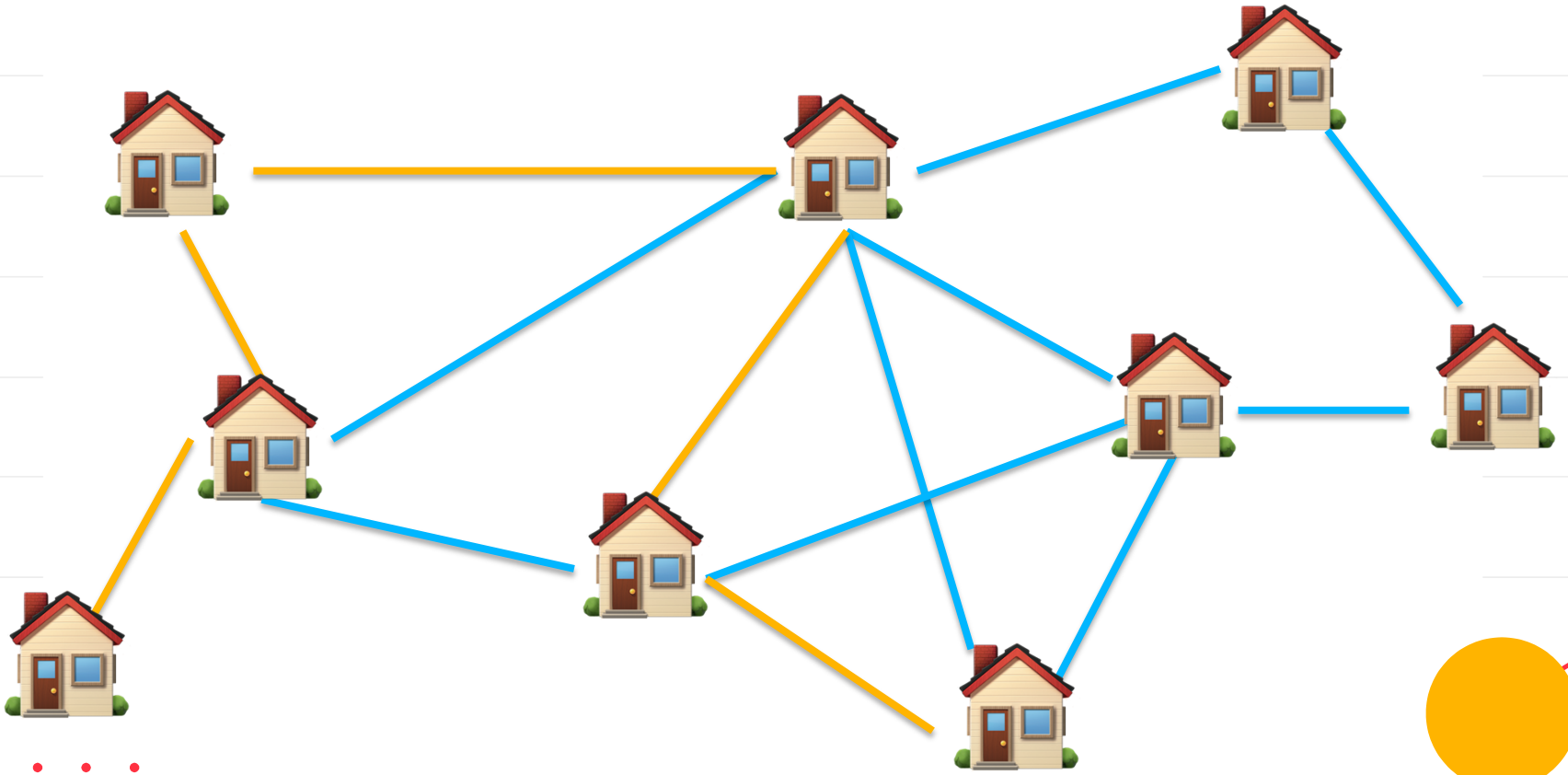
Practice: Hamiltonian Path



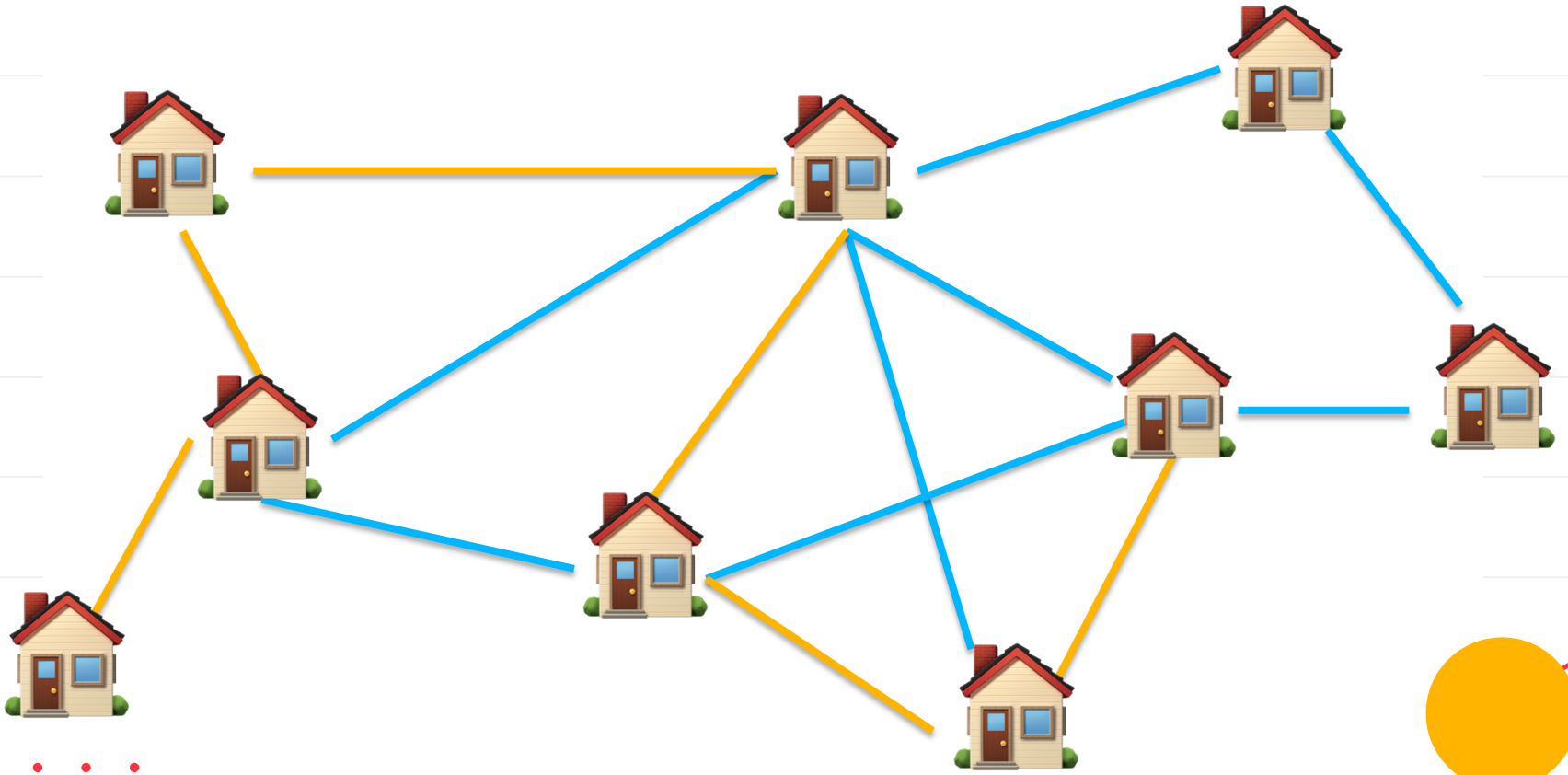
Practice: Hamiltonian Path



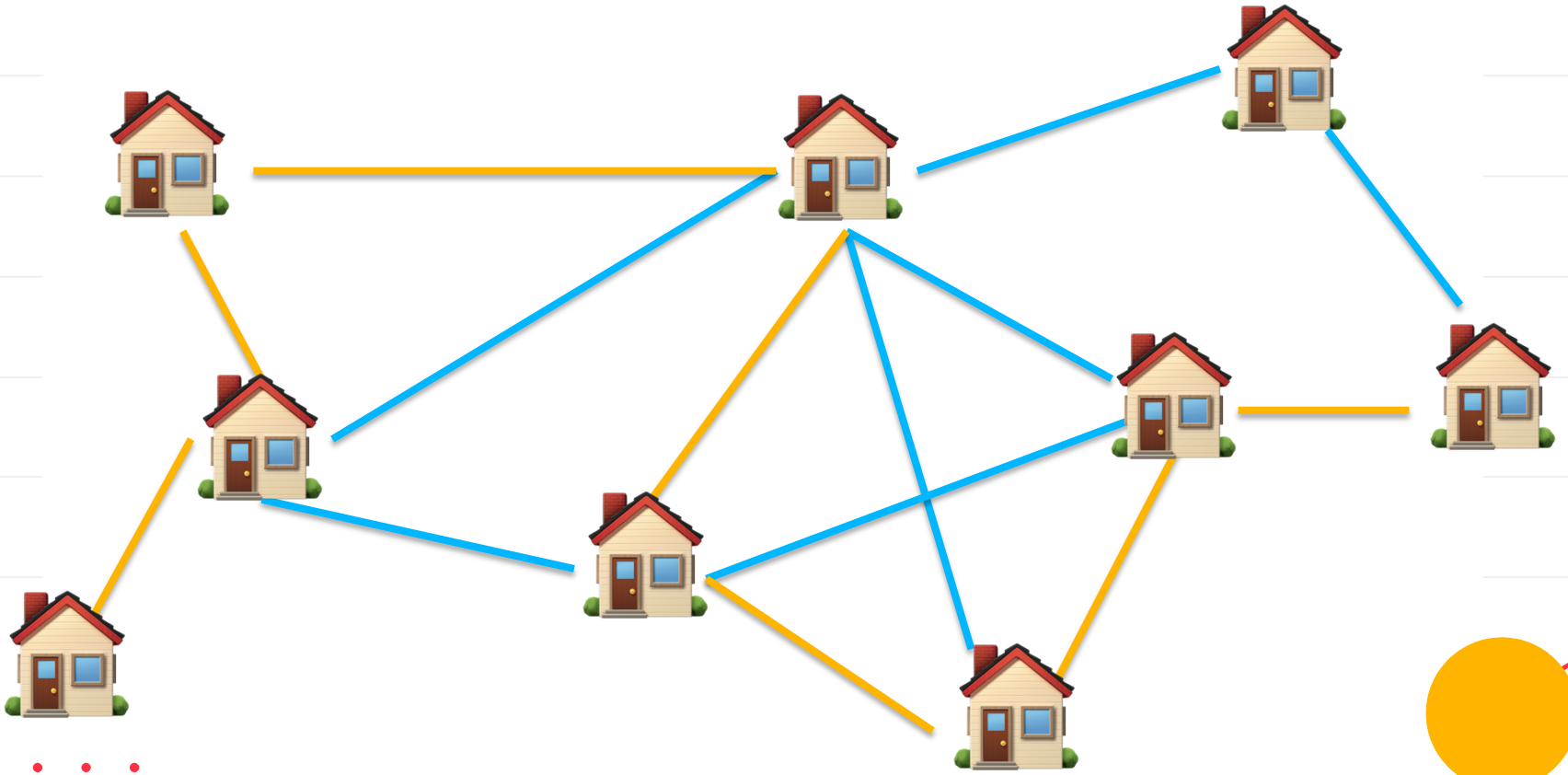
Practice: Hamiltonian Path



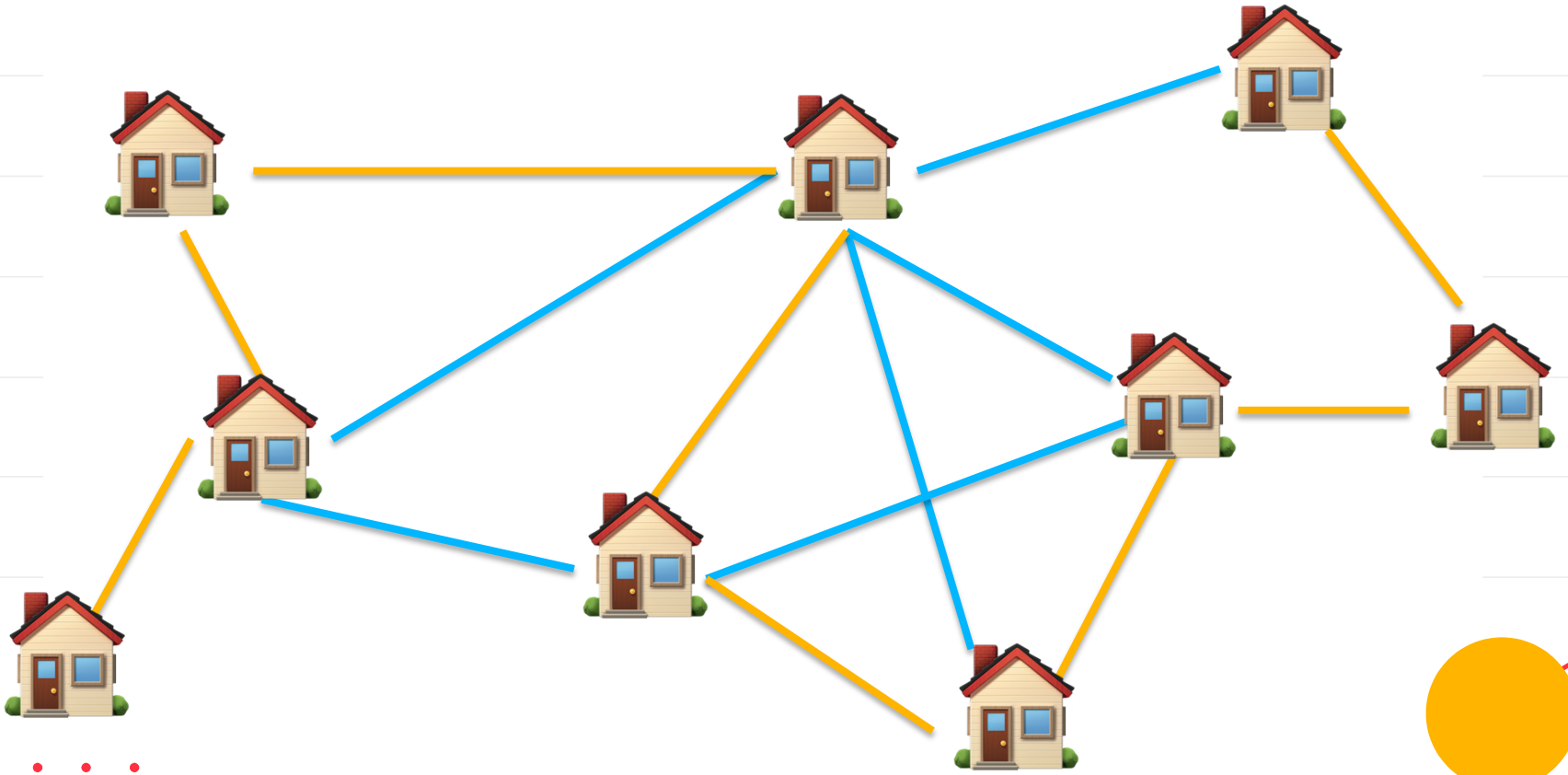
Practice: Hamiltonian Path



Practice: Hamiltonian Path



Practice: Hamiltonian Path



Practice: Hamiltonian Path

We find a transformation witnessing Ham $\leq_p$ SAT

A path has n slots



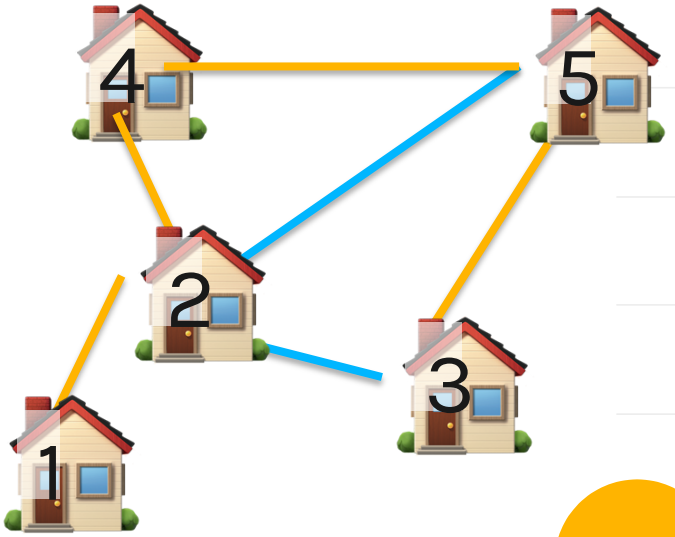
We create n^2 variables x_{ij}.
If x_{ij} is true, node i occupies slot j

Two nodes cannot occupy the same slot

Each slot must be occupied

No node can occupy two different slots

Adjacent slots must be occupied by nodes connected by an edge



Practice: Hamiltonian Path

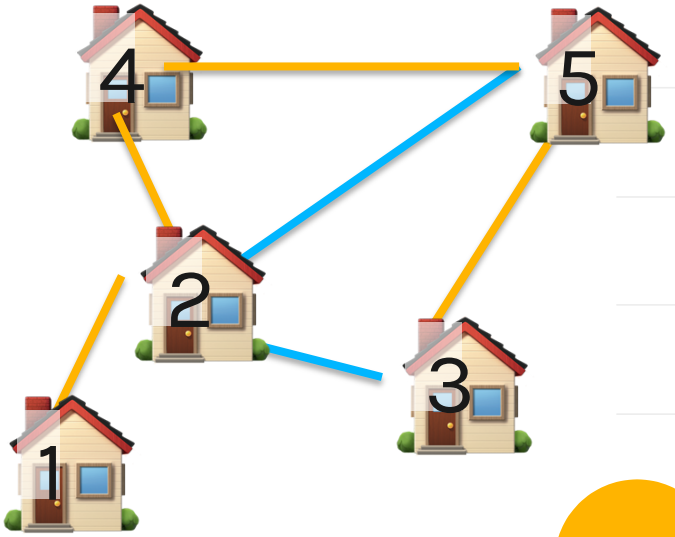
We find a transformation witnessing Ham $\leq_p$ SAT

Two nodes cannot occupy the same slot

$\texttt{exactlyOne}(x_{1k}, x_{2k}, \ldots, x_{nk})$

for each slot k

n^3 clauses

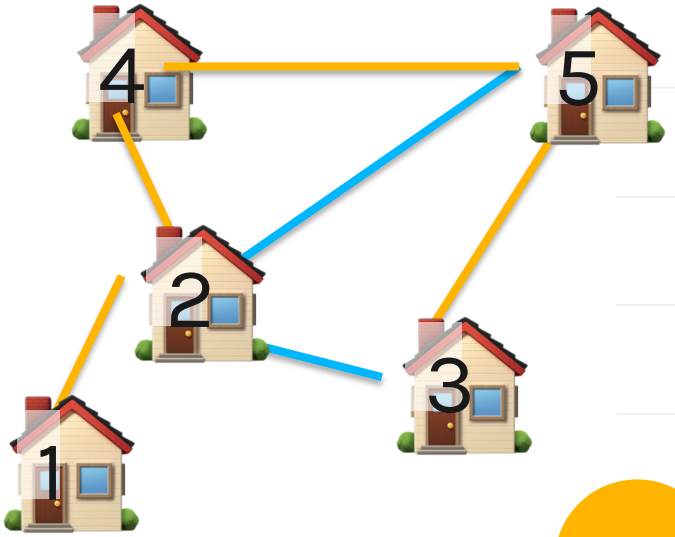


Practice: Hamiltonian Path

We find a transformation witnessing Ham $\leq_p$ SAT

Each slot must be occupied

already covered by the previous clauses



Practice: Hamiltonian Path

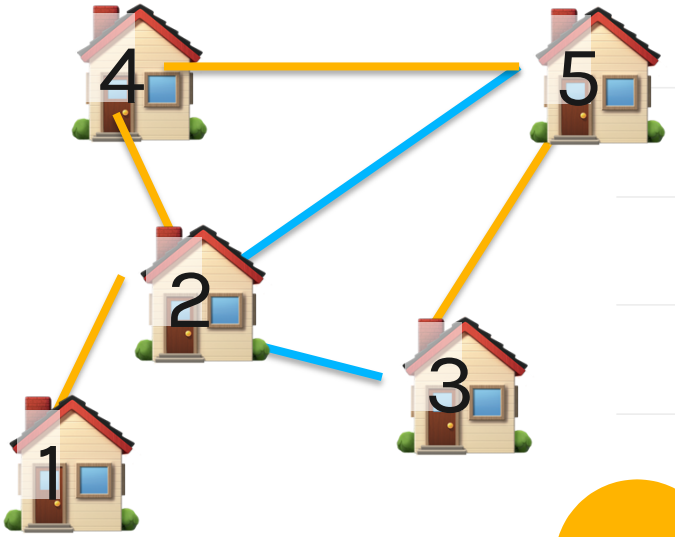
We find a transformation witnessing Ham $\leq_p$ SAT

No node can occupy two different slots

$\texttt{exactlyOne}(x_{i1}, x_{i2}, \ldots, x_{in})$

for each node i

n^3 clauses



Practice: Hamiltonian Path

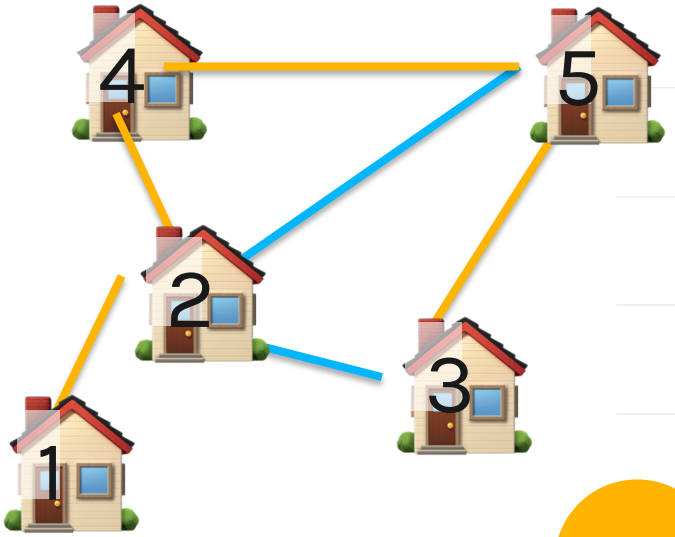
We find a transformation witnessing Ham $\leq_p$ SAT

Adjacent slots must be occupied by nodes connected by an edge



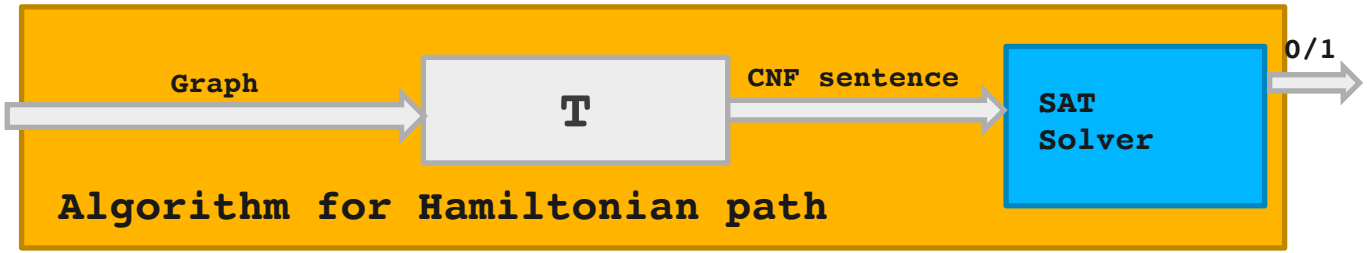
$$\bigwedge_{\substack{(i,j) \notin E \\ k < n}} (\neg x_{ik} \vee \neg x_{jk+1})$$

If no edge between `i` and `j` exists, then some node is not adjacent to the other



Practice: Hamiltonian Path

We must prove that there exists a Hamiltonian path in G ***iff*** the CNF formula we have built from G is satisfiable.



$\text{Ham} \leq_p \text{SAT}$

How is SAT useful to us?

We want to solve a difficult problem f, for which we do not know an algorithm:

We find a transformation witnessing $f \leq_p SAT$
We transform the input to our problem in a SAT formula and run a SAT solver.

usage 1

$LP \leq_p SAT$

$Ham \leq_p SAT$

How is SAT useful to us?

usage 2

We want to solve a difficult problem f, for which we do not know an algorithm:

We want to prove that a problem f, is NP-hard.

We find a transformation witnessing SAT $\leq_p$ f

We can also choose other NP-hard problems, not necessarily SAT.

SAT $\leq_p$ 3-SAT

SAT $\leq_p$ Subset-Sum

SAT $\leq_p$ k-Clique

3-SAT

Is a sentence in CNF, where each clause has exactly 3 literals satisfiable?

SAT $\leq_p$ 3-SAT

For each SAT sentence we build some 3-SAT sentence

The SAT sentence is satisfiable iff the 3-SAT sentence we build is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| = 1$ $\quad C_i = L$

We invent two variables, and the sentence:

$$D_i = (L \vee x_1 \vee x_2) \wedge (L \vee \neg x_1 \vee x_2) \wedge (L \vee x_1 \vee \neg x_2) \wedge (L \vee \neg x_1 \vee \neg x_2)$$

C_i is satisfiable iff D_i is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| = 2$ $\quad C_i = L_1 \vee L_2$

We invent one variable, and the sentence:

$$D_i = (L_1 \vee L_2 \vee x) \wedge (L_1 \vee L_2 \vee \neg x)$$

C_i is satisfiable iff D_i is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$

If $|C_i| = 3$ $\quad D_i = C_i$

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| > 3$ $\quad C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

We invent $k-3$ variables to be used as follows:

$$\begin{aligned} D_i = \; & (L_1 \vee L_2 \vee x_1) \wedge \\ & (\neg x_1 \vee L_3 \vee x_2) \wedge \\ & (\neg x_2 \vee L_4 \vee x_3) \wedge \\ & \ldots \\ & (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\ & (\neg x_{k-3} \vee L_{k-1} \vee L_k) \end{aligned}$$

$C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

We prove C_i is satisfiable iff D_i is satisfiable

$$
\begin{aligned}
D_i = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

$C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

$C = L_1 \vee L_2 \vee \ldots L_i \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

$C = L_1 \vee L_2 \vee \dots$

Make x_j true, for all $j \le i-1$

$D = (L_1 \vee L_2 \vee x_1) \wedge$
$(\neg x_1 \vee L_3 \vee x_2) \wedge$
$(\neg x_2 \vee L_4 \vee x_3) \wedge$
$\dots$
$(\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge$
$\dots$
$(\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge$
$(\neg x_{k-3} \vee L_{k-1} \vee L_k)$

This clause is true.

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

Make x_j true, for all $j \ge i$

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

($\Leftarrow$) Suppose D is satisfiable

If no x_i is true, then $L_1 \vee L_2$ is true.

$C = L_1 \vee L_2 \vee \ldots L_i \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇐) Suppose D is satisfiable

If no x_i is false, then $L_{k-1} \vee L_k$ is true.

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇐) Suppose D is satisfiable

If x_1 is false then $L_1 \vee L_2$ is true.

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

($\Leftarrow$) Suppose D is satisfiable

If some x_i is true and some x_{i+1} is false, then L_{i+2} is true.

If x_{k-3} is true then $L_{k-1} \vee L_k$ is true.

We have proved that 3SAT is NP-complete.



Analiza Algoritmilor

SAT and a few polynomial reductions

Matei Popovici

Boolean sentences

Start with a set of propositions

`Props = {p, q, r, … , x, y, z, …}`

Sometimes they are called variables.
It is not a mistake but it is awkward.

Boolean sentences

Start with a set of propositions

$\mathtt{Props} = \{p, q, r, \ldots, x, y, z, \ldots\}$

A proposition can be true or false.

An interpretation tells us which propositions are true and which are false.

$\mathtt{I} : \mathtt{Props} \rightarrow \{0,1\}$ or

$\mathtt{I} \subseteq \mathtt{Props}$

Propositions in $\mathtt{I}$ are held to be true and the rest – false.

Boolean sentences

A sentence φ is a proposition ($\varphi = p$)

or a conjunction ($\varphi = \varphi_1 \wedge \varphi_2$)

or a disjunction ($\varphi = \varphi_1 \vee \varphi_2$)

or a negation ($\varphi = \neg\varphi_1$)

or parentheses ($\varphi = (\varphi_1)$)

parentheses constructors per se, but they help us disambiguate propositions just like in arithmetic

Boolean sentences

The sentence $\varphi_1 \rightarrow \varphi_2$ means $\neg\varphi_1 \vee \varphi_2$

The sentence $\varphi_1 \leftrightarrow \varphi_2$ means $(\varphi_1 \rightarrow \varphi_2) \wedge (\varphi_2 \rightarrow \varphi_1)$

Conjunctive Normal Form (CNF)

The sentence φ is in CNF iff

$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$ where

$C_i = L_{i1} \wedge L_{i2} \wedge \ldots \wedge L_{ik}$ where

$L_{ij} = p$ or $\neg p$ where p is any proposition

Conjunctive Normal Form (CNF) vs arbitrary form

$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$

Representing a sentence requires a tree

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

We can transform this sentence in CNF

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg(q \vee \neg r))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q \wedge r)))$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = (p \vee q) \\ (p \vee \neg r) \\ (\neg p \vee \neg q) \\ (\neg p \vee r)$$

each line is a conjunction

Conjunctive Normal Form (CNF) vs arbitrary form

$$
\begin{aligned}
\varphi = (\ & p \vee q \quad\ \ ) \\
(\ & p \vee \quad\ \ \neg r) \\
(\neg & p \vee \neg q \quad\ ) \\
(\neg & p \vee \quad\ \ r)
\end{aligned}
$$

each line is a conjunction

Conjunctive Normal Form (CNF) vs arbitrary form

$$\varphi = \begin{matrix} 1 & 1 & 0 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \\ -1 & 0 & 1 \end{matrix}$$

each element L_{ij} tells us if proposition j appears positive, negated, or not at all, in clause i

Boolean sentences

A sentence is called **satisfiable** iff there **exists** an interpretation (for each proposition of the formula), which makes the formula true.

Is this formula **satisfiable**?

$$\varphi = (p \vee q \wedge \neg r) \wedge \neg(p \wedge (q \vee \neg r))$$

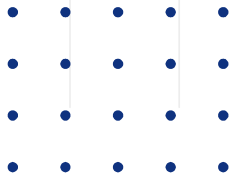
Hard to ***eyeball*** the answer



Boolean sentences

Is this formula **satisfiable**?

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$



Boolean sentences

Is this formula satisfiable?

$$\varphi = (p \vee q) \wedge (p \vee \neg r) \wedge (\neg p \vee \neg q) \wedge (\neg p \vee r)$$

We search for an interpretation which makes all clauses true

In each clause at least one literal must be true

SAT:

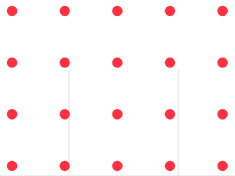
Given a sentence in CNF, is it satisfiable?

SAT ∈ *NP*

```
SAT-algorithm(phi, n, m):
I = array of size m.

for p = 1,m:
    I[p] = choice({-1,1})

for i = 1,n:
    current_clause = false
    for l = 1,m:
        if phi[i][j]*I[j] > 0:
            current_clause = true
    if !current_clause:
        fail
success
```



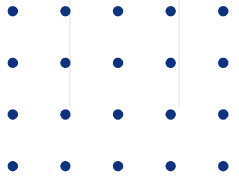
SAT:

Given a sentence in CNF, is it satisfiable?

SAT is ***NP-complete***

Cook-Levine Theorem (1970)





How is SAT useful to us?

usage 1

We want to solve a difficult problem f, for which we do not know an algorithm:

- We find a transformation witnessing $f \leq_p SAT$
  We transform the input to our problem in a SAT formula and run a SAT solver.

usage 2

We want to prove that a problem f, is NP-hard.

- We find a transformation witnessing $SAT \leq_p f$
  - We can also choose other NP-hard problems, not necessarily SAT.

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$T_{11} + T_{12} + \dots + T_{1n1} \leq c_1$$
$$T_{21} + T_{22} + \dots + T_{2n2} \leq c_2$$
$$\dots$$
$$T_{k1} + T_{k2} + \dots + T_{knk} \leq c_k$$

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$x_1 + x_2 + x_3 = 1$ *exactly one variable is 1*

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)$$

not in CNF

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)$$

$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee \text{rest}$

$$(x_1 \vee \text{rest}) \wedge (\neg x_2 \vee \text{rest}) \wedge (\neg x_3 \vee \text{rest})$$

$(x_1 \vee \text{rest}) \wedge$
$(\neg x_2 \vee \text{rest}) \wedge$
$(\neg x_3 \vee \text{rest})$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))$$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))$$

$$(x_1 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest}) \wedge$$
$$(\neg x_2 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest}) \wedge$$
$$(\neg x_3 \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee \text{rest})$$

$$((x_1 \vee \neg x_1 \vee \text{rest}) \wedge (x_1 \vee x_2 \wedge \text{rest}) \wedge (x_1 \vee \neg x_3 \wedge \text{rest})) \wedge$$
$$((\neg x_2 \vee \neg x_1 \vee \text{rest}) \wedge (\neg x_2 \vee x_2 \vee \text{rest}) \wedge (\neg x_2 \vee \neg x_3 \vee \text{rest})) \wedge$$
$$((\neg x_3 \vee \neg x_1 \vee \text{rest}) \wedge (\neg x_3 \vee x_2 \vee \text{rest}) \wedge (\neg x_3 \vee \neg x_3 \vee \text{rest}))$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(x_1 \vee x_2 \vee \text{rest}) \wedge \\
&(x_1 \vee \neg x_3 \vee \text{rest})) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(\neg x_2 \vee x_2 \vee \text{rest}) \wedge \\
&(\neg x_2 \vee \neg x_3 \vee \text{rest})) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee \text{rest}) \wedge \\
&(\neg x_3 \vee x_2 \vee \text{rest}) \wedge \\
&(\neg x_3 \vee \neg x_3 \vee \text{rest}))
\end{aligned}
$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(x_1 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(x_1 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_2 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_2 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_3 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge \\
&(\neg x_3 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)))
\end{aligned}
$$

$$((x_1 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(x_1 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(x_1 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge$$
$$((\neg x_2 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_2 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3))) \wedge$$
$$((\neg x_3 \vee \neg x_1 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee x_2 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)) \wedge$$
$$(\neg x_3 \vee \neg x_3 \vee (\neg x_1 \wedge \neg x_2 \wedge x_3)))$$

$$
\begin{aligned}
&((x_1 \vee \neg x_1 \vee \neg x_1) \wedge (x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_1 \vee x_3)) \wedge \\
&((x_1 \vee x_2 \vee \neg x_1) \wedge (x_1 \vee x_2 \vee \neg x_2) \wedge (x_3 \vee x_1 \vee x_2)) \wedge \\
&((x_1 \vee \neg x_3 \vee \neg x_1) \wedge (x_1 \vee \neg x_3 \vee \neg x_2) \wedge (x_1 \vee \neg x_3 \vee x_3)) \wedge \\
&((\neg x_2 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_1 \vee x_3)) \wedge \\
&((\neg x_2 \vee x_2 \vee \neg x_1) \wedge (\neg x_2 \vee x_2 \vee \neg x_2) \wedge (\neg x_2 \vee x_2 \vee x_3)) \wedge \\
&((\neg x_2 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_3)) \wedge \\
&((\neg x_3 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_1 \vee x_3)) \wedge \\
&((\neg x_3 \vee x_2 \vee \neg x_1) \wedge (\neg x_3 \vee x_2 \vee \neg x_2) \wedge (\neg x_3 \vee x_2 \vee x_3)) \wedge \\
&((\neg x_3 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_3 \vee x_3)))
\end{aligned}
$$

$$(x_1 \vee \neg x_1 \vee \neg x_1) \wedge (x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_1 \vee x_3) \wedge$$
$$(x_1 \vee x_2 \vee \neg x_1) \wedge (x_1 \vee x_2 \vee \neg x_2) \wedge (x_3 \vee x_1 \vee x_2) \wedge$$
$$(x_1 \vee \neg x_3 \vee \neg x_1) \wedge (x_1 \vee \neg x_3 \vee \neg x_2) \wedge (x_1 \vee \neg x_3 \vee x_3) \wedge$$
$$(\neg x_2 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_1 \vee x_3) \wedge$$
$$(\neg x_2 \vee x_2 \vee \neg x_1) \wedge (\neg x_2 \vee x_2 \vee \neg x_2) \wedge (\neg x_2 \vee x_2 \vee x_3) \wedge$$
$$(\neg x_2 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_2 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3 \vee x_3) \wedge$$
$$(\neg x_3 \vee \neg x_1 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_1 \vee x_3) \wedge$$
$$(\neg x_3 \vee x_2 \vee \neg x_1) \wedge (\neg x_3 \vee x_2 \vee \neg x_2) \wedge (\neg x_3 \vee x_2 \vee x_3) \wedge$$
$$(\neg x_3 \vee \neg x_3 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_3 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_3 \vee x_3)$$

Original formula:

$$(x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3).$$

CNF transformation is **exponential in size** in the worst case

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$(\neg x_1 \vee \neg x_2) \wedge$ *one of* x_1, x_2 *is* ***zero****, and*
$(\neg x_1 \vee \neg x_3) \wedge$ *one of* x_1, x_3 *is* ***zero****, and*
$(\neg x_1 \vee \neg x_4) \wedge$ *one of* x_1, x_4 *is* ***zero****, and*
$(\neg x_2 \vee \neg x_3) \wedge$ *one of* x_2, x_3 *is* ***zero*** *, and*
$(\neg x_2 \vee \neg x_4) \wedge$ *one of* x_2, x_4 *is* ***zero*** *, and*
$(\neg x_3 \vee \neg x_4)$ *one of* x_3, x_4 *is* ***zero*** *.*

$O(n^2)$ literals for n variables

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

at most one variable is 1

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) = \bigwedge_{i \neq j} (\neg x_i \vee \neg x_j)$$

We want to solve a difficult problem **f**, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$x_1 + x_2 + x_3 + x_4 \leq 1$ *at most one variable is 1*

$x_1 + x_2 + x_3 = 1$ *exactly one variable is 1*

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

at most one variable is 1

$$x_1 + x_2 + x_3 = 1$$

$$\text{exactlyOne}(x_1, x_2, \ldots, x_n) =$$

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) \wedge$$

$$\text{atLeastOne}(x_1, x_2, \ldots, x_n)$$

We want to solve a difficult problem f, for which we do not know an algorithm

Linear programming

Is a set of inequalities over integers satisfiable?

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

$$x_1 + x_2 + x_3 = 1$$

at most one variable is 1

$$\text{exactlyOne}(x_1, x_2, \ldots, x_n) =$$

$$\text{atMostOne}(x_1, x_2, \ldots, x_n) \wedge$$

$$(x_1 \vee x_2 \vee \ldots \vee x_n)$$

This set of inequalities is satisfiable

$$x_1 + x_2 + x_3 + x_4 \leq 1$$

$$x_1 + x_2 + x_3 = 1$$

if and only if the following CNF formula is satisfiable

$$(\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_4) \wedge (\neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

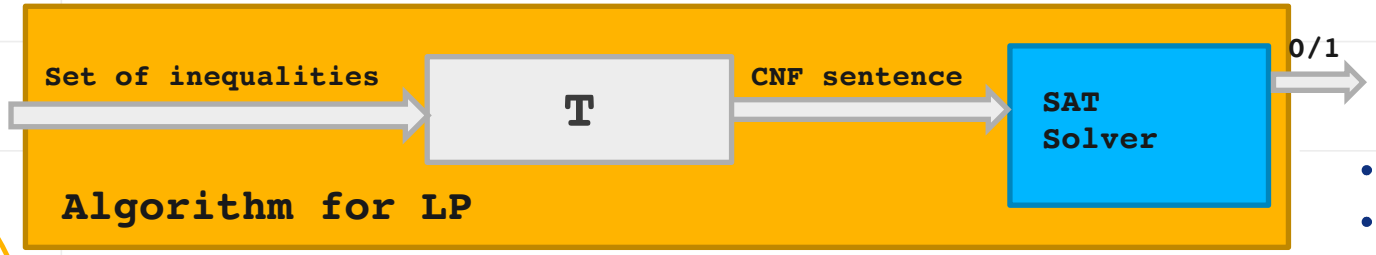
We also know of encodings for more general equations where c > 1

$$T_{11} + T_{12} + \ldots + T_{1n1} \leq c_1$$

They are more elaborate and go beyond the scope of this lecture

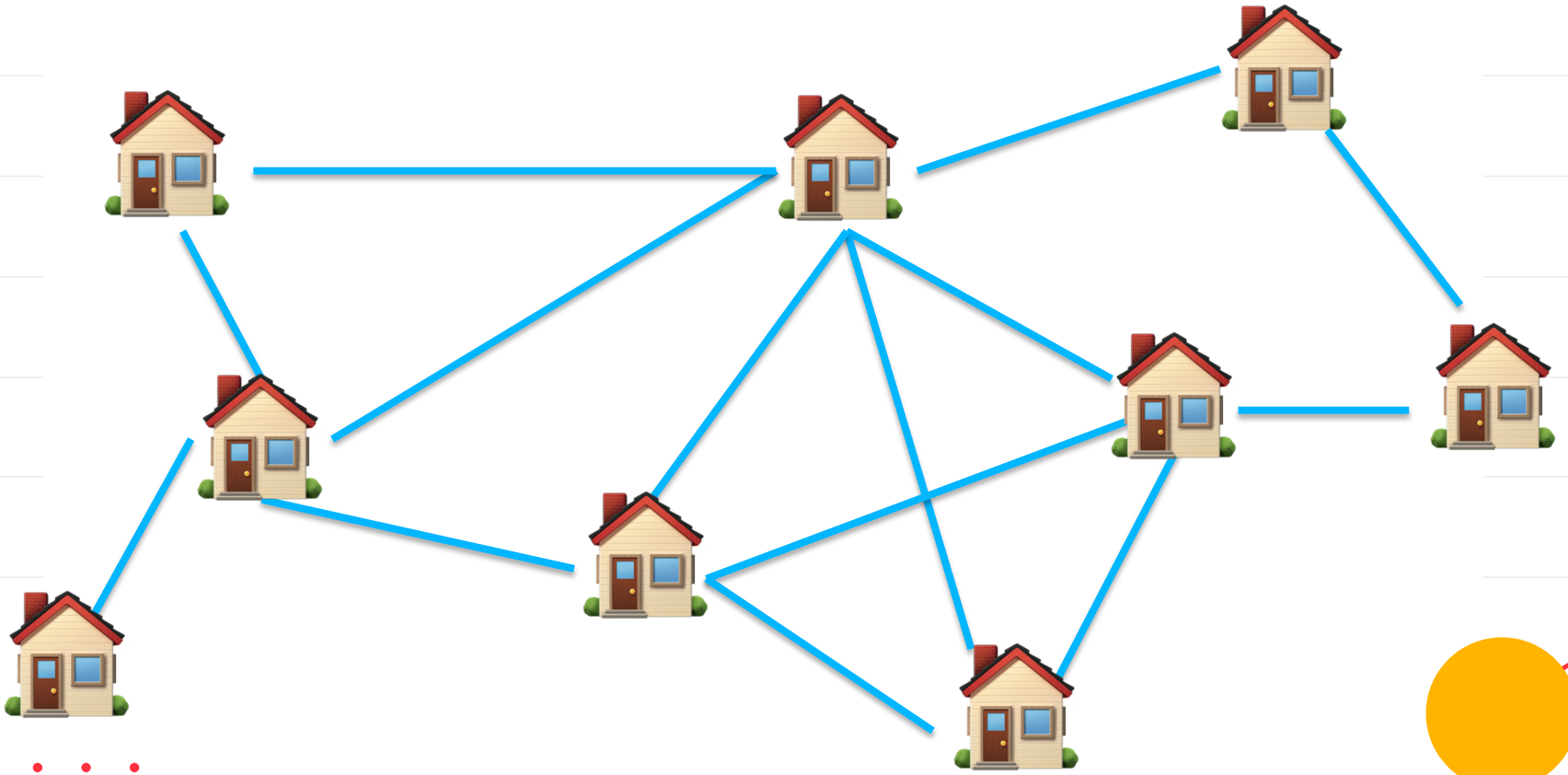
We must prove that the inequalities are satisfiable *iff* the CNF formula we have built is satisfiable.

$$T_{11} + T_{12} + \dots + T_{1n1} \leq c_1$$
$$T_{21} + T_{22} + \dots + T_{2n2} \leq c_2$$
$$\dots$$
$$T_{k1} + T_{k2} + \dots + T_{knk} \leq c_k$$

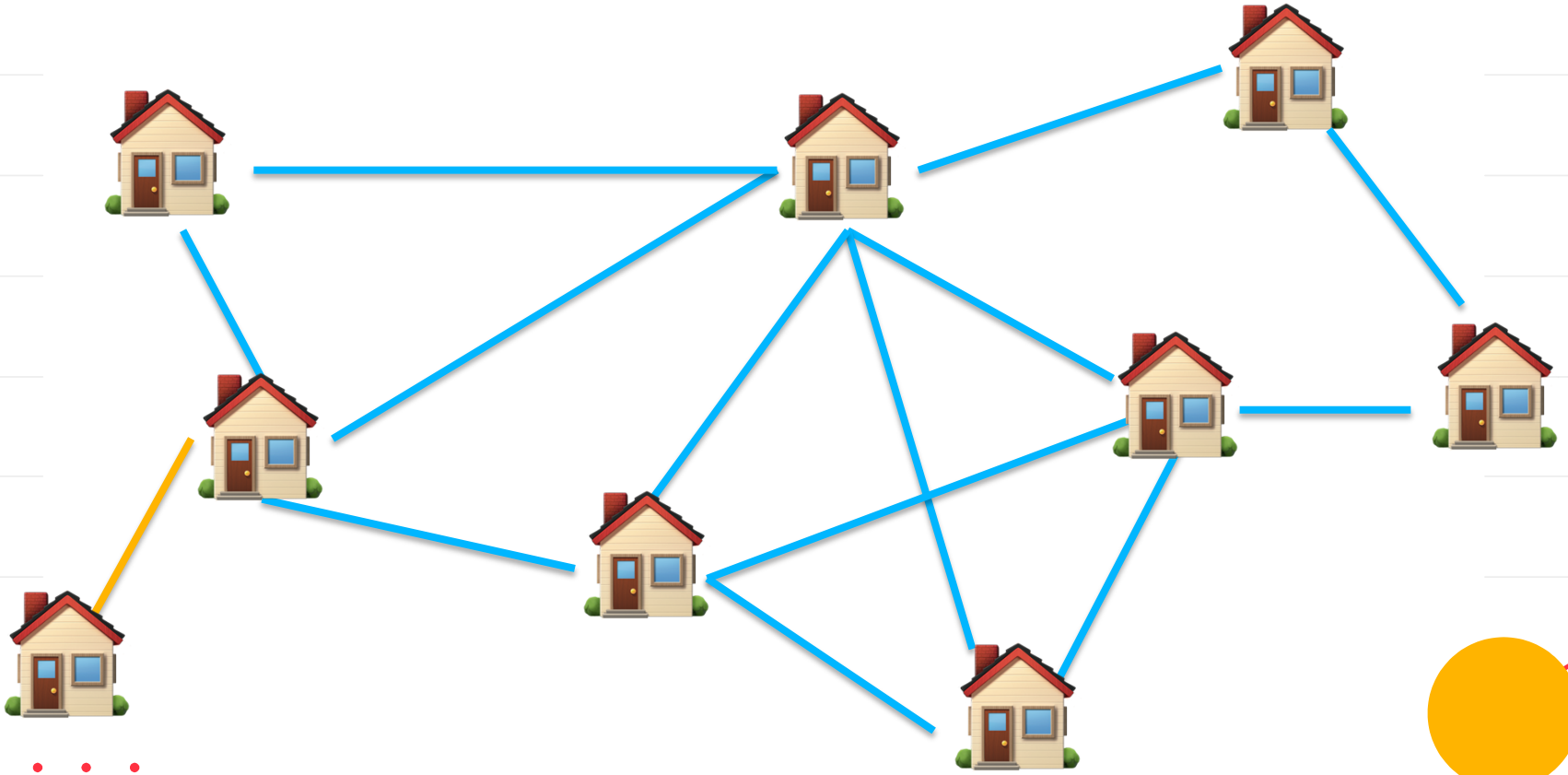


$LP \leq_p SAT$

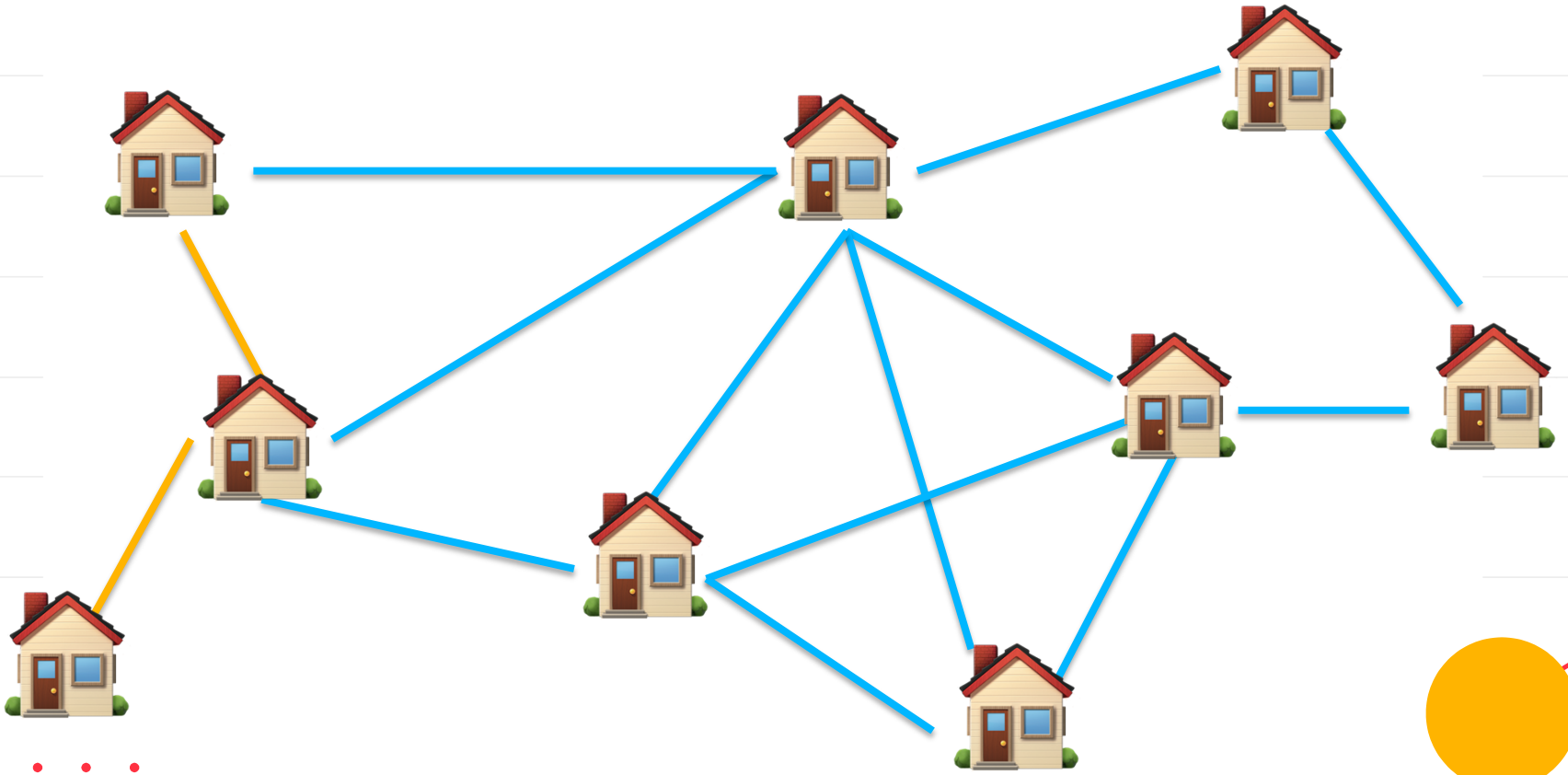
Practice: Hamiltonian Path



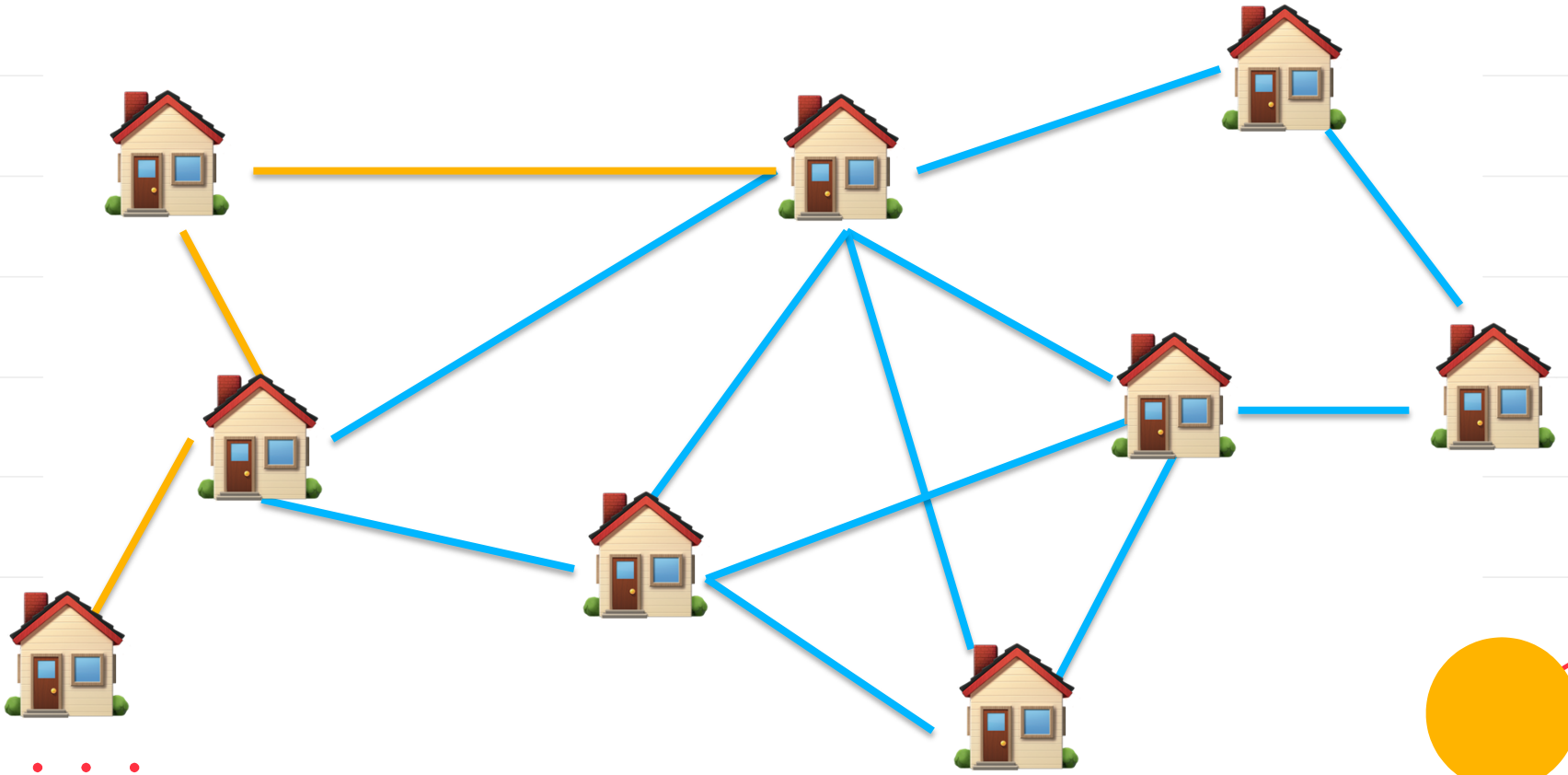
Practice: Hamiltonian Path



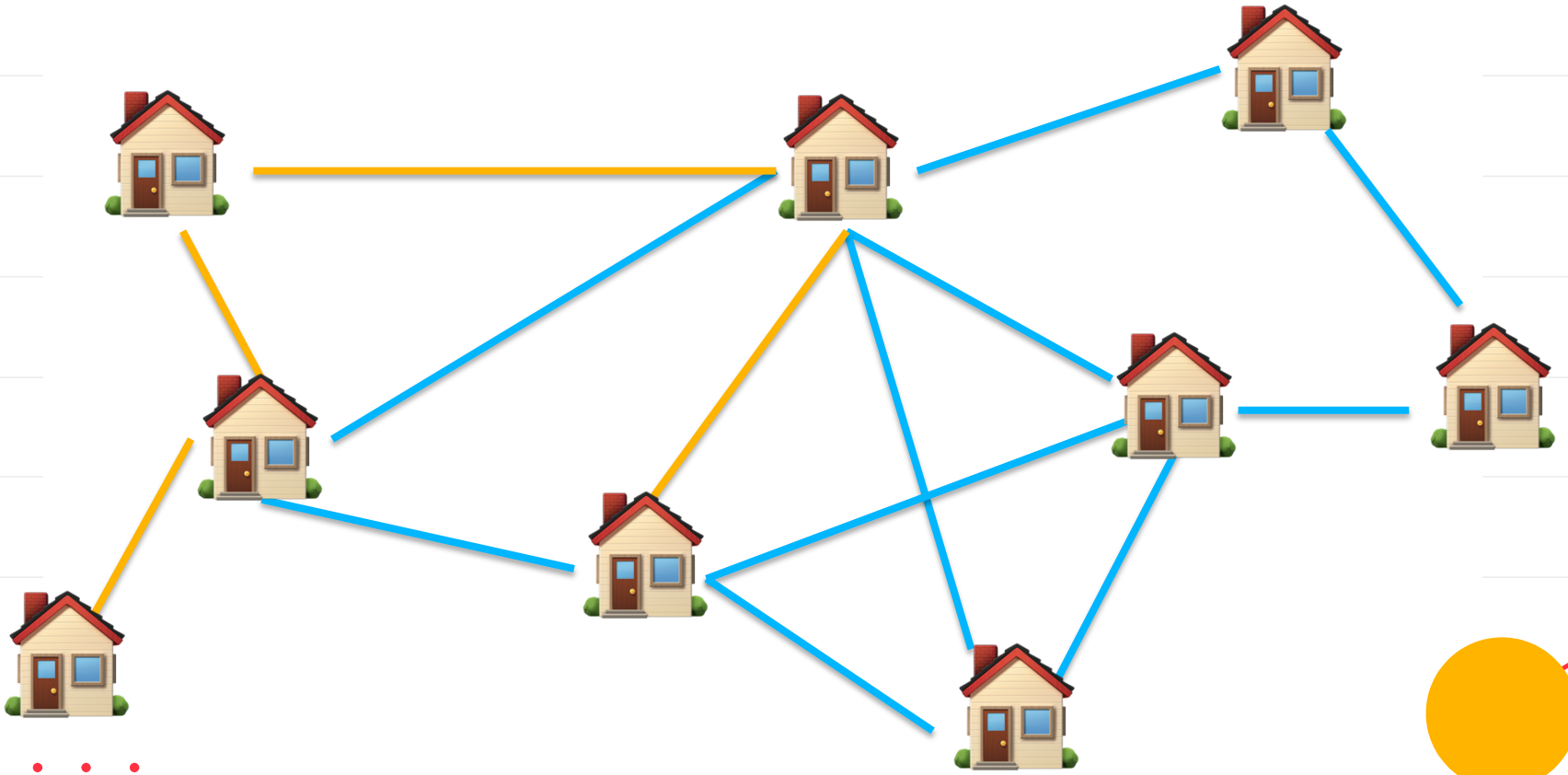
Practice: Hamiltonian Path



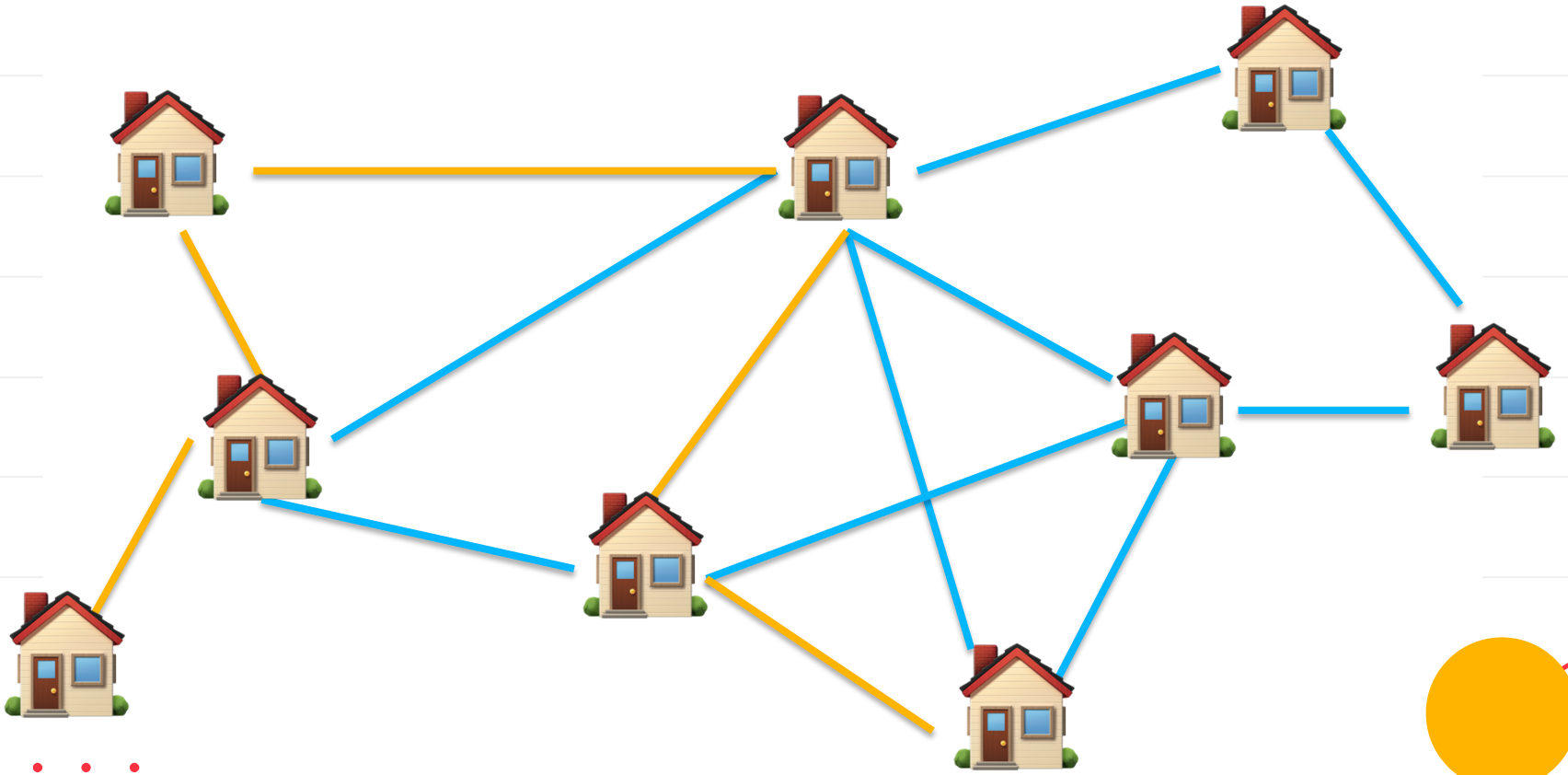
Practice: Hamiltonian Path



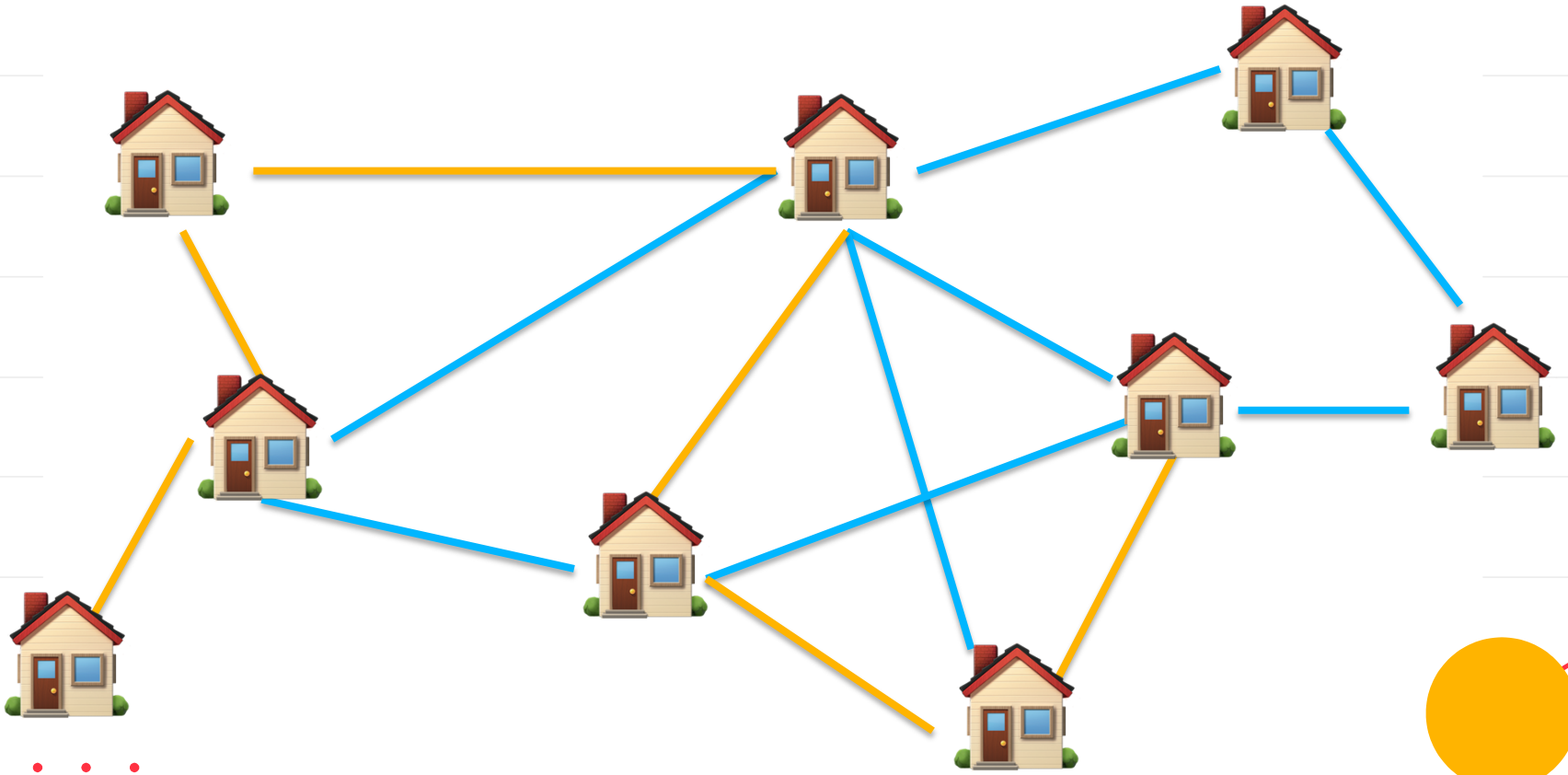
Practice: Hamiltonian Path



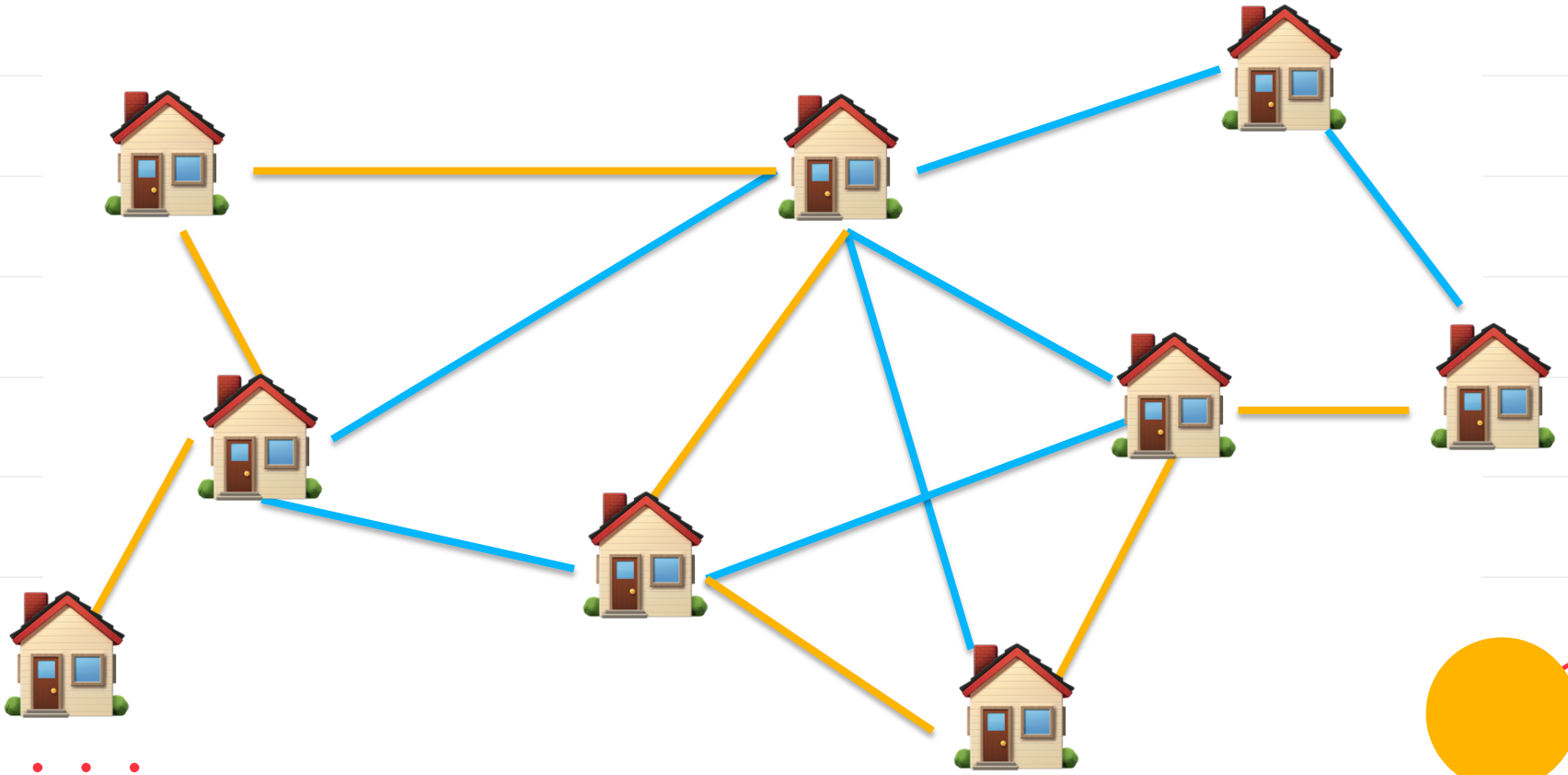
Practice: Hamiltonian Path



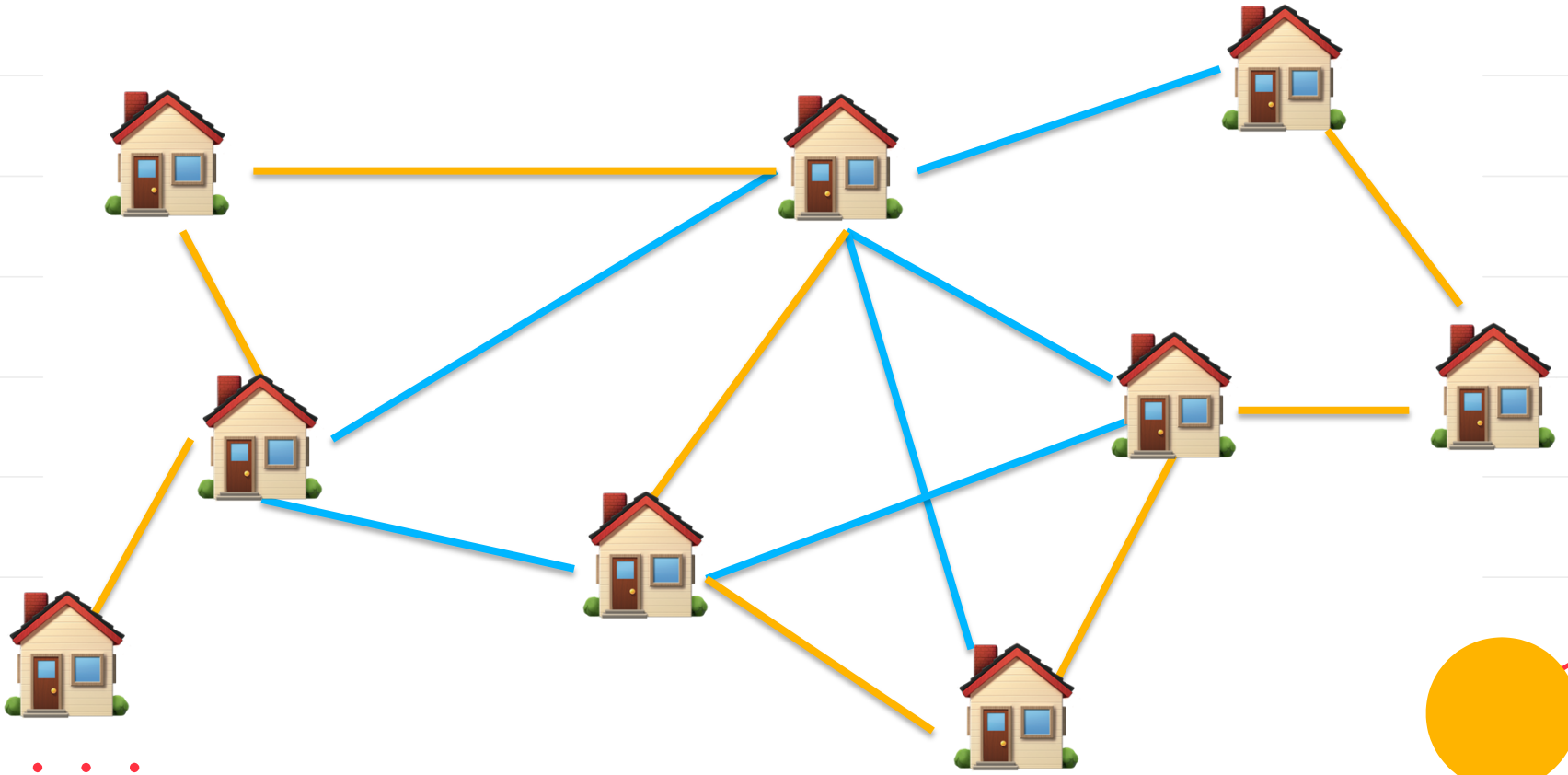
Practice: Hamiltonian Path



Practice: Hamiltonian Path



Practice: Hamiltonian Path



Practice: Hamiltonian Path

We find a transformation witnessing Ham $\leq_p$ SAT

A path has n slots



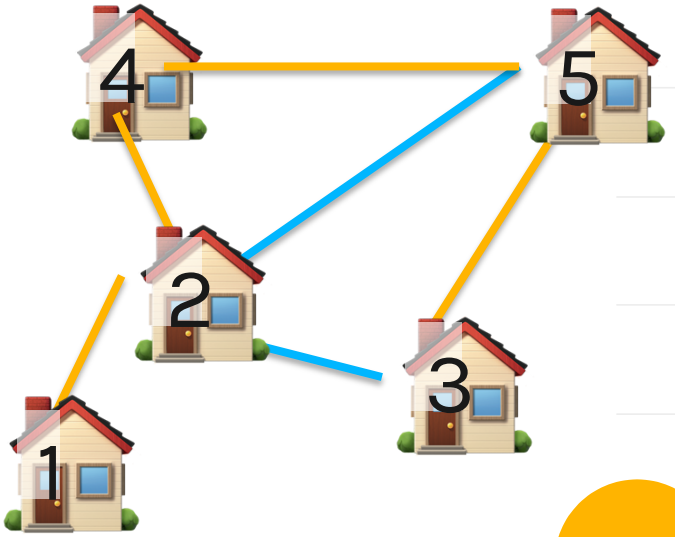
We create n^2 variables x_{ij}.
If x_{ij} is true, node i occupies slot j

Two nodes cannot occupy the same slot

Each slot must be occupied

No node can occupy two different slots

Adjacent slots must be occupied by nodes connected by an edge



Practice: Hamiltonian Path

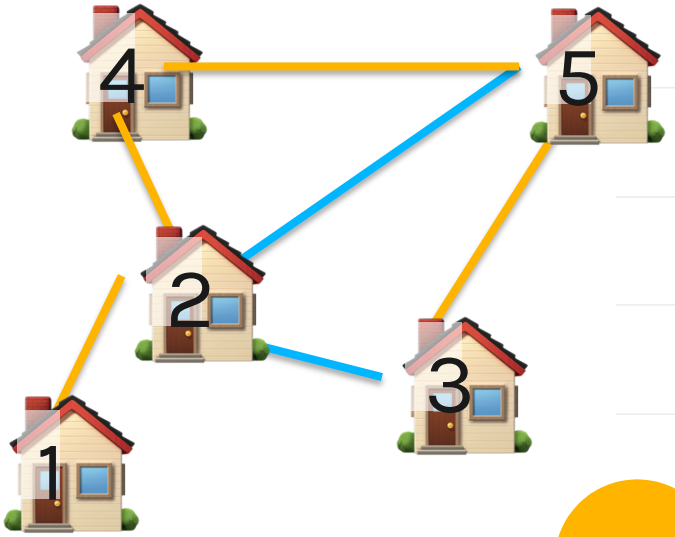
We find a transformation witnessing Ham $\leq_p$ SAT

Two nodes cannot occupy the same slot

$\texttt{exactlyOne}(x_{1k}, x_{2k}, \ldots, x_{nk})$

for each slot k

n^3 clauses

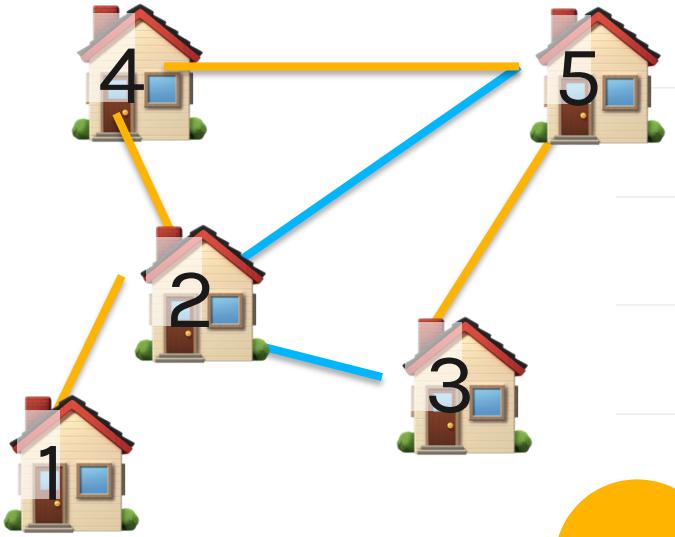


Practice: Hamiltonian Path

We find a transformation witnessing Ham $\leq_p$ SAT

Each slot must be occupied

already covered by the previous clauses



Practice: Hamiltonian Path

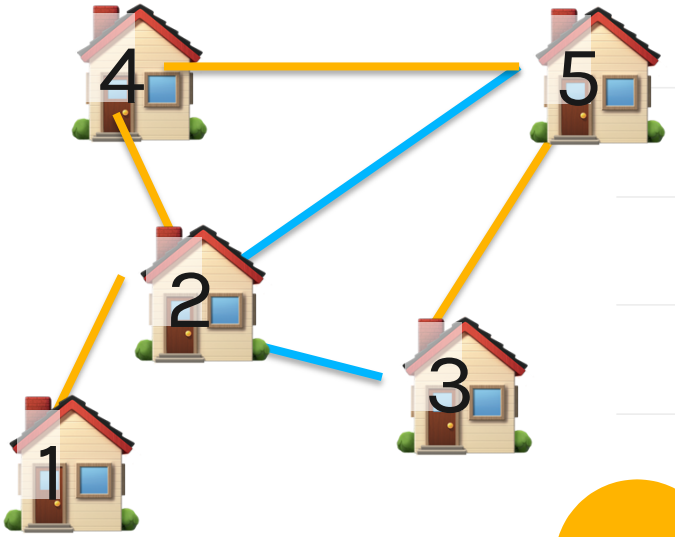
We find a transformation witnessing Ham $\leq_p$ SAT

No node can occupy two different slots

$\texttt{exactlyOne}(x_{i1}, x_{i2}, \ldots, x_{in})$

for each node i

n^3 clauses



Practice: Hamiltonian Path

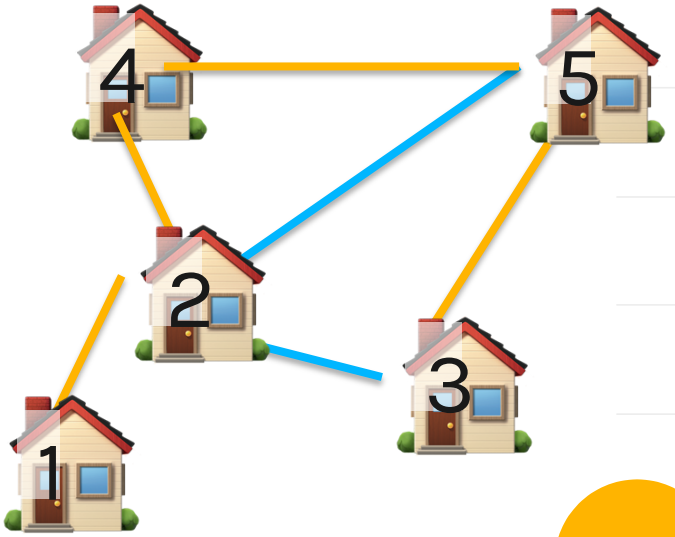
We find a transformation witnessing Ham $\leq_p$ SAT

Adjacent slots must be occupied by nodes connected by an edge



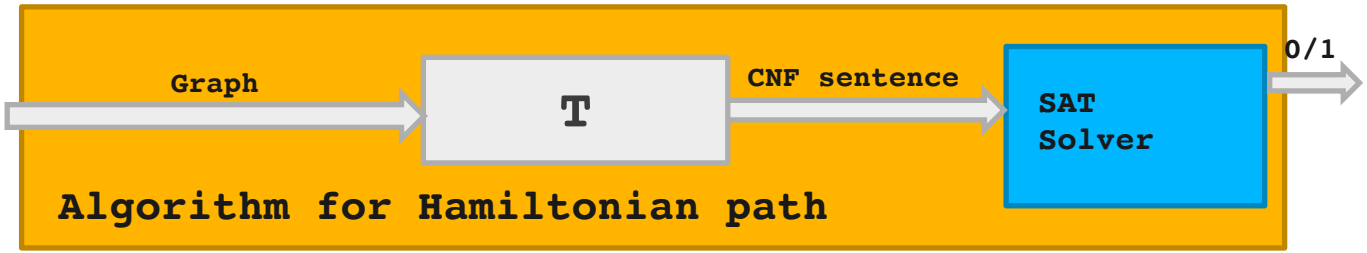
$$\bigwedge_{\substack{(i,j) \notin E \\ k < n}} (\neg x_{ik} \vee \neg x_{jk+1})$$

If no edge between `i` and `j` exists, then some node is not adjacent to the other



Practice: Hamiltonian Path

We must prove that there exists a Hamiltonian path in G ***iff*** the CNF formula we have built from G is satisfiable.



$\text{Ham} \leq_p \text{SAT}$

How is SAT useful to us?

We want to solve a difficult problem f, for which we do not know an algorithm:

We find a transformation witnessing $f \leq_p SAT$
We transform the input to our problem in a SAT formula and run a SAT solver.

usage 1

$LP \leq_p SAT$

$Ham \leq_p SAT$

How is SAT useful to us?

usage 2

We want to solve a difficult problem f, for which we do not know an algorithm:

We want to prove that a problem f, is NP-hard.

We find a transformation witnessing SAT $\leq_p$ f

We can also choose other NP-hard problems, not necessarily SAT.

SAT $\leq_p$ 3-SAT

SAT $\leq_p$ Subset-Sum

SAT $\leq_p$ k-Clique

3-SAT

Is a sentence in CNF, where each clause has exactly 3 literals satisfiable?

SAT $\leq_p$ 3-SAT

For each SAT sentence we build some 3-SAT sentence

The SAT sentence is satisfiable iff the 3-SAT sentence we build is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| = 1$ $\quad C_i = L$

We invent two variables, and the sentence:

$$D_i = (L \vee x_1 \vee x_2) \wedge (L \vee \neg x_1 \vee x_2) \wedge (L \vee x_1 \vee \neg x_2) \wedge (L \vee \neg x_1 \vee \neg x_2)$$

C_i is satisfiable iff D_i is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| = 2$ $\quad C_i = L_1 \vee L_2$

We invent one variable, and the sentence:

$$D_i = (L_1 \vee L_2 \vee x) \wedge (L_1 \vee L_2 \vee \neg x)$$

C_i is satisfiable iff D_i is satisfiable

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$

If $|C_i| = 3$ $\quad D_i = C_i$

SAT $\leq_p$ 3-SAT

Start off with some SAT sentence:

$$\varphi = C_1 \wedge C_2 \wedge \ldots \wedge C_n$$

If $|C_i| > 3$ $\quad C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

We invent $k-3$ variables to be used as follows:

$$\begin{aligned} D_i = \; & (L_1 \vee L_2 \vee x_1) \wedge \\ & (\neg x_1 \vee L_3 \vee x_2) \wedge \\ & (\neg x_2 \vee L_4 \vee x_3) \wedge \\ & \ldots \\ & (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\ & (\neg x_{k-3} \vee L_{k-1} \vee L_k) \end{aligned}$$

$C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

We prove C_i is satisfiable iff D_i is satisfiable

$$
\begin{aligned}
D_i = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

$C_i = L_1 \vee L_2 \vee \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

$C = L_1 \vee L_2 \vee \ldots L_i \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

$C = L_1 \vee L_2 \vee \dots$

Make x_j true, for all $j \le i-1$

$D = (L_1 \vee L_2 \vee x_1) \wedge$
$(\neg x_1 \vee L_3 \vee x_2) \wedge$
$(\neg x_2 \vee L_4 \vee x_3) \wedge$
$\dots$
$(\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge$
$\dots$
$(\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge$
$(\neg x_{k-3} \vee L_{k-1} \vee L_k)$

This clause is true.

(⇒) Suppose **C** is satisfiable

Some interpretation **I** makes **C** true.

I makes some L_i true.

Make x_j true, for all $j \ge i$

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

($\Leftarrow$) Suppose D is satisfiable

If no x_i is true, then $L_1 \vee L_2$ is true.

$C = L_1 \vee L_2 \vee \ldots L_i \ldots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \ldots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \ldots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇐) Suppose D is satisfiable

If no x_i is false, then $L_{k-1} \vee L_k$ is true.

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

(⇐) Suppose D is satisfiable

If x_1 is false then $L_1 \vee L_2$ is true.

$C = L_1 \vee L_2 \vee \dots L_i \dots \vee L_k$

$$
\begin{aligned}
D = {} & (L_1 \vee L_2 \vee x_1) \wedge \\
& (\neg x_1 \vee L_3 \vee x_2) \wedge \\
& (\neg x_2 \vee L_4 \vee x_3) \wedge \\
& \dots \\
& (\neg x_{i-2} \vee L_i \vee x_{i-1}) \wedge \\
& \dots \\
& (\neg x_{k-4} \vee L_{k-2} \vee x_{k-3}) \wedge \\
& (\neg x_{k-3} \vee L_{k-1} \vee L_k)
\end{aligned}
$$

($\Leftarrow$) Suppose D is satisfiable

If some x_i is true and some x_{i+1} is false, then L_{i+2} is true.

If x_{k-3} is true then $L_{k-1} \vee L_k$ is true.

We have proved that 3SAT is NP-complete.